Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Hardware-Specific Notes

These notes provide information about a specific hardware product or component, highlighting performance optimizations, feature descriptions, and configuration options. The hardware-specific notes consist of the following chapters:

Zebra 4Sight
Zebra AltiZ
Zebra AltiZ 4200
Zebra Clarity UHD
Zebra Concord PoE
Zebra GenTL Consumer
Zebra GevIQ
Zebra GigE Vision driver
Zebra Indio
Zebra Iris GTX
Zebra Radient eV-CL
Zebra Rapixo CL
Zebra Rapixo CXP
Zebra Rapixo CoF
Zebra USB3 Vision driver
Zebra Video4Linux2 Consumer

Zebra 4Sight

This chapter discusses features of Aurora Imaging Library that are particular to Zebra 4Sight series, and ways that optimize its performance. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra 4Sight overview
  - Summary of Zebra 4Sight EV7 features
  - Summary of Zebra 4Sight XV7 features
- Using Zebra 4Sight with Aurora Imaging Library
  - Using the Advanced I/O Engine of Zebra 4Sight EV7
  - Using EtherNet/IP, Modbus, CC-Link, or PROFINET on Zebra 4Sight EV7
- Using timers and rotary decoders with Zebra 4Sight EV7
  - Timers
  - Quadrature decoders
- Zebra 4Sight EV7 connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra 4Sight data flow diagrams
  - Zebra 4Sight EV7
  - Zebra 4Sight XV7
  - Zebra 4Sight EV6
  - Zebra 4Sight XV6
- Miscellaneous reference information for Zebra 4Sight
  - Msys
    - MsysControl/Inquire
- Miscellaneous user guide information for Zebra 4Sight
  - Licensing

Zebra 4Sight overview

Zebra 4Sight is a series of vision controllers (imaging computers). A vision controller is a system that connects hardware components to image processing software, providing robust vision processing capabilities. There are two families:

- **Zebra 4Sight EV7**. Zebra 4Sight EV7 is a fanless industrial vision controller that comes with hardware-assisted PROFINET support. Zebra 4Sight EV7 integrates processing and display capabilities, along with image capture from multiple GigE Vision and/or USB3 Vision cameras. Zebra 4Sight EV7 has 2.5 Gigabit Ethernet networking capabilities with support for Power-over-Ethernet (PoE), as well as industrial communication using the PROFINET, EtherNet/IP, CC-Link, or Modbus/TCP industrial protocol.

  Zebra 4Sight EV7 also has an Advanced I/O Engine; the engine provides use of 16 optically-isolated digital auxiliary I/O signals (8 inputs and 8 outputs), and includes I/O command lists, timers, quadrature decoders, and user outputs.

- **Zebra 4Sight XV7**. Zebra 4Sight XV7 is a non-industrial vision controller that offers expandability. Zebra 4Sight XV7 integrates processing and display capability, along with image capture from USB3 Vision and/or GigE Vision cameras. Zebra 4Sight XV7 supports networking and industrial communication (over GigE Ethernet ports or a RS-232/RS-422/RS-485 port).

  Zebra 4Sight XV7 also supports add-on boards compliant with the full-height, half-length PCIe form factor. Zebra 4Sight XV7 has four PCIe connectors for connecting full-height, half-length add-on boards. For example, you can connect Zebra frame grabbers and/or Zebra digital I/O boards.

  > Zebra 4Sight XV7 does not have hardware-assisted PROFINET; however, you can connect a Zebra I/O add-on board that supports hardware-assisted PROFINET communication, such as the Zebra Indio.

Summary of Zebra 4Sight EV7 features

The following table outlines the features currently available for Zebra 4Sight EV7.

Features	Zebra 4Sight EV7
CPU	Yes (Intel Core i3-1220PE or i5-1250PE)
Host memory	Up to 64 Gbytes
Networking	Four (4) 2.5 Gigabit Ethernet ports with PoE Two (2) standard Gigabit Ethernet ports (1 of which provides hardware-assisted PROFINET)
USB	Four (4) USB 3.2 SuperSpeed ports Two (2) USB 2.0 ports

Number of supported UARTS	Two (2) serial ports (one RS-232 and one RS-232/RS-485)
Number of auxiliary signals	Eight (8) inputs Eight (8) outputs
PROFINET Engine	Yes
Number of timers	Sixteen (16)
Number of quadrature decoders	Two (2)
Number of I/O command lists	Two (2)
Display connectors	Two (2) DisplayPort connectors

> You can access the latest version of the Zebra 4Sight EV7 Installation and Hardware Reference at zebra.com/4sight-ev7-info. Click on the **Documentation** tab to access it.

Summary of Zebra 4Sight XV7 features

The following table outlines the features currently available for Zebra 4Sight XV7.

Features	Zebra 4Sight XV7
CPU	Yes (Intel i5-14501E or Intel i7-14700 or i9-14900)
Host memory	Up to 192 Gbytes
Interface type	PCIe
Networking [1]	Four (4) GigE Ethernet interfaces (100/1000/2500 BaseT Ethernet)
Number of supported UARTS	One (1) serial port connector (RS-232, RS-422, or RS-485)

Display connectors	Two (2) DisplayPort connectors One (1) HDMI connector One (1) VGA connector

[1] Zebra 4Sight XV7 does not support hardware-assisted PROFINET. To add PROFINET support to Zebra 4sight XV7, connect an add-on board with PROFINET support (for example, Zebra Indio).

> All of the information found in this chapter also applies to the predecessors of Zebra 4Sight XV7 (for example, Zebra 4Sight XV6). You can access the latest version of the Zebra 4Sight XV6 Installation and Hardware Reference at zebra.com/4sight-xv6-info. Click on the **Documentation** tab to access it.

Using Zebra 4Sight with Aurora Imaging Library

Aurora Imaging Library treats Zebra 4Sight units as a normal computer.

To use the processing power of Zebra 4Sight, allocate a Host system using MsysAlloc() with M_SYSTEM_HOST.

Zebra 4Sight EV7 has auxiliary signals capabilities. For more information, see the the Using the Advanced I/O Engine of Zebra 4Sight EV7 subsection of this section.

Zebra 4Sight EV7 allows you to use either GigE Vision-compliant cameras or USB3 Vision-compliant cameras directly connected to the unit.

Camera type	System name	Aurora Imaging Library system-specific information (Aurora Imaging Library Reference)
GigE Vision	M_SYSTEM_GIGE_VISION	GigE Vision
USB3 Vision	M_SYSTEM_USB3_VISION	USB3 Vision

Depending on the model of your Zebra 4Sight unit, it can come preinstalled with either the Windows 10 IoT or Windows 11 IoT Enterprise operating system and a runtime license for both Aurora Imaging Library and Aurora Design Assistant, or just Aurora Imaging Library. The runtime license includes the industrial and robot communication module and allows for Distributed Aurora Imaging Library configurations; this might change over time, contact your Zebra representative for more information.

If you are using a Zebra 4Sight XV7, you can install additional boards in the unit, such as a Zebra frame grabber or I/O board. To use the features of an installed board, you must first allocate the board as an Aurora Imaging Library system. For more information, refer to the Hardware-specific Notes for that particular board (for example, see the Using Zebra Indio with Aurora Imaging Library section of the Zebra Indio chapter in the Hardware-specific Notes for Zebra Indio).

> Note that when installing a redistribution (runtime) version of Aurora Imaging Library on Zebra 4Sight units, the license number will be based on the hardware fingerprint of your unit. This means that you will not be able to copy the licensed version of Aurora Imaging Library to another computer.

Using the Advanced I/O Engine of Zebra 4Sight EV7

Zebra 4Sight EV7 has an Advanced I/O Engine that controls the auxiliary I/O interface. The engine includes 16 timers, 2 quadrature decoders, 2 I/O command lists, 8 auxiliary output signals, and 8 auxiliary input signals. To route auxiliary I/O signals and configure interrupts, use MsysControl(). To inquire this information, use MsysInquire(). For more information on some of these features, refer to Chapter 56: I/O signals and communicating with external devices.

Using EtherNet/IP, Modbus, CC-Link, or PROFINET on Zebra 4Sight EV7

Zebra 4Sight EV7 supports industrial communication with external devices using protocols such as EtherNet/IP, Modbus (over TCP/IP), CC-Link, or PROFINET. To use hardware-assisted PROFINET, you must use the port that has access to the PROFINET engine (LAN2). The interface of the PROFINET engine is recognized as a second network device with its own IP and MAC settings (when the PROFINET service is enabled) even though it shares the same LAN

connection. To enable PROFINET, open the Aurora Imaging Configurator utility, expand the **Communication** tab, select the **PROFINET** page, and add a PROFINET protocol instance; note that instance names are case sensitive. Once the protocol instance is added, the status of the protocol instance should be **Ready**. If the status is **Waiting NIC**, there is no cable connected to the Ethernet port. By default, PROFINET is disabled.

For more information, see the Steps to perform industrial communication section of Chapter 57: Industrial communication.

Using timers and rotary decoders with Zebra 4Sight EV7

Zebra 4Sight EV7 has 16 timers and 2 quadrature decoders that can be used to synchronize with external devices. Timers are used with I/O signals to coordinate events with external devices. Quadrature decoders are used to decode quadrature input (2-bit Gray code derived from two signals) from a quadrature encoder.

Timers

The 16 timers on Zebra 4Sight EV7 can be configured using MsysControl(). The timers are configured in a similar way to what is described in the Timers and coordinating events section of Chapter 56: I/O signals and communicating with external devices. There are some differences, however, such as:

- A timer arming mechanism can be configured and enabled for each timer. If timer arming is enabled, the timer will ignore its trigger signal (MsysControl() with M_TIMER_TRIGGER_SOURCE) until a signal transition, specified using MsysControl() with M_TIMER_ARM_ACTIVATION, occurs on the signal specified using MsysControl() with M_TIMER_ARM_SOURCE. To enable timer arming, use MsysControl() with M_TIMER_ARM.
- The clock source can be specified for both the delay portion and duration portion of the timer output signal. For example, a rotary decoder output signal can be used to set the delay between the timer trigger and the active portion of the timer's output signal, and the unit's clock generator can be used to set the duration of the active portion of the timer's output signal.
- The timer can measure the duration of a pulse on a signal. To enable this functionality, use MsysControl() with M_TIMER_USAGE set to M_PULSE_MEASUREMENT. When a timer is set in this mode, the duration of pulses that occur on the timer's trigger source (M_TIMER_TRIGGER_SOURCE) are measured with respect to the clock source for the active portion of the timer (M_TIMER_CLOCK_SOURCE). To measure active-high pulses, set M_TIMER_TRIGGER_ACTIVATION to M_LEVEL_HIGH; to measure active-low pulses, set M_TIMER_TRIGGER_ACTIVATION to M_LEVEL_LOW. The measured value can be inquired using MsysInquire() with M_TIMER_VALUE. Note that when in this mode, the timer's output always remains low, and the values set using M_TIMER_DELAY_CLOCK_SOURCE, M_TIMER_DELAY, and M_TIMER_DURATION are ignored.

Quadrature decoders

The two quadrature decoders on Zebra 4Sight EV7 can be configured using MsysControl() with M_ROTARY_ENCODER_.... The quadrature decoders are configured in a similar way to what is described in the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. However, the following settings are unavailable when using Zebra 4Sight EV7: setting the direction of movement (M_ROTARY_ENCODER_DIRECTION), setting a signal source that changes the rotary decoder counter value (M_ROTARY_ENCODER_FORCE_VALUE_SOURCE), setting whether to enable the rotary decoder to store the counter value at the end of the last grab or frame interrupt (M_ROTARY_ENCODER_FRAME_END_READ), and setting a multiplying factor to each increment/decrement of the rotary decoder counter (M_ROTARY_ENCODER_MULTIPLIER).

There are two data latches associated with the rotary decoders on Zebra 4sight EV7. You can store the counter value of a rotary decoder in either of these data latches whenever a signal is received from a timer or auxiliary I/O, using MsysControl() with settings from the table For controlling the settings of a data latch associated with rotary encoders. You can retrieve the stored value within a hook function (hooked using MsysHookFunction()) using MsysGetHookInfo() with M_SYS_DATA_LATCH_VALUE.

> Note that the function must be hooked to the same type of event that triggers the latch. For example, if the latch is triggered by a timer (M_SYS_DATA_LATCH_TRIGGER_SOURCE with M_TIMERn), the hook function must be hooked to a timer event (M_TIMER_START or M_TIMER_END).

Zebra 4Sight EV7 connectors and signal names

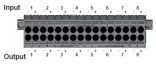
This section serves as a reference to match Zebra 4Sight EV7's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. To set/inquire all the settings for this computer's auxiliary signals, use MsysControl() / MsysInquire().

For information on other connectors and a comprehensive list of all available input and output signals, refer to the computer's installation and hardware reference manual.

Board connectors

Only the following connector has auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Image	Description
Auxiliary I/O terminal block connector		The auxiliary I/O interface connector is a terminal block connector. It is composed of 8 isolated auxiliary output signals and 8 isolated auxiliary input signals. These auxiliary signals can be used to send and receive digital control signals to and from external devices, respectively.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

System	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
System [1]	Indicates the following.
I/O #: M_AUX_IO0	Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
Pin information	Connector: Terminal block (Output) Pin: 1+

Direction	Output
User-bit information	User-bit #: M_USER_BIT0;
Timer information	Timer: M_TIMERn [2]
Hardware manual signal name	AUX_ISOIND_OUT1
I/O #: M_AUX_IO1	Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
Pin information	Connector: Terminal block (Output) Pin: 2+
Direction	Output
User-bit information	User-bit #: M_USER_BIT1;
Timer information	Timer: M_TIMERn [2]
Hardware manual signal name	AUX_ISOIND_OUT2
I/O #: M_AUX_IO2	Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
Pin information	Connector: Terminal block (Output) Pin: 3+
Direction	Output
User-bit information	User-bit #: M_USER_BIT2;
Timer information	Timer: M_TIMERn [2]
Hardware manual signal name	AUX_ISOIND_OUT3
I/O #: M_AUX_IO3	Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
Pin information	Connector: Terminal block (Output) Pin: 4+
Direction	Output
User-bit information	User-bit #: M_USER_BIT3;
Timer information	Timer: M_TIMERn [2]
Hardware manual signal name	AUX_ISOIND_OUT4
I/O #: M_AUX_IO4	Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
Pin information	Connector: Terminal block (Output) Pin: 5+
Direction	Output

	User-bit information	User-bit #: M_USER_BIT4;
	Timer information	Timer: M_TIMERn [2]
	Hardware manual signal name	AUX_ISOIND_OUT5
I/O #: M_AUX_IO5		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: Terminal block (Output) Pin: 6+
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT5;
	Timer information	Timer: M_TIMERn [2]
	Hardware manual signal name	AUX_ISOIND_OUT6
I/O #: M_AUX_IO6		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: Terminal block (Output) Pin: 7+
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT6;
	Timer information	Timer: M_TIMERn [2]
	Hardware manual signal name	AUX_ISOIND_OUT7
I/O #: M_AUX_IO7		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: Terminal block (Output) Pin: 8+
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT7;
	Timer information	Timer: M_TIMERn [2]
	Hardware manual signal name	AUX_ISOIND_OUT8
I/O #: M_AUX_IO8		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 1+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN1

I/O #: M_AUX_IO9		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 2+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN2
I/O #: M_AUX_IO10		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 3+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN3
I/O #: M_AUX_IO11		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 4+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN4
I/O #: M_AUX_IO12		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 5+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN5
I/O #: M_AUX_IO13		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 6+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN6
I/O #: M_AUX_IO14		Industrial auxiliary signal (input), which supports: user input or quadrature input.
	Pin information	Connector: Terminal block (Input) Pin: 7+
	Direction	Input
	Hardware manual signal name	AUX_ISOIND_IN7

	I/O #: M_AUX_IO15		Industrial auxiliary signal (input), which supports: user input or quadrature input.
		Pin information	Connector: Terminal block (Input) Pin: 8+
		Direction	Input
		Hardware manual signal name	AUX_ISOIND_IN8

[1] The digitizer device number is not applicable for Zebra 4Sight EV7.

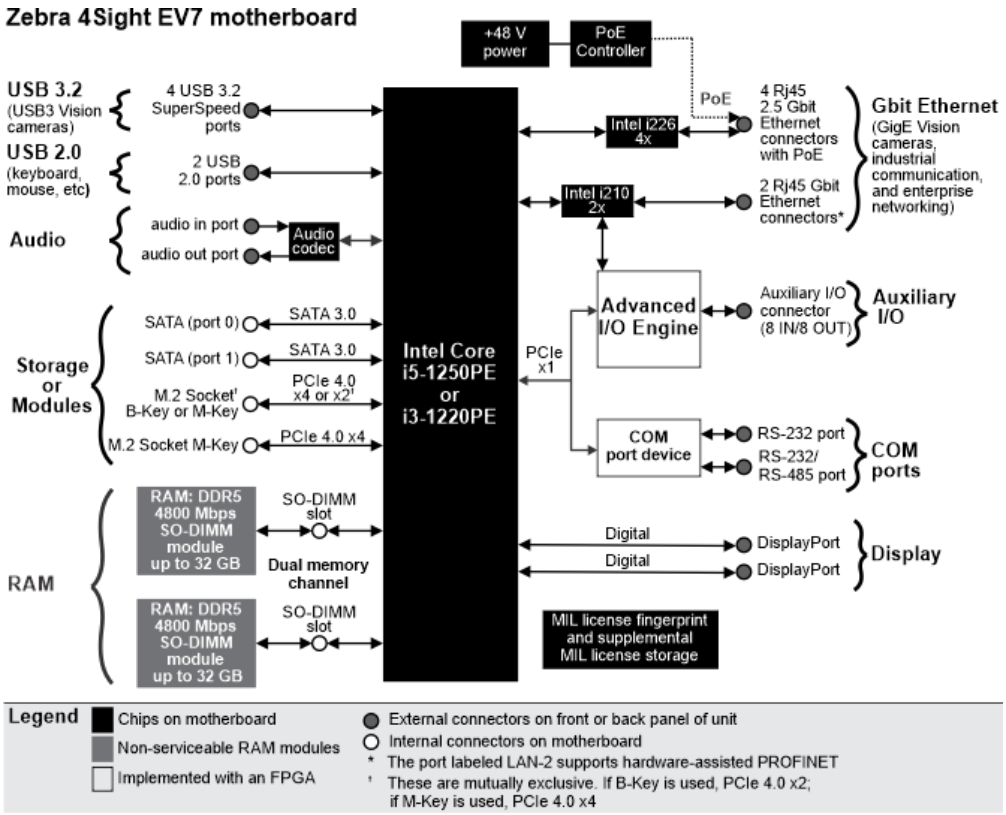
[2] Where *n* is a value between 1 and 16.

Zebra 4Sight data flow diagrams

The following sections contain the data flow diagrams for Zebra 4Sight EV7, Zebra 4Sight XV7, Zebra 4Sight EV6, and Zebra 4Sight XV6.

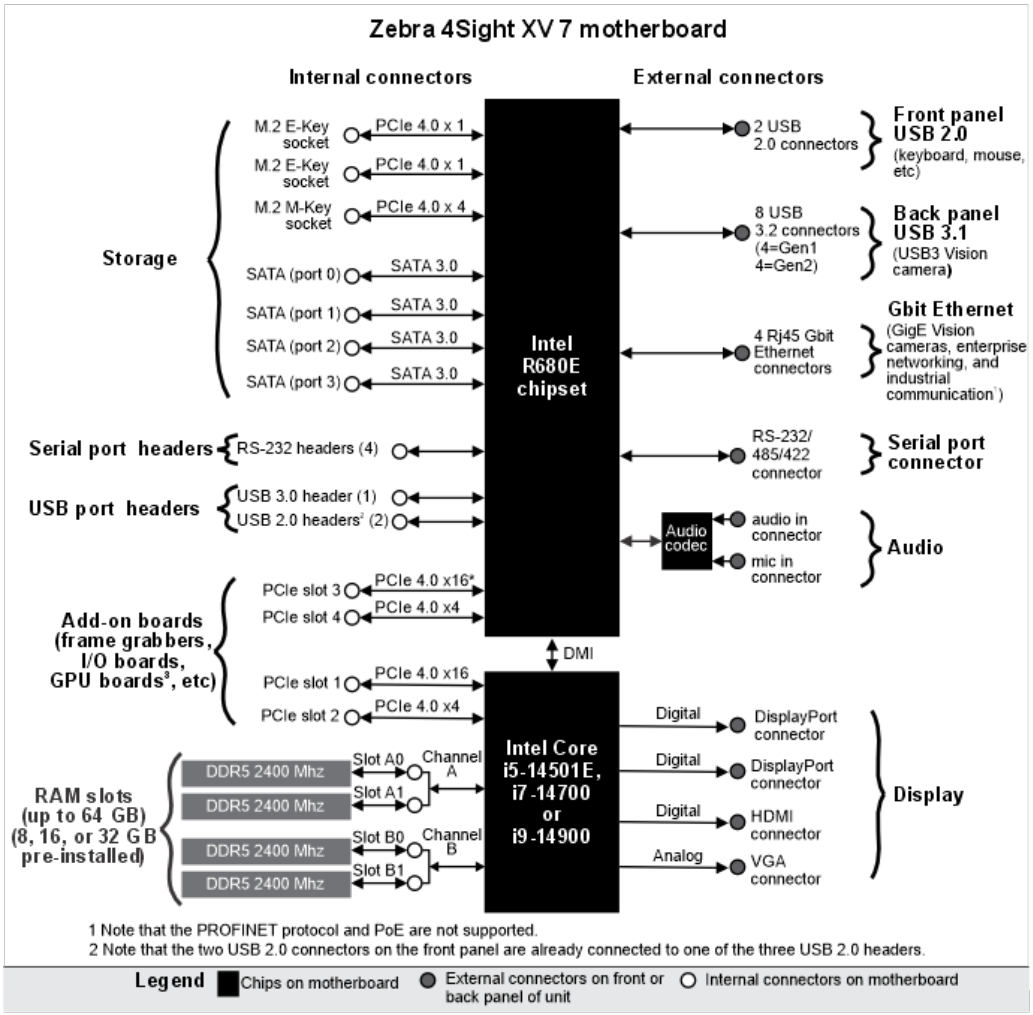
Zebra 4Sight EV7

The following diagram illustrates the data flow of a Zebra 4Sight EV7 unit.



Zebra 4Sight XV7

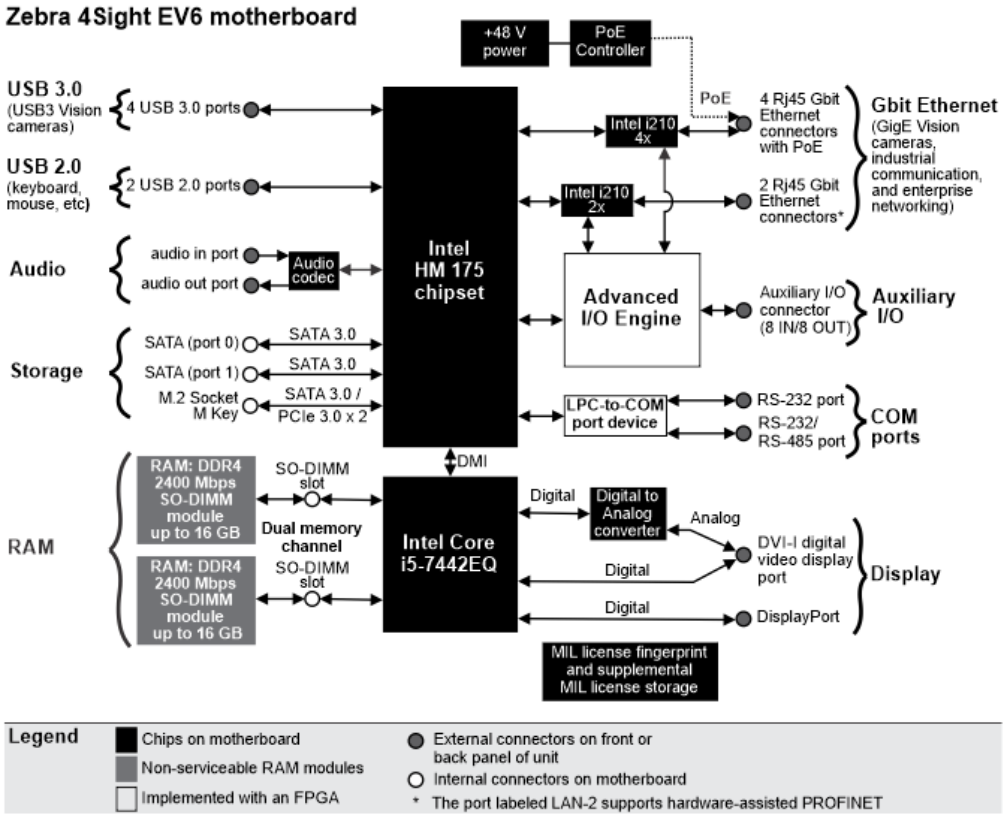
The following diagram illustrates the data flow of a Zebra 4Sight XV7 unit.



* Note that this slot is mechanically x16, but electrically only supports x8

Zebra 4Sight EV6

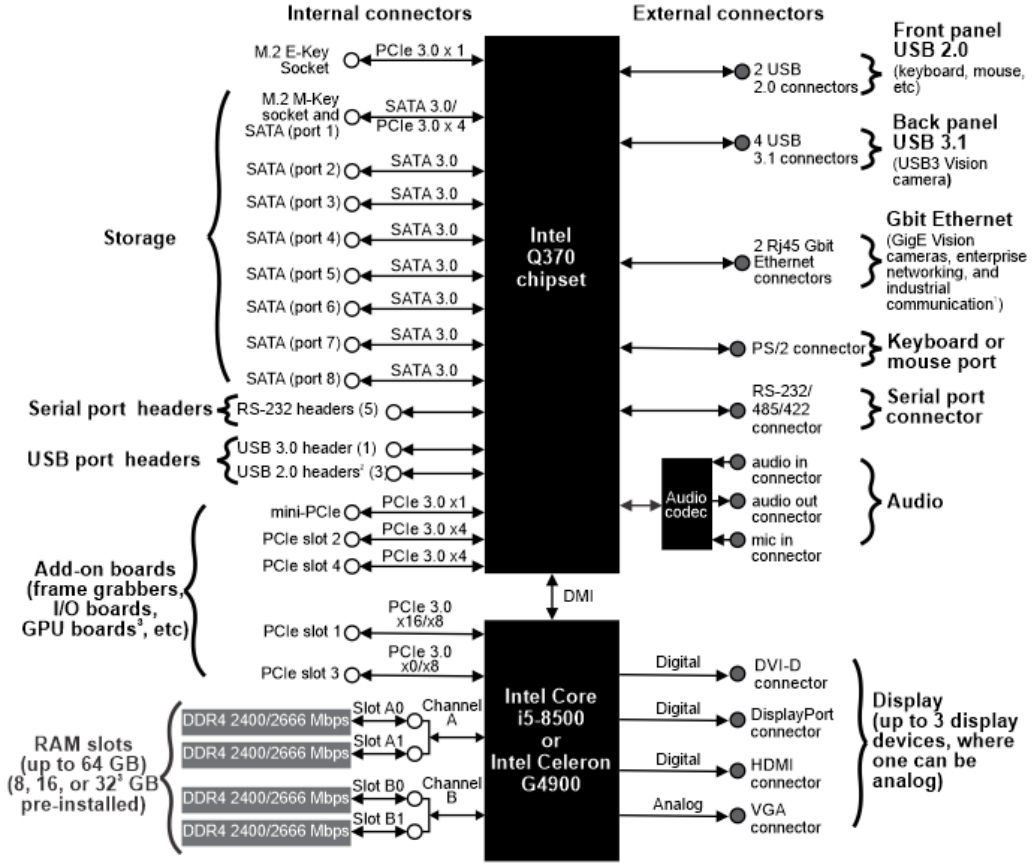
The following diagram illustrates the data flow of a Zebra 4Sight EV6 unit.



Zebra 4Sight XV6

The following diagram illustrates the data flow of a Zebra 4Sight XV6 unit.

Zebra 4Sight XV6 motherboard



1 Note that the PROFINET protocol and PoE are not supported.
2 Note that the two USB 2.0 connectors on the front panel are already connected to one of the three USB 2.0 headers.
3 Note that the XV6I5M32CT model includes a second SSD, 32 Gbytes of RAM, and a GPU board.

Legend ■ Chips on motherboard ● External connectors on front or back panel of unit ○ Internal connectors on motherboard

Miscellaneous reference information for Zebra 4Sight

This section includes additional command reference information for Zebra 4Sight. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Msys.

Msys

This section presents information about Zebra 4Sight and the system module.

MsysControl/Inquire

The following lists additional 4Sight support when using MsysControl() / MsysInquire().

- Support for inquire of an output: **M_USER_BIT_STATE + M_USER_BIT[0...15]**. Possible inquire values include:
  - **M_OFF**.
  - **M_ON**.
- Support for inquire of an output: **M_USER_BIT_STATE_ALL**. Possible inquire values include:
  - **M_OFF**.
  - **M_ON**.
- Support for inquire of an output: **M_IO_STATUS + M_AUX_IO[0...15]**.

  Support for inquire of an input: **M_IO_STATUS + M_AUX_IO[16...31]**.
- Support for **M_IO_INTERRUPT_ACTIVATION + M_AUX_IO[16...31]**. Possible control/inquire values include:
  - **M_DEFAULT** or **M_EDGE_RISING**.
  - **M_EDGE_FALLING**.
- Support for **M_IO_INTERRUPT_STATE + M_AUX_IO[16...31]**. Possible control/inquire values include:
  - **M_DEFAULT** or **M_DISABLE**.
  - **M_ENABLE**.
- Support for **M_IO_MODE + M_AUX_IO[n]**. Possible control/inquire values include:
  - **M_OUTPUT** for n = 0...15.
  - **M_INPUT** for n = 15...31.
- Support for reading the current encoder position with MsysInquire().

  MsysInquire(Sys, M_ROTARY_ENCODER_POSITION+M_ROTARY_ENCODERn, &Position).

  Select *n* as 1 or 2 to choose the target encoder.

  Position is an **AIL_INT**.
- Support for reading the current encoder position based on a signal event with MsysGetHookInfo(). Possible events include:

- The following associates a latch with a specific rotary encoder:

  MsysControl(Sys, M_SYS_DATA_LATCH_TYPE+M_LATCHm, M_ROTARY_ENCODERn).

  Select *m* as 1 or 2 to choose the target latch storage.

  Select *n* as 1 or 2 to choose the target encoder.

- The following determines which signal will latch. In this case, the rotary encoder position:

  MsysControl(Sys, M_SYS_DATA_LATCH_TRIGGER_SOURCE+M_LATCHm, LatchTriggerSource).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchTriggerSource is an **AIL_INT** initialized with **M_AUX_IOi**, where i is from 8 to 15 or **M_TIMERj**, where j is from 1 to 16, on Zebra Indio.

- The following determines which signal edge will latch the encoder position:

  MsysControl(Sys, M_SYS_DATA_LATCH_TRIGGER_ACTIVATION+M_LATCHm, LatchTriggerActivation).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchTriggerActivation is an **AIL_INT** initialized with **M_EDGE_RISING**, **M_EDGE_FALLING**, or **M_ANY_EDGE**.

- The following activates (or deactivates) the given latch mechanism:

  MsysControl(AilSystem, M_SYS_DATA_LATCH_STATE+M_LATCHm, LatchState).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchState is an **AIL_INT** initialized with **M_ENABLE** or **M_DISABLE**.

- The following allows you to retrieve the latched value in the context of a hook-handler function registered with MsysHookFunction():

  MsysGetHookInfo(EventId, M_SYS_DATA_LATCH_VALUE+M_LATCHm, LatchValue).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchValue is an **AIL_UINT64** that returns the rotary encoder position that is latched.

  Note that if **M_SYS_DATA_LATCH_TRIGGER_SOURCE** was specified as **M_AUX_IOi**, the MsysGetHookInfo() call is done in the scope of a hook function to **M_IO_CHANGE**. If **M_SYS_DATA_LATCH_TRIGGER_SOURCE** was specified as **M_TIMERj**, the MsysGetHookInfo() call is done in the scope of a hook function to **M_TIMER_START** or **M_TIMER_END**.

Miscellaneous user guide information for Zebra 4Sight

This section includes additional user guide information for Zebra 4Sight. The information found in this section might be a reiteration of content previously documented.

In general, related Zebra auxiliary FPGA supports RS485 half-duplex auto mode and the maximum output toggle rate is 60 KHz.

Licensing

When activating a temporary license or registering the software license for Aurora Imaging Configurator, a firmware update might be launched, or there could be issues when writing the license to the system's EEPROM. To prevent these problems, first close Aurora Imaging Configurator. Then, manually start the firmware updater tool, located in the installed Tools folder (for example, *C:\Program Files\Aurora Imaging Library\11\Tools*). After the update is complete, you can re-launch Aurora Imaging Configurator.

Zebra 4Sight XV6/XV7 supports Aurora Imaging Library licensing fingerprint storage on the disk (for example, on the SSD). Re-initializing the disk will remove the license, however, the license can be recovered and reapplied. Contact your Zebra representative for more information.

Zebra AltiZ

This chapter discusses features of Aurora Imaging Library that are particular to Zebra AltiZ. Camera-specific information regarding specific functions is integrated into the functions' reference topics under Zebra GigE Vision driver. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra AltiZ overview
  - Summary of Zebra AltiZ features
- Using Zebra AltiZ with Aurora Imaging Library

Zebra AltiZ overview

The Zebra AltiZ is a family of 3D profile sensors that use camera-laser triangulation to acquire high-precision 3D data. There are four models: AZ1D4SR, AZ1D4SB, AZ1D4MR, and AZ1D4LR.

The primary differences between the Zebra AltiZ models are: laser color and class, working distance, available Z-range, Z-axis resolution, and X-axis resolution.

Specifications	Zebra AltiZ family members			
	AZ1D4SR	AZ1D4SB	AZ1D4MR	AZ1D4LR
Laser color and class	Red/2M	Blue/2M	Red/3R	Red/3R
Working distance	100 mm	100 mm	185 mm	160 mm
Available Z-range	70 mm	70 mm	225 mm	385 mm
Z-axis resolution	4 µm - 8 µm	4 µm - 8 µm	9.5 µm - 35 µm	10 µm - 89 µm
X-axis resolution	39 µm	39 µm	84 µm	158 µm

Zebra AltiZ can interface with a controlling computer and other devices over Ethernet (1000Base-T) using the GigE Vision protocol. Zebra AltiZ can also interface with other devices using its auxiliary digital I/O lines (auxiliary I/O signals).

Zebra AltiZ supports the GenICam GenDC specification with 3D datasets. It also supports custom GenICam features, which are outlined in the Zebra AltiZ Installation and Technical Reference.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/altiz-info. Click on the **Documentation** tab to access it.

Summary of Zebra AltiZ features

The following table outlines the features currently available with each model of the Zebra AltiZ.

Features	Zebra AltiZ
Number of supported triggers	4 general auxiliary
Asynchronous reset mode	Yes

Number of timers	4
Number of auxiliary signals (lines)	6 (4 in and 2 out)
Quadrature decoder [1]	1

[1] Also known as quadrature encoder interface.

Using Zebra AltiZ with Aurora Imaging Library

To use your Zebra AltiZ with Aurora Imaging Library, you must allocate an Aurora Imaging Library GigE Vision system (M_SYSTEM_GIGE_VISION), using MsysAlloc(). This allocation opens general communication with all the GigE Vision-compliant cameras found on your subnet (through one or more Gigabit Ethernet network adapters in your computer). Then, allocate a digitizer for each Zebra AltiZ that you want to use to grab data and/or access directly, using MdigAlloc(). In the case where you have only one Zebra AltiZ, use MdigAlloc() with M_DEV0. If, however, you have multiple grabberless interface cameras, you should use the M_GC_CAMERA_ID() macro and specify the camera's name; this will make for more portable code.

Each process (executable) can allocate only one Aurora Imaging Library GigE Vision system; an error will occur if you try to allocate more than one. Multiple processes can allocate an Aurora Imaging Library GigE Vision system.

Since you interact with Zebra AltiZ as you would another GigE Vision camera, Zebra AltiZ information in the Aurora Imaging Library Reference can be found in the paragraphs and values marked as being supported by the GigE Vision driver.

To grab data from Zebra AltiZ, use MdigGrab() with an Aurora Imaging Library container. For information, refer to Chapter 42: Grabbing from 3D sensors.

If the grabbed data from your Zebra AltiZ is in the form of a point cloud and the direction of the Z-axis of your working coordinate system is not suitable for your application, you can invert or flip its direction. For more information, refer to the Aligning or flipping the coordinate system section of Chapter 42: Grabbing from 3D sensors.

To inquire the current value of the required GigE Vision camera feature, use MdigInquireFeature(). If a GigE Vision feature is not already set to the required value, use MdigControlFeature() to change the value of the feature. Functionality is also supported with the conventional Aurora Imaging Library functions MdigControl() and MdigInquire(). Note to control/inquire about the auxiliary I/O lines (signals), rotary encoder interface, and timers, you need to use MdigControlFeature() and MdigInquire(). For information on the available GenICam features, see the Zebra AltiZ Installation and Technical Reference.

Aurora Imaging Capture Works' Feature Browser accesses the device description file (XML) of any GigE Vision-compliant camera/3D sensor, including Zebra AltiZ, and provides an interface to change or inquire the device's information. For information on how to change or inquire the features of your Zebra AltiZ using Aurora Imaging Library, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Zebra AltiZ 4200

This chapter discusses features of Aurora Imaging Library that are particular to Zebra AltiZ 4200. Camera-specific information regarding specific functions is integrated into the functions' reference topics under Zebra GigE Vision driver. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra AltiZ 4200 overview
  - Summary of Zebra AltiZ 4200 features
- Using Zebra AltiZ 4200
  - Using Zebra Altiz 4200 with Capture Works

Zebra AltiZ 4200 overview

Zebra AltiZ 4200 is a family of 3D profile sensors that use camera-laser triangulation (line profiling) to acquire high-precision 3D data.

There are three models: AZ2S-2SB, AZ2S-3SB, and AZ2S-5SB. The primary specifications and notable differences between the Zebra AltiZ 4200 models are: clearance distance, working distance, available Z-range, X-axis field of view.

Specifications [1]	Zebra AltiZ 4200 family members		
	AZ2S-2SB	AZ2S-3SB	AZ2S-5SB
Clearance distance (min)	38.13 mm	79.67 mm	82.37 mm
Working distance	56.13 mm	97.67 mm	100.37 mm
Available Z-range	20.16 mm	44.53 mm	69.42 mm
min Z - max Z	56.13 mm - 76.29 mm	97.67 mm - 142.20 mm	100.37 mm - 169.79 mm
X-axis Field of View (near - far)	26.22 mm - 32.34 mm	43.25 mm - 58.84 mm	69.28 mm - 93 mm

[1] Note that these are approximate values and might slightly vary from unit to unit.

Zebra AltiZ 4200 supports the GenICam Standard Feature Naming Convention (SFNC) version 2.7 and the GenICam GenDC 1.0 specification with 3D datasets. It also supports custom GenICam features, which are outlined in the Zebra AltiZ 4200 Installation and Technical Reference. Zebra AltiZ 4200 can interface with a controlling computer and other devices over Ethernet (2.5GBASE-T) using the GigE Vision protocol. Zebra AltiZ 4200 can also interface with other devices using its auxiliary digital I/O lines (auxiliary I/O signals).

> You can access the latest version of the Installation and Hardware Reference at zebra.com/altiz-4200-info. Click on the **Documentation** tab to access it.

Summary of Zebra AltiZ 4200 features

The following table outlines the features currently available with each model of the Zebra AltiZ 4200.

Features	Zebra AltiZ 4200
Number of supported triggers	4 general auxiliary
Asynchronous reset mode	Yes

Number of timers	4
Number of auxiliary signals (lines)	6 (4 in and 2 out)
Quadrature decoder [2]	1

[2] Also known as quadrature encoder interface.

Using Zebra AltiZ 4200

To use your Zebra AltiZ 4200 with Aurora Imaging Library, you must allocate an Aurora Imaging Library GigE Vision system (M_SYSTEM_GIGE_VISION), using MsysAlloc(). This allocation opens general communication with all the GigE Vision-compliant cameras found on your subnet (through one or more Gigabit Ethernet network adapters in your computer). Then, allocate a digitizer for each Zebra AltiZ 4200 that you want to use to grab data and/or access directly, using MdigAlloc(). In the case where you have only one Zebra AltiZ 4200, use MdigAlloc() with M_DEV0. If, however, you have multiple grabberless interface cameras, you should use the M_GC_CAMERA_ID() macro and specify the camera's name; this will make for more portable code.

Each process (executable) can allocate only one Aurora Imaging Library GigE Vision system; an error will occur if you try to allocate more than one. Multiple processes can allocate an Aurora Imaging Library GigE Vision system.

Since you interact with Zebra AltiZ 4200 as you would another GigE Vision camera, Zebra AltiZ 4200 information in the Aurora Imaging Library Reference can be found in the paragraphs and values marked as being supported by the GigE Vision driver.

To grab data from Zebra AltiZ 4200, use MdigGrab() with an Aurora Imaging Library container. For information, refer to Chapter 42: Grabbing from 3D sensors.

If the grabbed data from your Zebra AltiZ 4200 is in the form of a point cloud and the direction of the Z-axis of your working coordinate system is not suitable for your application, you can invert or flip its direction. For more information, refer to the Aligning or flipping the coordinate system section of Chapter 42: Grabbing from 3D sensors.

To inquire the current value of the required GigE Vision camera feature, use MdigInquireFeature(). If a GigE Vision feature is not already set to the required value, use MdigControlFeature() to change the value of the feature. Functionality is also supported with the conventional Aurora Imaging Library functions MdigControl() and MdigInquire(). Note to control/inquire about the auxiliary I/O lines (signals), rotary encoder interface, and timers, you need to use MdigControlFeature() and MdigInquireFeature(). For information on the available GenICam features, see the Zebra AltiZ 4200 Installation and Technical Reference.

Using Zebra Altiz 4200 with Capture Works

Aurora Imaging Capture Works' Feature Browser accesses the device description file (XML) of any GigE Vision-compliant camera/3D sensor, including Zebra AltiZ 4200, and provides an interface to change or inquire the device's information. For information on how to change or inquire the features of your Zebra AltiZ 4200 using Aurora Imaging Library, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Zebra Clarity UHD

This section discusses features of Aurora Imaging Library that are particular to Zebra Clarity UHD and ways that optimize the board's performance. Zebra Clarity UHD information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Clarity UHD overview
  - Acquisition features
  - Other features
  - Summary of Zebra Clarity UHD features
- Using Zebra Clarity UHD
- Cameras that can be connected to Zebra Clarity UHD
  - Data input channels of acquisition paths
  - Switching between cameras of the same type
  - Switching between cameras of different types
  - Grabbing a single field
- Automatic camera detection with Zebra Clarity UHD
- Zebra Clarity UHD tools
  - Zebra Clarity UHD Bench utility
  - Zebra Clarity UHD System Monitor tool
- Zebra Clarity UHD board flow diagrams
- Miscellaneous reference information for Zebra Clarity UHD
  - H.264 encoding
  - Specifying image buffer
- Miscellaneous user guide information for Zebra Clarity UHD
  - Grabbing capabilities
  - Encoding capabilities
  - Particularities

Zebra Clarity UHD overview

Zebra Clarity UHD is a high-performance frame grabber, supporting standard definition (SD) analog video, high-definition (HD) analog and digital video, and ultra-high-definition (UHD) digital video input. Zebra Clarity UHD can be purchased with the additional feature of a H.264 encoder for video compression/decompression.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/clarity-uhd-info.

Acquisition features

Zebra Clarity UHD has eight independent acquisition paths which support the acquisition of up to 4 Gbytes (4096 Mbytes/sec) combined bandwidth from 10 possible connectors. The number of simultaneous grabs that a Zebra Clarity UHD can perform depends on the available connections and data formats used.

Buffer pixel format	Maximum number of streams that can be grabbed and displayed
Monochrome 8-bit	4 x 2160p60 streams and 4 x 1080p60 streams, or 6 x 2160p30 streams and 2 x 1080p60 streams, or 8 x 1080p60 streams
YUV 4:2:0 8-bit	4 x 2160p60 streams, or 6 x 2160p30 streams and 2 x 1080p60 streams, or 8 x 1080p60 streams
YUV 4:2:2 8-bit	3 x 2160p60 streams, or 6 x 2160p30 streams, or 8 x 1080p60 streams
YUV 4:2:2 10-bit	2 x 2160p60 streams and one 1080p60 stream, or 4 x 2160p30 streams and one 1080p60 stream, or 8 x 1080p60 streams
RGB 8-bit planar or YUV 4:4:4 8-bit planar	2 x 2160p60 streams, or 4 x 2160p30 streams, or 8 x 1080p60 streams

BGR32 8-bit packed	one 2160p60 stream and 2 x 1080p60 streams, or 3 x 2160p30 streams, or 6 x 1080p60 streams
RGB 10-bit packed	3 x 2160p30 streams, or 6 x 1080p60 streams

Other features

Zebra Clarity UHD supports cropping (ROI capture), and subsampling (that is, downscaling) by integer factors of 1 to16.

When grabbing from an SDI source into a monochrome 8-bit, YUV16, or BGR32 packed buffer, Zebra Clarity UHD automatically converts the incoming data, using the proper color space equations, depending on the input resolution (ITU-R BT.601 for SD, ITU-R BT.709 for HD, and ITU-R BT.2020 for UHD). Note that, typically SDI streams are natively in YUV format. When grabbing into a YUV16 packed buffer, the input data is not converted.

If Zebra Clarity UHD has been purchased with the H.264 encoder option, the board can offload video compression/decompression from the Host, as well as perform video compression of the grabbed video input.

Summary of Zebra Clarity UHD features

The following table outlines the features currently available on Zebra Clarity UHD.

Features	Zebra Clarity UHD
PCIe board type (Form factor)	x8 half-length board
Acquisition format	Monochrome 8-bit, YUV16 (YCrCB) packed, and BGR32 packed
Number of acquisition paths	8 independent
Number of supported hardware triggers	0
Number of auxiliary signals	0
On-board memory	4 Gbytes of DDR3 SDRAM
Digitizer LUT	No

Number of display paths	0
Color space conversion specifications	ITU-R BT.601, ITU-R BT.709, ITU-R BT.2020
Additional features	Formatting features (subsampling and cropping). Hooks for camera plug/unplug. Automatic input detection. Temperature and fan monitoring.
Optional features	H.264 video compression and decompression available on the Zebra Clarity UHD CLA4GHDSAE model

Using Zebra Clarity UHD

To use any Zebra Clarity UHD, you must allocate it as an Aurora Imaging Library Clarity UHD system using MsysAlloc() with M_SYSTEM_CLARITY_UHD. This allocation opens communications with your Zebra Clarity UHD and allows Aurora Imaging Library to use its resources. You can allocate an Aurora Imaging Library Clarity UHD system for your board in multiple processes (executables). However, multiple processes cannot allocate a digitizer for the same acquisition path simultaneously.

Zebra Clarity UHD supports the following grab buffer pixel formats:

Aurora Imaging Library grab buffer format	Description
1 band 8-bit	Monochrome 8-bit
M_YUV16 + M_PACKED	YUV 4:2:2 8-bit
M_YUV24 + M_PLANAR	YUV 4:4:4 8-bit planar
RGB 3 band 8-bit	RGB 8-bit planar
M_BGR32 + M_PACKED	BGR32 8-bit packed buffer

You can also grab into a dynamic buffer. For more information, see the Grabbing capabilities subsection of the Miscellaneous user guide information for Zebra Clarity UHD section later in this chapter.

If Zebra Clarity UHD has been purchased with the H.264 encoder option, you can use the Aurora Imaging Library Sequence module to perform on-board H.264 video compression or decompression. To compress a sequence, you must allocate a sequence context using MseqAlloc() with M_SEQ_COMPRESS; to decompress a sequence, you must allocate a sequence context using MseqAlloc() with M_SEQ_DECOMPRESS. For more information on H.264 video compression and decompression, see Inputs and outputs of an H.264 compression or decompression.

Cameras that can be connected to Zebra Clarity UHD

You can allocate up to eight independent Aurora Imaging Library digitizers on your Zebra Clarity UHD, using MdigAlloc().

The connector used determines the digitizer device number assigned. In all but two cases, each connector uses an independent acquisition path. However, M_DEV2 corresponds to both an HDMI and an analog connector; both of these connectors use the same independent acquisition path. Therefore, if both connectors are connected to a camera, you cannot grab simultaneously from both cameras. This is also the case for M_DEV3.

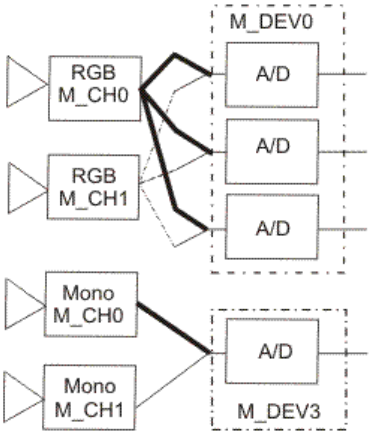
Digitizer device number	Connector Name	Resolutions supported
M_DEV0	Mini-HDMI (type C) connector 0	Supports grabbing at up to DVI-D and HDMI 2.0 non-interlaced (up to 2160p60) and interlaced resolutions (1080i50, 1080i59.94, and 1080i60).
M_DEV1	Mini-HDMI (type C) connector 1	
M_DEV2	Mini-HDMI (type C) connector 2	Supports grabbing at up to DVI-D and HDMI 1.4 non-interlaced (up to 1920x1200p60) and interlaced resolutions (1080i50, 1080i59.94, and 1080i60).
	Analog connector 0	Supports grabbing at up to DVI-A (analog) non-interlaced (up to 1920x1200p60) and interlaced resolutions (1080i50 fps, 1080i59.94, and 1080i60); in addition, it supports grabbing at SD (analog) resolutions for RS170 (720 x 486 i30), CCIR (720 x 576 i25), NTSC (720 x 486 i30), NTSC/YC (720 x 486 i30), PAL (720 x 576 i25), PAL/YC (720 x 576 i25), NTSC RGB (640 x 480 i30), PAL RGB (768 x 576 i25).
M_DEV3	Mini-HDMI (type C) connector 3	Same as M_DEV2.

	Analog connector 1	
M_DEV4	Mini Display Port 1.2 connector 0	Supports grabbing at Display Port 1.2 non-interlaced resolutions of up to 2160p60 and interlaced resolutions of 1080i50, 1080i59.94, and 1080i60.
M_DEV5	Mini Display Port 1.2 connector 1	
M_DEV6	SDI connector 0	Supports grabbing at up to: SD resolutions (NTSC (720x486 @30fps) and PAL (720x576 @25 fps). HD resolutions (720p50, 720p59.94, 720p60, 1080p23.98, 1080p24, 1080p25, 1080p29.97, 1080p30, 1080p50, 1080p59.94, 1080p60, 1080i50, 1080i59.94, 1080i60). 2K DCI resolutions (2K 23.98p, 2K 24p, 2K 25p). UHD resolutions (2160p23.98, 2160p24, 2160p25, 2160p29.97, 2160p30, 2160p50, 2160p59.94, 2160p60). 4K DCI resolutions (4K 23.98p, 4K 24p, 4K 25p).
M_DEV7	SDI connector 1	

In addition, if not used with a component RGB camera, M_DEV2 and M_DEV3 have three data channels. You can connect a composite color/monochrome camera to each channel and switch between them, using MdigControl() with M_CHANNEL. You can only grab from one data channel per M_DEVn at a time.

Data input channels of acquisition paths

For M_DEV2 and M_DEV3, you can choose to use the 3 data input channels independently (meaning that you grab from only one channel at a time). If you have a camera that is connected to a channel other than the first of its digitizer, you must specify the channel, using MdigControl() with M_CHANNEL.



Switching between cameras of the same type

When connecting several cameras of the same data format to different data input channels of a digitizer, allocate a single digitizer with the appropriate DCF for the first camera and use MdigControl() with M_CHANNEL to switch between the others of the same type.

Switching between cameras of different types

When connecting cameras of different data formats to different data input channels of a digitizer, you must use a different DCF for each camera. You could allocate a digitizer, grab the required frame, free the digitizer, and then allocate the digitizer again with the second DCF; this can increase the time required for the operation. Aurora Imaging Library can circumvent this problem by using a fast DCF-switching technique which is outlined in the steps below:

1. Make as many calls to MdigAlloc() as you have cameras, with different formats, from which you want to grab.
2. Specify the required digitizer settings using MdigControl() and MdigHookFunction() for each allocated digitizer.
3. Specify the channel to use for the grab using M_CHANNEL.
4. Call MdigGrab() with an allocated digitizer identifier. Note that if this call uses a digitizer identifier different from the previous call, a DCF switch will occur.

> If there is a grab in progress with one digitizer, calling any of the following functions with any other digitizer will result in an error: MdigGrab(), MdigGrabContinuous(), MdigInquire(), MdigProcess(), MdigAlloc(), and MdigFree(). To synchronize your grabs with different digitizers, use MthrWait().

Grabbing a single field

With interlaced scanning cameras, 2 fields are grabbed by default; therefore one call to MdigGrab() will grab both the odd and even fields. You can change the number of fields to 1 and have Aurora Imaging Library treat each field as one frame using MdigControl() with M_GRAB_FIELD_NUM, grabbing every second row and storing them in sequential rows. Therefore, the grab time is reduced by half. This control type can only be set to 1 or 2, and should only be used for interlaced video. When set to 1, each field is treated like a frame and the following digitizer events occur relative to the grabbed field: M_GRAB_FRAME_START, M_GRAB_END, and M_GRAB_FRAME_END.

To achieve 60 fps in NTSC or 50 fps in PAL, the control type M_GRAB_START_MODE must be set to M_FIELD_START.

Automatic camera detection with Zebra Clarity UHD

With Zebra Clarity UHD, Aurora Imaging Library provides autodetect DCFs and specific resolution DCFs. To use an autodetect DCF when allocating a digitizer, a camera must be connected to the specified acquisition path (M_DEVn). To use a specific resolution DCF, a camera does not necessarily need to be connected before the allocation.

You can hook a function to the event that occurs when a camera is connected or disconnected from the board using MsysHookFunction() with the M_CAMERA_PRESENT hook type. In the hook-handler function, you can inquire whether the event occurred upon the connection or disconnection of the camera using MsysGetHookInfo() with M_CAMERA_PRESENT; you can establish to which acquisition path the camera was connected or disconnected from using MsysGetHookInfo() with M_NUMBER. Based on this information, you can allocate or free the digitizer for the camera in the hook-handler function.

The following example demonstrates how to use automatic camera detection with Zebra Clarity UHD: *MultiCameraDisplay.cpp*. To run the example, you can use **Example Launcher**.

Zebra Clarity UHD tools

There is one Zebra Clarity UHD external application (utility) available (Zebra Clarity UHD Bench) and one Zebra Clarity UHD tool integrated in the Aurora Imaging Configurator utility (Zebra Clarity UHD System Monitor tool). To access these utilities and this tool, launch the Aurora Imaging Configurator utility. Then, select the **Boards Clarity UHD** item from the tree structure in the presented interface. The utilities are located in the **Launch external applications** and **Zebra Clarity UHD System Monitor** panes, respectively.

Zebra Clarity UHD Bench utility

The Zebra Clarity UHD Bench utility calculates the real-time transfer speed of the PCIe slot (in Mbytes/sec) from:

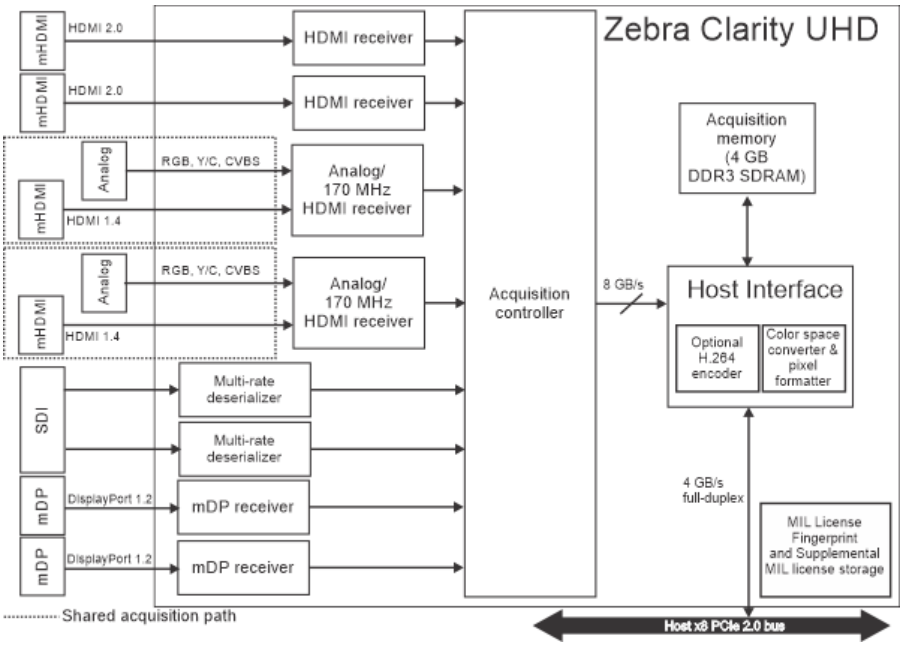
- Zebra Clarity UHD to Host.
- Host to Zebra Clarity UHD.

Zebra Clarity UHD System Monitor tool

The Zebra Clarity UHD system monitor tool is integrated in the Aurora Imaging Configurator utility. It is used to display the temperature of the acquisition controller and the Host interface of the board.

Zebra Clarity UHD board flow diagrams

The following diagram illustrates the data flow of Zebra Clarity UHD.



Miscellaneous reference information for Zebra Clarity UHD

This section includes additional Zebra Clarity UHD information regarding the command reference. The information found in this section might be a reiteration of content previously documented.

H.264 encoding

The following section outlines additional settings regarding H.264 encoding.

Function(s)	Parameter/value	Information
MseqAlloc()	InitFlag	Must be set to M_DEFAULT.
MseqControl() / MseqInquire()	M_CODEC_TYPE	**M_HARDWARE** + **M_CLARITY_UHD_H264**
	M_STREAM_BIT_RATE	The default value is 15000 kbits/sec.
	M_STREAM_BIT_RATE_MAX	The default value is 30000 kbits/sec.
	M_STREAM_GROUP_OF_PICTURE_SIZE	The default value is 90 frames per group.
	M_STREAM_LEVEL	Additional supported level: M_LEVEL_5_2.
	M_STREAM_PROFILE	Additional supported profiles: • **M_AUTOMATIC** is the default, which selects the profile that matches the pixel format of the buffer passed to MseqFeed(). • **M_PROFILE_EXTENDED** • **M_PROFILE_HIGH422**

Zebra Clarity UHD supports encoding buffers in multiple buffer formats. This buffer can be used when calling MseqFeed().

The most efficient encoding buffer pixel formats when using M_PROFILE_HIGH are:

- **M_YUV12** + **M_PLANAR** + **M_ON_BOARD**.
- **M_DYNAMIC** + **M_ON_BOARD** with **M_PFNC_TARGET_FORMAT** set to **PFNC_YCbCr411_8**.

The most efficient encoding buffer pixel formats when using **M_PROFILE_HIGH422** are:

- **M_YUV16** + **M_PACKED** + **M_ON_BOARD**.

- **M_DYNAMIC** + **M_ON_BOARD** with **M_PFNC_TARGET_FORMAT** set to **PFNC_YCbCr422_10p**.

Other combinations of buffer formats and profiles are supported, but will result in an internal buffer conversion, which reduces encoding performance.

Specifying image buffer

The following combination value is used in MbufAlloc1d() to specify the intended purpose of the image buffer.

M_DYNAMIC	Specifies an image buffer whose bit-depth and data format are specified by the camera (see **M_PFNC_TARGET_FORMAT**). To determine the required size of the image buffer, use MdigInquire() with M_SIZE_X and M_SIZE_Y, respectively. This value can only be used with a buffer with the M_IMAGE + M_GRAB attribute.

Miscellaneous user guide information for Zebra Clarity UHD

This section includes additional user guide information for Zebra Clarity UHD. The information found in this section might be a reiteration of content previously documented.

Grabbing capabilities

In addition to the grab buffer pixel formats listed in the the Using Zebra Clarity UHD section earlier in this chapter, the formats in the following table are also supported:

Before grabbing into the buffer, if you allocate a grab buffer with an **M_DYNAMIC** attribute, you must specify the format in which to convert and store the input data using MdigControl() with **M_PFNC_TARGET_FORMAT**. The format can be changed in between grabs. A buffer with an **M_DYNAMIC** attribute cannot be used with most other Aurora Imaging Library functions. However, you can access the data using MbufInquire() with M_HOST_ADDRESS and process the data directly using a custom function. You can also use the buffer with the Sequence module (Mseq).

Aurora Imaging Library grab buffer format	MdigControl() with **M_PFNC_TARGET_FORMAT**	Description
M_DYNAMIC	PFNC_BGRa10p	10-bit color depth with a BGRa format (B:10,G:10,R:10, A:2). This format is not supported when grabbing from a 3840x2160@50Hz source or higher.
M_DYNAMIC	PFNC_YCbCr411_8	8-bit color depth with a YUV 4:2:0 format. Note that the actual data is in YUV 4:2:0 (not YUV 4:1:1) because it is not yet defined in the PFNC specification. The data is in 2 planes. One for the luminance and the other for the chrominance.
M_DYNAMIC	PFNC_YCbCr422_10p	10-bit packed color depth with a V210 format (YUV 4:2:2 10-bit)

Encoding capabilities

The following table lists the maximum number of streams that can be simultaneously grabbed, displayed, and encoded depending on the stream resolution, frame rate, and the destination buffer pixel format.

Buffer pixel format	Encoding profile (MseqControl() with M_STREAM_PROFILE), using the default encoding settings	Maximum number of streams that can be grabbed, displayed, and encoded.
	M_PROFILE_HIGH	one 2160p60 stream or

Monochrome 8-bit	Buffer is internally converted to YUV 4:2:0 before encoding	3 x 2160p30 streams or
		6 x 1080p60 streams
YUV 4:2:0 8-bit	**M_PROFILE_HIGH**	2 x 2160p60 streams or
		4 x 2160p30 streams or
		8 x 1080p60 streams
YUV 4:2:2 8-bit	**M_PROFILE_HIGH422**	one 2160p60 stream or
		3 x 2160p30 streams or
		6 x 1080p60 streams
YUV 4:2:2 10-bit	**M_PROFILE_HIGH422** (Encoding in 10-bit)	one 2160p60 stream or
		2 x 2160p30 streams or
		5 x 1080p60 streams
RGB 8-bit planar	**M_PROFILE_HIGH** Buffer is internally converted to YUV 4:2:0 before encoding	one 2160p60 stream or
		3 x 2160p30 streams or
		6 x 1080p60 streams
BGR32 8-bit packed	**M_PROFILE_HIGH** Buffer is internally converted to YUV 4:2:0 before encoding	one 2160p60 stream or
		3 x 2160p30 streams or
		5 x 1080p60 streams
RGB 10-bit	**M_PROFILE_HIGH** Buffer is internally converted to YUV 4:2:0 before	2 x 2160p30 streams or

packed	encoding	5 x 1080p60 streams

Particularities

The following are some particularities specific to the Clarity UHD.

- If autodetect DCF does not recognize some analog component YPbPr cameras, you can manually specify the DCF using the Aurora Imaging Configurator or using MdigAlloc().
- When grabbing from an SDI source into a monochrome 8-bit or color BGR32 packed buffer, the Clarity UHD automatically selects the proper color space equations depending on the input resolution (for example, ITU-R BT.601 for SD, ITU-R BT.709 for HD and ITU-R BT.2020 for UHD). Note that when grabbing into a YUV16 packed buffer, the input data is preserved in its original format. The color space can be inquired using MbufInquire() with M_YCBCR_RANGE.
- For the SDI inputs, dual-link, 3D, and RGB 4:4:4 are not supported.

Zebra Concord PoE

This chapter discusses features of Aurora Imaging Library that are particular to Zebra Concord PoE base model and Zebra Concord PoE with ToE. Zebra Concord PoE information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Concord PoE overview
  - Summary of Zebra Concord PoE features
  - Note on nomenclature
- Using Zebra Concord PoE
  - Using the Advanced I/O Engine of Zebra Concord PoE with ToE
- Using an Aurora Imaging Library Concord PoE system with the Zebra Concord PoE base model
- Sending a Trigger-over-Ethernet packet without Host intervention
  - Sending a Trigger-over-Ethernet packet as an action command
  - Sending a Trigger-over-Ethernet packet as a GigE Vision software trigger
- Zebra Concord PoE with ToE connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Concord PoE board flow diagrams

Zebra Concord PoE overview

The Zebra Concord PoE family consists of two models: the Zebra Concord PoE base model and Zebra Concord PoE with ToE.

Zebra Concord PoE base model is a Gigabit Ethernet network interface board that provides optimum support for interfacing with GigE Vision cameras and supplies Power-over-Ethernet (PoE) to GigE Vision cameras that support PoE. There are two versions of the Zebra Concord PoE base model: the Dual-port version and the Quad-port version.

Zebra Concord PoE with ToE includes all the features of the Zebra Concord PoE base model, with the addition of an Advanced I/O Engine. The Advanced I/O Engine provides use of 8 auxiliary signals, and includes I/O command lists, timers, quadrature decoders, user outputs, and a Trigger-over-Ethernet (ToE) module. The Trigger-over-Ethernet module can transmit a Trigger-over-Ethernet packet, either as an action command or a GigE Vision software trigger, upon reception of an internal or external event. There are two versions of Zebra Concord PoE with ToE: the Dual-port version and the Quad-port version.

In addition to being prelicensed for use with the Aurora Imaging Library drivers for GigE Vision cameras, the Zebra Concord PoE board can act as a fingerprint for licensing supplemental Aurora Imaging Library functionality. The board can also store a supplemental Aurora Imaging Library license, which facilitates the process of moving to a new computer.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/concord-poe-info.

Summary of Zebra Concord PoE features

The following table outlines the features currently available for Zebra Concord PoE family.

	Zebra Concord PoE base model	Zebra Concord PoE with ToE
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature() [1]	Yes	Yes
On-board memory	N/A	N/A
Number of acquisition ports	2 (Dual) or 4 (Quad)	2 (Dual) or 4 (Quad)
Number of auxiliary signals	0	8 (6 in, 2 out)
Trigger-over-Ethernet (ToE) module	No	Yes
Number of timers	0	16

Number of quadrature decoders for input from linear or rotary encoders	0	2
Number of I/O command lists	0	2
Number of reference latches per I/O command list	0	4
Power over Ethernet (PoE) support	Yes	Yes

[1] When using a GigE Vision system (M_SYSTEM_GIGE_VISION, using MsysAlloc()).

Note on nomenclature

This manual refers to all models of Zebra Concord PoE as the Zebra Concord PoE. When necessary, this manual distinguishes between the boards using their full names (Zebra Concord PoE base model and Zebra Concord PoE with ToE).

Using Zebra Concord PoE

To use Zebra Concord PoE with Aurora Imaging Library for acquisition, you must allocate an Aurora Imaging Library GigE Vision system (M_SYSTEM_GIGE_VISION), using MsysAlloc(). This allocation opens communication with GigE Vision compliant cameras (through one or more Gigabit Ethernet network adapters in your computer). You can then access and grab from multiple connected GigE Vision cameras by allocating a digitizer for each connected camera (using MdigAlloc() with M_DEVn). Each process (executable) can allocate only one Aurora Imaging Library GigE Vision system. Each camera can be used by one process (executable) at a time. For additional information, refer to the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

To use other functionality available on Zebra Concord PoE with ToE or inquire about the status (for example, BoardType) of either model of the board, you must allocate an Aurora Imaging Library Concord PoE system, using MsysAlloc() with M_SYSTEM_CONCORD_POE.

> Be aware that to access the Aurora Imaging Library license that is included on the Zebra Concord PoE board, you must install the Zebra Concord PoE driver.

Using the Advanced I/O Engine of Zebra Concord PoE with ToE

Zebra Concord PoE with ToE has an Advanced I/O Engine that controls the auxiliary I/O interface. The engine includes 16 timers, 2 rotary decoders, 2 I/O command lists, a user-output register, 2 auxiliary output signals, 6 auxiliary input signals and a Trigger-over-Ethernet (ToE) module. To route I/O signals, configure interrupts, and set the mode of an auxiliary signal, use MsysControl(). To inquire this information, use MsysInquire() For more information on some of these features, refer to Chapter 56: I/O signals and communicating with external devices. For information on how to use the ToE module to send a Trigger-over-Ethernet packet (as an action command or GigE Vision software trigger) to the camera, refer to the Sending a Trigger-over-Ethernet packet without Host intervention section later in this chapter.

Using an Aurora Imaging Library Concord PoE system with the Zebra Concord PoE base model

To use Zebra Concord PoE for acquisition, you must allocate and use an Aurora Imaging Library GigE Vision system (MsysAlloc() with M_SYSTEM_GIGE_VISION); refer to information denoted for a GigE Vision system. You only need to allocate and use an Aurora Imaging Library Concord PoE system (MsysAlloc() with M_SYSTEM_CONCORD_POE) to use the other functionality on the board and to inquire about the board itself. This means that you rarely use an Aurora Imaging Library Concord PoE system with the Zebra Concord PoE base model.

For this reason, in the reference, information pertinent to using an Aurora Imaging Library Concord PoE system is identified with a column for the Zebra Concord PoE with ToE; most of the functions and constants are typically only available to this model. If the information is applicable to the Zebra Concord PoE base model, there will be a note included to specify this.

The following are the only functions and constants that can be used with an Aurora Imaging Library Concord PoE system (using MsysAlloc() with M_SYSTEM_CONCORD_POE) when allocated for a Zebra Concord PoE base model:

Function	Constants supported
MsysAlloc()	M_SYSTEM_CONCORD_POE with M_DEVn or user-defined board name.
MsysFree()	All functionality is available.
MsysControl()	M_DEVICE_NAME, M_POWER_OVER_CABLE.
MsysInquire()	M_BOARD_TYPE, M_DEVICE_NAME, M_DEVICE_NAME_MAX_SIZE, M_POWER_OVER_CABLE, M_SERIAL_NUMBER, M_SERIAL_NUMBER, M_SYSTEM_DESCRIPTOR, M_SYSTEM_DESCRIPTOR, M_SYSTEM_TYPE.

Action commands are also supported on Zebra Concord PoE base model; however, you need to use an Aurora Imaging Library GigE Vision system (using MsysAlloc() with M_SYSTEM_GIGE_VISION).

Sending a Trigger-over-Ethernet packet without Host intervention

Although both models of Zebra Concord PoE support sending Trigger-over-Ethernet packets as described in Introducing action commands, the Zebra Concord PoE with ToE model can also do so without Host intervention.

Zebra Concord PoE with ToE includes a Trigger-over-Ethernet (ToE) module that can send a Trigger-over-Ethernet packet (action command or GigE Vision software trigger) over Ethernet, upon reception of an internal or external event, without Host intervention. The ToE module reduces latency, jitter, and is useful for applications that require a high degree of trigger accuracy. Since most GigE Vision cameras include a hardware implementation for receiving action commands, sending an action command is likely to provide a better result than sending a GigE Vision software trigger. For an explanation on action commands, see the Introducing action commands subsection of the Triggering simultaneous actions in multiple GigE Vision cameras section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. If your camera does not support action commands, or does not have hardware implemented for receiving them, then using a GigE Vision software trigger is an alternative option.

Sending a Trigger-over-Ethernet packet as an action command

To configure the ToE module to send a Trigger-over-Ethernet packet as an action command, you follow similar steps as when sending an action command in software. However, you set up the action command on an Aurora Imaging Library Concord PoE system instead of an Aurora Imaging Library GigE system; you still set up the cameras using digitizers allocated on an Aurora Imaging Library GigE system. Specifically:

1. Allocate an Aurora Imaging Library Concord PoE system, using MsysAlloc() with M_SYSTEM_CONCORD_POE.

2. Allocate an Aurora Imaging Library GigE system, using MsysAlloc() with M_SYSTEM_GIGE_VISION. Then, allocate a digitizer on this system for each GigE camera that is physically connected to the Zebra Concord PoE and will receive the action command, using MdigAlloc().

3. Set up the action command on the Aurora Imaging Library Concord PoE system. To do so, make successive calls to MsysControl() with the Aurora Imaging Library Concord PoE system identifier and each of the following:

   • M_GC_ACTION_DEVICE_KEY to identify the group of cameras to receive the action command.

   • M_GC_ACTION_GROUP_MASK to limit which cameras in the group should generate the action signal upon receiving the command. Set this to 0xFFFFFFFF if all cameras in the group should generate the action signal.

   • M_GC_ACTION_GROUP_KEY to specify the key corresponding to the internal action signal on the camera.

   • M_TRIGGER_SOURCE to specify the event that should cause the action command to be sent (for example, an auxiliary input signal).

4. Set up the GigE cameras to receive the action command, using successive calls to MdigControlFeature() with their digitizer identifiers (allocated on the Aurora Imaging Library GigE system) and each of the following features: ActionDeviceKey, ActionGroupMask, and ActionGroupKey.

5. Set up the cameras to acquire the next frame upon receiving the action command. To do so, make successive calls to MdigControlFeature() with the camera's corresponding digitizer and each of the following features:

   • The TriggerSelector feature to select the type of trigger to configure (for example, FrameStart).

   • The ActionSelector feature to associate an internal action signal (for example, Action0) with the action group key.

   • The TriggerSource feature to set the trigger source of the frame start to the internal action signal (for example, Action0).

- The TriggerMode feature to activate triggering the frame start upon the reception of the action command.

6. Register with Aurora Imaging Library the cameras associated with the action command. To do so, make successive calls to MsysControl() with the Aurora Imaging Library Concord PoE system, M_ADD_DESTINATION, and each of the Aurora Imaging Library digitizers. Aurora Imaging Library uses this information to know on which port to send the packet and to track and make use of the camera's action results.

7. Activate sending a Trigger-over-Ethernet packet as an action command upon the reception of the specified trigger event (for example, an auxiliary input signal). Use MsysControl() with M_TRIGGER_STATE set to M_ENABLE.

The following code snippet demonstrate how to set up the ToE module on Zebra Concord PoE with ToE to send an action command, and how to set the camera to grab a frame upon receiving the action command:

```
void SetupToEUsingActions(AIL_ID AilConcordPoESystem, const std::vector<ToEDevice>& Devices)
{
  MosPrintf(AIL_TEXT("\nSetting-up GigE Vision devices and the Zebra ConcordPoE board.\n\n"));

  /* Set-up action command on Zebra Concord PoE with ToE */
  AIL_INT DeviceKey = 0x56781234, GroupKey = 0x24, GroupMask = 0xFFFFFFFF;
  MsysControl(AilConcordPoESystem, M_GC_ACTION0 + M_GC_ACTION_DEVICE_KEY, DeviceKey);
  MsysControl(AilConcordPoESystem, M_GC_ACTION0 + M_GC_ACTION_GROUP_MASK, GroupMask);
  MsysControl(AilConcordPoESystem, M_GC_ACTION0 + M_GC_ACTION_GROUP_KEY, GroupKey);
  MsysControl(AilConcordPoESystem, M_GC_ACTION0 + M_TRIGGER_SOURCE, M_AUX_IO2);

  /* Set-up action command on the GigE Vision devices */
  for (size_t i = 0; i < Devices.size(); i++)
  {
    AIL_INT64 ActionNumber = 0;
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("ActionSelector"), M_TYPE_INT64, &Ac
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("ActionDeviceKey"), M_TYPE_INT64, &D
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("ActionGroupMask"), M_TYPE_INT64, &G
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("ActionGroupKey"), M_TYPE_INT64, &Gr

    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerSelector"), M_TYPE_STRING, A
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerSource"), M_TYPE_STRING, AIL
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerMode"), M_TYPE_STRING, AIL_T

    MsysControl(AilConcordPoESystem, M_GC_ACTION0 + M_ADD_DESTINATION, Devices[i].AilDigitizerId);
  }
}
```

Sending a Trigger-over-Ethernet packet as a GigE Vision software trigger

To configure the ToE module to send a Trigger-over-Ethernet packet as a GigE Vision software trigger, do the following:

1. Allocate an Aurora Imaging Library Concord PoE system, using MsysAlloc() with M_SYSTEM_CONCORD_POE.

2. Allocate an Aurora Imaging Library GigE Vision system, using MsysAlloc() with M_SYSTEM_GIGE_VISION. Then, allocate a digitizer on this system for each GigE Vision camera that should receive the GigE Vision software trigger, using MdigAlloc().

3. Set up the ToE module to send a Trigger-over-Ethernet packet, as a GigE Vision software trigger, upon a specified event. To do so, make successive calls to MsysControl() with the Aurora Imaging Library Concord PoE system identifier and each of the following:

• M_GC_TRIGGER_SOFTWARE0 + M_GC_TRIGGER_SELECTOR to specify the type of trigger that should take place on the cameras upon receiving the Trigger-over-Ethernet packet (for example, frame start).

• M_GC_TRIGGER_SOFTWARE0 + M_TRIGGER_SOURCE to specify the event that should cause the Trigger-over-Ethernet packet (GigE Vision software trigger) to be sent (for example, an auxiliary input signal).

4. Set up the cameras to receive the Trigger-over-Ethernet packets, using successive calls to MdigControlFeature() with their digitizer identifiers (allocated on the Aurora Imaging Library GigE Vision system) and each of the following features:

   • The TriggerSelector feature to select the type of trigger to configure (for example, FrameStart).

   • The TriggerSource feature to set the trigger source of the frame start to Software.

   • The TriggerMode feature to activate triggering the frame start upon the reception of the GigE Vision software trigger.

5. Specify the cameras that should receive the GigE Vision software trigger. To do so, make successive calls to MsysControl() with the Aurora Imaging Library Concord PoE system, M_ADD_DESTINATION, and each of the Aurora Imaging Library digitizers.

6. Activate sending a Trigger-over-Ethernet packet as a GigE Vision software trigger upon the reception of the specified trigger event (for example, an auxiliary input signal). Use MsysControl with M_TRIGGER_STATE set to M_ENABLE.

The following code snippet demonstrate how to set up the ToE module on Zebra Concord PoE with ToE to send a GigE Vision software trigger, and how to set the camera to grab a frame upon receiving the GigE Vision software trigger:

```
void SetupToEUsingSoftware(AIL_ID AilConcordPoESystem, const std::vector<ToEDevice>& Devices)
{
  MosPrintf(AIL_TEXT("\nSetting-up GigE Vision devices and the Zebra ConcordPoE board.\n\n"));

  /* Set-up GigE software trigger on Zebra Concord PoE with ToE */
  MsysControl(AilConcordPoESystem, M_GC_TRIGGER_SOFTWARE0 + M_TRIGGER_SOURCE, M_TIMER1);
  MsysControl(AilConcordPoESystem, M_GC_TRIGGER_SOFTWARE0 + M_GC_TRIGGER_SELECTOR, AIL_TEXT("FrameStart"));

  /* Set-up GigE software trigger on the GigE Vision devices */
  for (size_t i = 0; i < Devices.size(); i++)
  {
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerSelector"), M_TYPE_STRING, A
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerSource"), M_TYPE_STRING, AIL
    MdigControlFeature(Devices[i].AilDigitizerId, M_FEATURE_VALUE, AIL_TEXT("TriggerMode"), M_TYPE_STRING, AIL_T

    MsysControl(AilConcordPoESystem, M_GC_TRIGGER_SOFTWARE0 + M_ADD_DESTINATION, Devices[i].AilDigitizerId);
  }

  MsysControl(AilConcordPoESystem, M_GC_TRIGGER_SOFTWARE0 + M_TRIGGER_STATE, M_ENABLE);
}
```

Zebra Concord PoE with ToE connectors and signal names

This section serves as a reference to match Zebra Concord PoE with ToE's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MsysControl() / MsysInquire(), respectively.

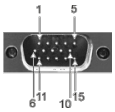
Auxiliary I/O signals can have one or more functionalities (for example, outputting the state of an I/O command register bit, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Concord PoE with ToE board provides an external auxiliary I/O connector and Gigabit Ethernet connectors.

Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
External auxiliary I/O connector	HD-15		The external auxiliary I/O connectors is used to transmit/receive auxiliary signals from the Advanced I/O engine.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

System			Description
	I/O #		
		Pin information	
		Direction	

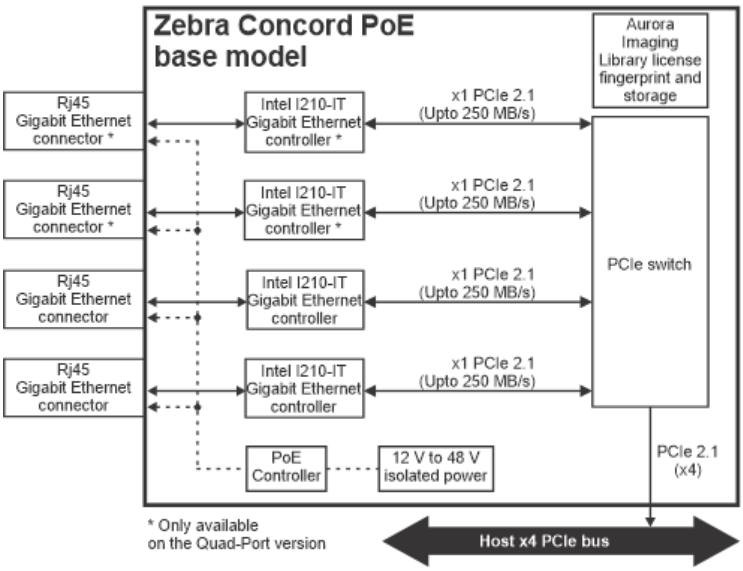
		User-bit information	
		Trigger information	
		Timer information	
		Hardware manual signal name	
System			Indicates the following.
	I/O #: M_AUX_IO0		Opto-isolated auxiliary signal 0 (output), which supports: user output, timer output, and receiving the state of an I/O command register bit.
		Pin information	Connector: HD-15 Pin: 12+, 11-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT0
	I/O #: M_AUX_IO1		Opto-isolated auxiliary signal 1 (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: HD-15 Pin: 6+, 1-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT1;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT1
	I/O #: M_AUX_IO2		Opto-isolated auxiliary signal 2 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: HD-15 Pin: 5+, 4 (AUX_OPTOIND_IN_COM_2-3)
		Direction	Input

	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN2
I/O #: M_AUX_IO3		Opto-isolated auxiliary signal 3 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: HD-15 Pin: 3+, 4 (AUX_OPTOIND_IN_COM_2-3)
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN3
I/O #: M_AUX_IO4		Opto-isolated auxiliary signal 4 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: HD-15 Pin: 9+, 8 (AUX_OPTOIND_IN_COM_4-5)
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN4
I/O #: M_AUX_IO5		Opto-isolated auxiliary signal 5 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: HD-15 Pin: 2+, 8 (AUX_OPTOIND_IN_COM_4-5)
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN5
I/O #: M_AUX_IO6		Opto-isolated auxiliary signal 6 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: HD-15 Pin: 15+, 14 (AUX_OPTOIND_IN_COM_6-7)
	Direction	Input

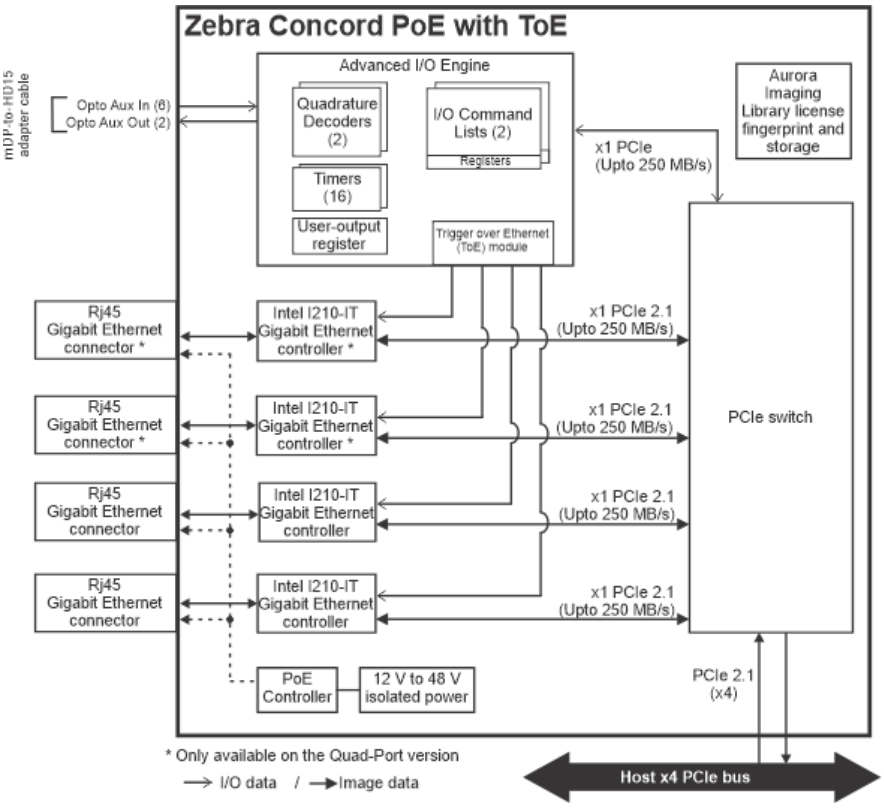
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN6
	I/O #: M_AUX_IO7		Opto-isolated auxiliary signal 7 (input), which supports: user input, I/O command list counter source, timer clock, timer arm, reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: HD-15 Pin: 13+, 14 (AUX_OPTOIND_IN_COM_6-7)
		Direction	Input
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN7

Zebra Concord PoE board flow diagrams

The following diagram illustrates the data flow of the Zebra Concord PoE base model.



The following diagram illustrates the data flow of Zebra Concord PoE with ToE.



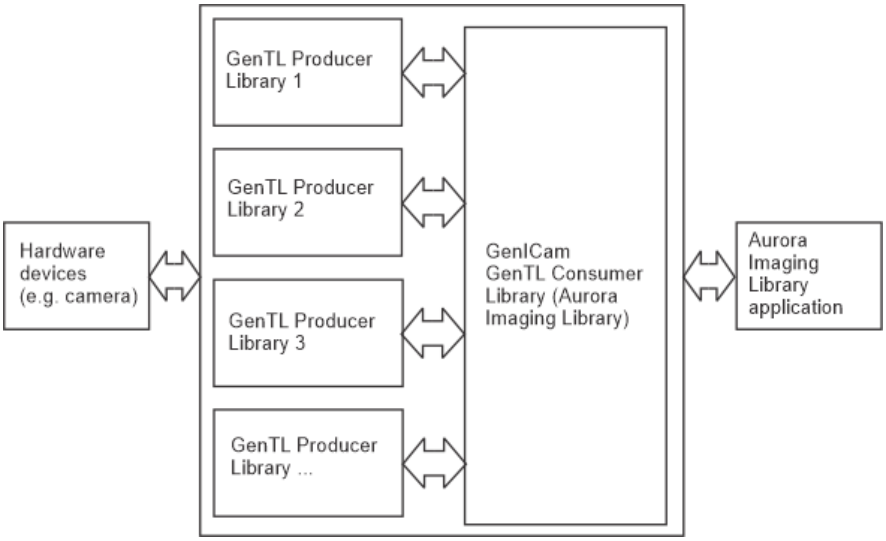
Zebra GenTL Consumer

This chapter discusses features of Aurora Imaging Library that are particular to the Zebra driver for GenTL (Aurora Imaging Library GenTL Consumer), and optimizing its performance. Information regarding using this driver with specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra GenTL Consumer (library) overview
  - Support features and standards
- Using Zebra GenTL Consumer (library)
  - GenICam-compliant device description files
  - Allocating Aurora Imaging Library GenTL systems
  - Mixed-type GenTL Producers
  - Controlling and Inquiring GenTL features
  - Aurora Imaging Library buffer handling with the Zebra GenTL Consumer (library)
  - General GenTL Producer configuration and installation requirements
  - GenIcam SFNC

Zebra GenTL Consumer (library) overview

The Zebra GenTL Consumer (library) allows you to use a third-party GenTL Producer (library) with Aurora Imaging Library. The Zebra GenTL Consumer (based on the GenICam GenTL standard) provides a software-generic way to communicate with one or more third-party devices through multiple interface technologies (such as, using GigE Vision, Camera Link, USB3 Vision, or a custom/non-standard interface to connect the camera and the Host).



The Zebra GenTL Consumer requires you to first install a third-party, vendor-supplied, standard compliant, GenTL Producer. Once installed, the Aurora Imaging Configurator utility lists the available GenTL Producer libraries. It is possible that multiple Producer libraries can access the same device (camera). For example, if you install multiple GenTL Producer libraries for GigE Vision-compliant devices (from different vendors), it is possible that any one of them can be used to grab from a GigE Vision-compliant device.

A GenTL Producer is a software library implementing the GenTL interface to enable an application or a software library to access and configure hardware in a generic way (for example, to stream image data from a camera). The Aurora Imaging Library GenTL Consumer is a software library that communicates with multiple GenTL Vision-compliant devices via the specified GenTL Producer.

You can install more than one GenTL Producer on your computer. These libraries are indexed and sorted by Aurora Imaging Library. You can inquire the indices of the installed GenICam GenTL Producers in the Aurora Imaging Configurator utility, or inquire the count of different GenTL Producers installed using MappInquire() with M_GENTL_PRODUCER_COUNT.

If you are using a GigE Vision or USB3 Vision-compliant device, or a camera connected to a Zebra frame grabber, you should use the native library for maximum performance (rather than the Zebra GenTL Consumer). The Zebra GenTL Consumer should be used with devices (cameras) that use a third-party, proprietary communication protocol, and for which the vendor has supplied a GenTL Producer.

Support features and standards

The Zebra GenTL driver supports GenTL version 1.6. It also supports **M_DYNAMIC** and **M_CONTAINER** grab buffers.

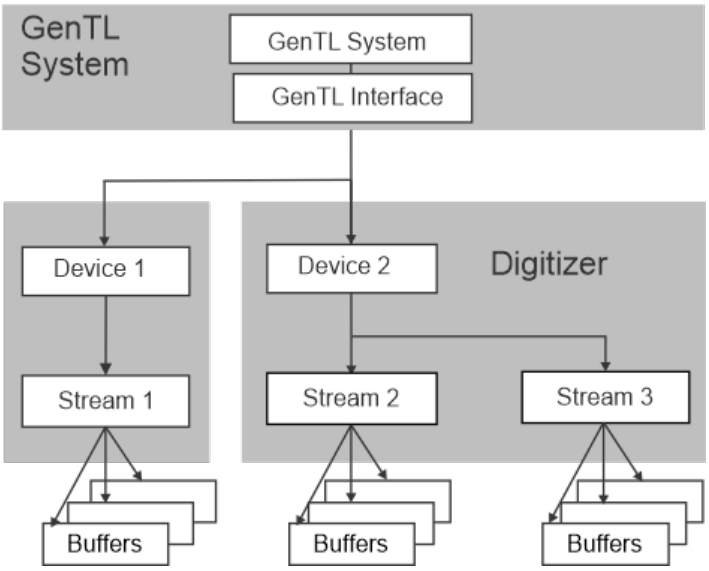
The Zebra GenTL driver supports the following standards:

- GenICam GenTL Standard version 1.6.
- GenICam GenApi version 3.4.2.
- GenICam GenDC Standard version 1.1.

- GenICam Standard Feature Naming Convention (SFNC) version 2.7.

Using Zebra GenTL Consumer (library)

To use your Zebra GenTL Consumer (library) with Aurora Imaging Library, you must first install at least one third-party GenTL Producer (library). The Aurora Imaging Library GenTL Consumer communicates with the third-party GenTL Producer that allows you to configure or grab from your compliant devices (camera).



Aurora Imaging Capture Works' Feature Browser can access all of the GenICam GenTL-compliant device description files (XML), providing an interface to view and change the system information, digitizer, device (camera), and buffer.

GenICam-compliant device description files

The Zebra GenTL Producer (library) uses five distinct sets of GenICam-compliant device description files (XML).

- **Per Aurora Imaging Library system:**
  - **GenTL system configuration information**. The system refers to the allocated GenTL Producer. There is one GenTL system configuration information file per GenTL Producer.
  - **GenTL interface configuration information**. The interface, in this case, is the type of physical connection (port) between the Host and the device (such as Camera Link, Ethernet, or USB3 connections). Note that this could include auxiliary I/O signals such as trigger input, user input or user output, depending on the signal.
- **Per Aurora Imaging Library digitizer:**
  - **GenTL device configuration information**. The GenTL device is a proxy to the remote device controlling the GenTL Producer's view of the device. Typically, there is one GenTL device configuration information file for each digitizer allocated on your Aurora Imaging Library system.
  - **GenTL remote device configuration information**. The GenTL remote device is your actual device (camera). Typically, there is one remote device configuration information file for each camera allocated to a specific Aurora Imaging Library digitizer. With Aurora Imaging Library, this is a one-to-one relationship.
  - **GenTL stream configuration information**. The GenTL stream is the data stream generated by your device (camera) is handled by the GenTL Producer. Typically, there is one GenTL stream configuration information file per data stream between your device and your digitizer.
- **Per Aurora Imaging Library buffer:**
  - **GenTL buffer configuration information** (optional). The GenTL buffer is how the GenTL Consumer

handles buffers. Typically, there is one GenTL buffer configuration information file per acquisition target (that is, per Aurora Imaging Library buffer used by your Aurora Imaging Library system).

Allocating Aurora Imaging Library GenTL systems

You can then allocate an Aurora Imaging Library GenTL system using MsysAlloc() with M_SYSTEM_GENTL. Set SystemNum to M_DEVn + M_GENTL_PRODUCER(n), where M_DEVn corresponds to the rank of the adapter board and M_GENTL_PRODUCER(n) specifies the index of the GenTL Producer to allocate. When you install more than one GenTL Producer on your computer, the GenTL Producers are indexed and sorted by Aurora Imaging Library. You can establish the number of GenTL Producers, using MappInquire() with M_GENTL_PRODUCER_COUNT or using the General Default Values page of the Aurora Imaging Configurator utility. Allocating a GenTL system opens general communication with all connected third-party devices that can communicate with the GenTL Producer. You can then allocate a digitizer for each device (camera) using MdigAlloc().

You can allocate more than one Aurora Imaging Library GenTL system (MsysAlloc() with M_SYSTEM_GENTL) in a process (executable); and each must have a unique SystemNum. Note that for GigE Vision, USB3 Vision, and custom transport layer type Producers, you must only use M_DEV0 to allocate the GenTL system. Allocating one Producer will discover all devices of this type regardless of their adapter board. For all other transport layer type Producers, M_DEVn targets the *nth* frame grabber of that type. For example, you can allocate two systems that use the same GenTL Producer but target different frame grabbers that have the same type as the Producer. GenICam GenTL Producers of mixed-type can be used multiple times; the mixed-type supports multiple interface technologies, and can be used once per interface type. For example, a mixed-type library can support both a proprietary Ethernet (non-GigE Vision) camera and a proprietary USB (non-USB3 Vision) camera; therefore, a GenICam GenTL mixed-type Producer can be allocated twice. Otherwise, a GenTL Producer can only be used once per Aurora Imaging Library application.

Mixed-type GenTL Producers

When allocating an Aurora Imaging Library GenTL system for a mixed-type GenTL Producer, you can also set the InitFlag parameter to the type of interface technology that you want to use. For example, if you have a mixed-type GenTL Producer that supports both GigE Vision and USB3 Vision, but you only want GigE Vision interfaces, set the InitFlag to M_GEV. If you do not specify an interface type, all interface types supported by the Producer are used.

Controlling and Inquiring GenTL features

In most cases, you should control and inquire GenTL features using the conventional Aurora Imaging Library functions (for example, MsysControl() / MsysInquire(), MdigControl() / MdigInquire(), and MbufControl() / MbufInquire()). However, some GenTL-compliant device have features that can only be accessed using MsysControlFeature() / MsysInquireFeature(), MdigControlFeature() / MdigInquireFeature(), and MbufControlFeature() / MbufInquireFeature(). For more information about accessing the features of your GenTL-compliant device, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Aurora Imaging Library buffer handling with the Zebra GenTL Consumer (library)

The GenICam GenTL standard specifies that a buffer must be announced before it can be used to receive a grabbed image. To overcome this requirement, the Zebra GenTL Consumer automatically announces ten internal buffers when any digitizer is allocated (using MdigAlloc()). Note that allocated grab buffers will also be automatically announced to the Producer when MdigGrab(), MdigGrabContinuous(), or MdigProcess() is called. This allows you to use MdigGrab(), MdigGrabContinuous(), or MdigProcess() to capture frames. These functions use internal buffers to buffer the incoming data before it is copied to the specified Aurora Imaging Library buffer, or if a grab cannot be made directly to the allocated buffer. For example, if you allocate a 3-band color grab buffer and the device (camera) outputs a

monochrome image, the image will be grabbed into an internal buffer and copied into the allocated 3-band color buffer. Note that you are still require to allocate and specify an Aurora Imaging Library buffer. Note that when using MdigProcess(), each buffer queued is automatically announced by Aurora Imaging Library.

Note that when using MdigGrab(), there is a delay announcing the buffer because pending grabs must finish before announcing a new buffer. You can manually announce buffers using MdigControl() with M_GENTL_ANNOUNCE_BUFFER. When using MdigGrabContinuous() or MdigProcess(), there is no delay present; this is because grabs are not immediately executed when these functions are called. When you are finished with the buffer, revoke it using MdigControl() with M_GENTL_REVOKE_BUFFER before freeing it. Note that if you use M_GENTL_ANNOUNCE_BUFFER, Aurora Imaging Library will not use its internal buffers.

General GenTL Producer configuration and installation requirements

The GenTL Producer communicates with and is used by the Zebra GenTL Consumer (library) to configure or grab from your GenICam-compliant devices (camera). You can configure a GenTL Producer in the Aurora Imaging Configurator utility by expanding the **General** item in the tree structure and navigating to the **Default Values** subitem where you can select the GenTL Producer number that corresponds to the correct GenTL Producer library. Alternatively, you can specify the GenTL Producer to allocate the Aurora Imaging Library GenTL system by setting SystemNum to M_DEVn + M_GENTL_PRODUCER(n), where M_DEVn corresponds to the rank of the board and M_GENTL_PRODUCER(n) specifies the index of the GenTL Producer to allocate. You can determine the number of GenTL libraries installed on your computer, using MappInquire() with M_GENTL_PRODUCER_COUNT. If the expected GenTL Producer library is not visible in the Aurora Imaging Configurator utility, the cause is typically that the device's GenTL Producer file (CTI file extension) or the `GENICAM_GENTL64_PATH` environment variable does not include the path to the correct CTI file. Make sure that the GenTL Producer is installed either by the software provided with the device, or install a compatible generic GenTL Producer. In addition, verify that the `GENICAM_GENTL64_PATH` environment variable has the appropriate CTI file path. Each directory that you list in the environment variable must contain a CTI file (GenTL Producer).

> Note that GenTL Producer information is highly dependent on the vendor of the GenICam-compliant device. For specific information related to your device, refer to the device's documentation.

GenIcam SFNC

For more information on the GenICam GenTL standard features available, refer to GenICam's standard feature naming convention (SFNC), found online at: emva.org/standards-technology/genicam/. Additional GenICam GenTL SFNC-extension features might be available; check your camera's manual and Aurora Imaging Capture Works' Feature Browser for details. Note that the standard features specified in the GenICam SFNC work with the Aurora Imaging Library Digitizer module.

Zebra GevIQ

This section discusses features of Aurora Imaging Library that are particular to Zebra GevIQ board and ways that optimize its performance. Zebra GevIQ specific information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra GevIQ overview
  - Summary of Zebra GevIQ features
- Using Zebra GevIQ
  - Grabbing and the Aurora Imaging Library buffer formats to use for each camera pixel format
  - Performing Bayer color conversion in hardware
- Zebra GevIQ utilities and tools
  - Zebra GevIQ Bench utility
  - Aurora Imaging Capture Works
  - Aurora Imaging Gecho Viewer
  - Zebra GigE Vision discovery service
  - Zebra GevIQ PCIe Information tool
  - Zebra GevIQ System Monitor tool
  - Ethernet switches and Configuring your IP
- Minimum latency and grabbing all frames
  - Detecting missed frames
  - Efficiently copying an on-board buffer to a destination buffer in Host memory
  - Efficiently converting buffer formats while copying
- Zebra GevIQ board flow diagram
- Miscellaneous user guide information for Zebra GevIQ
  - GigE Vision Stream Channel
    - Stream channel support
    - Link-Aggregation
  - GenICam GenTL Producer support
  - Aurora Imaging Capture Works
    - Configuration file editor
    - Ethernet adapter configuration tool
    - Saving components
    - Renaming devices
    - Device window display mode
  - Power conservation
  - Particularities

Zebra GevIQ overview

Zebra GevIQ is a smart network adapter that provides two 10 or 25 GbE interfaces (ports) for connection with 1/2.5/5/10/25 Gbits/sec GigE Vision-compatible cameras. Zebra GevIQ is a low-profile PCIe 3.0 x8 board that includes two SFP cages for connecting SFP+/SFP28 transceiver modules. The board can also be used as a standard network interface controller (NIC), and supports Aurora Imaging Library licensing fingerprint storage. Zebra GevIQ supports GigE Vision Standard version 2.2, GenICam GenApi version 3.4.2, GenICam Standard Feature Naming Convention (SFNC) version 2.7, and GenICam GenTL version 1.6.

Zebra GevIQ supports up to two GigE Vision cameras connected directly to the Zebra GevIQ board, or can offload up to 32 stream channels when connected behind a switch. Area-scan and line-scan monochrome or color video sources are supported, as long as they are GigE Vision-compliant (including video sources with a Bayer color filter).

GigE Vision streaming protocol packets are offloaded to Zebra GevIQ, where images are reconstructed on-board and then sent to the Host using scatter-gather DMA transfers if the destination buffer is in paged memory. Zebra GevIQ can send data to the Host at a maximum theoretical transfer rate of 8 Gbytes/sec using scatter-gather DMA transfers. Optimum conditions for high speed transfer include using the board in a PCIe 3.0 slot with 8 active lanes and using a 256-byte payload. If the destination buffer is non-paged, then standard DMA transfers are used.

To measure the effective available bandwidth of the PCIe slot in your computer with your Zebra GevIQ board, Zebra provides the Zebra GevIQ Bench utility. This utility is accessible using the Aurora Imaging Configurator utility. For more information on Zebra GevIQ Bench utility other tools, refer to the Zebra GevIQ utilities and tools section later in this chapter.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/geviq-info.

Summary of Zebra GevIQ features

The following table outlines the features currently available for Zebra GevIQ.

	Zebra GevIQ
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature()	Yes
On-board memory	4 Gbytes of DDR4 SDRAM
PCIe (3.0) board type	x8 low-profile board
Transceiver modules (SFP+/SFP28)	2
Number of stream channels	Up to 32 when connected behind a switch
Acquisition section	GigE Vision

GigE Vision compatible camera speeds	1/2.5/5/10/25 Gbit/sec
Color space converter/Image formatter	Yes [1]
Bayer decoder	Yes (GRBG, GBRG, BGGR, RGGB supported)
Supports Aurora Imaging Library licensing fingerprint storage	Yes

[1] It can convert 8-bit or 16-bit monochrome, or 24-bit or 48-bit packed BGR data to monochrome, packed BGR, packed BGRa, planar RGB, or YUV (YUYV) format.

Using Zebra GevIQ

To use Zebra GevIQ, you must allocate it as an Aurora Imaging Library GevIQ system (using MsysAlloc() with M_SYSTEM_GEVIQ). This allocation opens general communication with (allows you to discover) all GigE Vision-compliant cameras (or other devices) connected to Zebra GevIQ directly or behind a switch. You must then allocate a digitizer for each camera that you want to use to grab images and/or access directly, using MdigAlloc(). If you have multiple GigE Vision-compliant interface cameras, you should use the M_GC_CAMERA_ID() macro and specify the device's name; this will make for more portable code. You can use the Aurora Imaging Capture Works utility to view a list of available GigE Vision-compliant cameras and their user-defined name. For information on this utility, refer to the Zebra GevIQ utilities and tools section later in this chapter. For information regarding working with multiple GigE Vision-compliant devices, refer to the Working with one or more GigE Vision-compliant cameras subsection of the Using GigE Vision with Aurora Imaging Library section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes. You can allocate an Aurora Imaging Library GevIQ system for your board in multiple processes (executables).

> Note that you must only allocate an M_SYSTEM_GEVIQ system per Zebra GevIQ board and not for each SFP transceiver per board.

Zebra GevIQ is a network adapter that supports digital video sources compliant with the GigE Vision 2.2 standard. Zebra GevIQ can grab monochrome, RGB color, and Bayer color-encoded data. The board can also perform pixel unpacking, image sub-sampling, and color-conversion. Depending on how your video sources are configured/connected to your Zebra GevIQ (for example, using a switch), an SFP connector could have multiple offloaded stream channels behind it. For information regarding working with multiple GigE Vision-compliant devices using a switch, refer to the Using a Gbit Ethernet switch with GigE Vision-compliant cameras section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Grabbing and the Aurora Imaging Library buffer formats to use for each camera pixel format

The following table outlines which Aurora Imaging Library buffer format to use with each supported camera pixel format. Note that these are the officially supported camera pixel formats.

Camera pixel format [3]	Aurora Imaging Library buffer format (type and attribute)	Operation
Mono8, Mono10, Mono12, Mono14, Mono16, Mono32	1-band 8-bit, 1-band 16-bit, 1-band 32-bit, 3-band 8-bit [1] , 3-band 16-bit [2]	Data will automatically be bit-shifted to have most-significant bits (MSB) in destination buffer
BayerXX8, BayerXX10, BayerXX12, BayerXX14, BayerXX16 where XX = GR, GB, BG or RG	1-band 8-bit, 1-band 16-bit	Demosaicing followed by RGB to Y (bit shift for MSB)
	3-band 8-bit, 3-band 16-bit	Demosaicing (bit shift for MSB)
	3-band YUV16 + M_PACKED	Demosaicing followed by RGB to YUV

RGB8, RGBA8,	1-band 8-bit	RGB to Y
	3-band 8-bit	Direct copy
	3-band YUV16 + M_PACKED	RGB to YUV
RGB10, RGBA10, RGB12, RGBA12, RGB14, RGBA14, RGB16, RGBA16	1-band 8-bit	RGB to Y (bit shift for MSB)
	1-band 16-bit	RGB to Y
	3-band 16-bit M_PLANAR	Packed destination buffers are not supported
YUV422_8, YCbCr601_422_8, YCbCr709_422_8	1-band 8-bit	Only Y band
	3-band YUV16 + M_PACKED	Direct copy

[1] 3-band 8-bit buffers can be in any of the following formats: M_RGB24 + M_PLANAR, M_RGB24 + M_PACKED, M_BGR24 + M_PACKED, or M_BGR32 + M_PACKED.

[2] 3-band 16-bit buffers can be in either an M_RGB48 + M_PLANAR or M_RGB48 + M_PACKED format.

[3] All of the values available for RGB are also available in BGR format.

> Note that for Zebra GevIQ, allocating a buffer with M_GRAB will force the buffer in pageable memory (M_PAGED) by default.

Performing Bayer color conversion in hardware

When Zebra GevIQ grabs color images from a video source with a Bayer color filter, it performs Bayer color conversion in hardware, as it transfers the images to the Host. If the images require white balancing, Zebra GevIQ can perform this automatically if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_DISABLE.

The M_BAYER... control types of MdigControl() can only be used when grabbing from a camera that has a Bayer color filter; otherwise, an error will be generated.

Zebra GevIQ utilities and tools

The Zebra GevIQ can leverage the use of Aurora Imaging Capture Works (external application) and a series of tools integrated in the Aurora Imaging Configurator utility (such as, the PCIe Information tool and the System Monitor tool) to help setup, diagnose, and debug your system.

To access these utilities and tools, launch the Aurora Imaging Configurator utility. Expand the **Boards** item in the tree structure and then click on the **GevIQ** subitem from the tree structure in the presented interface. The utilities and tools are located in the **Launch external applications** and **Zebra GevIQ System Monitor** pages, respectively.

Zebra GevIQ Bench utility

The Zebra GevIQ Bench utility calculates the real-time transfer speed of the PCIe slot (in Gbytes/sec) for Zebra GevIQ-to-Host:

- **Non-paged DMA**: A copy from a Zebra GevIQ on-board buffer to a buffer located in Host non-paged memory.
- **Scatter gather DMA**: A copy from a Zebra GevIQ on-board buffer to a buffer located in Host paged memory using scatter gather DMA.

Another key feature of the Zebra GevIQ Bench utility is the interrupt latency tool, which measures the system interrupt latency (the time required for the system to respond to an interrupt). While the Zebra GevIQ is under load (for example, grabbing images using the Aurora Imaging Capture Works utility), the tool will output the current, minimum, and maximum ISR (Interrupt Service Routine) latency in microseconds.

Aurora Imaging Capture Works

Aurora Imaging Capture Works is an interactive utility to configure and test devices that make use of a GenICam-based interface standard, and supports the GigE Vision interface standard (besides other standards). Aurora Imaging Capture Works adds, in particular, support for GenICam GenDC containers and multi-part payload types. Aurora Imaging Capture Works also adds support for 3D displays. Aurora Imaging Capture Works will list all detected GigE Vision-compliant devices connected to each allocated board. It can start or stop capturing images, display acquired images, save the last grabbed image, send a software trigger, as well as browse and control the selected device's features. You can view and change acquisition properties, and view acquisition statistics.

Aurora Imaging Capture Works key features:

- Camera-centric user interface with support for camera enumeration and presence.
- Feature browser to display and control camera, digitizer, and system settings.
- DCF creation to save camera and digitizer settings.
- Efficient display of grabbed images with many options and view modes.
- Display of line and arc profiles.
- Display of image histogram.
- Memory viewer to display pixel values.

More information is available from the Aurora Imaging Capture Works manual available online.

Aurora Imaging Gecho Viewer

Aurora Imaging Gecho Viewer is an event logging application that gives you the ability to monitor and troubleshoot

acquisition performance.

Zebra GigE Vision discovery service

The Zebra GigE Vision discovery service allows your Aurora Imaging Library application and the Aurora Imaging Capture Works utility to automatically detect when GigE Vision-compliant cameras are added to or removed from your network. For more information, refer to the Zebra GigE Vision discovery service subsection of the Zebra GigE Vision tools section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Zebra GevIQ PCIe Information tool

The Zebra GevIQ PCIe Information tool returns information about the board's PCIe connection.

Use the acquisition (device) selector to set the stream channel about which to inquire (M_DEVn). Note that this selector is shared with the Zebra GevIQ System Monitor tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- Current number of PCIe lanes.
- Current speed of the PCIe lanes.
- Maximum number of PCIe lanes.
- Maximum speed of the PCIe lanes.

Zebra GevIQ System Monitor tool

The Zebra GevIQ System Monitor tool displays board properties of the specified digitizer (M_DEVn).

Use the acquisition (device) selector to set the stream channel about which to inquire (M_DEVn). Note that this selector is shared with the Zebra GevIQ PCIe Information tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- The current temperature of the camera connected to the specified digitizer (M_DEVn).
- The maximum recorded temperature of the camera connected to the specified digitizer (M_DEVn).

Ethernet switches and Configuring your IP

Zebra GevIQ supports digital video sources compliant with the GigE Vision 2.2 standard. Depending on how your video sources are configured/connected to your Zebra GevIQ (for example, using a switch), a single port can offload multiple stream channels. For information regarding working with multiple GigE Vision-compliant devices using a switch, refer to the Using a Gbit Ethernet switch with GigE Vision-compliant cameras section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Zebra GevIQ network adapter settings are already optimized for best performance and do not need to be adjusted (for example, in the event of corrupted frames). However, for generalized information regarding acquisition reliability and common causes (such as firewall settings, insufficient inter-packet delay, packet resend capabilities, and more), refer to the Troubleshooting acquisition reliability issues section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

For information regarding IP configuration of your Zebra GevIQ and GigE Vision-complaint camera, refer to the Configuring IP addresses section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Minimum latency and grabbing all frames

When grabbing with either MdigGrab() or MdigProcess(), Zebra GevIQ is optimized to have the smallest latency possible between the last pixel sent from the camera to the last pixel written into the buffer. This allows the buffer to be available for processing with a minimum amount of delay.

By default, Zebra GevIQ grabs into buffers in Host memory. On-board memory is used to protect against PCIe latency. To perform real-time grabs, your rate of acquisition (grabbing bandwidth) must be lower than the PCIe maximum transfer rate (PCIe bandwidth). If the acquisition rate is higher than the transfer rate across the PCIe bus, frames will be skipped rather than allowing a grab over-run to occur. To determine the image transfer speed from Zebra GevIQ to the Host, use the Zebra GevIQ Bench utility.

Detecting missed frames

To be certain that no frames were missed when MdigProcess() was last used, use MdigInquire() with M_PROCESS_FRAME_MISSED. This value reads the number of frames that were transmitted by the camera when no buffer was available to receive the frame; for example, if a grab is triggered and the associated hooked function is slow, then a frame can be missed.

Efficiently copying an on-board buffer to a destination buffer in Host memory

When copying an on-board buffer into a destination buffer in Host memory, the copy operation is performed by your Zebra GevIQ using the board's DMA write engine, rather than the Host. Your Zebra GevIQ will perform the operation faster than the Host. To ensure that Zebra GevIQ performs the operation, use the following:

- To copy an on-board buffer into a destination buffer in Host memory, use MbufCopy().

Efficiently converting buffer formats while copying

When copying an on-board buffer into a destination buffer in Host memory, the color space converter and image formatter can automatically convert the bit-depth and color format of the source buffer to the bit-depth and color format of the destination buffer.

For on-board conversion of buffers to take place, the following on-board buffer and destination buffer depth and color formats are supported.

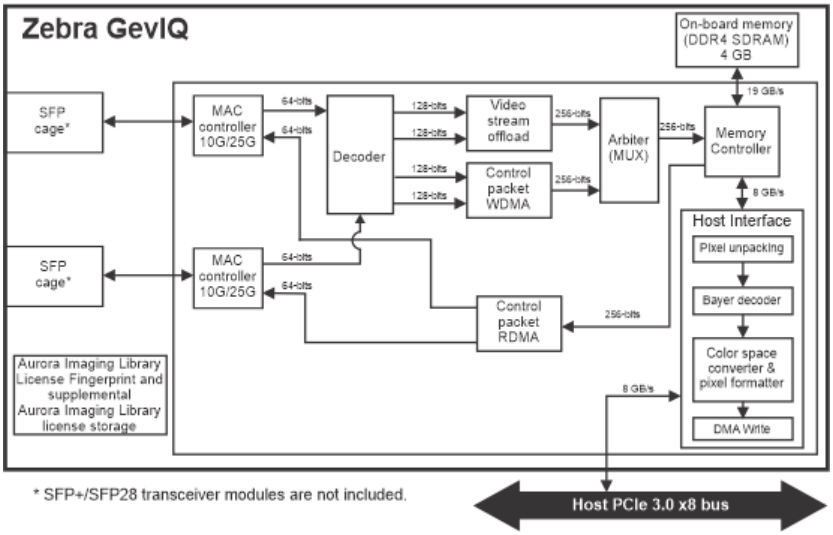
On-board buffer depth and color format (specified in the DCF)	Destination buffer depth and color format						
	8-bit monochrome	16-bit monochrome	M_PACKED + M_BGR24	M_PACKED + M_BGR32	M_YUV16_YUYV / M_YUV16	M_PLANAR + M_RGB24	M_PLANAR + M_RGB48
8-bit monochrome	Yes	No	Yes	Yes	Yes	Yes	No

16-bit monochrome	Yes	Yes	Yes	Yes	Yes	Yes	Yes
M_PACKED + M_BGR24	Yes	No	Yes	Yes	Yes	Yes	No
M_RGB48 + M_PACKED	Yes	Yes	Yes	Yes	Yes	Yes	Yes

For information about YUV or RGB/BGR buffers, refer to Chapter 23: Data buffers. For the calculations used to convert to YUV, refer to the Zebra GevIQ Hardware and Installation manual.

Zebra GevIQ board flow diagram

The following diagram illustrates the data flow of Zebra GevIQ.



Miscellaneous user guide information for Zebra GevIQ

This section includes additional user guide information for Zebra GevIQ. The information found in this section might be a reiteration of content previously documented.

GigE Vision Stream Channel

The following sections describe GigE Vision Stream Channel support and features for the Zebra GevIQ Smart GigE Vision Adapter:

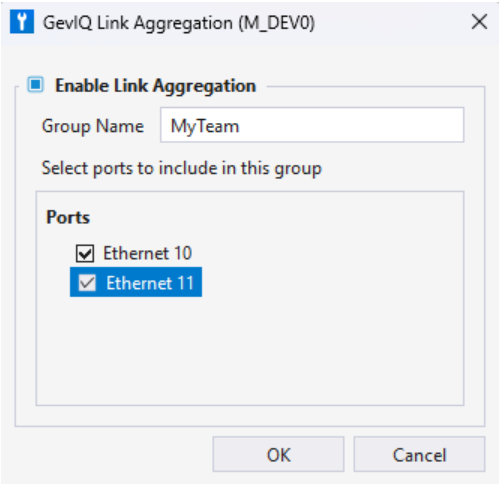
Stream channel support

Only the first stream is supported when interfacing with a camera that supports multiple stream channels.

Link-Aggregation

You can offload Link-Aggregated devices. Link-aggregation is a networking technique that combines multiple network connections to increase the total data transfer capacity and throughput. Link-Aggregation can be enabled by defining a Link-Aggregation Group (LAG) where physical network connections are grouped and appear to the user as a single network connection having higher transfer capacity and throughput.

To define a LAG, open Aurora Imaging Configurator and navigate to **GevIQ** under the **Boards** item and click on **Link Aggregation**. The **GevIQ Link Aggregation** dialog appears.



In the dialog, specify a group name and select the ports to include. Click **OK**. A reboot of the PC is typically required for the configuration change to take effect. In the example above, Ethernet 10 and Ethernet 11 will be replaced with a new connection that aggregates both ports.

Once defined, the link-aggregation configuration is stored in the FLASH memory of the Zebra GevIQ Smart GigE Vision Adapter and remains active until deleted.

To disable link aggregation, deselect **Enable Link Aggregation** in the above dialog and reboot.

The round-robin distribution algorithm is supported for static Link-Aggregation (as defined in the GigE Vision Specification).

GenICam GenTL Producer support

The Zebra GevIQ Smart GigE Vision Adapter software stack provides a GenICam GenTL producer. It can be used with third party libraries that implement a GenTL consumer.

Aurora Imaging Capture Works

The following are additional features that are available to Zebra GevIQ in Aurora Imaging Capture Works:

Configuration file editor

The following steps provide a basic methodology for using the device configuration file (DCF) editor:

1. In Aurora Imaging Capture Works, allocate the device. This will grant you access to the configuration file editor.
2. Navigate to the **Configuration File** tab, which is beside the feature browser.
3. Click the **Camera and Digitizer Configuration** button.
4. In the popup dialog, configure the configuration file. For example. you can drag and drop the camera and/or digitizer features into the Camera and Digitizer Configuration section to include them in the file.
5. Save your changes.
6. Use your configuration file when allocating a device. This file can be used to configure the device's settings.

Ethernet adapter configuration tool

Capture Works will highlight network adapters that are not correctly configured for GigE Vision acquisition with a yellow icon. A tooltip will offer the user recommended modifications. The user can easily rectify configuration issues by right-clicking on the network connection and choosing the **Optimize** option from the dropdown menu. Note that administrator privileges are necessary for these changes to be implemented.

In addition, an Ethernet Adapter Configuration tool is accessible from the Tools menu. This tool enables the optimization of multiple Ethernet adapters simultaneously and the ability to modify the IP configuration settings (including the IP address and subnet mask of the network adapters).

Alternatively, the Ethernet Adapter Configuration tool is also accessible from the command-line interface. Type `CaptureWorks.exe --help` on the command line for more details.

Zebra GevIQ Smart GigE Vision Adapter is optimized for GigE Vision acquisition. As such, the Capture Works Ethernet adapter configuration tool applies to network connections found in consumer end products when used with the software-based Aurora Imaging Library GigE Vision system and not the GevIQ network connections.

Saving components

You can save an individual component (such as an image) from a multi-component grab container. This allows you to isolate and save a single component from a collection. To use this feature, follow these steps:

1. In Aurora Imaging Capture Works, select the component to save via the display window.
2. Click the **Save** button in the top-right corner of the display window.

Renaming devices

Compatible devices can be re-named using the **F2** key or by right-clicking on the device and selecting the **Rename** option.

Device window display mode

You can change the display mode of the device window. The conventional Vendor-Model display scheme can be replaced with user-assigned names instead. To activate this feature, follow these steps:

1. Access the **Tools** menu and select **Options**.
2. In the Options dialog, select the **View** tab.
3. Change the Device Window Display Format to User-defined name.

Power conservation

Microsoft Windows power management can turn off Ethernet devices to save power. If you are using Zebra GevIQ Ethernet Adapters, ensure this option is turned off to maintain network connectivity. To do this, follow these steps:

1. Open Windows Device Manager.
2. Expand the Network Adapters section.
3. Right-click on a Zebra GevIQ Ethernet Adapter and select **Properties**.
4. In the Properties window, select the **Power Management** tab.
5. Uncheck the **Allow the computer to turn off this device to save power** option.

> These steps must be repeated for all Zebra GevIQ Ethernet Adapters listed under Network Adapters in the Device Manager so that Ethernet adapters remain powered on and connected to the network, even when the system is trying to conserve power.

To achieve maximum performance, the following Windows power options should be configured:

- High Performance power plan should be selected.
- Under advanced power options, ensure **PCI Express Link State Power Management** is turned off.

Particularities

The following are some particularities when using Zebra GevIQ:

M_GRAB buffers are allocated in paged memory by default.

CPU usage will be higher due to post-processing when:

- The camera PixelFormat is in YUV format, and a color-space conversion is required.
- A camera streams in legacy Chunk Data payload type.

The following digitizer controls are disabled when the payload transmitted by the camera is not an image payload type (as defined in the GigE Vision specification):

- **M_GRAB_SCALE_X**.
- **M_GRAB_SCALE_Y**.
- **M_SOURCE_SIZE_X**.
- **M_SOURCE_SIZE_Y**.
- **M_SOURCE_OFFSET_X**.
- **M_SOURCE_OFFSET_Y**.

If available, **M_GRAB_DIRECTION_X** and **M_GRAB_DIRECTION_Y** digitizer controls are mapped to ReverseX and ReverseY camera features.

Microsoft Kernel DMA protection is not available. See Kernel DMA Protection | Microsoft Learn for more details. If the Zebra GevIQ driver fails to start with code 55, you will need to disable kernel DMA protection in the UEFI/BIOS.

Packet resends are not available.

Zebra GigE Vision driver

This chapter discusses features of Aurora Imaging Library that are particular to the Zebra GigE Vision driver, and ways that optimize its performance. Information regarding using this driver with specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra GigE Vision driver overview
- Using GigE Vision with Aurora Imaging Library
  - Working with more than one Gigabit Ethernet network adapter
  - Working with one or more GigE Vision-compliant cameras
  - Working without a Zebra board as your Gigabit Ethernet network adapter
- Zebra GigE Vision tools
  - Aurora Imaging Capture Works
  - Zebra GigE Vision discovery service
- Configuring your GigE Vision camera and Gigabit Ethernet network adapter
  - Automatic format switching
  - Gigabit Ethernet network adapter settings
  - Optimizing further
- Configuring IP addresses
- Link aggregation
- Using IP multicast
  - Configuring a multicast controller
  - Configuring one or more multicast workers
    - Configuring one or more multicast monitors
- Ethernet switches, routers, IGMP, and IP multicast
  - Internet Group Management Protocol (IGMP)
  - Ethernet switches and multicast
  - IGMP snooping and Ethernet switches
  - Using IP multicast without a router
- Using a Gbit Ethernet switch with GigE Vision-compliant cameras
- Troubleshooting acquisition reliability issues
- Triggering simultaneous actions in multiple GigE Vision cameras
  - Introducing action commands
  - Establish if your camera supports action commands
  - Acknowledgment of reception of an action command
  - Masking out one or more cameras
  - Scheduling time
  - Examples of action commands
    - Example of sending a single action command
    - Example of sending multiple action commands
    - Example of using group masks to ignore an action command on one camera
  - Example distributed with Aurora Imaging Library
- Miscellaneous user guide information for Zebra GigE Vision driver
  - Working with a 3D device
    - Grabbing from a 3D device
    - Grabbing into an Aurora Imaging Library container
    - Grabbing into an Aurora Imaging Library buffer
    - Grabbing using older utilities and certain examples
  - Capture Works
    - Ethernet adapter configuration tool
    - GigE Vision device IP configuration tool

    - Known issues
    - Standards compliance
- Miscellaneous reference information for Zebra GigE Vision driver
    - Mbuf
        - Additions to MbufAlloc...
        - Additions to MbufInquire
        - Additions to MbufInquireContainer
    - Mdig
        - Additions to MdigAlloc
        - Additions to MdigControl
        - Additions to MdigControlFeature
        - Additions to MdigGetHookInfo
        - Additions to MdigHookFunction
        - Additions to MdigInquire
        - Additions to MdigInquireFeature
    - Msys
        - Additions to MsysControl
        - Additions to MsysGetHookInfo
        - Additions to MsysHookFunction
        - Additions to MsysInquire

Zebra GigE Vision driver overview

The Zebra GigE Vision driver allows you to use a Gigabit Ethernet (1000 BaseT or faster) network adapter to grab from GigE Vision-compliant cameras with Aurora Imaging Library. The Gigabit Ethernet network adapter can be a third-party device, or a Zebra product (such as the Zebra Concord PoE).

Using GigE Vision with Aurora Imaging Library

To use your Zebra GigE Vision driver with Aurora Imaging Library, you must allocate an Aurora Imaging Library GigE Vision system (M_SYSTEM_GIGE_VISION), using MsysAlloc(). This allocation opens general communication with (allows you to discover) all GigE Vision-compliant cameras (or other devices) found on your network(s) (through all Gigabit Ethernet, or faster, network adapters in your computer). Note that GigE Vision cameras connected to a Zebra GevIQ frame grabber are not accessible (discoverable) by the GigE Vision system and vice versa.

You must then allocate a digitizer for each camera that you want to use to grab images and/or access directly, using MdigAlloc(). In the case where you have only one GigE Vision-compliant interface camera, use MdigAlloc() with M_DEV0. If, however, you have multiple GigE Vision-compliant interface cameras, you should use the M_GC_CAMERA_ID() macro and specify the device's name; this will make for more portable code. You can use the Aurora Imaging Capture Works utility to view a list of available GigE Vision-compliant cameras and their user-defined name. For information on this utility, refer to the Zebra GigE Vision tools section later in this chapter. For information regarding working with multiple GigE Vision-compliant devices, refer to the Working with one or more GigE Vision-compliant cameras subsection of this section.

Each process (executable) can allocate only one Aurora Imaging Library GigE Vision system; an error will occur if you try to allocate more than one. Multiple processes can allocate an Aurora Imaging Library GigE Vision system.

Typically, multiple processes cannot allocate a digitizer for the same GigE vision-compliant camera simultaneously. However, this is not a restriction when using IP multicasting, as long as only one digitizer is allocated as a multicast controller digitizer (by using MdigAlloc() with M_GC_MULTICAST_CONTROLLER). For information regarding IP multicasting, refer to the Using IP multicast section later in this chapter.

All GigE Vision-compliant devices support configuration using the GenICam standard. To learn the list of GenICam-standard features available, refer to GenICam's standard feature naming convention (SFNC), found online at: emva.org/standards-technology/genicam/. Your camera might support additional GenICam features that do not appear in the GenICam SFNC; check your camera's manual and Aurora Imaging Capture Works' Feature Browser for details. Aurora Imaging Capture Works' Feature Browser accesses the GigE Vision-compliant camera's device description file (XML), providing an interface to view and change the camera's information.

In most cases you should control and inquire GigE Vision devices features using the conventional Aurora Imaging Library functions MdigInquire() and MdigControl(). However, some GigE Vision-compliant device have features that can only be accessed using MdigInquireFeature() and MdigControlFeature() functions. For more information about accessing the features of your GigE Vision-compliant device, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

GigE Vision information in the Aurora Imaging Library Reference can be found in the paragraphs and values marked as being supported by the Zebra GigE Vision driver.

Working with more than one Gigabit Ethernet network adapter

When your computer has more than one Gigabit Ethernet network adapter installed, you can use each network adapter to access cameras on different networks. Alternatively, multiple network adapters can simply be used to increase the number of cameras to which you have access. Unless otherwise specified, this documentation uses the model of one network adapter connected to a local network on which one or more GigE Vision-compliant cameras exist.

When dealing with more than one Gigabit Ethernet network adapter, the list of your GigE Vision-compliant devices in the Aurora Imaging Capture Works is subdivided by the network adapter to which they are visible. Both the network adapters and their associated GigE Vision-compliant devices are listed in the order of their MAC addresses.

For more detailed information regarding network configurations and GigE Vision-compliant camera setup, refer to the Aurora Imaging Capture Works white paper (located in your *Aurora Imaging Library\Tools* folder).

Working with one or more GigE Vision-compliant cameras

When there is more than one GigE Vision-compliant network camera available, the network discovery process (in the Zebra GigE Vision discovery service) ranks the GigE Vision-compliant cameras according to their MAC address. For example, the camera having the lowest MAC address is ranked as device zero; the rank increments by one for each camera having a subsequently greater MAC address. Aurora Imaging Library uses the camera's rank when you allocate a digitizer with a device number (using MdigAlloc() with M_DEVn); so the rank 0 camera becomes M_DEV0. When you replace a GigE Vision-compliant camera on your network, its rank might not be the same as the camera you are replacing. This means that the rank of all the GigE Vision-compliant cameras could shift to allow the new camera into the sequence the next time you stop and restart your application. When you try to allocate a digitizer for the new camera (to replace the old camera), if the rank is different, the allocation might fail or worse, it might succeed, but the images captured by the camera actually allocated will not be of what you expect.

To avoid this problem, assign the camera a user-defined name, using Aurora Imaging Capture Works (on the **Device information** tab), and then use this name when you allocate a digitizer for this camera.

To allocate a digitizer for a camera with a user-defined name, use MdigAlloc() with M_GC_DEVICE_NAME and specify your camera's name. Then, when a camera needs to be replaced, set the user-defined name of the new camera to match the name of the camera that it replaced. Allocating your GigE Vision-compliant camera using the camera's name allows your application to work with the new camera without modifying either your application or your application's configuration. Alternatively, if your camera uses a persistent (static) IP address, you can use that IP address to identify your camera (using M_GC_DEVICE_IP_ADDRESS).

A new GigE Vision-compliant camera (added to the network as a new camera or a replacement) is not allocated by default and, although it is present, it is not available to your application until you allocate it using MdigAlloc(). If an allocated camera is removed from your network, free it using MdigFree().

To watch for new cameras on your network, use MsysHookFunction() with M_CAMERA_PRESENT. Note that verifying the camera's state (through the hooked function) should be done before your application relies on information from the camera.

Use MsysInquire() with M_DISCOVER_DEVICE_COUNT to determine the number of cameras available. This number is updated when you allocate your system, refresh the list of discovered devices, using MsysControl() with M_DISCOVER_DEVICE, and again if called by a function hooked to an M_CAMERA_PRESENT event. Note that this number is not affected by the number of digitizers allocated for the cameras.

Working without a Zebra board as your Gigabit Ethernet network adapter

To successfully allocate an Aurora Imaging Library GigE Vision system without a Zebra board as your Gigabit Ethernet network adapter, you must already have purchased the Zebra GigE Vision driver license (regardless if you have a license for Aurora Imaging Library development, Aurora Imaging Library runtime, or Aurora Imaging Library Lite). To determine whether you have a Zebra GigE Vision driver license, use MappInquire() with M_LICENSE_MODULES; it should return M_LICENSE_INTERFACE.

Zebra GigE Vision tools

Besides general utilities (such as Aurora Imaging Intellicam and the Aurora Imaging Configurator), the Zebra GigE Vision driver comes with Aurora Imaging Capture Works (external application) and the Zebra GigE Vision discovery service.

Aurora Imaging Capture Works

Aurora Imaging Capture Works is an interactive utility to configure and test devices that make use of a GenICam-based interface standard, and supports the GigE Vision transport layer standard (among other standards). Aurora Imaging Capture Works is compatible with GenICam GenDC containers and multi-part payload types. The Aurora Imaging Library GigE Vision system decodes and stores data into container (M_CONTAINER) buffers. Aurora Imaging Capture Works also adds support for 3D displays. Aurora Imaging Capture Works will list all detected GigE Vision-compliant devices connected to the host (or PC). It can start or stop capturing images, display acquired images, save the last grabbed image, send a software trigger, as well as browse and control the selected device's features. You can view and change acquisition properties, and view acquisition statistics.

Aurora Imaging Capture Works key features:

- Camera-centric user interface with support for camera enumeration and presence.
- Feature browser to display and control camera, digitizer, and system settings.
- DCF creation to save camera and digitizer settings.
- Efficient display of grabbed images with many options and view modes.
- Display of line and arc profiles.
- Display of image histogram.
- Memory viewer to display pixel values.
- Ethernet adapter configuration tool to optimize network adapters and modify IP configuration settings.
- GigE Vision device IP configuration tool.

More information is available from the Aurora Imaging Capture Works manual available from the Zebra website. To access this documentation, you can also search online for "Aurora Imaging Capture Works manual".

Zebra GigE Vision discovery service

The Zebra GigE Vision discovery service allows your Aurora Imaging Library application and the Aurora Imaging Capture Works utility to automatically detect when GigE Vision-compliant cameras are added to or removed from your network. The service is enabled by default when you install the Zebra GigE Vision driver. To disable or re-enable the service, launch the Aurora Imaging Configurator utility. Then, select the **Boards GigE Vision** item from the tree structure in the presented interface, click on the **Use Camera Discovery Service** check box in the presented page, and click on the **Apply** button.

While the service is running, your Aurora Imaging Library application can execute a hook-handler function whenever a GigE Vision-compliant camera is added to or removed from your network (using MsysHookFunction() with M_CAMERA_PRESENT).

While the service is not running, your Aurora Imaging Library application will only detect which GigE Vision-compliant cameras are on your network at the moment you allocate an Aurora Imaging Library GigE Vision system.

You can manually refresh the list of discovered GigE Vision compliant cameras on your network using MsysControl()

with M_DISCOVER_DEVICE or by clicking the **Perform Device Discovery** toolbar button in Aurora Imaging Capture Works. You can perform this without running the service.

> Note that if you have disabled the discovery service, your firewall might block the discovery of GigE Vision devices.

Configuring your GigE Vision camera and Gigabit Ethernet network adapter

There are several features that can improve the performance of your Zebra GigE Vision driver and associated GigE Vision-compliant camera.

> Note that, some GigE Vision-compliant cameras include libraries that, when installed, might install network filter drivers or override your network adapter's native driver. While these third-party libraries have their own benefits, we cannot guarantee their compatibility with Aurora Imaging Library. For best results, the computer being used to communicate with your GigE Vision-compliant camera should always use Aurora Imaging Library for image capture, as well as your network adapter's native driver (for example, an Intel driver for an Intel network adapter).

Automatic format switching

By default, the Zebra GigE Vision driver will tell the camera to transmit images in the same format as the grab buffer; this is called automatic format switching. If the grab buffer is allocated in a format that is not compliant with the camera's current pixel format, the Zebra GigE Vision driver changes the camera's pixel format to a format more compliant with the grab buffer before performing the copy operation. Automatic format switching prevents color space conversion, which can cause additional processing and delays in delivering the image from the camera to your computer. Unfortunately, there are cases where automatic format switching cannot occur (such as when the operation requires grabbing into internal buffers, grabbing to a container (M_CONTAINER) or dynamic (**M_DYNAMIC**) buffer, and when performing Bayer conversion in the camera or when the camera is in chunk mode).

When automatic format switching cannot be performed, the image is copied (in the camera's current pixel format) into a compatible automatically-allocated internal buffer. The Host will then copy the internal buffer to the specified grab buffer to force the conversion to the format of the specified grab buffer.

Ideally, you should try to avoid automatic format switching, by inquiring the camera's current pixel format using MdigInquire() with M_SOURCE_DATA_FORMAT before allocating your grab buffers (that is, using MbufAlloc...() with M_IMAGE and the returned value from MdigInquire() with M_SOURCE_DATA_FORMAT. Note that, the camera's current format typically varies from camera to camera.

Gigabit Ethernet network adapter settings

If your Ethernet controller settings are not optimal, Aurora Imaging Capture Works flags these issues and proposes adjustments to enhance acquisition performance. The following settings can be modified.

- **Receive buffers**. Gigabit Ethernet network adapters use receive buffers, also known as receive descriptors, to store received packets. If the number of packets sent by your camera exceeds the number of available receive buffers, the additional packets will be dropped. The number of receive buffers assigned to your network adapter therefore impacts the reliability of your camera's image transmission. Typical bandwidth usage of GigE Vision cameras requires that a large number of receive buffers be assigned to your network adapter. It is recommended to adjust this setting to its maximum value (typically 2048 for Intel Gigabit Ethernet network adapters).

- **Packet size**. Network traffic is transmitted in packets, and the default maximum packet size for most network adapters is set to 1500 bytes. In situations where larger packets make up the majority of network traffic, such as with GigE Vision image streams, enabling jumbo packets can improve overall network performance by reducing CPU utilization. Jumbo packets typically support packet sizes up to 9014 bytes.

  > Note that, if you want to use jumbo packets and you use an Ethernet switch in your network, you should also enable jumbo packets on your switch, since jumbo packet support is typically disabled by default.

- **Ethernet flow control**. Ethernet flow control can pause and restart the flow of packets from your camera. This reduces the chance of on-board memory becoming too full to properly deal with the flow of packets, and reduces the chance of data loss (dropped packets). A number of network adapters support Ethernet flow control. Enabling your network adapter's Ethernet flow control settings is recommended.
- **Interrupt moderation**. Interrupt moderation controls how often interrupts from the network adapter are generated and handled by the system. Setting the interrupt moderation value impacts the processing of received Ethernet frames; lower values typically result in faster processing but can increase CPU load, while higher values might introduce latency but improve CPU utilization by reducing the rate of interruptions. It is important to customize these settings to suit your application. It is important to evaluate the tradeoff between these for your specific application requirements.

To manually configure your network adapter with this information under Microsoft Windows, refer to the Microsoft Windows documentation.

Optimizing further

If your Gigabit Ethernet network adapter requires further optimization (that is, its performance is still not adequate after modifying the network adapter settings), try changing the inter-packet delay.

Situations can occur that cause delays in the reception of packets by your network adapter. Inter-packet delays help by spreading packet transmission over time, giving the receiving network adapter enough time to deal with incoming packets and reduces the chance of an overworked receiving network adapter dropping packets.

For an example of how to calculate inter-packet delays, refer to the Zebra Aurora Imaging Library page in GitHub.

Configuring IP addresses

There are three ways to configure the IP address of your Gbit Ethernet network adapter(s):

- **Using DHCP** (corporate network). DHCP relies on a network server to automatically assign a unique IP address to each network device. If using DHCP, you do not need to configure your network devices.
- **Using a point-to-point connection**. A point-to-point connection (link-local addressing or LLA, or AutoIP) relies on your computer to generate a unique IP address for your computer's network adapter and connected GigE Vision cameras on the 169.254 subnet (that is, an IP address starting with 169.254.X.X).
- **Using a static (or persistent) IP address**. A static IP address is used when there is no DHCP available, and you want control over the IP address used. You can also use a static IP address to reduce the time it takes to assign all the associated devices' IP addresses. If your computer uses a static IP address, other connected devices on your network should also use a static IP address.

A device might become inaccessible if the IP address used is incompatible with its network segment. In most cases, Aurora Imaging Library identifies this issue and assigns a valid IP address. However, there are situations where the library cannot resolve configuration issues, particularly when the network segment in question has a DHCP service. The allocated IP address can lead to problems if the DHCP server assigns the same IP address to a different device, which causes an IP address conflict and the device becomes inaccessible. The GigE Vision driver will identify these devices as "unreachable", and in Aurora Imaging Capture Works, these devices will be denoted with a red icon.

You can resolve this issue using the GigE Vision device IP configuration tool in Aurora Imaging Capture Works. To do so, right-click on the device and select **IP resolve**. This will temporary assign a valid IP address to the device, making it addressable, and the device's IP configuration will be set to a mode that is compatible with its network segment.

Then, it will send a command requesting the device to initiate its IP configuration cycle using the new configuration mode. If a DHCP server is available, the device will request an IP address using the DHCP method, which will properly configure the device. Alternatively, you can access the **Tools GigE Vision Device IP Configuration** menu item for more control over how to resolve the conflict.

Link aggregation

Some GigE Vision cameras have more than one network interface because the total bandwidth required exceeds 1 Gbit/sec. In this case, they transmit their images over both interfaces and expect the receiving computer to have two network adapters that are configured for link aggregation to receive the video stream. Link aggregation is a method of combining multiple network connections in parallel, to increase the throughput beyond what a single connection can sustain. Note that, while some cameras automatically support link aggregation (determined by the number of connections made from the camera to the network), the settings of your Gigabit Ethernet network adapter must be updated to allow link aggregation.

The Zebra GigE Vision driver deduces that a GigE Vision camera is transmitting across multiple network interfaces, and that the network adapter is using link-aggregation, when the reported link speed is greater than 1 Gbit/sec but less than 10 Gbit/sec, excluding link speeds of 2.5 and 5 Gbit/sec. Detecting this type of connection between the GigE Vision camera and network adapter allows the Zebra GigE Vision driver to automatically adjust its packet resend engine to fit the requirements of the connection between your camera and network adapter, since they tend to stream packets out of order.

Link aggregation has been incorporated into version 2.0 of the GigE Vision specification; this has formalized its use with GigE Vision cameras and Gbit Ethernet network adapters.

Using IP multicast

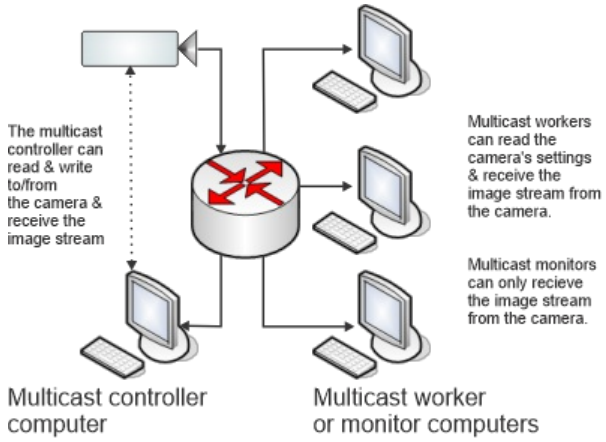
When sending images from your GigE Vision camera to multiple computers, the overall bandwidth requirements can be significant; this can lead to significantly reduced network performance for other computers on your network. One way to reduce the bandwidth requirements, without reducing the overall frame rate or image size, is to use IP multicast between your GigE Vision camera and your computers' Gigabit Ethernet network adapter.

IP multicast is a method of sending data from one computer to a group of computers (on the Internet or on local networks) in a single transmission. Streaming media applications often use multicasting on the Internet or on private networks. Using IP multicast, a GigE Vision-compliant camera can stream data to multiple destinations without having to send multiple copies of the data; in this case, the network routers and Ethernet switches perform the data replication instead. On a local network, the Internet Group Management Protocol (IGMP) typically controls multicast delivery. In a typical network, IGMP resides on a router.

With IP multicasting, beside the camera having its own IP address, the camera is also assigned a multicast IP address. The camera sends its multicast stream to the multicast IP address. Each computer that waits to receive this multicast stream, joins the group associated with the multicast IP address (called a multicast group). Each computer must either know the multicast IP address used by the camera or have the camera's IP address and access to read the camera's settings to establish the multicast address. Each computer communicates with its networks' multicast service (typically implemented in a router) to join the list of destinations for the particular multicast group through the Zebra GigE Vision driver. This list of destinations that a multicast stream is sent to is known as a distribution tree. When the camera sends its multicast stream to the multicast address, the stream is forwarded through your network, according to the distribution tree stored on each router or switch, until it reaches the Ethernet switch closest to one or more destination computers. Once there, the image stream is sent directly to the destination computer and not to all computers attached to the Ethernet switch (as would be the case with an IP broadcast). The Ethernet routers on your network manage additions and deletions to the multicast group and its associated distribution tree; and translate the multicast IP address into multiple IP addresses using IGMP.

A multicast address is an IP address within a predetermined range from 224.0.0.0 to 239.255.255.255. This predetermined range and its subnets have been established by the Internet Engineering Task Force (IETF) best current practice document and are defined in RFC 5771 and RFC 2365. If you allocate a digitizer for a GigE Vision camera and specify the camera should use IP multicast (that is, be in multicast mode), the Zebra GigE Vision driver selects a multicast address in the 239.0.0.0 to 239.255.255.255 range; this is a range of IP addresses that is not used on the Internet. If you use an IP address outside this range (but inside the predetermined multicast IP range), conflicts are more likely to occur. If your IGMP-aware device (that is, a router, switch, or server) uses a different range of IP addresses, you can reset Aurora Imaging Library's multicast IP addresses using MdigControl() with M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING (for the image stream) and M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING (for the message stream).

Note, when using IP multicast to stream images from the camera to several computers, you must allocate a multicast controller digitizer on one computer for the GigE Vision-compliant camera, and allocate a multicast worker digitizer or a monitor digitizer on one or more different computers for the same camera. The Aurora Imaging Library application that allocates the multicast controller digitizer is the only one that can modify the camera's settings. The Aurora Imaging Library application that allocates the multicast worker digitizer can only inquire the camera's settings, receive the image stream and GigE Vision messages (events), and request stream packets be resent. The Aurora Imaging Library application that allocates a multicast monitor digitizer can do everything that the multicast worker digitizer can do, except inquire the camera's settings.



To use IP multicast with your cameras, the following considerations should be taken:

- You need a network consisting of a GigE Vision-compliant camera, an Ethernet switch (with IGMP enabled), and at least two computers (each with their own Gigabit Ethernet network adapter).
- Only one computer can have an Aurora Imaging Library application that allocates a multicast controller digitizer. This application is the only one that can control, read from, write to, and receive the image stream from the camera.
- One or more computers can have an Aurora Imaging Library application that allocates a multicast worker or monitor digitizer. These applications can read settings and information from (multicast worker digitizer only) and receive the image stream from the camera.

Two channels are considered when using IP multicast on your camera: a stream channel and a message channel. This section predominantly focuses on the stream channel. The message channel is similarly configured as the stream channel (in that it also has a multicast IP address and port). The message channel carries GigE Vision messages from the camera and is available only if the camera manufacturer has enabled the feature. GigE Vision messages are camera-specific. Refer to your camera's documentation for more information.

Configuring a multicast controller

The multicast controller digitizer behaves like a standard Aurora Imaging Library digitizer. A multicast controller digitizer assigns the multicast address of the cameras's stream channel and the multicast address of the message channel (if supported). Once configured, the multicast controller digitizer receives images and GigE Vision messages from the camera through its multicast addresses.

By default, if the digitizer was allocated using M_DEVn, the multicast controller digitizer sets the IP multicast address to be 239.255.0.(*n* +1). For example, if you allocate a multicast controller digitizer using M_DEV0, the multicast addresses used will be: 239.255.0.1 for the stream channel, and 239.255.1.1 for the message channel. For a multicast controller digitizer allocated using M_DEV15 (provided you have 16 or more cameras connected to your computer), the addresses used will be 239.255.0.16 for the stream channel, and 239.255.1.16 for the message channel.

To allocate and use a multicast controller digitizer, perform the following:

1. Allocate the multicast controller digitizer using MdigAlloc() with M_GC_MULTICAST_CONTROLLER.
2. In the case of an IP conflict between the Aurora Imaging Library-determined multicast IP address and a multicast IP address already in use on your network, you can change the multicast IP address and its associated port number using MdigControl() with M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING and M_GC_LOCAL_STREAM_PORT (for the image stream) or M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING and M_GC_LOCAL_MESSAGE_PORT (for the message stream).
3. Configure your grab and any required processing, according to your specifications.

4. Use either MdigGrab(), MdigGrabContinuous(), or MdigProcess() to grab the image with the multicast controller digitizer; the network switches and routers will direct the image stream from the GigE Vision camera to all the members of the multicast group (that is, the multicast controller, and any multicast workers and monitor digitizers in your network).

Configuring one or more multicast workers

The multicast worker is restricted in what it can do. A worker digitizer can get information (using MdigInquireFeature() or MdigInquire()) and GigE Vision messages (using hooked-functions and MdigGetHookInfo()), from a camera, but it cannot set values on the camera (using MdigControlFeature() or MdigControl()). In addition, it can receive the images that the camera is transmitting to the multicast controller computer; to do so, the multicast worker must call MdigGrab(), MdigGrabContinuous(), or MdigProcess().

A multicast worker digitizer is typically used to view images on a secondary station and as such, data integrity is not typically critical. A worker digitizer can issue packet resend requests (if supported by your camera). Packet resend requests are issued when missing packets are detected. Due to the nature of multicast, resent packets will be forwarded to all members of the multicast group (the controller, worker, and monitor digitizers). Disable packet resend on the worker digitizer; this eliminates the possibility that packet resend requests, issued by the worker, affect acquisition reliability on the controller. To prevent a worker digitizer from issuing packet resend requests, you can use MdigControl() on the worker digitizer with M_GC_PACKET_RESEND set to M_DISABLE. You must set this control type before calling MdigGrab(), MdigGrabContinuous(), or MdigProcess(). Disabling packet resends on the worker digitizer does not prevent packet resends on the controller digitizer.

The multicast worker digitizer's DCF (passed to MdigAlloc()) is not used. A multicast worker digitizer can receive events, and acquire images from the camera. However, the worker digitizer can perform image acquisition only after another computer (with the controller digitizer) initiates a grab and the camera is streaming data. If the controller digitizer has not initiated image acquisition, the worker digitizer will not receive an image when issuing a grab command (using MdigGrab(), MdigGrabContinuous() or MdigProcess()); instead it will wait. To avoid waiting indefinitely, set a worker digitizer's M_GRAB_TIMEOUT to an appropriate amount of time.

To allocate and use a multicast worker digitizer, perform the following:

1. Allocate the multicast worker digitizer using MdigAlloc() with M_GC_MULTICAST_WORKER.

2. Verify that the multicast controller is still connected to the camera, using MdigInquire() with M_GC_MULTICAST_CONTROLLER_CONNECTED.

   If the multicast controller digitizer disconnects, use M_GC_MULTICAST_CONTROLLER_CONNECTED periodically to establish when it reconnects; then, update the multicast worker digitizer's information using MdigControl() with M_GC_UPDATE_MULTICAST_INFO.

3. While the multicast controller is grabbing, use either MdigGrab(), MdigGrabContinuous(), or MdigProcess() with the multicast worker digitizer to grab the image from the image stream.

Configuring one or more multicast monitors

A multicast monitor is more restricted than a multicast worker, since it is allocated without the right to read or modify the camera's settings. Multicast monitor digitizers can still send packet resend requests (if supported by your GigE Vision camera), receive image streams and GigE Vision messages, just like a multicast worker, but it cannot query the camera to get its multicast IP address, port address, or query the image size or pixel format before the first frame is grabbed. Due to this, you must set some information manually for the multicast monitor digitizer. Use a multicast monitor in situations where your Aurora Imaging Library system cannot query the multicast IP address from the GigE Vision camera due to security restrictions on your network.

Note that, if your GigE Vision camera is in chunk mode, it cannot be used with a multicast monitor.

To allocate and use a multicast monitor digitizer, perform the following:

1. Allocate the multicast monitor digitizer using MdigAlloc() with M_GC_MULTICAST_MONITOR. Note that you cannot allocate a monitor digitizer if your camera is in **chunk-mode**.
2. Specify the multicast IP address that the multicast controller digitizer has assigned to the camera to carry messages (events). To do so, use MdigControl() with M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING.
3. Specify the multicast UDP port that the multicast controller digitizer has assigned to the camera to carry messages. To do so, use MdigControl() with M_GC_LOCAL_MESSAGE_PORT.
4. Specify the multicast IP address that the multicast controller digitizer has assigned to the camera to carry images. To do so, use MdigControl() with M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING.
5. Specify the multicast UDP port that the multicast controller digitizer has assigned to the camera to carry images. To do so, use MdigControl() with M_GC_LOCAL_STREAM_PORT.
6. Update the multicast information for the multicast monitor digitizer, using MdigControl() with M_GC_UPDATE_MULTICAST_INFO. Note that with a multicast monitor digitizer, you cannot detect when the controller digitizer disconnects.
7. Set the X-size, Y-size, and pixel format of the grab image for the digitizer, using MdigControl() with M_SOURCE_SIZE_X, M_SOURCE_SIZE_Y, and M_GC_PIXEL_FORMAT, respectively. Alternatively, you can set these values in the DCF (for an example, see the *gigevision_multicast_monitor.dcf*).
8. While the multicast controller is grabbing, use either MdigGrab(), MdigGrabContinuous() or MdigProcess() with the multicast monitor digitizer to grab the image from the image stream.

Ethernet switches, routers, IGMP, and IP multicast

When configuring your GigE Vision camera and Gbit Ethernet network adapter for multicasting, there are a few networking concepts with which you should be familiar. This section provides some background information for the requirements of a network designed to deliver a multicast service.

A network designed to deliver a multicast service can include the following hardware components:

- A multicast transmitter (for example, your GigE Vision camera).
- Ethernet routers.
- Ethernet switches.
- Host multicast receivers (for example, your Aurora Imaging Library GigE Vision applications with a controller, worker, or monitor multicast digitizer).

Each component fills a role. To deliver a multicast service, the Internet Group Management Protocol (IGMP) is used your local network.

Internet Group Management Protocol (IGMP)

IGMP is a standard that specifies how a Host can register with a router to receive specific multicast traffic. IGMP operates between the client computer (such as, the computers running your Aurora Imaging Library GigE Vision applications using a controller, worker, or monitor multicast digitizer) and a local multicast router.

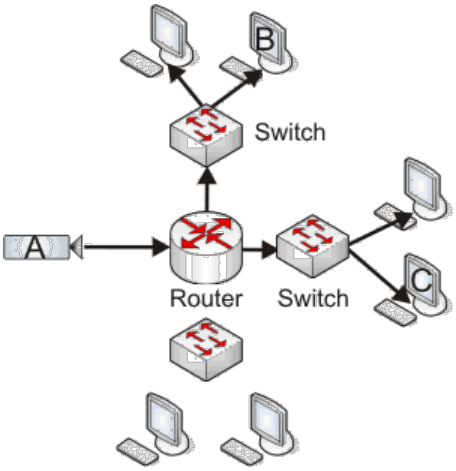
An Aurora Imaging Library digitizer allocated with M_GC_MULTICAST_CONTROLLER, M_GC_MULTICAST_WORKER, or M_GC_MULTICAST_MONITOR will automatically send IGMP requests to a network router asking for membership to a multicast group. The network router will then issue periodic membership queries to the registered members of a multicast group. Upon reception of these queries, Aurora Imaging Library will respond with IGMP report messages. IGMP allows a router to correctly forward multicast traffic.

Ethernet switches and multicast

To correctly establish packet forwarding, an Ethernet switch uses and maintains an address table. Each packet that enters an Ethernet switch is added to this table; the incoming packet's source MAC address is put in the address table along with the port from which the packet came. To know on which port to output the packet, the switch will lookup the packet's destination MAC address in the address table. If the address is found, the packet will be output on the appropriate port. If the destination MAC address is not found, the packet is flooded on all ports. Eventually the address table will allow the switch to avoid flooding ports because almost every packet received can be looked up in the table.

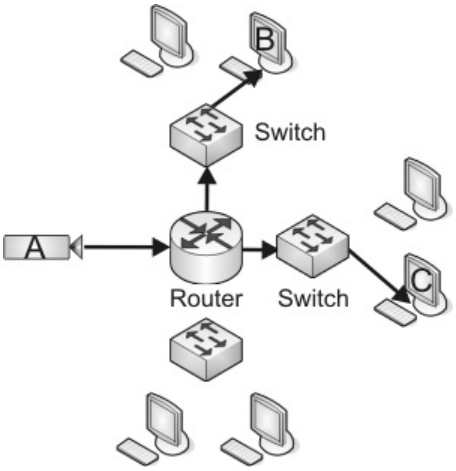
Unfortunately this mechanism causes Ethernet switches to flood the broadcast domain with multicast traffic. This is because a switch usually learns MAC addresses by looking into the source address field of all the Ethernet packets it receives. A multicast MAC address is never used as source address for a packet. Such addresses do not appear in the MAC address table, and the switch has no method for learning them. This consumes a lot of bandwidth since all multicast packets received on one port of the switch are output to all other ports of the switch.

The diagram below illustrates multicast datagrams transmitted over a network. The GigE Vision camera (A) is sending multicast IP data to clients B and C. The router correctly forwards the data to the networks of B and C because of the IGMP messages gathered. While the router correctly forwards the data to the client's networks, because of the IGMP messages gathered, the LAN switches flood the data to all PCs connected to them.



IGMP snooping and Ethernet switches

To eliminate the flooding problem discussed previously, some Ethernet switches support IGMP snooping. With IGMP snooping, the switch intercepts IGMP packets and uses the information it gathers to establish appropriate forwarding rules for the multicast traffic.



IGMP snooping is disabled by default in LAN switches. To avoid your Ethernet switch causing issues with IP multicast flooding, you must enable IGMP snooping in your Ethernet switch using the managed (or other) interface of your switch.

Using IP multicast without a router

IGMP operates between a computer and a local router. If your network does not have a multicast router, IGMP will fail since there is no source for IGMP membership queries. Some Ethernet switches support being the source for IGMP membership queries (IGMP querying). If you do not have a multicast router in your network, you must use an Ethernet switch that supports IGMP querying and IGMP snooping (typically, if one is supported, then they both are).

If both IGMP snooping and IGMP querying are supported by your Ethernet switch, IP multicast becomes possible without the need of a router. Make sure both IGMP snooping and IGMP querying are enabled in your switch since these features are typically disabled by default; enable them using the managed (or other) interface of your Ethernet switch. When IGMP querying is enabled, it sends out periodic IGMP queries that trigger IGMP report messages from computers that want to receive multicast traffic. IGMP querying works with IGMP snooping since your Ethernet switch listens to these IGMP queries to establish appropriate forwarding.

Using a Gbit Ethernet switch with GigE Vision-compliant cameras

If you need to connect to more cameras than you have network adapters (or network ports if using a multi-port network adapter) in your computer, a switch can provide the required expansion capabilities without increasing the complexity of your network (as would be the case with adding a router) while optimizing network throughput. When using a Gbit Ethernet switch, you must ensure that your Gbit Ethernet switch's performance metrics are adequate for your application's needs, particularly with regard to the bandwidth requirements of your applications. For more information, refer to your Gbit Ethernet switch's documentation for the switching capacity and packets per second capacity of your switch.

In addition, your Gbit Ethernet switch should support the following: jumbo packets, Ethernet (IEEE 802.3x) flow control, and port grouping (if using link-aggregated GigE Vision-compliant cameras through a switch).

Jumbo packets and Ethernet flow control are typically disabled by default and must be enabled if needed.

If port grouping (link aggregation) is required, it must be manually configured using the switch's configuration utility.

Finally, if you need to use IP multicast, make sure your switch supports the following: IGMP snooping and IGMP querying (unless your network has a multicast router to handle IGMP). IGMP snooping and IGMP querying are typically disabled by default and must be enabled if needed.

Troubleshooting acquisition reliability issues

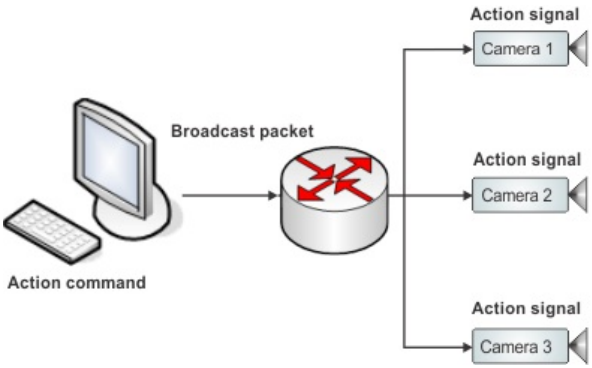
A number of factors can cause acquisition reliability issues. You can use the Aurora Imaging Capture Works utility to determine if you are having reliability issues. In this utility, select a camera and start the acquisition. Then, select the **Acquisition Statistics** tab. This tab will show whether any acquired frames (images) are corrupt as a result of missing packets. The bottom of the **Acquisition Statistics** page contains the last frame that reported an error, along with error codes that can help in diagnosing the issue.

The following is a list of the common causes of corrupted frames:

- Your Gigabit Ethernet network adapter is not well-suited to your GigE Vision camera's bandwidth requirements. Use a high-bandwidth Network Interface Card (or a Zebra GevIQ frame grabber).
- Your network adapter settings have not been adjusted for GigE Vision acquisition. You need to adjust your adapter with recommended settings.
- A network device between your GigE Vision camera and your network adapter, such as an Ethernet switch, is dropping packets. See the Using an Ethernet switch with GigE Vision cameras section.
- The total bandwidth of all GigE Vision cameras streaming image data to the same network adapter exceeds its link speed.
- An insufficient inter-packet delay.
- Your camera does not have the packet resend capability.
- Your Ethernet cables are not rated Category 5e (Cat 5e) or better.

Triggering simultaneous actions in multiple GigE Vision cameras

Triggering simultaneous actions across multiple GigE Vision devices (such as, cameras) on your network is a difficult and time-consuming process that, when dealing with unicast packets across a network, involves unavoidable delays and difficulties. The GigE Vision specification allows for synchronizing actions using broadcast packets that can trigger actions (such as, acquisitions) on all the cameras programmed to receive these broadcast packets; to trigger actions, these packets contain an action command.



Introducing action commands

A broadcast packet that contains an action command contains the information to identify the cameras, on your network, that should receive and accept the broadcast packet and the information to trigger an associated action on the cameras. To configure and send out the broadcast packet from your Aurora Imaging Library application, use MsysControl() with M_GC_ACTION... + M_GC_ACTIONn where *n* is the number of the action command in Aurora Imaging Library.

Your GigE Vision cameras must be configured to receive and act upon an action command sent by your Aurora Imaging Library application. To receive the action command, the action device key of the camera (`ActionDeviceKey` feature) must match that of the action command. To act upon the action command, you must select the internal action signal that should be generated upon reception of the command. The internal action signal will then trigger any associated action (such as, triggering acquisition). To associate an action (feature) with the reception of an action command, set the internal action signal as the trigger source of the action using either MdigControlFeature() or Aurora Imaging Capture Works' Feature Browser, with the associated SNFC trigger source feature name (for example, setting the `TriggerSource` feature of the Acquisition control category to `Action`*n* , where *n* is the action number).

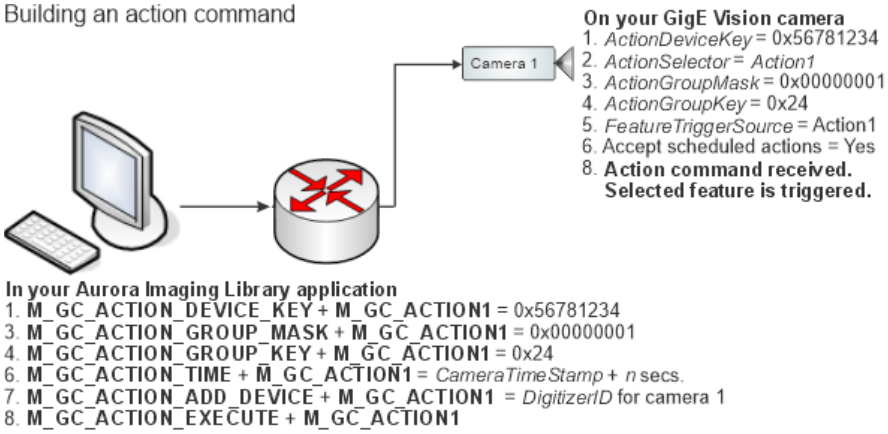
> Note that all references in this section to GenICam SFNC feature names assume your camera is compliant with GenICam SFNC 2.0; check your camera's documentation or a listing of your camera's device description file to establish whether your camera uses a feature with a different name (or range of values) than those mentioned in this section.

The following table provides a quick overview of the steps required to use Aurora Imaging Library to send action commands to your GigE Vision cameras.

What to do	In your Aurora Imaging Library application (using MsysControl() and adding M_GC_ACTIONn to the control type)	On each of your GigE Vision cameras	Definition
1. Specify the group of cameras on which to perform the action.	Set your Aurora Imaging Library device key (using M_GC_ACTION_DEVICE_KEY).	Set each camera's action device key (using the `ActionDeviceKey` feature).	The camera's action device key is a 32-bit integer value that identifies the group of cameras on which the synchronized actions should occur. The Aurora Imaging Library device key and the camera's action device key must match; otherwise, the camera ignores the action command.
2. Select the internal action signal to generate upon reception of a specific action command.		Select the internal action signal that you want to associate with an action group key (using the `ActionSelector` feature).	The action selector establishes which action signal to generate.
3. Specify whether to include or exclude cameras already in the group.	Specify your action group mask (using M_GC_ACTION_GROUP_MASK).	For the selected action signal, set the action group mask (using the `ActionGroupMask` feature).	The action group mask is a bit-encoded 32-bit integer value that allows you to exclude (mask out) cameras so they will not generate the selected action signal. See the Masking out one or more cameras subsection of this section.
4. Specify the key corresponding to the action signal on the camera.	Set your Aurora Imaging Library action group key (using M_GC_ACTION_GROUP_KEY).	For the selected action signal, set the camera's action group key (using the `ActionGroupKey` feature).	The action group key is a 32-bit integer value that identifies the Aurora Imaging Library action command to your GigE Vision camera. It associates the action command with the selected action signal.

5. Set the internal action signal as the trigger source of an action(s) on your camera.		Set the trigger source of an action (feature) to the internal action signal (for example, set the `TriggerSource` feature of the acquisition control category to `Action0`).	The internal action signal is generated when your camera receives the associated Aurora Imaging Library action command. For something to happen (such as image acquisition), set the trigger source of an action (feature) on your camera to the internal action signal.
6. Specify to perform the action at a predetermined time (scheduling) using a time offset.	Set your Aurora Imaging Library action time (using M_GC_ACTION_TIME).	Set the camera to accept scheduled action commands (using the `GevIEEE1588` feature).	The scheduled action command uses a time offset that identifies when the action should occur on the camera. This optional value specifies when the action command should execute on your camera. See the Scheduling time subsection of this section.
7. Add your GigE Vision camera(s) to your action command, in Aurora Imaging Library.	Identify the cameras to associate with the action command by specifying their Aurora Imaging Library digitizer identifier (using M_GC_ACTION_ADD_DEVICE).		For Aurora Imaging Library to track and make use of the camera's action results, you must also add each camera explicitly to the action command using M_GC_ACTION_ADD_DEVICE + M_GC_ACTIONn. This is described later in Acknowledgment of reception of an action command.
8. Send the action command from your Aurora Imaging Library application to your GigE Vision camera.	Use M_GC_ACTION_EXECUTE.		Aurora Imaging Library sends the broadcast packet containing the action command. Typically, this control type is specified during the time-critical loop of your application.

The following image illustrates the steps (above). The numbers refer to the steps on which they are set.



Establish if your camera supports action commands

To send action commands to your GigE Vision cameras, each camera must support the following:

- **Action commands.** To verify that your camera supports action commands, use MdigInquire() with M_GC_CONTROL_PROTOCOL_CAPABILITY. It should return M_GC_ACTION_SUPPORT, M_GC_UNCONDITIONAL_ACTION_SUPPORT, or M_GC_SCHEDULED_ACTION_SUPPORT.

- **The feature on your camera supports action signals as a trigger source.** The feature that you need to trigger (such as, acquisition) must support action signals as a trigger source. To verify this, refer to your camera's device description file. For example, the `TriggerSource` feature of the Acquisition control category should support the internal signal named `Action` *n*, where *n* is the internal action signal number. Note that some cameras might only support a single action signal, while others might support several. The action command number (M_GC_ACTIONn) is an index specific to Aurora Imaging Library and does not necessarily match the number of the internal action signal set as the trigger source for the action on your camera. The association between Aurora Imaging Library and your camera's specific action signal is made using the action group key.

Acknowledgment of reception of an action command

When an action command is sent to a camera, the camera will respond with an acknowledgment of reception (a special type of packet). This packet signifies that your camera successfully received Aurora Imaging Library's action command. When an action command is sent to several cameras, Aurora Imaging Library expects the number of acknowledgments to equal the number of cameras added to the action command using M_GC_ACTION_ADD_DEVICE (minus those removed using M_GC_ACTION_REMOVE_DEVICE).

For example, on a network of four cameras all added using M_GC_ACTION_ADD_DEVICE and with the same action device key, if one action command is sent but is only received by two cameras, Aurora Imaging Library expects the number of acknowledgments to be 4, but only 2 would be returned. This generates an Aurora Imaging Library error.

Masking out one or more cameras

In the case where you need one (or more) camera(s) to temporarily ignore an action command, you can mask out the action command by changing the action group mask of the command either in Aurora Imaging Library or on the camera. For the camera to accept and generate an internal action signal, the action group mask of the action command in Aurora Imaging Library and that on the camera, when combined in a bitwise AND operation, must result in a non-zero value. If the result is a zero, the camera ignores the action command.

Action command 1	The action group mask's 32-bit value

Action group mask in Aurora Imaging Library	00000000 00000000 00000000 00000001
Action group mask on camera 1	00000000 00000000 00000000 00000001
Result: Camera 1 triggers the feature	00000000 00000000 00000000 00000001

The best way to use the action group mask is to treat each bit as an identifier of the camera on which the action command should be accepted. Set the action group mask of all action commands on each camera to a value where all bits are set to 0 except the identifier bit. Then, in Aurora Imaging Library, set the action group mask such that only the bits identifying the cameras to receive the command are set to 1. For example:

Action command 1	The action group mask's 32-bit value
Action group mask in Aurora Imaging Library	00000000 00000000 00000000 00001110
Action group mask on camera 1	00000000 00000000 00000000 00000001
Action group mask on camera 2	00000000 00000000 00000000 00000010
Action group mask on camera 3	00000000 00000000 00000000 00000100
Action group mask on camera 4	00000000 00000000 00000000 00001000
Action group mask on camera 5 Only camera 2, 3, and 4 trigger the action.	00000000 00000000 00000000 00010000

For a further example, refer to the Example of using group masks to ignore an action command on one camera subsection of this section.

When masking out one or more action commands on one or more cameras, you must also modify the number of acknowledgments that Aurora Imaging Library expects to receive, using MsysControl() with M_GC_ACTION_ACKNOWLEDGE_NUMBER + M_GC_ACTIONn, where *n* is the number of the action command in Aurora Imaging Library.

Scheduling time

To schedule an action command so that it will execute at the same time across multiple cameras, the following features must be available on your camera:

- **Scheduled actions**. To verify that scheduled actions are available on your camera, use MdigInquire() with M_GC_CONTROL_PROTOCOL_CAPABILITY. It should return M_GC_SCHEDULED_ACTION_SUPPORT.

- **IEEE 1588**. IEEE 1588 is a standard that defines the use of PTP (the precision time protocol). PTP synchronizes clocks across multiple network devices (such as, multiple GigE Vision cameras). To verify that IEEE 1588 is available on your camera, use MdigInquire() with M_GC_CONTROL_PROTOCOL_CAPABILITY. It should return M_GC_IEEE_1588_SUPPORT.

If the above are available, you must then enable IEEE 1588 on each of your cameras, for example, using `GevIEEE1588` with Aurora Imaging Intellicam (or using MdigControlFeature()).

Once enabled, IEEE 1588 will synchronize the clocks of all of your cameras. Note that this can take some time. When the synchronization is complete, querying the `Gev1588status` feature for each camera should return one camera as timeTransmitter, and all the rest as timeReceivers. You can then set up the action command in your Aurora Imaging Library application to generate the internal action signal at a specific time and know that the action will occur at the exact moment on all cameras receiving the command at the required time. With all your cameras clocks synchronized, you only have to account for the longest network delay possible between sending a packet to your camera, and the camera receiving it, when selecting a time.

> Note that, when you enable IEEE1588, the timestamp frequency of the camera will change. Therefore, your application should read the timestamp after you enable IEEE1588. In addition, all the 1588 devices must be connected to the same switch on your network. The switch must have a multicast service and it must be enabled to use 1588.

To select a time and provide that time to the action command, use MsysControl() with M_GC_ACTION_TIME. To determine the required time, perform the following:

1. After 1588 is enabled, read the timestamp from any target camera using MdigInquire() with M_GC_CAMERA_TIME_STAMP.

   If you want your action to occur relative to the timing of another action (for example, an exposure start), especially when the action should reoccur periodically, read the timestamp from within the callback function hooked to the other action. In this case, use MdigGetHookInfo() with M_GC_CAMERA_TIME_STAMP to read the time at which the event occurred on the camera.

2. Determine the longest possible transmission and processing latency of sending a packet from your computer to your camera. Add this amount of time (in seconds) to the timestamp retrieved previously.

   Experimentation is required to accurately determine how long it takes for your action command to reach the camera and the action to occur. If the action command reaches the camera after the scheduled time, an error is returned, but the action will still occur.

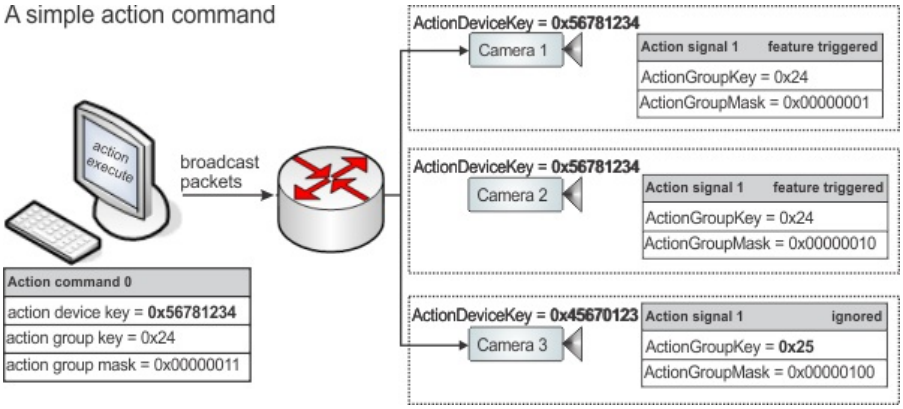
Examples of action commands

This subsection details a set of examples showing how action commands trigger actions (features) on the camera, based on the configuration of the action command in your Aurora Imaging Library application.

Example of sending a single action command

In the following example, three GigE Vision cameras on the network are configured to receive action commands from an Aurora Imaging Library application. When you send an action command (using M_GC_ACTION_EXECUTE + M_GC_ACTION0), Aurora Imaging Library sends a broadcast packet containing the action command.

In this case, action command 0 generates internal action signal 1 so it triggers the feature whose trigger source is set to action 1 on Cameras 1 and 2. Camera 3 ignores action command 0 because Camera 3 has a different action device key than the other cameras. This configuration is designed so that Camera 3 will always ignore action command 0.

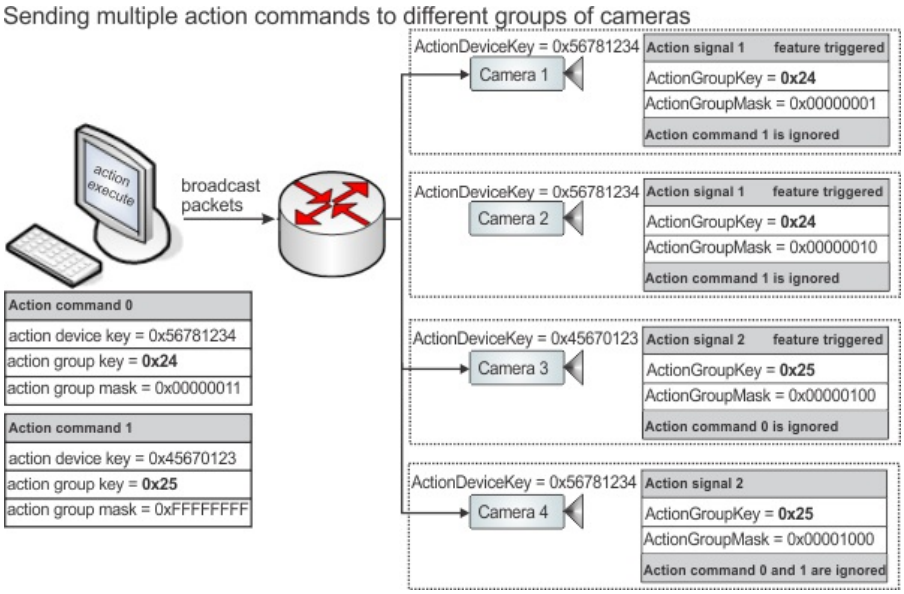


Example of sending multiple action commands

In the following example, three GigE Vision cameras on the network are set up with the same action device keys (Cameras 1, 2, and 4). When you send the two action commands to your cameras (using M_GC_ACTION_EXECUTE + M_GC_ACTIONn, where *n* is the number of the action command in Aurora Imaging Library), Aurora Imaging Library sends two broadcast packets (one per action command). The network copies the packets to all four GigE Vision cameras. On the cameras that accept the broadcast packets, the camera generates an internal action signal (either action signal 1 or action signal 2) that will trigger one or more features (depending on their trigger source).

In this case, action command 0 triggers the feature whose trigger source is set to action signal 1 on Cameras 1 and 2. Camera 4 ignores action command 0 because no action signal is associated with the action group of action command 0. To allow Aurora Imaging Library's action command 0 to trigger a feature on Camera 4, the action group key (0x24) would have to be associated with an action signal on Camera 4.

Action command 1 triggers the feature whose trigger source is set to action signal 2 only on Camera 3, since it is the only camera with both an action device key and action group key associated with action command 1. To allow Aurora Imaging Library's action command 1 to trigger a feature on Camera 4, the action device key of Camera 4 would have to be set to 0x45670123.

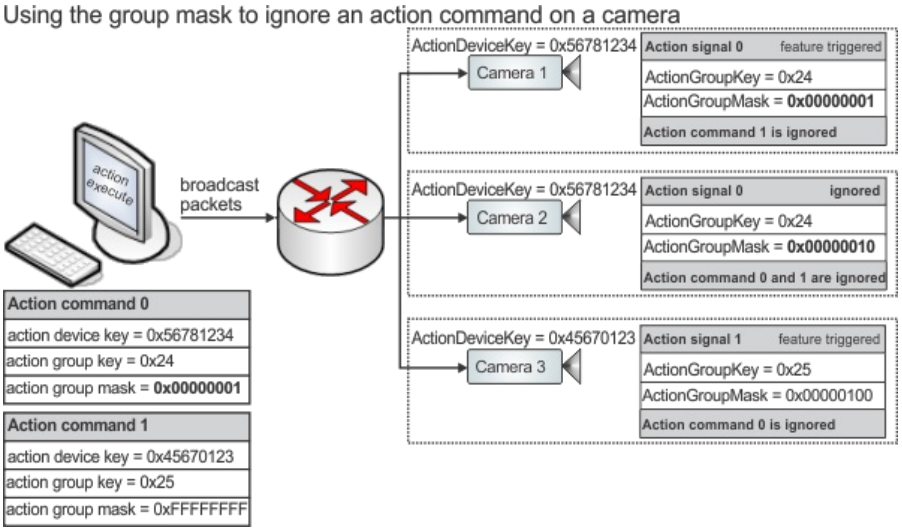


Example of using group masks to ignore an action command on one camera

In the following example, an action group mask is used to have Camera 2 temporarily ignore action command 0. It sets the group mask in Aurora Imaging Library's action command 0 so that the bit representing Camera 2 is 0. When ANDed in a bitwise fashion with the camera's group mask for action signal 0, the result is a 0 instead of a 1.

Action command 0	The action group mask's 32-bit value
Action group mask in Aurora Imaging Library	00000000 00000000 00000000 00000001
Action group mask on Camera 2	00000000 00000000 00000000 00000010
Result: Camera 2 ignores the action.	00000000 00000000 00000000 00000000

This forces Camera 2 to ignore action command 0, even though the device key and group key match between action command 0 in your Aurora Imaging Library application and the associated action signal on your camera.



Example distributed with Aurora Imaging Library

The Aurora Imaging Library hardware system specific example *ActionTrigger.CPP* illustrates how to send an action command to multiple cameras, configuring both the action command in Aurora Imaging Library and on each of the GigE Vision cameras on your network. In addition, the example tries to use scheduled actions, testing first that IEEE 1588 (Precision Time Protocol) is available and then enabling it on the cameras. If IEEE 1588 is not available, the example reverts to sending the action command without scheduling. To run this and other examples, use Aurora Imaging Example Launcher.

Miscellaneous user guide information for Zebra GigE Vision driver

This section includes additional user guide information for Zebra GigE Vision driver. The information found in this section might be a reiteration of content previously documented.

Working with a 3D device

The following includes useful information when working with a 3D device.

Grabbing from a 3D device

Data from a 3D device is grabbed using functions from the Digitizer module such as MdigGrabContinuous(), MdigGrab(), and MdigProcess(). In most respects, grabbing data from a 3D device is similar to grabbing images from a 2D camera. However, to work with 3D data in Aurora Imaging Library, you need to use a container. The reason is that 3D devices can transmit multiple components (range, confidence, etc.) during a grab.

If your 3D device transmits data using the GenICam GenDC or the GigE Vision multi-part payload types, you can grab directly into a container that was previously allocated using MbufAllocContainer().

Grabbing into an Aurora Imaging Library container

When grabbing into a container for the first time, Aurora Imaging Library will free all existing components and allocate the components required to store the current frame of data. The library will typically reuse these automatically allocated components during subsequent grabs, unless you add or remove components from the container. If the latter is the case, all components will be freed and new components will again be allocated for the grabbed data.

Aurora Imaging Library will not grab into components of the container that you have allocated manually using MbufAllocComponent(), MbufCopyComponent(), or MbufCreateComponent(). Rather, the library will create components, as required.

Grabbing into an Aurora Imaging Library buffer

If you grab into a traditional buffer from a device that outputs data in a format suitable for a container, the first intensity component (i.e., a standard image) in the frame of data will be grabbed. If there is no intensity component, the first component that is not metadata will be grabbed.

Grabbing using older utilities and certain examples

Limitations are to be expected when using older utilities (such as Aurora Imaging Intellicam), and certain examples (such as MdigGrab, MdigGrabSequence and MdigProcess), to grab from particular 3D devices. These utilities and examples do not support grabbing multiple components. That is, they are not able to grab all of the components at once. If present, the first intensity component is always used. Otherwise, the first component that can be stored into an image buffer is used.

Capture Works

The Capture Works interactive utility has been updated with the following.

Ethernet adapter configuration tool

Capture Works will highlight network adapters that are not correctly configured for GigE Vision acquisition with a yellow icon. A tooltip will offer the user recommended modifications. The user can easily rectify configuration issues by right-clicking on the network connection and choosing the "optimize" option from the dropdown menu. Please note that administrator privileges are necessary for these changes to be implemented.

In addition, an Ethernet Adapter Configuration tool is accessible from the Tools menu. This tool enables the optimization of multiple Ethernet adapters simultaneously and the ability to modify the IP configuration settings (including the IP address and subnet mask of the network adapters).

Alternatively, the Ethernet Adapter Configuration tool is also accessible from the command-line interface. Type CaptureWorks.exe --help on the command line for more details.

GigE Vision device IP configuration tool

GigE Vision devices require an IP address for operation. The acquisition of an IP address by a device can be achieved through three methods: Static IP, DHCP, and Link-Local Address. A device can become inaccessible if the IP address in use is incompatible with its network segment. In many cases, Aurora Imaging Library identifies this issue and assigns a valid IP address, thereby resolving the configuration problem transparently.

However, there are situations where Aurora Imaging Library is unable to rectify the configuration problem, particularly when the network segment in question has a DHCP service. The IP address allocated by the library could eventually lead to problems as the DHCP server might assign the same IP address to a different device, resulting in an IP address conflict. A device that is misconfigured on such a network segment will become inaccessible.

In previous versions of Aurora Imaging Library, unreachable devices were not reported. Consequently, it was the user's responsibility to troubleshoot this type of problem. However, the GigE Vision driver now identifies such devices as "unreachable", and Capture Works will denote these devices with a red icon.

To resolve these issues, the GigE Vision device IP configuration tool included in Capture Works can be utilized. Simply right-click on the device and select the "IP resolve" option to automatically rectify the problem. The device will be temporarily assigned a valid IP address, making it addressable. At this point, Capture Works will modify the device's IP configuration to a mode that is compatible with its network segment and send a command requesting the device to initiate its IP configuration cycle using the new configuration mode. If a DHCP server is available, the device will request an IP address using the DHCP method, resulting in a properly configured device.

Alternatively, and for more control over the method used to resolve the conflict, users can access the GigE Vision device IP configuration option from the Tools menu.

Known issues

There are known acquisition issues when Microsoft Teams is running resulting in stream data packet resends and corrupted frames. Shutting down Microsoft Teams resolves the issue; note that simply closing its window is not sufficient to resolve the problem. Microsoft Teams can be shutdown through the Windows system tray.

Standards compliance

The following is a list of supported standards:

- Support for GigE Vision 2.2, including:
    - Support for GigE Vision control, stream, and message channels.
    - Support for multicasting on stream and message channels.
    - Support for GenICam chunk mode.

    - Support for peripheral devices (e.g. non-streaming devices).
- Support for GenICam 3.4.
- Support for Standard Feature Naming Convention (SFNC) version 2.7.
- Support for GenDC 1.1.
- Support for GigE Vision action and scheduled action command with precision time protocol (PTP IEEE 1588).
- Support for Microsoft Network Driver Interface Specification 6.20.
- Seamless support for link aggregated devices. Note that Intel has dropped support in their Windows 11 drivers but link aggregation is supported on the GevIQ frame grabber regardless of the OS.

Miscellaneous reference information for Zebra GigE Vision driver

This section includes additional command reference information for Zebra GigE Vision driver. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Mbuf.
- Mdig.
- Msys.

Mbuf

This section presents information about Zebra GigE Vision driver and the buffer module.

- Support for grab containers. See container support in the Mbuf module of the Help for details.

Additions to MbufAlloc...

The following lists additional GigE Vision support when using MbufAlloc...().

- Support for **M_IMAGE** + **M_GRAB** + **M_DYNAMIC** buffer attribute.

Additions to MbufInquire

The following lists additional GigE Vision support when using MbufInquire().

- Support for **M_GRAB_TIME_STAMP**.

  Inquires the time when Aurora Imaging Library finished grabbing into the buffer, expressed in seconds. Inquire returns an **AIL_DOUBLE**.

- Support for **M_GRAB_TIME_STAMP_NS**.

  Inquires the time when Aurora Imaging Library finished grabbing into the buffer, expressed in nanoseconds. Inquire returns an **AIL_INT64**.

- Support for **M_CAMERA_TIME_STAMP**.

  Inquires the time at which the device generated the frame of data, expressed in seconds. Inquire returns an **AIL_DOUBLE**.

- Support for **M_CAMERA_TIME_STAMP_NS**.

  Inquires the time at which the device generated the frame of data expressed in nanoseconds. Inquire returns an **AIL_INT64**.

- Support for **M_COMPONENT_TIME_STAMP**.

  BufId must be that of a component. When grabbing into a container, from a device transmitting data in GenDC format, this inquire gets the time at which the device generated the data component expressed in seconds. Zero is returned if there is no component time stamp available. Inquire returns an **AIL_DOUBLE**. Note: components belonging to the same container can have different component time stamps.

- Support for **M_COMPONENT_TIME_STAMP_NS**.

  BufId must be that of a component. When grabbing into a container, from a device transmitting data in GenDC format, this inquire gets the time at which the device generated the data component expressed in nanoseconds. Zero is returned if there is no component time stamp available. Inquire returns an **AIL_INT64**. Note: components belonging to the same container can have different component time stamps.

- Support for **M_COMPONENT_TIME_STAMP_IS_PTP**.

  BufId must be that of a component. If **M_COMPONENT_TIME_STAMP_IS_PTP** returns **M_TRUE**, then **M_COMPONENT_TIME_STAMP_NS** is relative to the epoch January 1st 1970 0:00:00.

- Support for **M_LAYOUT_MODIFICATION_COUNT**.

  BufId must be that of a container. This value is modified every time a component is added or removed from the container. Inquire returns an **AIL_INT**.

- Support for **M_PFNC_VALUE**.

  Returns the buffer's format as defined in the GenICam Pixel Format Naming Convention (PFNC).

- Support for **M_PFNC_SUPPORT**.

  Returns **M_YES** if the PFNC format is a native Aurora Imaging Library format.

  Returns **M_WITH_COMPENSATION** if the PFNC format is not a native Aurora Imaging Library format but can be converted to an Aurora Imaging Library native format. A buffer of this type can only be used as a source buffer.

  Returns **M_NO** if Aurora Imaging Library cannot handle this type of format. In this case the user has to access the buffer's memory directly. **MbufInquire** with **M_HOST_ADDRESS** can be used to get a pointer to the buffer's data.

Additions to MbufInquireContainer

The following lists additional GigE Vision support when using MbufInquireContainer().

- Support for **M_COMPONENT_TIME_STAMP**.

  BufId must be that of a container and the **Component** parameter must not be set to **M_CONTAINER**. When grabbing into a container, from a device transmitting data in GenDC format, this inquire gets the time at which the device generated the data component expressed in seconds. Zero is returned if there is no component time stamp available. Inquire returns an **AIL_DOUBLE**. Note: components belonging to the same container can have different component time stamps.

- Support for **M_COMPONENT_TIME_STAMP_NS**.

  BufId must be that of a container and the **Component** parameter must not be set to **M_CONTAINER**. When grabbing into a container, from a device transmitting data in GenDC format, this inquire gets the time at which the device generated the data component expressed in nanoseconds. Zero is returned if there is no component time stamp available. Inquire returns an **AIL_INT64**. Note: components belonging to the same container can have different component time stamps.

- Support for **M_COMPONENT_TIME_STAMP_IS_PTP**.

  the **Component** parameter must not be set to **M_CONTAINER**. If **M_COMPONENT_TIME_STAMP_IS_PTP** returns **M_TRUE**, then **M_COMPONENT_TIME_STAMP_NS** is relative to the epoch January 1st 1970 0:00:00.

- Support for **M_LAYOUT_MODIFICATION_COUNT**.

  BufId must be that of a container and the **Component** parameter must be set to **M_CONTAINER**. This value is

modified every time a component is added or removed from the container. Inquire returns an **AIL_INT**.

Mdig

This section presents information about Zebra GigE Vision driver and the digitizer module.

Additions to MdigAlloc

The following lists additional GigE Vision support when using MdigAlloc().

- The **InitFlag** parameter now supports the following settings:
  - Support for **M_GC_XML_FORCE_DOWNLOAD**.

    Forces the Zebra driver to download the GigE Vision device's XML description file at MdigAlloc(). If this setting is omitted, the cached version present in the Aurora Imaging Library XML repository is used instead.
  - Support for **M_GC_XML_DOWNLOAD_SKIP**.

    Prevents the Zebra driver from downloading the GigE Vision device's XML description file at MdigAlloc(). The cached version present in the Aurora Imaging Library XML repository is used instead. The **M_GC_XML_DOWNLOAD_SKIP** flag is ignored if:
    - The XML description file is not present in the Aurora Imaging Library XML repository.
    - The XML description file name and/or file size, as reported by the GigE Vision device, has changed.
  - Support for **M_GC_MULTICAST_CONTROLLER**.

    Specifies that the allocated device will be the controller digitizer in a multicasting controller-worker relationship. When this setting is used, the GigE Vision device's stream and message channel (if present) will be programmed to use a destination IPv4 address in the multicast address range (from 224.0.0.0 to 239.255.255.255). By default, Aurora Imaging Library chooses an administratively scoped IP multicast address in the range 239.0.0.0 to 239.255.255.255. IP multicast also requires a network designed to deliver a multicast service. See the IP multicast section of the Aurora Imaging Capture Works help for more details.
  - Support for **M_GC_MULTICAST_WORKER**.

    Specifies that the allocated device will be a worker digitizer in a multicasting controller-worker relationship. See the IP multicast section of the Aurora Imaging Capture Works help for more details.
  - Support for **M_GC_MULTICAST_MONITOR**.

    Specifies that the allocated device will be a monitor digitizer in a multicasting controller-worker relationship.
  - Support for **M_GC_PACKET_SIZE_NEGOTIATION_SKIP**.

    Prevents the Zebra driver from negotiating a packet size with the GigE Vision device at MdigAlloc(). Instead, the GigE Vision device's packet size register is read and this value is used by the Zebra driver. If the read packet size is too small, acquisition reliability will suffer. Using this scheme, the packet size can be forced by storing it in the DCF or by using MdigControl() with **M_GC_PACKET_SIZE**. In either case, acquisition reliability will be affected if the forced packet size is too small or if the packet size is not supported by the network infrastructure.
  - Support for **M_DEV_NUMBER**.

    Supports customary allocation using device numbers.

- **M_GC_CAMERA_ID(MT("))** can now be used with the **DigNum** parameter (MdigAlloc()), in conjunction with the following **InitFlag** parameter settings: **M_GC_DEVICE_IP_ADDRESS** or **M_GC_DEVICE_USER_NAME**. For example, the following 2 calls to MdigAlloc() will allocate a camera whose IP address is 169.254.23.237 and whose user-defined name is MyCameraName, respectively.
    - **MdigAlloc(..., M_GC_CAMERA_ID(MT("169.254.23.237")), ..., M_GC_DEVICE_IP_ADDRESS, ...)**.
    - **MdigAlloc(..., M_GC_CAMERA_ID(MT("MyCameraName")), ..., M_GC_DEVICE_USER_NAME, ...)**.

Additions to MdigControl

The following lists additional GigE Vision support when using MdigControl().

- Support for **M_GC_NETWORK_INTERFACE_CONFIGURATION**.

  Controls the GigE Vision device's persistent network interface configuration. The configuration bits that can be used can be any of the following:

    - **M_GC_PAUSE_RECEPTION_SUPPORT**.
    - **M_GC_PAUSE_GENERATION_SUPPORT**.
    - **M_GC_LINK_LOCAL_ADDRESS_SUPPORT**.
    - **M_GC_DHCP_SUPPORT**.
    - **M_GC_PERSISTENT_IP_SUPPORT**.

- Support for **M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING**.

  Controls the multicast address used by the device to stream image data. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**. Supported control values are any valid IPv4 multicast address in the range 224.0.0.0 to 239.255.255.255.

- Support for **M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING**.

  Controls the multicast address used by the device to send message channel event data. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**. Supported control values are any valid IPv4 multicast address in the range 224.0.0.0 to 239.255.255.255.

- Support for **M_GC_STREAM_PORT**.

  Controls the multicast UDP port number used for the stream channel. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**.

- Support for **M_GC_MESSAGE_PORT**.

  Controls the multicast UDP port number used for the message channel. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**.

- Support for **M_GC_NUMBER_OF_STREAM_CHANNELS**.

  Controls the number of stream channels on the GigE Vision device. This control can only be used when the digitizer is allocated with the **M_GC_MULTICAST_MONITOR InitFlag**.

- Support for **M_GC_STREAMING_MODE**.

  Controls the camera's image stream activation mechanism. Supported control values are:

    - **M_DEFAULT**: Lets the Zebra driver be in control when the camera's image stream is started/stopped.
    - **M_AUTOMATIC**: Same as **M_DEFAULT**.

- **M_MANUAL**: Lets the user be in control when the camera's image stream is started/stopped.

- Support for **M_GC_STREAMING_START**.

  Starts the camera's image stream. **M_GC_STREAMING_MODE** must be set to **M_MANUAL**. Supported control values are:

  - **M_DEFAULT**.

- Support for **M_GC_STREAMING_STOP**.

  Stops the camera's image stream. Supported control values are:

  - **M_DEFAULT**.

- Support for **M_GC_STREAMING_STOP_CHECK_PERIOD**.

  Control value: **AIL_INT**. Represents the time period in milliseconds (ms), or **M_DEFAULT**. Default is 1000 ms. This is the period at which the driver checks if the camera's image stream should be stopped.

- Support for **M_GC_STREAMING_STOP_DELAY**.

  Control value: **AIL_INT**. Represents the delay, in ms, before stopping the camera's image stream. Default is 0 ms. When the **M_GC_STREAMING_STOP_CHECK_PERIOD** time elapses, this delay is applied before a check is made to stop the stream.

- Support for **M_GC_PIXEL_FORMAT**.

  Controls the camera's pixel format. The pixel format value must be a valid GigE Vision pixel format. See GVSP_Pixel_Formats.h for a list of supported values.

- Support for **M_GC_FEATURE_POLLING**.

  Controls the feature browser's polling thread. Your GigE Vision device can define polling values for features that can change without the user's input. An example of this is the "DeviceTemperature" feature. The polling thread can be used when the device's feature browser is displayed. Its role is to poll features that specify a polling period. Supported control values are:

  - **M_DEFAULT**: Same as **M_ENABLE**.
  - **M_ENABLE**: Enables the polling thread. Camera features that report polling support will be read periodically.
  - **M_DISABLE**: Disables the polling thread.

- Support for **M_DIGITIZER_INTERNAL_BUFFERS_NUM**.

  Controls the number of internal grab buffers allocated and used when the Zebra driver cannot directly grab into the user's buffer. Internal grab buffers are needed when the GenICam chunk mode is active. They are also needed when the grab buffer's format is not compatible with the camera's current pixel format. Supported control values are:

  - **M_DEFAULT**: Five internal grab buffers will be allocated.
  - A positive integer between 2 and 1024 representing the number of internal grab buffers to use. An error will get generated if the amount of non paged memory available is insufficient to allocate the buffers.

- Support for **M_GC_ACQUISITION_MODE**.

  Controls the camera's **AcquisitionMode** feature. Can be set to **M_CONTINUOUS** / **M_DEFAULT** or **M_SINGLE_FRAME**.

- Support for **M_GC_HEARTBEAT_STATE** (if supported by the camera).

  Controls if the heartbeat mechanism is used to keep the GigE Vision Control channel open. Supported values: **M_ENABLE**, **M_DISABLE**, **M_DEFAULT** (same as **M_ENABLE**). Can only be used with GigE Vision cameras that support the heartbeat disable feature.

- Support for **M_GC_PACKET_TIMEOUT**.

  Value is in ms. Default is 10 ms. This is the maximum amount of time to wait before flagging a packet as dropped.

- Support for **M_GC_FRAME_TIMEOUT**.

  Value is in ms. Default is 100 ms. This is the maximum amount of time to wait for the remaining packets of a frame after the trailer packet is received. If packets are missing, the frame is flagged as corrupted.

- Support for **M_GC_PACKET_MAX_RETRIES**.

  Default is 3. This is the maximum number of retries for a packet. If reached, the frame is corrupted.

- Support for **M_GC_FRAME_MAX_RETRIES**.

  Default is 30. This is the maximum number of packet retries per frame. If reached, the frame is corrupted.

- Support for **M_GC_STATISTICS_RESET**.

  Control value must be **M_DEFAULT**. This resets the statistics.

- Support for **M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER**.

  Default value is 0. Controls the number of packets to wait before asking for a retransmission. This is useful when using link aggregated cameras where packets are received out of order.

- Support for **M_USER_BIT_FORMAT** +Number.

  Controls the current electrical format of the specified physical input or output Line. Number can range from 0 or 1 up to the maximum number of Lines in the camera. Supported control values are:

  - **M_NO_CONNECT**: The line is not connected.
  - **M_TRI_STATE**: The Line is currently in Tri-State mode (Not driven).
  - **M_TTL**: The Line is currently accepting or sending TTL level signals.
  - **M_RS422**: The Line is currently accepting or sending RS422 level signals.
  - **M_LVDS**: The Line is currently accepting or sending LVDS level signals.
  - **M_OPTO**: The Line is Opto-Coupled.

  An Aurora Imaging Library error will be generated for this control type if the specified Line number does not exist in the camera or if the Line format cannot be changed by the camera.

  An Aurora Imaging Library error will be generated for any of the previous control values if the specified format does not exist for the specified Line number.

- Support for **M_USER_BIT_MODE** +Number.

  Controls if the physical Line is used to input or output a signal. Number can range from 0 or 1 up to the maximum number of Lines in the camera. Supported control values are:

  - **M_INPUT**: The selected physical line is used to input an electrical signal.
  - **M_OUTPUT**: The selected physical line is used to output an electrical signal.

An Aurora Imaging Library error will be generated for this control type if the specified Line number does not exist in the camera or if the Line mode cannot be changed by the camera.

An Aurora Imaging Library error will be generated for any of the previous control values if the specified mode does not exist for the specified Line number.

- Support for **M_AUX_SIGNAL_SOURCE** +Number.

  Selects which internal signal to output on the selected Line. The corresponding **M_USER_BIT_MODE** must be **M_OUTPUT**. Number can range from 0 or 1 up to the maximum number of Lines in the camera. Supported control values are:

  - **M_USER_BIT**: The line will output a user-bit signal.
  - **M_TIMER1**: The line will output if the chosen timer is in an active state.
  - **M_TIMER2**: The line will output if the chosen timer is in an active state.

  An Aurora Imaging Library error will be generated for this control type if the specified Line number does not exist in the camera or if the Line mode is set to **M_INPUT**.

  An Aurora Imaging Library error will be generated for any of the previous control values if the specified signal source does not exist for the specified Line number.

- Support for **M_USER_BIT_VALUE** +Number.

  Sets the value of the specified user bit. Number can range from 0 or 1 up to the maximum number of user-defined signals in the camera. Supported control values are:

  - **M_ON**: The selected physical line is used to input an electrical signal.
  - **M_OFF**: The selected physical line is used to output an electrical signal.

  An Aurora Imaging Library error will be generated for this control type if the specified Line number does not exist in the camera.

- Support for **M_GC_FEATURE_BROWSER**.

  Creates the device feature browser dialog from an Aurora Imaging Library console application. Supported control values are:

  - **M_DEFAULT**: Opens the feature browser dialog. The calling thread is blocked.
  - **M_OPEN** + **M_SYNCHRONOUS**: Same as **M_DEFAULT**.
  - **M_OPEN** + **M_ASYNCHRONOUS**: Opens the feature browser dialog. The calling thread returns as soon as the feature browser is created.
  - **M_CLOSE**: Closes the feature browser dialog.

Additions to MdigControlFeature

The following lists additional GigE Vision support when using MdigControlFeature().

- The **FeatureType** parameter has been changed to **UserVarType**. This was done to simplify writing code with MdigControlFeature() / MdigInquireFeature(). **UserVarType** must always reflect the type of the pointer passed to the **UserVarPtr** parameter. Legacy code is transparently supported, but we recommend you update your code. Note that **M_TYPE_REGISTER** now becomes **M_TYPE_UINT8**, **M_TYPE_ENUMERATION** now becomes **M_TYPE_INT64** or **M_TYPE_STRING**, and **M_TYPE_COMMAND** now becomes **M_DEFAULT**. Data type conversions are made, whenever possible, in cases where the feature's "native" data type is different than the

UserVarType supplied. Regardless of a feature's "native" data type it can always be read as a string.

The following is a list of example calls using the new **UserVarType**:

- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("Width"), M_TYPE_INT64, &Int64Var)**.
- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("Gain"), M_TYPE_DOUBLE, &DoubleVar)**.
- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("ReverseX"), M_TYPE_BOOLEAN, &BoolVar)**.
- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("PixelFormat"), M_TYPE_STRING, AIL_TEXT("Mono8"))**.
- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("LUTValueAll"), M_TYPE_UINT8, Uint8Array)**.
- **MdigControlFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("AcquisitionStart"), M_DEFAULT, M_NULL)**.

- Support for **M_FEATURE_CHANGE_HOOK**.

  Identifies the specified **FeatureName** to trigger the **M_FEATURE_CHANGE** hook callback. You must be hooked to the **M_FEATURE_CHANGE** hook type using MdigHookFunction().

Additions to MdigGetHookInfo

The following lists additional GigE Vision support when using MdigGetHookInfo().

- Hooking to a GenICam feature change event.

  M_GC_FEATURE_CHANGE_NAME can be used from an **M_FEATURE_CHANGE** hook function. The function returns the name of the GenICam feature that triggered the hook. **UserVarPtr** must point to a user allocated array of type **AIL_TEXT_CHAR**.

  M_GC_FEATURE_CHANGE_NAME_SIZE can be used from an **M_FEATURE_CHANGE** hook function. The function returns the number of characters in the string returned by **M_GC_FEATURE_CHANGE_NAME**. **UserVarPtr** must point to an **AIL_INT**.

- Support for **M_GC_FRAME_STATUS_CODE**.

  Returns an **AIL_INT**. Returns the status code from the last image acquired. The status code is extracted from the GigE Vision device's stream packets.

- Support for **M_GC_PACKETS_MISSED**.

  Returns an **AIL_INT**. Returns the number of missing packets in the corrupted frame. If 0, the frame is not corrupted.

- Support for **M_GC_PACKETS_RECEIVED**.

  Returns an **AIL_INT**. This is the number of packets in the frame.

- Support for **M_GC_PACKETS_RESENDS_NUM**.

  Returns an **AIL_INT**. This is the number of resend commands sent to the camera for a frame. One resend command can contain multiple consecutive packets.

- Support for **M_GC_PACKETS_RECOVERED**.

Returns an **AIL_INT**. This is the number of packets recovered in the frame.

- Support for **M_GC_FRAME_ERROR_CODE**.

  Returns an **AIL_INT**. This is the error code returned by the camera for a frame.

- Support for **M_GC_FRAME_LINE_COUNT**.

  Returns an **AIL_INT**. This is the number of lines in the frame.

- Support for **M_GC_FRAME_BLOCK_ID**.

  Returns an **AIL_INT**. This is the frame BlockID value from the camera. The BlockID in the camera is implemented as a 16-bit running counter.

- Support for **M_GC_FRAME_TIMESTAMP**.

  Returns an **AIL_DOUBLE** in seconds. This is the frame timestamp from the camera.

- Support for **M_COUNTER_INDEX**.

  Returns an **AIL_INT**. This is the index of the counter that generated the event.

- Support for **M_TIMER_INDEX**.

  Returns an **AIL_INT**. This is the index of the timer that generated the event.

- Support for **M_LINE_INDEX**.

  Returns an **AIL_INT**. This is the index of the line that generated the event.

- Support for **M_GC_EVENT_TYPE**.

  Returns an **AIL_INT**. This is the raw GigE Vision event sent by the camera.

Additions to MdigHookFunction

The following lists additional GigE Vision support when using MdigHookFunction().

- Hooking to a GenICam feature change event.

  M_GC_FEATURE_CHANGE can be used as a hook type. The hook is triggered when a GenICam feature gets invalidated. This usually occurs when a feature or a dependent feature is written.

- Support for **M_GC_EVENT**.

  Hooks an Aurora Imaging Library function callback to a generic GigE Vision camera event. Note that the underlying event must be enabled in the camera with MdigControlFeature().

- Support for **M_GC_EVENT** + **M_ACQUISITION_TRIGGER**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionTrigger" camera event.

- Support for **M_GC_EVENT** + **M_ACQUISITION_START**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionStart" camera event.

- Support for **M_GC_EVENT** + **M_ACQUISITION_END**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionEnd" camera event.

- Support for **M_GC_EVENT** + **M_ACQUISITION_TRANSFER_START**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionTransferStart" camera event.

- Support for **M_GC_EVENT** + **M_ACQUISITION_TRANSFER_END**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionTransferEnd" camera event.

- Support for **M_GC_EVENT** + **M_ACQUISITION_ERROR**.

  Hooks an Aurora Imaging Library function callback to the GenICam "AcquisitionError" camera event.

- Support for **M_GC_EVENT** + **M_FRAME_TRIGGER**.

  Hooks an Aurora Imaging Library function callback to the GenICam "FrameTrigger" camera event.

- Support for **M_GC_EVENT** + **M_FRAME_START**.

  Hooks an Aurora Imaging Library function callback to the GenICam "FrameStart" camera event.

- Support for **M_GC_EVENT** + **M_FRAME_END**.

  Hooks an Aurora Imaging Library function callback to the GenICam "FrameEnd" camera event.

- Support for **M_GC_EVENT** + **M_FRAME_TRANSFER_START**.

  Hooks an Aurora Imaging Library function callback to the GenICam "FrameTransferStart" camera event.

- Support for **M_GC_EVENT** + **M_FRAME_TRANSFER_END**.

  Hooks an Aurora Imaging Library function callback to the GenICam "FrameTransferEnd" camera event.

- Support for **M_GC_EVENT** + **M_EXPOSURE_START**.

  Hooks an Aurora Imaging Library function callback to the GenICam "ExposureStart" camera event.

- Support for **M_GC_EVENT** + **M_EXPOSURE_END**.

  Hooks an Aurora Imaging Library function callback to the GenICam "ExposureEnd" camera event.

- Support for **M_GC_EVENT** + **M_COUNTER_START** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "CounterXStart" camera event. Number determines the index of the Counter used to generate the event. Number can range from 0 to 8.

- Support for **M_GC_EVENT** + **M_COUNTER_END** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "CounterXEnd" camera event. Number determines the index of the Counter used to generate the event. Number can range from 0 to 8.

- Support for **M_GC_EVENT** + **M_TIMER_START** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "TimerXStart" camera event. Number determines the index of the Timer used to generate the event. Number can range from 0 to 8.

- Support for **M_GC_EVENT** + **M_TIMER_END** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "TimerXEnd" camera event. Number determines the index of the Timer used to generate the event. Number can range from 0 to 8.

- Support for **M_GC_EVENT** + **M_LINE_RISING_EDGE** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "LineXRisingEdge" camera event. Number determines the index of the Line used to generate the event. Number can range from 0 to 32.

- Support for **M_GC_EVENT** + **M_LINE_FALLING_EDGE** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "LineXFallingEdge" camera event. Number

determines the index of the Line used to generate the event. Number can range from 0 to 32.

- Support for **M_GC_EVENT** + **M_LINE_ANY_EDGE** +Number.

  Hooks an Aurora Imaging Library function callback to the GenICam "LineXRisingEdge" and "LineXFallingEdge camera events. Number determines the index of the Line used to generate the event. Number can range from 0 to 32.

- Notes:

  M_UNHOOK can be used with all of the above **M_GC_EVENT** events to disable the underlying event in the camera.

  The above message channel events must be implemented in the GigE Vision camera, otherwise calling MdigHookFunction() with **M_GC_EVENT** +... will generate an Aurora Imaging Library error message.

Additions to MdigInquire

The following lists additional GigE Vision support when using MdigInquire().

- Support for **M_GC_PIXEL_FORMAT_STRING**.

  Returns the camera's **PixelFormat** as a string.

- Support for **M_GC_LOCAL_CONTROL_PORT**.

  Returns the host UDP port number for the control channel.

- Support for **M_GC_LOCAL_MESSAGE_PORT**.

  Returns the host UDP port number for the control channel.

- Support for **M_GC_LOCAL_STREAM_PORT**.

  Returns the host UDP port number for the control channel.

- Support for **M_GC_REMOTE_CONTROL_PORT**.

  Returns the camera's UDP port number for the control channel.

- Support for **M_GC_REMOTE_MESSAGE_PORT**.

  Returns the camera's UDP port number for the control channel.

- Support for **M_GC_REMOTE_STREAM_PORT**.

  Returns the camera's UDP port number for the control channel.

- Support for **M_GC_LOCAL_IP_ADDRESS**.

  Returns the IP address of the host NIC as an **AIL_UINT64**.

- Support for **M_GC_LOCAL_IP_ADDRESS_STRING**.

  Returns the IP address of the host NIC as a string.

- Support for **M_GC_LOCAL_MAC_ADDRESS**.

  Returns the MAC address of the host NIC as an **AIL_UINT64**.

- Support for **M_GC_LOCAL_MAC_ADDRESS_STRING**.

  Returns the MAC address of the host NIC as a string.

- Support for **M_GC_REMOTE_IP_ADDRESS**.

  Returns the IP address of the camera as an **AIL_UINT64**.

- Support for **M_GC_REMOTE_IP_ADDRESS_STRING**.

  Returns the IP address of the camera as a string.

- Support for **M_GC_ REMOTE _MAC_ADDRESS**.

  Returns the MAC address of the camera as an **AIL_UINT64**.

- Support for **M_GC_ REMOTE _MAC_ADDRESS_STRING**.

  Returns the MAC address of the camera as a string.

- Support for **M_GC_CONTROL_PROTOCOL_CAPABILITY**.

  Returns an **AIL_INT** that represents the control channel capabilities of the GigE Vision device. Capability bits returned can be any combination of the following:

  - **M_GC_USER_DEFINED_NAME_SUPPORT**.
  - **M_GC_SERIAL_NUMBER_SUPPORT**.
  - **M_GC_HEARTBEAT_DISABLE_SUPPORT**.
  - **M_GC_LINK_SPEED_REGISTER_SUPPORT**.
  - **M_GC_PORT_AND_IP_REGISTER_SUPPORT**.
  - **M_GC_MANIFEST_TABLE_SUPPORT**.
  - **M_GC_TEST_DATA_SUPPORT**.
  - **M_GC_DISCOVERY_ACK_DELAY_SUPPORT**.
  - **M_GC_WRITABLE_DISCOVERY_ACK_DELAY_SUPPORT**.
  - **M_GC_EXTENDED_STATUS_CODES_1_SUPPORT**.
  - **M_GC_PRIMARY_APP_SWITCHOVER_SUPPORT**.
  - **M_GC_UNCONDITIONAL_ACTION_SUPPORT**.
  - **M_GC_IEEE_1588_SUPPORT**.
  - **M_GC_EXTENDED_STATUS_CODES_2_SUPPORT**.
  - **M_GC_SCHEDULED_ACTION_SUPPORT**.
  - **M_GC_ACTION_SUPPORT**.
  - **M_GC_PENDING_ACK_SUPPORT**.
  - **M_GC_EVENT_DATA_SUPPORT**.
  - **M_GC_EVENT_SUPPORT**.
  - **M_GC_PACKET_RESEND_SUPPORT**.
  - **M_GC_WRITE_MEM_SUPPORT**.
  - **M_GC_CONCATENATION_SUPPORT**.

- Support for **M_GC_STREAM_PROTOCOL_CAPABILITY**.

  Returns an **AIL_INT** that represents the stream protocol capabilities of the GigE Vision device. Capability bits returned can be any combination of the following:

  - **M_GC_FIREWALL_TRAVERSAL_SUPPORT**.
  - **M_GC_LEGACY_16BIT_BLOCK_SUPPORT**.

- Support for **M_GC_MESSAGE_PROTOCOL_CAPABILITY**.

  Returns an **AIL_INT** that represents the message protocol capabilities of the GigE Vision device. Capability bits returned can be any combination of the following:

  - **M_GC_FIREWALL_TRAVERSAL_SUPPORT**.

- Support for **M_GC_NETWORK_INTERFACE_CAPABILITY**.

  Returns an **AIL_INT** that represents the GigE Vision device networking interface capabilities. Capability bits returned are can be any combination of the following:

  - **M_GC_PAUSE_RECEPTION_SUPPORT**.
  - **M_GC_PAUSE_GENERATION_SUPPORT**.
  - **M_GC_LINK_LOCAL_ADDRESS_SUPPORT**.
  - **M_GC_DHCP_SUPPORT**.
  - **M_GC_PERSISTENT_IP_SUPPORT**.

- Support for **M_GC_NETWORK_INTERFACE_CONFIGURATION**.

  Controls the GigE Vision device's persistent network interface configuration. The inquire returns an **AIL_INT** which represents the currently enabled configuration. The configuration bits that can be used can be any of the following:

  - **M_GC_PAUSE_RECEPTION_SUPPORT**.
  - **M_GC_PAUSE_GENERATION_SUPPORT**.
  - **M_GC_LINK_LOCAL_ADDRESS_SUPPORT**.
  - **M_GC_DHCP_SUPPORT**.
  - **M_GC_PERSISTENT_IP_SUPPORT**.

- Support for **M_GC_PHYSICAL_LINK_CONFIGURATION_CAPABILITY**.

  Returns an **AIL_INT** that represents the GigE Vision device physical link configuration capabilities. Capability bits returned can be any combination of the following:

  - **M_GC_SINGLE_LINK_SUPPORT**.
  - **M_GC_MULTIPLE_LINK_SUPPORT**.
  - **M_GC_STATIC_LINK_AGGREGATION_SUPPORT**.
  - **M_GC_DYNAMIC_LINK_AGGREGATION_SUPPORT**.

- Support for **M_GC_STREAM_CHANNEL_CAPABILITY**.

  Returns an **AIL_INT** that represents the stream channel capabilities of the GigE Vision device. Capability bits returned can be any combination of the following:

- **M_GC_BIG_AND_LITTLE_ENDIAN_SUPPORT**.
- **M_GC_IP_REASSEMBLY_SUPPORT**.
- **M_GC_MULTI_ZONE_SUPPORT**.
- **M_GC_PACKET_RESEND_OPTION_SUPPORT**.
- **M_GC_ALL_IN_SUPPORT**.
- **M_GC_UNCONDITIONAL_STREAMING_SUPPORT**.
- **M_GC_EXTENDED_CHUNK_DATA_SUPPORT**.

- Support for **M_GC_STREAM_CHANNEL_MULTICAST_ADDRESS_STRING**.

  Returns the multicast address used by the device to stream image data. The inquire returns an **AIL_TEXT_CHAR** string.

- Support for **M_GC_MESSAGE_CHANNEL_MULTICAST_ADDRESS_STRING**.

  Returns the multicast address used by the device to send message channel event data. The inquire returns an **AIL_TEXT_CHAR** string.

- Support for **M_GC_MULTICAST_CONTROLLER_CONNECTED**.

  Returns an **AIL_INT**. Returns **M_TRUE** if a controller digitizer is connected to the device.

- Support for **M_GC_STREAM_PORT**.

  Returns the multicast UDP port number used for the stream channel. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**. The inquire returns an **AIL_INT**.

- Support for **M_GC_MESSAGE_PORT**.

  Returns the multicast UDP port number used for the message channel. The device must have been allocated with the **M_GC_MULTICAST_CONTROLLER InitFlag**. The inquire returns an **AIL_INT**.

- Support for **M_GC_NUMBER_OF_STREAM_CHANNELS**.

  Returns the number of stream channels supported by the GigE Vision device. The inquire can always be used. The inquire returns an **AIL_INT**.

- Support for **M_GC_STREAMING_MODE**.

  Returns the camera's image stream activation mechanism. Inquire returns an **AIL_INT**.

- Support for **M_GC_PIXEL_FORMAT**.

  Returns the camera's pixel format. The pixel format value must be a valid GigE Vision pixel format. See GVSP_Pixel_Formats.h for a list of supported values. The inquire returns an **AIL_INT**.

- Support for **M_GC_FEATURE_POLLING**.

  Inquire returns an **AIL_INT**. Returns the feature browser's polling thread. Your GigE Vision device can define polling values for features that can change without the user's input. An example of this is the "DeviceTemperature" feature. The polling thread can be used when the device's feature browser is displayed. Its role is to poll features that specify a polling period.

- Support for **M_DIGITIZER_INTERNAL_BUFFERS_NUM**.

  Inquire returns an **AIL_INT**. Returns the number of internal grab buffers allocated and used when the Zebra driver cannot directly grab into the user's buffer. Internal grab buffers are needed when the GenICam chunk

mode is active. They are also needed when the grab buffer's format is not compatible with the camera's current pixel format.

- Support for **M_SOURCE_DATA_FORMAT**.

  Inquire returns an **AIL_INT64**. Returns a value representing the Aurora Imaging Library buffer format that is compatible with the camera's current **PixelFormat**. Useful for allocating grab buffers and avoiding costly color space conversions, Ex:

  - **Mono8 PixelFormat** will return **M_MONO8** + **M_PLANAR**.
  - **Mono10 PixelFormat** will return **M_MONO16** + **M_PLANAR**.
  - **BayerBG8 PixelFormat** will return **M_YUV16_YUYV** + **M_PACKED**.
  - **BayerBG12Packed PixelFormat** will return **M_RGB48** + **M_PACKED**.
  - **YUYVPacked PixelFormat** will return **M_YUV16_YUYV** + **M_PACKED**.

- Support for **M_GC_TOTAL_PACKET_CACHE_HITS**.

  Inquire returns an **AIL_INT**. Represents the total number of packets that passed through the packet cache.

- Support for **M_GC_TOTAL_FRAME_CACHE_HITS**.

  Inquire returns an **AIL_INT**. Represents the total number of partial or complete frames that passed through the packet cache.

- Support for **M_GC_NIC_FRIENDLY_NAME**.

  Inquire returns an **AIL_TEXT_CHAR** string. Returns the network adapter's friendly name connected to your GigE Vision device.

- Support for **M_GC_FRAME_STATUS_CODE**.

  Inquire returns an **AIL_INT**. Returns the status code from the last image acquired. The status code is extracted from the GigE Vision device's stream packets.

- Support for **M_GC_ACQUISITION_MODE**.

  Inquire returns an **AIL_INT**. Returns the camera's **AcquisitionMode** feature.

- Support for **M_GC_FRAME_TIMESTAMP**.

  Inquire returns an **AIL_DOUBLE**. Inquires the timestamp, in seconds, of the last frame grabbed.

- Support for **M_GC_COUNTER_TICK_FREQUENCY**.

  Inquire returns an **AIL_INT64**. Inquires the camera's tick frequency register. A value of 0 Hz indicates the camera does not support time stamps.

- Support for **M_GC_HEARTBEAT_STATE** (if supported by the camera).

  Inquire returns an **AIL_INT**. Supported values: **M_ENABLE**, **M_DISABLE**, **M_DEFAULT** (same as **M_ENABLE**). Inquires if the heartbeat mechanism is used to keep the GigE Vision Control channel open. Can only be used with GigE Vision cameras that support the heartbeat disable feature.

- Support for **M_GC_NIC_IP_ADDRESS**.

  Inquire returns an **AIL_INT64**. Returns the host NIC's IP address used with the camera.

- Support for **M_GC_NIC_MAC_ADDRESS**.

  Inquire returns an **AIL_INT64**. Returns the host NIC's MAC address used with the camera.

- Support for **M_GC_PACKET_TIMEOUT**.

  Inquire returns an **AIL_INT**. Value is in ms. Default is 10 ms. This is the maximum amount of time to wait before flagging a packet as dropped.

- Support for **M_GC_FRAME_TIMEOUT**.

  Inquire returns an **AIL_INT**. Value is in ms. Default is 100 ms. This is the maximum amount of time to wait for the remaining packets of a frame after the trailer packet is received. If packets are missing, the frame is flagged as corrupted.

- Support for **M_GC_PACKET_MAX_RETRIES**.

  Inquire returns an **AIL_INT**. Default is 3. This is the maximum number of retries for a packet. If reached, the frame is corrupted.

- Support for **M_GC_FRAME_MAX_RETRIES**.

  Inquire returns an **AIL_INT**. Default is 30. This is the maximum number of packet retries per frame. If reached, the frame is corrupted.

- Support for **M_GC_TOTAL_PACKETS_MISSED**.

  Inquire returns an **AIL_INT**. This is the total number of packets that are missing. Does not include recovered packets.

- Support for **M_GC_TOTAL_PACKETS_RECEIVED**.

  Inquire returns an **AIL_INT**. This is the total number of packets received.

- Support for **M_GC_TOTAL_PACKETS_RESENDS_NUM**.

  Inquire returns an **AIL_INT**. This is the total number of emitted PACKETRESEND commands. One resend command can contain multiple consecutive packets.

- Support for **M_GC_TOTAL_PACKETS_RECOVERED**.

  Inquire returns an **AIL_INT**. This is the total number of packets recovered.

- Support for **M_GC_TOTAL_FRAMES_GRABBED**.

  Inquire returns an **AIL_INT**. This is the total number of frames grabbed.

- Support for **M_GC_TOTAL_FRAMES_CORRUPTED**.

  Inquire returns an **AIL_INT**. Returns the number of corrupted frames.

- Support for **M_GC_TOTAL_PACKETS_RECEIVED_OUT_OF_ORDER**.

  Inquire returns an **AIL_INT**. This is the total number of packets that were received out of order, which usually happens with link aggregated cameras. If this number is high, increase the value of **M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER**.

- Support for **M_GC_MAX_NBR_PACKETS_OUT_OF_ORDER**.

  Inquire returns an **AIL_INT**. Default value is 0. Controls the number of packets to wait before asking for a retransmission. This is useful when using link aggregated cameras where packets are received out of order.

- Support for **M_PROCESS_PENDING_GRAB_NUM**.

  Inquire returns an **AIL_INT**. Returns the number of buffers remaining in the list of buffers used by MdigProcess(). Note that this number includes the buffer used by the grab in progress. When dealing with round-robin grabbing, this number will typically be relatively close to the total number of allocated grab buffers.

- Support for **M_USER_BIT_FORMAT** +Number.

  Inquire returns an **AIL_INT**. Inquires the current electrical format of the specified physical input or output Line. Number can range from 0 or 1 up to the maximum number of Lines in the camera.

- Support for **M_USER_BIT_MODE** +Number.

  Inquire returns an **AIL_INT**. Inquires if the physical Line is used to input or output a signal. Number can range from 0 or 1 up to the maximum number of Lines in the camera.

- Support for **M_AUX_SIGNAL_SOURCE** +Number.

  Inquire returns an **AIL_INT**. Inquires which internal signal to output on the selected Line. The corresponding **M_USER_BIT_MODE** must be **M_OUTPUT**. Number can range from 0 or 1 up to the maximum number of Lines in the camera.

- Support for **M_USER_BIT_VALUE** +Number.

  Inquire returns an **AIL_INT**. Returns the current status of the selected input or output Line. Number can range from 0 or 1 up to the maximum number of user-defined signals in the camera.

Additions to MdigInquireFeature

The following lists additional GigE Vision support when using MdigInquireFeature().

- Support for **M_FEATURE_ENUM_ENTRY_TOOLTIP** + N.

  Returns the nth enum entry's tooltip string. The string size can be inquired by adding the **M_STRING_SIZE** combination constant.

- Support for **M_FEATURE_ENUM_ENTRY_DESCRIPTION** + N.

  Returns the nth enum entry's description string. The string size can be inquired by adding the **M_STRING_SIZE** combination constant.

- Support for **M_FEATURE_ENUM_ENTRY_ACCESS_MODE** + N.

  Returns the nth enum entry's access mode. See **M_FEATURE_ACCESS_MODE** for details.

- Support for **M_FEATURE_ENUM_ENTRY_VISIBILITY** + N.

  Returns the nth enum entry's visibility value. See **M_FEATURE_VISIBILITY** for details.

- Support for **M_FEATURE_ENUM_ENTRY_CACHING_MODE** + N.

  Returns the nth enum entry's caching mode. See **M_FEATURE_CACHING_MODE** for details.

- Support for **M_FEATURE_ENUM_ENTRY_STREAMABLE** + N.

  Returns the nth enum entry's streamable value. See **M_FEATURE_STREAMABLE** for details.

- Support for **M_FEATURE_SELECTOR_COUNT**.

  Returns the number of features selected by the specified **FeatureName** parameter. Note that the **FeatureName** parameter must specify a selector type feature.

- Support for **M_FEATURE_SELECTOR_NAME** + N.

  Returns the nth feature name, as a string, selected by the specified feature. Note that the **FeatureName** parameter must specify a selector type feature.

- Support for **M_FEATURE_VALID_VALUE_COUNT**.

Integer and floating-point type features can support a fixed list of valid values instead of the traditional minimum, maximum, and increment values. **M_FEATURE_VALID_VALUE_COUNT** returns the number of valid values supported. A value of 0 returned indicates the feature does not support a list of valid values; minimum, maximum and increment values must be used instead.

- Support for **M_FEATURE_VALID_VALUE** + N.

  Returns the nth valid value for an integer of floating-point type feature.

- The **FeatureType** parameter has been changed to **UserVarType**. This was done to simplify writing code with MdigControlFeature() / MdigInquireFeature(). **UserVarType** must always reflect the type of the pointer passed to the **UserVarPtr** parameter. Legacy code is transparently supported, but we recommend you update your code. Note that **M_TYPE_REGISTER** now becomes **M_TYPE_UINT8**, **M_TYPE_ENUMERATION** now becomes **M_TYPE_INT64** or **M_TYPE_STRING**, and **M_TYPE_COMMAND** now becomes **M_DEFAULT**. Data type conversions are made, whenever possible, in cases where the feature's "native" data type is different than the **UserVarType** supplied. Regardless of a feature's "native" data type it can always be read as a string. See Board-specific examples for details.

  The following is a list of example calls using the new **UserVarType**:

  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("Width"), M_TYPE_INT64, &Int64Var)**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("Gain"), M_TYPE_DOUBLE, &DoubleVar)**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("ReverseX"), M_TYPE_BOOLEAN, &BoolVar)**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE + M_STRING_SIZE, AIL_TEXT("PixelFormat"), M_TYPE_AIL_INT, &IntVar)**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("PixelFormat"), M_TYPE_STRING, TextCharArray)**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("LUTValueAll"), M_TYPE_UINT8, Uint8Array)**.

- **M_FEATURE_USER_ARRAY_SIZE** can now be used with MdigInquireFeature() when the data type returned is a string or an array of bytes (register). The **M_FEATURE_USER_ARRAY_SIZE** macro is used to pass the size of the user-allocated buffer passed to the **UserVarPtr** parameter of MdigInquireFeature(). **M_FEATURE_USER_ARRAY_SIZE** is passed using the **UserVarType** parameter.

  The following is a list of example calls using **M_FEATURE_USER_ARRAY_SIZE**:

  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("PixelFormat"), M_TYPE_STRING + M_FEATURE_USER_ARRAY_SIZE(N), TextCharArray);** N being equal to the number of **AIL_TEXT_CHAR** in the **TextCharArray**.
  - **MdigInquireFeature(Digitizer, M_FEATURE_VALUE, AIL_TEXT("LUTValueAll"), M_TYPE_UINT8 + M_FEATURE_USER_ARRAY_SIZE(N), Uint8Array); N being equal to the number of Uint8 in the Uint8Array**.

- **M_FEATURE_ENUM_ENTRY_DISPLAY_NAME** can now be used to inquire possible enumeration string entry to use for display purposes. See **M_FEATURE_ENUM_ENTRY_NAME**.

- Support for **M_FEATURE_CHANGE_HOOK**.

  Identifies the specified **FeatureName** to trigger the **M_FEATURE_CHANGE** hook callback. You must be hooked to the **M_FEATURE_CHANGE** hook type using MdigHookFunction().

- Support for **M_INCREMENT** when using **M_TYPE_AIL_INT64**.

  Returns an **AIL_INT64**, representing the feature's increment value.

- Support for **M_INCREMENT** when using **M_TYPE_AIL_INT32**.

  Returns an **AIL_INT32**, representing the feature's increment value.

- Support for **M_INCREMENT** when using **M_TYPE_DOUBLE**.

  Returns an **AIL_DOUBLE**, representing the feature's increment value.

Msys

This section presents information about Zebra GigE Vision driver and the system module.

Additions to MsysControl

The following lists additional GigE Vision support when using MsysControl().

- Support for **M_DISCOVER_DEVICE**.

  Performs a device discovery. **ControlValue** must be **M_DEFAULT**. Note: executing this control can trigger an **M_CAMERA_PRESENT** hook.

- Support for **M_GC_ACTION_DEVICE_KEY** + N.

  Adds an action device key to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**.

- Support for **M_GC_ACTION_GROUP_KEY** + N.

  Adds an action group key to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**.

- Support for **M_GC_ACTION_GROUP_MASK** + N.

  Adds an action group mask to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**.

- Support for **M_GC_ACTION_ACKNOWLEDGE_NUMBER** + N.

  Lets the GigE Vision driver know how many action command acknowledge packets will be received for action context N. If not specified, the number of calls to **M_GC_ACTION_ADD_DEVICE** for this action context will be used as the number of acknowledge packets expected. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**.

- Support for **M_GC_ACTION_EXECUTE** + N.

  Broadcast a GigE Vision Action command packet or scheduled action command packet containing the device key, group key, and group mask associated to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Control value must be **M_DEFAULT**. Note that the device key, group key, and group mask must have previously been programmed in each GigE Vision device associated to this action context using MdigControlFeature() with "ActionDeviceKey", "ActionSelector", "ActionGroupKey", and "ActionGroupMask". See SFNC (Standard Feature Naming Convention) for details.

- Support for **M_GC_ACTION_ADD_DEVICE** + N.

  Associates a GigE Vision device with action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. The control value must be a valid Digitizer ID.

- Support for **M_GC_ACTION_REMOVE_DEVICE** + N.

  Removes a GigE Vision device from action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. The control value must be a valid Digitizer ID.

- Support for **M_GC_ACTION_CLEAR_DEVICES** + N.

  Removes all GigE Vision devices associated to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. The control value must be **M_DEFAULT**.

- Support for **M_GC_ACTION_TIME** + N.

  Associates an action time to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. This control must be used only if it is intended to send a scheduled action command. The time specified must be a time in the future. The GigE Vision devices associated to this action context must have IEEE 1588 PTP enabled using MdigControlFeature() with "GevIEEE1588", "GevIEEE1588ClockAccuracy", and "GevIEEE1588Status". See SFNC (Standard Feature Naming Convention) for details. The current time can be inquired with **MdigInquire(M_GC_CAMERA_TIME_STAMP)** or **MdigGetHookInfo(M_GC_CAMERA_TIME_STAMP)**. The current time is valid only if IEEE 1588 PTP is enabled on all devices when the inquire is made.

Additions to MsysGetHookInfo

The following lists additional GigE Vision support when using MsysGetHookInfo().

- Support for **M_CAMERA_PRESENT**.

  Returns an **AIL_INT**. Returns **M_TRUE** if a camera has been added or re-connected to the system. Returns **M_FALSE** if a camera has been removed from the system.

- Support for **M_NUMBER**.

  Returns an **AIL_INT**. Returns the system assigned digitizer device number associated with the camera that generated an **M_CAMERA_PRESENT** event.

- Support for **M_TIME_STAMP**.

  Returns an **AIL_DOUBLE**. Returns the operating system's time stamp of the event that generates a service request, in sec. The time stamp is generated by the operating system's performance counter.

- Support for **M_GC_USER_NAME_LENGTH**.

  Returns an **AIL_INT**. Returns the maximum number of characters in the camera's user-defined name string.

- Support for **M_GC_USER_NAME**.

  Returns an **AIL_INT**. Returns the user-defined name string for the camera.

- Support for **M_GC_IP_ADDRESS**.

  Returns an **AIL_INT64**. Returns the 32-bit internet protocol (IPv4) address of the GigE Vision device. Note that these addresses are returned in a 64-bit variable.

- Support for **M_GC_MAC_ADDRESS**.

  Returns an **AIL_INT**. Returns the system assigned digitizer device number associated with the camera that generated an **M_CAMERA_PRESENT** event.

Additions to MsysHookFunction

The following lists additional GigE Vision support when using MsysHookFunction().

- Can now hook a callback to **M_CAMERA_PRESENT**. Note that **M_CAMERA_PRESENT** hook support requires the use of the AILGigEService process. If the service is not started, an Aurora Imaging Library error will be generated when calling MsysHookFunction() with **M_CAMERA_PRESENT**.

Additions to MsysInquire

The following lists additional GigE Vision support when using MsysInquire().

- Support for **M_DISCOVER_DEVICE_COUNT**.

  Inquires the number of devices present. Inquire returns an **AIL_INT**.

- Support for **M_DISCOVER_DEVICE_DIGITIZER_NUMBER** + N.

  Inquires the digitizer device number associated to the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_INT**.

- Support for **M_DISCOVER_DEVICE_MANUFACTURER_NAME** + N.

  Inquires the device's manufacturer name, for the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_MODEL_NAME** + N.

  Inquires the device's model name, for the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_MANUFACTURER_INFO** + N.

  Inquires the device's manufacturer-specific information, for the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_USER_NAME** + N.

  Inquires the device's user programmable device name. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_VERSION** + N.

  Inquires the device version of the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_SERIAL_NUMBER** + N.

  Inquires the device serial number of the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_INTERFACE_NAME** + N.

  Inquires the name of the host interface associated to the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_DISCOVER_DEVICE_ADDRESS** + N.

  Inquires the address of the nth device. N must be between 0 and **M_DISCOVER_DEVICE_COUNT** – 1. Inquire returns an **AIL_STRING**.

- Support for **M_GC_ACTION_DEVICE_KEY** + N.

  Inquires the action device key of action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_INT**.

- Support for **M_GC_ACTION_GROUP_KEY** + N.

  Inquires the action group key of action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_INT**.

- Support for **M_GC_ACTION_GROUP_MASK** + N.

  Inquires the action group mask of action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_INT**.

- Support for **M_GC_ACTION_ACKNOWLEDGE_NUMBER** + N.

  Inquires how many action command acknowledge packets will be received for action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_INT**.

- Support for **M_GC_ACTION_ADD_DEVICE** + N.

  Inquires the GigE Vision device added to action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_ID**.

- Support for **M_GC_ACTION_REMOVE_DEVICE** + N.

  Inquires the GigE Vision device removed from action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_ID**.

- Support for **M_GC_ACTION_TIME** + N.

  Inquires the action time of action context N. N can range from **M_GC_ACTION0** to **M_GC_ACTION31**. Inquire returns an **AIL_DOUBLE**. The current time can be inquired with **MdigInquire(M_GC_CAMERA_TIME_STAMP)** or **MdigGetHookInfo(M_GC_CAMERA_TIME_STAMP)**. The current time is valid only if IEEE 1588 PTP is enabled on all devices when the inquire is made.

Zebra Indio

This section discusses features of Aurora Imaging Library that are particular to Zebra Indio and ways that optimize the board's performance. Zebra Indio information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Indio overview
  - Summary of Zebra Indio features
- Using Zebra Indio with Aurora Imaging Library
  - Using the Advanced I/O Engine of Zebra Indio
  - Using EtherNet/IP, Modbus, or PROFINET
- Zebra Indio connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Indio board flow diagram
- Miscellaneous reference information for Zebra Indio
  - Mcom
  - Msys
    - MsysInquire
  - MsysIo
- Miscellaneous user guide information for Zebra Indio

Zebra Indio overview

Zebra Indio is a versatile, industrial I/O and communications, PCIe board. It has an Advanced I/O Engine that provides 16 optically-isolated, digital auxiliary I/O signals (8 inputs and 8 outputs), and includes I/O command lists, timers, quadrature decoders, and user outputs. It also supports GigE Vision capture, as well as Industrial communication using the PROFINET, EtherNet/IP, or Modbus/TCP industrial protocol. To support PROFINET communication, Zebra Indio also includes a PROFINET Engine.

In addition to being prelicensed for use with the Aurora Imaging Library drivers for GigE Vision cameras, the Zebra Indio board can act as a fingerprint for licensing supplemental Aurora Imaging Library functionality. The board can also store a supplemental Aurora Imaging Library license, which facilitates the process of moving to a new computer.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/indio-info.

Summary of Zebra Indio features

The following table outlines the features currently available on Zebra Indio.

Features	Zebra Indio
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature() [1]	Yes
On-board memory	128 MBytes DDR3 SDRAM
Number of acquisition ports	1
Number of auxiliary signals	16 (8 in, 8 out)
PROFINET Engine	Yes
Number of timers	16
Number of quadrature decoders for input from linear or rotary encoders	2
Number of I/O command lists	2
Number of reference latches per I/O command list	4

Power-over-Ethernet (PoE) support	Yes

[1] When using a GigE Vision system (M_SYSTEM_GIGE_VISION, using MsysAlloc()).

Using Zebra Indio with Aurora Imaging Library

Zebra Indio allows you to use a GigE Vision-compliant camera for acquisition. To use a GigE Vision-compliant camera for acquisition with Zebra Indio and Aurora Imaging Library, you must allocate an Aurora Imaging Library GigE Vision system (M_SYSTEM_GIGE_VISION), using MsysAlloc(). This allocation opens communication with GigE Vision compliant cameras. You can then access and grab from multiple connected GigE Vision cameras by allocating a digitizer for each connected camera (using MdigAlloc() with M_DEVn). Each process (executable) can allocate only one Aurora Imaging Library GigE Vision system. Each camera can be used by one process (executable) at a time. For additional information, refer to the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

To use other functionality available on Zebra Indio (for example, I/O signals and industrial communication) or to inquire about the status (for example, board type) of the board, you must allocate an Aurora Imaging Library Indio system, using MsysAlloc() with M_SYSTEM_INDIO.

Zebra Indio is preinstalled with a runtime license for Aurora Imaging Library, which includes the Industrial and Robot Communication module (Mcom) and interface module. Be aware that to access the Aurora Imaging Library license that is included on the Zebra Indio board, you must install the Zebra Indio driver.

Using the Advanced I/O Engine of Zebra Indio

Zebra Indio has an Advanced I/O Engine that controls the auxiliary I/O interface. The engine includes 16 timers, 2 rotary decoders, 2 I/O command lists, a user-output register, 8 auxiliary output signals, and 8 auxiliary input signals. To route auxiliary I/O signals and configure interrupts, use MsysControl(). To inquire this information, use MsysInquire(). For more information on some of these features, refer to Chapter 56: I/O signals and communicating with external devices.

Using EtherNet/IP, Modbus, or PROFINET

You can use the Gigabit Ethernet port for communication with external devices using the EtherNet/IP, Modbus (over TCP/IP), or PROFINET industrial protocol. In this case, you should not use the port for acquisition. Unless using PROFINET, you should not use the port for general network traffic. Both EtherNet/IP and Modbus over TCP/IP communication require installation of the Intel i210 Network Adapter Driver available from downloadcenter.intel.com. When PROFINET is used, communication is assisted using the Zebra PROFINET Engine; its interface is recognized as a second network device with its own IP and MAC seetings (when the PROFINET service is enabled) even though it shares the same LAN connection. For more information, see the Steps to perform industrial communication section of Chapter 57: Industrial communication.

Zebra Indio connectors and signal names

This section serves as a reference to match Zebra Indio's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MsysControl() / MsysInquire(), respectively.

Auxiliary I/O signals can have one or more functionalities (for example, outputting the state of an I/O command register bit, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Indio board provides an auxiliary I/O connector and a Gigabit Ethernet connector.

Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Auxiliary I/O connector	DB-37		The external auxiliary I/O connector is a D-subminiature 37-pin female connector. It is used to transmit/receive auxiliary signals.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

System	Description
I/O #	
Pin information	
Direction	
User-bit information	

		Trigger information	
		Timer information	
		Hardware manual signal name	
System			Indicates the following.
	I/O #: M_AUX_IO0		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: DB-37 Pin: 2+, 1-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT0
	I/O #: M_AUX_IO1		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: DB-37 Pin: 4+, 3-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT1;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT1
	I/O #: M_AUX_IO2		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: DB-37 Pin: 7+, 6-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT2;
		Timer information	Timer: M_TIMERn

	Hardware manual signal name	AUX_OPTOIND_OUT2
I/O #: M_AUX_IO3		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: DB-37 Pin: 9+, 8-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT3;
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_OUT3
I/O #: M_AUX_IO4		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: DB-37 Pin: 12+, 11-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT4;
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_OUT4
I/O #: M_AUX_IO5		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: DB-37 Pin: 14+, 13-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT5;
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_OUT5
I/O #: M_AUX_IO6		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
	Pin information	Connector: DB-37 Pin: 17+, 16-

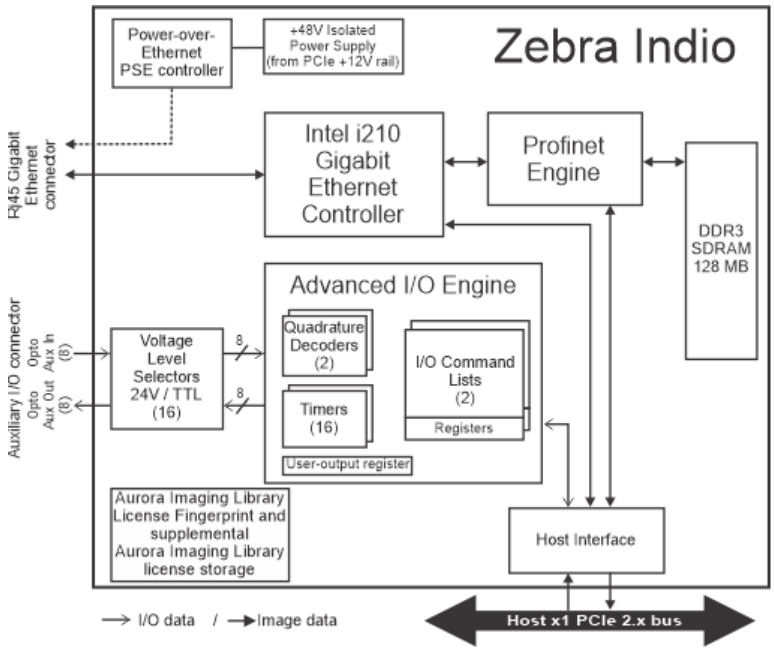
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT6;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT6
	I/O #: M_AUX_IO7		Industrial auxiliary signal (output), which supports: user output, timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: DB-37 Pin: 19+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT7;
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_OUT7
	I/O #: M_AUX_IO8		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: DB-37 Pin: 21+, 20-
		Direction	Input
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN8
	I/O #: M_AUX_IO9		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: DB-37 Pin: 23+, 22-
		Direction	Input
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN9

I/O #: M_AUX_IO10		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: DB-37 Pin: 26+, 25-
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN10
I/O #: M_AUX_IO11		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: DB-37 Pin: 28+, 27-
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN11
I/O #: M_AUX_IO12		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: DB-37 Pin: 30+, 29-
	Direction	Input
	Timer information	Timer: M_TIMERn
	Hardware manual signal name	AUX_OPTOIND_IN12
I/O #: M_AUX_IO13		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
	Pin information	Connector: DB-37 Pin: 32+, 31-
	Direction	Input
	Timer information	Timer: M_TIMERn

		Hardware manual signal name	AUX_OPTOIND_IN13
	I/O #: M_AUX_IO14		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: DB-37 Pin: 35+, 34-
		Direction	Input
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN14
	I/O #: M_AUX_IO15		Industrial auxiliary signal (input), which supports: user input, I/O command list counter source, timer clock, timer arm, I/O command list reference latch trigger, quadrature input bit 0 or 1, or quadrature decoder counter reset source.
		Pin information	Connector: DB-37 Pin: 37+, 36-
		Direction	Input
		Timer information	Timer: M_TIMERn
		Hardware manual signal name	AUX_OPTOIND_IN15

Zebra Indio board flow diagram

The following diagram illustrates the data flow of Zebra Indio.



Miscellaneous reference information for Zebra Indio

This section includes additional command reference information for Zebra Indio. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Mcom.
- Msys.
- MsysIo.

Mcom

This section presents information about Zebra Indio and the communication module.

To initiate communication on Zebra Indio using the Mcom module, the Aurora Imaging Library system identifier should be of **M_SYSTEM_INDIO** that was allocated using MsysAlloc(). For more information, refer to Host system column in the MsysIo command reference.

Msys

This section presents information about Zebra Indio and the system module.

To configure the auxiliary signals, use the **M_IO_...** control types.

To use an auxiliary input signal as a trigger source, use MsysControl() or MsysIoControl() with an **M_..._TRIGGER_SOURCE** control type. Your application can also act upon and interpret the state of an auxiliary input signal.

To poll the state of an auxiliary input signal, use MsysInquire() with **M_IO_STATUS**. The state of an auxiliary input signal can also generate an interrupt. To do so, use MsysControl() with **M_IO_INTERRUPT_STATE** and then use MsysHookFunction() with **M_IO_CHANGE** to hook a function to this event (that is, to set up an event handler).

There are 16 timers available. Use MsysHookFunction() with **M_TIMER_START** or **M_TIMER_END** to hook a function to the respective start or end of a timer.

MsysInquire

The following lists additional Indio support when using MsysInquire().

- Support for inquiring the current encoder position:

  MsysInquire(Sys, M_ROTARY_ENCODER_POSITION+M_ROTARY_ENCODERn, &Position).

  Select *n* as 1 or 2 to choose the target encoder.

  Position is an AIL_INT.

- Support for inquiring the encoder position based on a signal event. Possible events include:

  - The following associates a latch with a rotary encoder:

    MsysControl(Sys, M_SYS_DATA_LATCH_TYPE+M_LATCHm, M_ROTARY_ENCODERn).

    Select *m* as 1 or 2 to choose the target latch storage.

Select *n* as 1 or 2 to choose the target encoder.

- The following determines which signal will latch. In this case, the rotary encoder position:

  MsysControl(Sys, M_SYS_DATA_LATCH_TRIGGER_SOURCE+M_LATCHm, LatchTriggerSource).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchTriggerSource is an **AIL_INT** initialized with **M_AUX_IOi**, where i is from 8 to 15 or **M_TIMERj**, where j is from 1 to 16, on Zebra Indio.

- The following determines which signal edge will latch the encoder position:

  MsysControl(Sys, M_SYS_DATA_LATCH_TRIGGER_ACTIVATION+M_LATCHm, LatchTriggerActivation).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchTriggerActivation is an **AIL_INT** initialized with **M_EDGE_RISING**, **M_EDGE_FALLING**, or **M_ANY_EDGE**.

- The following activates (or deactivates) the given latch mechanism:

  MsysControl(AilSystem, M_SYS_DATA_LATCH_STATE+M_LATCHm, LatchState).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchState is an **AIL_INT** initialized with **M_ENABLE** or **M_DISABLE**.

- The following allows you to retrieve the latched value in the context of a hook-handler function registered with MsysHookFunction():

  MsysGetHookInfo(EventId, M_SYS_DATA_LATCH_VALUE+M_LATCHm, LatchValue).

  Select *m* as 1 or 2 to choose the target latch storage.

  LatchValue is an **AIL_UINT64** that returns the rotary encoder position that is latched.

  Note that if **M_SYS_DATA_LATCH_TRIGGER_SOURCE** was specified as **M_AUX_IOi**, the MsysGetHookInfo() call is done in the scope of a hook function to **M_IO_CHANGE**. If **M_SYS_DATA_LATCH_TRIGGER_SOURCE** was specified as **M_TIMERj**, the MsysGetHookInfo() call is done in the scope of a hook function to **M_TIMER_START** or **M_TIMER_END**.

MsysIo

This section presents information about Zebra Indio and the I/O sub-module of the system module.

To initiate an Aurora Imaging Library I/O command list on Zebra Indio using the MsysIo module, the Aurora Imaging Library system identifier should be of **M_SYSTEM_INDIO** that was allocated using MsysAlloc(). For more information, refer to Host system column in the MsysIo command reference.

Miscellaneous user guide information for Zebra Indio

This section includes additional user guide information for Zebra Indio. The information found in this section might be a reiteration of content previously documented.

Zebra Indio has sixteen discrete signals (8 inputs and 8 outputs) on the auxiliary I/O interface. All 8 auxiliary input signals support user input or quadrature input and have interrupt-generation capabilities. All 8 auxiliary output signals support user output, timer output, or outputting the state of an I/O command register bit.

Zebra Indio also integrates a network port. The port features an Intel I210 Ethernet controller with support for Power-over-Ethernet (PoE). This port is intended for communicating through Aurora Imaging Library using either the EtherNet/IP, Modbus over TCP/IP, or PROFINET protocols. PROFINET communication is hardware-assisted and does not require the installation of the Intel Network Adaptor Driver. EtherNet/IP and Modbus over TCP/IP communication requires the Intel Network Adaptor Driver. It is available from downloadcenter.intel.com.

Zebra Indio's network port can also be used for interfacing with a GigE Vision camera. This requires an Aurora Imaging Library installation with the Aurora Imaging Library System for GigE Vision selected. The use of the port for GigE Vision also requires installing the Intel Network Adaptor Driver, which is available from downloadcenter.intel.com. Its optimum use for GigE Vision requires additional configuration, which is explained in the Configuring your GigE Vision camera and Gigabit Ethernet network adapter section of the Zebra GigE Vision driver chapter in the Hardware-specific Notes.

Hardware-assisted PROFINET LED is supported on Zebra Indio. It is off if hardware-assisted PROFINET is disabled in Aurora Imaging Configurator. The LED is red if hardware-assisted PROFINET is enabled, but no connection is established with PLC. If the connection is established, it is off.

To use Zebra Indio and have access to all its auxiliary I/O signals, you must allocate it as an Aurora Imaging Library Indio system (**M_SYSTEM_INDIO**) using MsysAlloc(). This allocation opens communication with Zebra Indio and allows Aurora Imaging Library to use its resources. More than one process (executable) on Zebra Indio can allocate an Aurora Imaging Library Indio system.

The Aurora Imaging Library industrial communication module allows only one PROFINET device to be allocated. Therefore, if you have more than one Zebra Indio board installed in a system, you can only enable one for hardware-assisted PROFINET communication through the Communication section of the Aurora Imaging Configurator utility.

When selecting Zebra Indio as the default system in Aurora Imaging Configurator, the Aurora Imaging Library examples that use MdigAlloc() will produce errors since the Zebra Indio system does not directly support video capture. If you need to grab using the Gigabit Ethernet port on Zebra Indio, you must select GigE Vision as the default system.

Note that **M_IO_FORMAT** in MsysInquire() is obsolete.

Zebra Iris GTX

This section discusses features of Aurora Imaging Library that are particular to the Zebra Iris GTX family of smart cameras and ways that optimize their performance. Zebra Iris GTX information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Iris GTX overview
  - Summary of Zebra Iris GTX features
- Using Zebra Iris GTX with Aurora Imaging Library
  - Using the Advanced I/O Engine with Zebra Iris GTX
  - Using EtherNet/IP, Modbus, CC-Link, or PROFINET
  - Using the analog intensity control signal with Zebra Iris GTX
  - Performing Bayer color conversion in hardware
  - Zebra Iris GTX exposure mode
- Controlling your lighting device indirectly
  - Using either a grab trigger or a continuous grab to start your lighting device
  - Controlling your lighting controller and/or the LED lighting device
- Automatically scheduling an output upon the occurrence of an event
- Auto-focusing the Varioptic lens
- Zebra Iris GTX connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Miscellaneous reference information for Zebra Iris GTX
  - Mdig
    - MdigControl/Inquire

Zebra Iris GTX overview

Zebra Iris GTX is a family of smart cameras that combine image acquisition, embedded processing, and display. Zebra Iris GTX includes an Ethernet port (100/1000 BaseT), which provides connection for supported industrial protocols, including: PROFINET, EtherNet/IP, CC-Link, and Modbus. For PROFINET support, Zebra Iris GTX includes a PROFINET Engine; on Zebra Iris GTX, the PROFINET service comes preinstalled (but not enabled). Zebra Iris GTX also features an Advanced I/O Engine, which provides use of auxiliary I/O signals, an analog intensity control signal, general timers, an exposure timer, a strobe timer, a quadrature decoder, and an I/O command list (with 2 reference latches).

Zebra Iris GTX uses a CMOS image sensor, which supports an asynchronous externally triggered global shutter.

The primary difference between the members of the Zebra Iris GTX family are: the sensor chip size, the effective resolution, color, and frame rate.

Zebra Iris GTX family members	Sensor chip size [1]	Effective resolution	Color/monochrome	Frame rate
GTX2000	1/2.2"-type	1920 x 1200 pixels	Monochrome	Up to 70.7 fps
GTX2000C	1/2.2"-type	1920 x 1200 pixels	Color	Up to 17.6 fps
GTX5000	2/3"-type	2592 x 2048 pixels	Monochrome	Up to 41.7 fps
GTX5000C	2/3"-type	2592 x 2048 pixels	Color	Up to 10.4 fps
GTX8000	1/1.1"-type	4096 x 2160 pixels	Monochrome	Up to 39.6 fps
GTX8000C	1/1.1"-type	4096 x 2160 pixels	Color	Up to 9.9 fps
GTX12000	1"-type	4096 x 3072 pixels	Monochrome	Up to 28 fps
GTX12000C	1"-type	4096 x 3072 pixels	Color	Up to 7.0 fps
GTX16000	1.1"-type	4000 x 4000 pixels	Monochrome	Up to 21.6 fps
GTX16000C	1.1"-type	4000 x 4000 pixels	Color	Up to 5.4 fps

[1] Sensor chips are always measured on the diagonal.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/iris-gtx-info. Click on the **Documentation** tab to access it.

Summary of Zebra Iris GTX features

The following table outlines the features currently available on each member of the Zebra Iris GTX family.

Features	Zebra Iris GTX
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature()	No
On-board memory	4 Gbytes LPDDR4x SDRAM
Number of auxiliary signals	7 (4 in, 3 out)
PROFINET Engine	Yes
Number of general timers	8
Number of strobe timers	1
Number of exposure timers	1
Number of quadrature decoders for input from linear or rotary encoders	1
Number of I/O command lists	1
Number of reference latches per I/O command list	2
Digitizer LUT	Yes
Asynchronous reset mode support	Yes

Using Zebra Iris GTX with Aurora Imaging Library

To use Zebra Iris GTX, you must allocate it as an Aurora Imaging Library Iris GTX system (M_SYSTEM_IRIS_GTX), using MsysAlloc(). This allocation opens communication with Zebra Iris GTX and allows Aurora Imaging Library to use its resources. To grab images from your Zebra Iris GTX, you must allocate a single digitizer, using MdigAlloc() with M_DEV0. You can allocate an Aurora Imaging Library Iris GTX system for your smart camera in multiple processes (executables). However, only one digitizer can be allocated at a time for your Zebra Iris GTX's image sensor (across all processes).

To set up grabbing with triggers, use MdigControl() with M_GRAB_TRIGGER....

Using the Advanced I/O Engine with Zebra Iris GTX

Zebra Iris GTX has an Advanced I/O Engine that controls the auxiliary I/O interface. The Advanced I/O Engine includes 7 auxiliary signals (4 inputs and 3 outputs), 1 analog intensity control signal, 8 general timers, 1 exposure timer, 1 strobe timer, 1 quadrature decoder, and 1 I/O command list (with 2 reference latches). For more information on some of these features, refer to Chapter 56: I/O signals and communicating with external devices.

> Note, to program the exposure timer, use MdigControl() with M_EXPOSURE...; whereas, to control the strobe timer, use MdigControl() with M_TIMER... + M_TIMER_STROBE.

Using EtherNet/IP, Modbus, CC-Link, or PROFINET

You can use the Gigabit Ethernet port for communication with external devices using the EtherNet/IP, Modbus (over TCP/IP), CC-Link, or PROFINET industrial protocol. For PROFINET communication, the port has access to the PROFINET Engine; the interface of this engine is recognized as a second network device with its own IP and MAC settings (when the PROFINET service is enabled) even though it shares the same LAN connection. By default, PROFINET is disabled. To enable it, open the Aurora Imaging Configurator utility, expand the **Communication** tab, select the **PROFINET** page, and add a PROFINET protocol instance; note that instance names are case sensitive. Once the protocol instance is added, the status of the protocol instance should be **Ready**. If the status is **Waiting NIC**, there is no cable connected to the Ethernet port.

For more information, see the Steps to perform industrial communication section of Chapter 57: Industrial communication.

Using the analog intensity control signal with Zebra Iris GTX

Zebra Iris GTX has an analog intensity control signal that provides a slow changing analog intensity (dimming) control signal from 0-10 VDC. This control signal can be used to set the intensity for a light controller (such as, an Advanced Illumination inline control system, a Smart Vision Lights brick light, or similar device). This signal should not be used to draw any current or drive power; it should only be used as a reference voltage for dimming control. To set the intensity of the light controller, connect the lighting controller's intensity pin directly to the analog intensity control pin and use MdigControl() with M_LIGHTING_BRIGHT_FIELD. To see an example of how to connect your light controller to the analog intensity control signal, refer to the Zebra Iris GTX Hardware and Installation manual.

Performing Bayer color conversion in hardware

Zebra Iris GTX color smart cameras use a sensor with a Bayer color filter (as specified by the DCF); when grabbing, the cameras perform Bayer color conversion in hardware before saving the images. If the images require white balancing, Zebra Iris GTX can perform this automatically in hardware if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance

coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

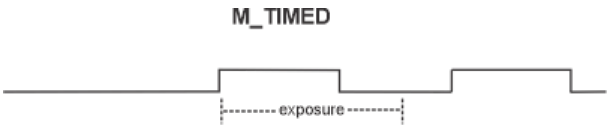
If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_ENABLE.

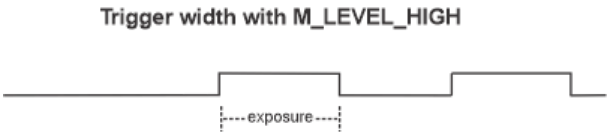
The M_BAYER... control types can only be used when grabbing from a color version of Zebra Iris GTX; otherwise, an error will be generated.

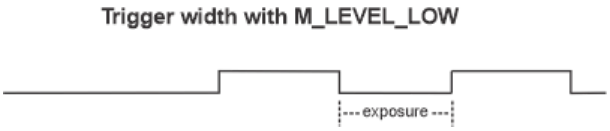
Zebra Iris GTX exposure mode

On Zebra Iris GTX, you can specify that the exposure duration is set with a timer or the width of the trigger signal, using MdigControl() with M_EXPOSURE_MODE set to M_TIMED or M_TRIGGER_WIDTH, respectively. When set to M_TIMED, the exposure duration will last as long as the time specified with M_EXPOSURE_TIME.

M_TRIGGER_WIDTH can only be used when M_GRAB_TRIGGER_ACTIVATION is set to M_LEVEL_HIGH or M_LEVEL_LOW. When set to M_LEVEL_HIGH, the exposure will be triggered upon the rising edge of the trigger signal, and will remain active while the trigger signal is high. When set to M_LEVEL_LOW, the exposure will be triggered upon the falling edge of the trigger signal, and will remain active while the trigger signal is low.

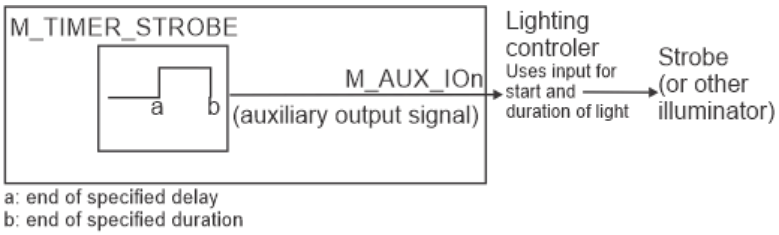






Controlling your lighting device indirectly

The auxiliary output signals of your Zebra Iris GTX can be linked to the exposure signal to control a lighting device that illuminates an object when the camera takes a picture.



To indirectly control a lighting device with the strobe timer signal, perform the following:

1. Connect a lighting controller to an auxiliary output signal.
2. Set the source of the auxiliary output signal to the strobe timer, using MsysControl() with M_IO_SOURCE set to M_TIMER_STROBE.
3. To control the strobe timer, use MdigControl() with M_TIMER... + M_TIMER_STROBE.

For more information about setting up a strobe timer, refer to the Timers and coordinating events section of Chapter 56: I/O signals and communicating with external devices.

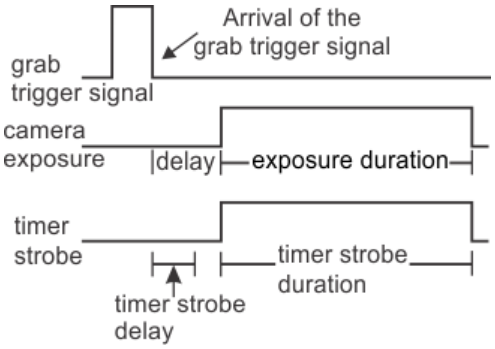
Using either a grab trigger or a continuous grab to start your lighting device

Whenever using a lighting device, it will always start cold (that is, completely discharged) and take a certain amount of time to reach the specified intensity.

How you account for this cold period when setting up the strobe timer, depends on how you are grabbing:

- **Continuous (or almost continuous) grabs**. If you are grabbing a sequence of images with a negligible amount of delay between one image and the next, you only have to deal with the lighting device being cold at the very beginning of the grab sequence. In this case, you can trigger the strobe timer when the camera starts exposing its sensor (using M_TIMER_TRIGGER_SOURCE set to M_EXPOSURE_START).
- **Triggered grabs**. If you are grabbing images based on a trigger that might arrive after a non-negligible delay, you will have to account for the lighting device recharging before reaching the specified intensity for each grab. In this case, you should trigger the strobe timer upon the grab trigger signal (using M_TIMER_TRIGGER_SOURCE set to M_GRAB_TRIGGER).

Typically, the lighting device should be illuminated for the duration of the camera's exposure time. Unlike with other products, when using the grab trigger signal to trigger the strobe timer, the specified duration of the strobe timer's output signal (set using M_TIMER_DURATION + M_TIMER_STROBE) specifies how long the timer should remain active during the exposure period (and therefore how long the lighting device should remain active during the exposure period). The active period of the strobe timer's output will actually start after the specified delay, but the count for how long it should remain active only starts when the camera starts exposing its image sensor. Once counting begins, the signal will typically be active for the specified duration. If the duration of the strobe timer is greater than the exposure time, the strobe timer's output signal will end when the exposure time ends (set using M_EXPOSURE_TIME).



This allows for hardware-specific variances to occur between when the grab trigger signal arrives and when the exposure of the camera's sensor actually starts.

The maximum amount of time it takes for your lighting device to arrive at the required intensity must be probed. This is because the amount of time is hardware-specific, and can vary greatly based on the type of lighting device, the period of the lighting device's reduced activity (caused by significant delays between grab triggers, or long processing delays during MdigProcess()) or inactivity (such as, between individual grabs with MdigGrab()).

When the exposure ends and the strobe timer's stops outputting an active signal, the lighting device begins to discharge and grow cold.

Controlling your lighting controller and/or the LED lighting device

The following is an example of how to configure a grab using the camera's exposure and the strobe timer.

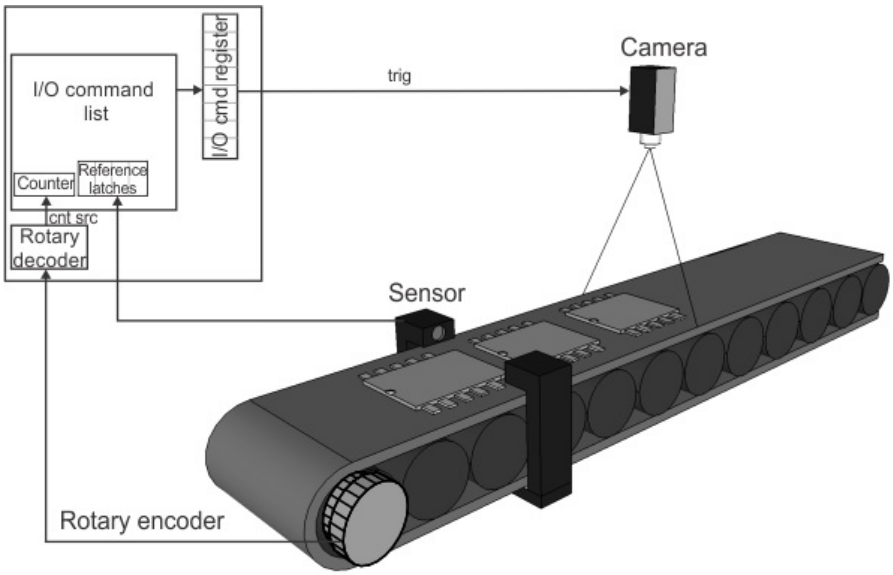
```
AIL_ID AilApplication,  /* Application identifier.  */
       AilSystem,       /* System identifier.       */
       AilDisplay,      /* Display identifier.      */
       AilDigitizer,    /* Digitizer identifier.    */
       AilImage;        /* Image buffer identifier. */

AIL_DOUBLE ExposureTime = 9000 * 1000;
AIL_DOUBLE ExposureTimeDelay = ExposureTime / 3;
AIL_DOUBLE StrobeLength = ExposureTime + ExposureTimeDelay;
AIL_DOUBLE StrobeTimeDelay = 0;
AIL_DOUBLE StrobeLevel = 128;

//Allocate defaults.
MappAllocDefault(M_DEFAULT, &AilApplication, &AilSystem, &AilDisplay,
                 &AilDigitizer, &AilImage);
//Set the grab trigger source to be an external signal.
MdigControl(AilDigitizer, M_GRAB_TRIGGER_SOURCE, M_AUX_IO8);

//Specify to trigger the grab on the falling edge of the grab trigger signal.
//Note that this could be set to the default, which is the same as M_EDGE_RISING.
MdigControl(AilDigitizer, M_GRAB_TRIGGER_ACTIVATION, M_EDGE_FALLING);

//Setup the exposure settings (in nanoseconds)
MdigControl(AilDigitizer, M_EXPOSURE_TIME, ExposureTime);
MdigControl(AilDigitizer, M_EXPOSURE_DELAY, ExposureTimeDelay);

//Invert the timer output.
MdigControl(AilDigitizer, M_TIMER_OUTPUT_INVERTER+M_TIMER_STROBE, M_ENABLE);

//Sets the grab trigger source signal as the trigger source.
MdigControl(AilDigitizer, M_TIMER_TRIGGER_SOURCE+M_TIMER_STROBE, M_GRAB_TRIGGER);

//Sets a delay before the active portion of the stobe (in nanoseconds).
MdigControl(AilDigitizer, M_TIMER_DELAY+M_TIMER_STROBE, StrobeTimeDelay);

//Sets the duration of the active portion of the strobe (in nanoseconds) during the exposure period.
//If set to M_INFINITE, the signal will always be ON.
MdigControl(AilDigitizer, M_TIMER_DURATION+M_TIMER_STROBE, StrobeLength);

//Enable timer 2 to output a pulse-train strobe signal and CCS output.
MdigControl(AilDigitizer, M_TIMER_STATE+M_TIMER_STROBE, M_ENABLE);

//Enable the strobe signal on M_AUX_IO4.
MsysControl(AilSystem, M_IO_SOURCE + M_AUX_IO4, M_TIMER_STROBE);

//Enable the grab trigger.
MdigControl(AilDigitizer, M_GRAB_TRIGGER_STATE, M_ENABLE);

//Grab continuously.
MdigGrabContinuous(AilDigitizer, AilImage);

//Wait.
MosGetch();

//Stop continuous grab.
MdigHalt(AilDigitizer);

//Disable the timer.
MdigControl(AilDigitizer, M_TIMER_STATE+M_TIMER_STROBE, M_DISABLE);

//Disable the grab trigger.
MdigControl(AilDigitizer, M_GRAB_TRIGGER_STATE, M_DISABLE);

//Free the defaults.
MappFreeDefault(AilApplication, AilSystem, AilDisplay, AilDigitizer, AilImage);
```

Automatically scheduling an output upon the occurrence of an event

Zebra Iris GTX supports I/O command lists. An I/O command list is an internal container for Aurora Imaging Library I/O commands. For information on I/O command lists, refer to the Using I/O command lists section of Chapter 56: I/O signals and communicating with external devices.

Besides supporting scheduling I/O commands on an individual basis, Zebra Iris GTX supports automatically registering an I/O command every time a specified latch is triggered, and using that latch's value as a reference. To do so, use MsysIoCommandRegister() with an operation, M_AUTO_REGISTER, and M_LATCHn.

In this example, parts are moving along a conveyor belt, but are fixed in their position on the conveyor belt. The parts are always taking 100 rotary steps forward before reaching a camera, which will always acquire an image of the parts. For simplicity, the conveyor belt can only move forward and the camera is an areascan camera.



An I/O command list is used to schedule the acquisition of the image. The rotary decoder output is used as the counter source of the I/O command list so that each time the rotary decoder counts 100 steps (distance that the part travels) after the sensor detects a part, an image is acquired.

```
// M_AUX_IO14: Receives rotary encoder bit 0
// M_AUX_IO15: Receives rotary encoder bit 1
// M_AUX_IO8 (input): Object detector
// M_AUX_IO0 (output): trigger to the camera, 100 positions after object detector
#define CAMERA_TRIGGER_OFFSET 100   // in rotary encoder steps

AIL_INT MFTYPE ProcessingFunction(AIL_INT HookType, AIL_ID HookId, void* HookDataPtr);

typedef struct _HOOK_PARAM
{
   AIL_ID AilSystem;
   AIL_ID CmdListId;
} HOOK_PARAM, *PHOOK_PARAM;
```

```
  // Setup triggered grab.
  MdigControl(AilDigitizer, M_GRAB_TRIGGER_SOURCE, M_AUX_IO0);
  MdigControl(AilDigitizer, M_GRAB_TRIGGER_ACTIVATION, M_EDGE_RISING);
  MdigControl(AilDigitizer, M_GRAB_TRIGGER_STATE, M_ENABLE);

  // Setup the rorary decoder; for simplicty, we assume the conveyor cannot move backwards.
  MsysControl(SysId, M_ROTARY_ENCODER_BIT0_SOURCE + M_ROTARY_ENCODER1, M_AUX_IO14);
  MsysControl(SysId, M_ROTARY_ENCODER_BIT1_SOURCE + M_ROTARY_ENCODER1, M_AUX_IO15);
  MsysControl(SysId, M_ROTARY_ENCODER_OUTPUT_MODE + M_ROTARY_ENCODER1, M_STEP_FORWARD);
  MsysControl(SysId, M_ROTARY_ENCODER_STATE + M_ROTARY_ENCODER1, M_ENABLE);

  // Link the grab trigger to an I/O command register bit.
  MsysControl(SysId, M_IO_SOURCE + M_AUX_IO0, M_IO_COMMAND_LIST1 + M_IO_COMMAND_BIT0);

  // Allocate an I/O command list where I/O commands are scheduled based on rotary decoder 1's output signal.
  CmdListId = MsysIoAlloc(SysId, M_IO_COMMAND_LIST1, M_IO_COMMAND_LIST, M_ROTARY_ENCODER1, M_NULL);
  if (CmdListId!= M_NULL)
  {

      // Latch the counter upon detection of the object
      MsysIoControl(CmdListId, M_REFERENCE_LATCH_TRIGGER_SOURCE+M_LATCH1, M_AUX_IO8);
      MsysIoControl(CmdListId, M_REFERENCE_LATCH_ACTIVATION+M_LATCH1, M_EDGE_RISING);
      MsysIoControl(CmdListId, M_REFERENCE_LATCH_STATE+M_LATCH1, M_ENABLE);

      // Every one hundred decoder steps after latch 1 is triggered, two I/O commands
      // are automatically added to the I/O command list to change register bit 0,
      // such that M_AUX_IO0 outputs an active high pulse.
      // Since M_AUX_IO0 is the grab trigger source, a frame will always be grabbed 100
      // decoder steps after latch 1 is triggered.
      MsysIoCommandRegister(CmdListId, M_PULSE_HIGH+M_AUTO_REGISTER, M_LATCH1, CAMERA_TRIGGER_OFFSET, 1, M_IO_

      // Start the grabbing and processing.
      // The processing function is called each time a frame is grabbed.
      MdigProcess(AilDigitizer, AilGrabBufferList, AilGrabBufferListSize,
          M_START, M_DEFAULT, ProcessingFunction, &HookParam);

      // Print a message and wait for a key press before stopping the processing.
      MosPrintf(AIL_TEXT("Press <Enter> to stop.                    \n\n"));
      MosGetch();

      HookParam.AilSystem = SysId;
      HookParam.CmdListId = CmdListId;

      // Stop the grabbing and processing.
      MdigProcess(AilDigitizer, AilGrabBufferList, AilGrabBufferListSize,
          M_STOP, M_DEFAULT, ProcessingFunction, &HookParam);

      // Release the auto-register, so that, if latch 1 is triggered, it will not cause
      // an I/O command to be added.
      MsysIoCommandRegister(CmdListId, M_PULSE_HIGH + M_AUTO_REGISTER_CANCEL, M_LATCH1, CAMERA_TRIGGER_OFFSET,

  }

// Free the I/O command list.
  MsysIoFree(CmdListId);
```

```
// Defines the processing function, called by MdigProcess()
AIL_INT MFTYPE ProcessingFunction(AIL_INT HookType, AIL_ID HookId, void* HookDataPtr)
   {
   PHOOK_PARAM pHookParam = (PHOOK_PARAM)HookDataPtr;
   AIL_ID ModifiedBufferId;

   /* Retrieve the AIL_ID of the grabbed buffer. */
   MdigGetHookInfo(HookId, M_MODIFIED_BUFFER+M_BUFFER_ID, &ModifiedBufferId);

   /* Execute the processing  */
   // ... PUT YOUR PROCESSING FUNCTION HERE

   return 0;
   }
```

Auto-focusing the Varioptic lens

Zebra Iris GTX supports using the Varioptic Caspian C-39NO-160-I2C or C-39N0-250-I2C lens. This lens can be auto-focused using MdigFocus(). Note that this function is only available in the full version of Aurora Imaging Library.

The Varioptic Caspian C-39NO-160-I2C lens can also be controlled using MdigControl() with M_FOCUS + M_WAIT to wait a predetermined amount of time for the camera to focus before calling the user-defined function associated with MdigFocus().

The Varioptic Caspian C-39NO-160-I2C lens has a range of 1024 lens motor positions. When using this or any other type of auto-focusing lens, to determine the minimum and maximum number of lens motor positions, use MdigInquire() with M_FOCUS + M_MIN_VALUE, or M_FOCUS + M_MAX_VALUE, respectively.

Zebra Iris GTX connectors and signal names

This section serves as a reference to match Zebra Iris GTX's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers.

Auxiliary I/O signals can have one or more functionalities (for example, trigger input, user input or user output, depending on the signal). Their possible functionalities are listed in their description in the pinout table below.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. To set/inquire the routing, or state of an auxiliary signal, use MsysControl() / MsysInquire() with M_IO_... control/inquire types, respectively. Whereas, the Aurora Imaging Library function that you should use to act upon an input signal or setup the source of an output signal depends on the functionality. For example, you can use MsysControl() with M_USER_BIT_... control types, MdigControl() with M_GRAB_TRIGGER_... control types, and you can use their corresponding MsysInquire() and MdigInquire() inquire types to inquire them.

To control a timer associated with the strobe signal, use MdigControl() with M_TIMER_... + M_TIMER_STROBE control types. To control the exposure timer, use MdigControl() with M_EXPOSURE.... To control the 8 general system timers available, use MsysControl() with M_TIMER_... + M_TIMERn, where *n* is a number from 1 to 8.

For a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

Only the following connector has auxiliary I/O signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Digital I/O and power connector	M12-12P		This connector is an M12 12-pin female connector. It is used to transmit and receive digital auxiliary I/O signals and provide power to your Zebra Iris GTX.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary I/O signals and the dedicated trigger signal with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

System	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	

		Hardware manual signal name	
System			Indicates the following.
	I/O #: M_AUX_IO0		Opto-isloated auxiliary signal 0 (output), which supports: user output, timer output, strobe timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: M12-12P Pin: 10+
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0;
		Hardware manual signal name	AUX_OPTOIND_OUT0
	I/O #: M_AUX_IO1		Opto-isloated auxiliary signal 1 (output), which supports: user output, timer output, strobe timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: M12-12P Pin: 12+
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT1;
		Hardware manual signal name	AUX_OPTOIND_OUT1
	I/O #: M_AUX_IO2		Opto-isloated auxiliary signal 2 (output), which supports: user output, timer output, strobe timer output, and outputting the state of an I/O command register bit.
		Pin information	Connector: M12-12P Pin: 3+
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT2;
		Hardware manual signal name	AUX_OPTOIND_OUT2
	I/O #: M_AUX_IO3		Opto-isolated auxiliary signal 3 (input), which supports: User-defined signal 0 (input 1 of 4), acquisition trigger, timer trigger source, and rotary encoder bit source.
		Pin information	Connector: M12-12P Pin: 5+
		Direction	Input
		Hardware manual signal name	AUX(TRIG)_OPTOIND_IN3
	I/O #: M_AUX_IO4		Opto-isolated auxiliary signal 4 (input), which supports: User-defined signal 1 (input 2 of 4), acquisition trigger, timer trigger source, and rotary encoder bit source.

		Pin information	Connector: M12-12P Pin: 9+
		Direction	Input
		Hardware manual signal name	AUX_OPTOIND_IN4
	I/O #: M_AUX_IO5		Opto-isolated auxiliary signal 5 (input), which supports: User-defined signal 2 (input 3 of 4), acquisition trigger, timer trigger source, and rotary encoder bit source.
		Pin information	Connector: M12-12P Pin: 7+
		Direction	Input
		Hardware manual signal name	AUX_OPTOIND_IN5
	I/O #: M_AUX_IO6		Opto-isolated auxiliary signal 6 (input), which supports: User-defined signal 3 (input 4 of 4), acquisition trigger, timer trigger source, and rotary encoder bit source.
		Pin information	Connector: M12-12P Pin: 8+
		Direction	Input
		Hardware manual signal name	AUX_OPTOIND_IN6

Miscellaneous reference information for Zebra Iris GTX

This section includes additional command reference information for Zebra Iris GTX. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Mdig.

Mdig

This section presents information about Zebra Iris GTX and the digitizer module.

MdigControl/Inquire

The following lists additional Iris GTX support when using MdigControl() / MdigInquire().

- The minimum supported exposure time that can be set using MdigControl() with **M_EXPOSURE_TIME** is 50 microseconds for monochrome sensors. The inquire of **M_EXPOSURE_TIME + M_MIN_VALUE** returns 50 microseconds.
- The Zebra Iris GTX supports **M_GRAB_SCALE_X** for all values from 1 to 1/16 and **M_GRAB_SCALE_Y** for 1 and 1/2.

  The value for **M_SOURCE_SIZE_Y** or **M_SOURCE_OFFSET_Y** must be a multiple of 4. Also, only **M_SOURCE_SIZE_Y** has an effect on the frame rate or grab period. The **M_SOURCE_SIZE_X** does not have an effect on the grab period.
- The value for **M_SOURCE_SIZE_Y** or **M_SOURCE_OFFSET_Y** must be a multiple of 4.

  M_SOURCE_SIZE_Y has an effect on the frame rate or grab period, whereas the **M_SOURCE_SIZE_X** does not have an effect on the grab period.
- **M_SOURCE_SIZE_Y** can be set to 4 only if **M_GRAB_SCALE_Y** is set to 1.

> You should avoid grabbing using the Zebra Iris GTX with a 10-bit grab resolution or allocating a grab buffer with the **M_MONO16** attribute.

Zebra Radient eV-CL

This section discusses features of Aurora Imaging Library that are particular to Zebra Radient eV-CL family of boards and ways that optimize their performance. Zebra Radient eV-CL family specific information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Radient eV-CL overview
  - Summary of Zebra Radient eV-CL features
  - Note on nomenclature
- Using Zebra Radient eV-CL with Aurora Imaging Library
  - Grabbing and the buffer formats to use for each camera pixel format on Zebra Radient eV-CL
  - Performing Bayer color conversion in hardware
  - Using frame burst with a multi-frame buffer
  - Using on-board flat-field correction
- Allocating independent Aurora Imaging Library digitizers on Zebra Radient eV-CL
  - Zebra Radient eV-CL
- Minimum latency and grabbing all frames
  - Detecting missed frames
  - Efficiently copying an on-board buffer to a destination buffer in Host memory
  - Efficiently converting buffer formats while copying
- Using Zebra Radient eV-CL data latches
  - Steps to configure and retrieve data from a data latch
  - Triggering the data latch
  - Data latch limitations
  - Retrieving data latch information
  - Data latch example
- Zebra Radient eV-CL utilities and tools
  - Usage Meter utility
  - Zebra Radient eV-CL Bench utility
  - PCIe Information tool
  - System Monitor tool
  - Zebra Radient eV-CL Deserializer setup tool
- Zebra Radient eV-CL SB/DB/QB connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Radient eV-CL SF/DF connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Radient eV-CL board flow diagrams
  - Zebra Radient eV-CL
- Miscellaneous reference information for Zebra Radient eV-CL
  - Mdig
    - MdigControl/MdigInquire
    - MdigControlFeature/MdigInquireFeature
    - MdigGetHookInfo
- Miscellaneous user guide information for Zebra Radient eV-CL
  - Supported features and standards
  - Particularities
  - Code snippets

Zebra Radient eV-CL overview

Zebra Radient eV-CL is a family of high-performance PCIe frame grabbers:

- **Zebra Radient eV-CL**. Zebra Radient eV-CL supports image capture from up to 4 video sources that use the Camera Link (CL) communication standard, depending on the version. There are 7 versions of Zebra Radient eV-CL available: Zebra Radient eV-CL SB, Zebra Radient eV-CL x4 DB, Zebra Radient eV-CL x4 SF, Zebra Radient eV-CL x8 SF, Zebra Radient eV-CL x8 DB, Zebra Radient eV-CL DF, and Zebra Radient eV-CL QB.

Zebra Radient eV-CL supports frame and line-scan, monochrome and color video sources. The color video sources can be RGB video sources or video sources with a Bayer color filter. Zebra Radient eV can decode Bayer color-encoded images and perform color space conversions while transferring the images to the Host. In addition, it can vertically or horizontally flip them and perform subsampling.

Zebra Radient eV-CL supports up to 4 Base or 2 Full/80-bit mode Camera Link connections, depending on the version of the board, as well as providing Power-over-Camera Link (PoCL) support.

> You can access the latest version of the Installation and Hardware Reference at zebra.com/radient-ev-cl-info.

Summary of Zebra Radient eV-CL features

The following table outlines the features currently available for Zebra Radient eV-CL.

	Zebra Radient eV-CL
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature()	Yes [1]
On-board memory [2]	256 Mbytes DDR3 SDRAM (Zebra Radient eV-CL SB) 512 Mbytes DDR3 SDRAM (Zebra Radient eV-CL x4 DB) 512 Mbytes DDR3 SDRAM (Zebra Radient eV-CL x4 SF) 1 Gbytes DDR3 SDRAM (Zebra Radient eV-CL x8 SF) 1 Gbytes DDR3 SDRAM (Zebra Radient eV-CL x8 DB) 1 Gbytes DDR3 SDRAM (Zebra Radient eV-CL DF) 1 Gbytes DDR3 SDRAM (Zebra Radient eV-CL QB)
Processing FPGA	No

Digitizer LUTs	8- and 10-bit (1/digitizer) or 12-bit (shared) [3]
Asynchronous camera reset support	Yes
Flat-field correction	Yes [4]
Acquisition section	Camera Link
Number of acquisition paths	4 (QB), 2 (DB, DF), 1 (SF, SB)
Number of auxiliary signals	32 (4 in, 1 out, and 3 in/out per I/O connector)
Number of timers	2/digitizer
Number of quadrature decoders for input from linear or rotary encoders	1/digitizer
Number of supported triggers	4/digitizer
Number of CoaXPress trigger signals	None
Number of UARTS	1
Number of data latches	16/digitizer (allocated using M_DEV0 or M_DEV1 only) [5]

[1] Only if the camera supports GenICam. For more information, refer to the Using GenICam with Camera Link cameras subsection of the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

[2] Zebra Radient eV has 128 Mbytes of acquisition memory mapped onto the PCIe bus. You can use a Host pointer to access this memory, or you can access it directly from another DMA capable PCIe device; this memory is referred to as shared memory.

[3] When performing a color grab, one 12-bit LUT is used.

[4] Not supported on Zebra Radient eV-CL SB.

[5] Not supported on Zebra Radient eV-CL SB.

Note on nomenclature

This manual refers to all versions of the board as Zebra Radient eV-CL. Zebra Radient eV-CL refers to Zebra Radient eV-CL SB, Zebra Radient eV-CL SF, Zebra Radient eV-CL DF, Zebra Radient eV-CL DB, and Zebra Radient eV-CL QB versions. When necessary, this manual distinguishes between the Zebra products using their full names.

Using Zebra Radient eV-CL with Aurora Imaging Library

To use Zebra Radient eV-CL, you must allocate it as an Aurora Imaging Library Radient eV-CL system (using MsysAlloc() with M_SYSTEM_RADIENTEVCL). This allocation opens communications with your Zebra Radient eV-CL and allows Aurora Imaging Library to use its resources.

You can allocate an Aurora Imaging Library Radient eV-CL system for your board in multiple processes (executables). If more than one process allocates a digitizer for the same acquisition path, grabs from the different processes will be queued in the order that they arrive (first in, first out).

Zebra Radient eV-CL features at least one quadrature decoder that can decode input from a linear or rotary encoder with quadrature output. For more information on this feature, refer to the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. This chapter also provides information on how to configure and use the I/O signals.

Zebra Radient eV-CL provides a GenICam CLProtocol driver to communicate with your Camera Link camera using the GenICam SFNC standard. For information regarding using Aurora Imaging Library to access the GenICam SFNC-compatible features of your camera, see the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Grabbing and the buffer formats to use for each camera pixel format on Zebra Radient eV-CL

For Zebra Radient eV-CL, the following table outlines which Aurora Imaging Library buffer format to use with each supported camera pixel format. Note that these are the officially supported camera pixel formats for Zebra Radient eV-CL.

Camera pixel format [1]	Aurora Imaging Library buffer format (type and attribute)	Operation
Mono8, Mono10, Mono12, Mono14, Mono16, Mono32	1-band 8-bit, 1-band 16-bit, 1-band 32-bit, 3-band 8-bit [2] , 3-band 16-bit [3]	Data will automatically be bit-shifted to have most-significant bits (MSB) in destination buffer
BayerXX8, BayerXX10, BayerXX12, BayerXX14, BayerXX16 where XX = GR, GB, BG or RG	1-band 8-bit, 1-band 16-bit	Demosaicing followed by RGB to Y (bit shift for MSB)
	3-band 8-bit, 3-band 16-bit	Demosaicing (bit shift for MSB)
	3-band YUV16 + M_PACKED	Demosaicing followed by RGB to YUV
	1-band 8-bit	RGB to Y

RGB8, RGBA8,	3-band 8-bit	Direct copy
	3-band YUV16 + M_PACKED	RGB to YUV
RGB10, RGBA10, RGB12, RGBA12, RGB14, RGBA14, RGB16, RGBA16	1-band 8-bit	RGB to Y (bit shift for MSB)
	1-band 16-bit	RGB to Y
	3-band 16-bit M_PLANAR	Packed destination buffers are not supported
YUV422_8, YCbCr601_422_8, YCbCr709_422_8	1-band 8-bit	Only Y band
	3-band YUV16 + M_PACKED	Direct copy

[1] All of the values available for RGB are also available in BGR format.

[2] 3-band 8-bit buffers can be in any of the following formats: M_RGB24 + M_PLANAR, M_RGB24 + M_PACKED, M_BGR24 + M_PACKED, or M_BGR32 + M_PACKED.

[3] 3-band 16-bit buffers can be in either an M_RGB48 + M_PLANAR or M_RGB48 + M_PACKED format.

Performing Bayer color conversion in hardware

When Zebra Radient eV-CL boards grab color images from a video source with a Bayer color filter (as specified by the DCF), it performs Bayer color conversion in hardware, as it transfers the images to the Host. If the images require white balancing, Zebra Radient eV-CL boards can perform this automatically if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_DISABLE.

The M_BAYER... control types of MdigControl() can only be used when grabbing from a camera that has a Bayer color filter (as specified by the DCF); otherwise, an error will be generated.

Using frame burst with a multi-frame buffer

All version of Zebra Radient eV-CL support frame burst technology. This technology allows you to grab a group of sequential frames into a multi-frame buffer with one grab command (MdigGrab(), or one grab of MdigProcess()); the defined number of frames are stored contiguously in the same buffer. The end-of-grab event only occurs once the entire group of frames has been grabbed, reducing the number of events that need to be handled. This is useful in cases where you have a high frame rate and need to ensure that no frames are missed. Note that a user-defined function hooked to the end-of-grab event (MdigProcess(), or MdigHookFunction() with M_GRAB_END) is executed

only once the entire group of frames has been grabbed. For information on creating a multi-frame image buffer to store sequential frames, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers.

Using on-board flat-field correction

Zebra Radient eV-CL supports flat-field correction on-board without Host intervention. Images taken in environments with uneven lighting can affect the quality of your processing. To correct this, an on-board flat-field correction can be applied to images so that the intensity across the image is even before processing begins. To use flat-field correction, you need to specify an on-board buffer that contains the gain values, and optionally another that contains the offset values, using MdigControl() with M_SHADING_CORRECTION_GAIN_ID and M_SHADING_CORRECTION_OFFSET_ID, respectively. The buffers should be the same size as the grabbed image, and the gain values should be in the specified fixed point format (M_SHADING_CORRECTION_GAIN_FIXED_POINT). The aggregate bandwidth to read the gain buffer and offset buffer from on-board memory cannot be bigger than the internal maximum bandwidth of the DMA port. There is a line limitation of 64 K pixels per line and 1 M lines per frame. Once you have set the buffers, enable the correction using MdigControl() with M_SHADING_CORRECTION.

> Note that for Zebra Radient eV-CL, flat-field correction is not supported on Zebra Radient eV-CL SB.

Allocating independent Aurora Imaging Library digitizers on Zebra Radient eV-CL

You can allocate up to 4 independent Aurora Imaging Library digitizers on your Zebra Radient eV-CL, using MdigAlloc(). Independent digitizers have different device numbers and use different acquisition paths.

Zebra Radient eV-CL

The number of independent digitizers that you can allocate on Zebra Radient eV-CL is dependent on the version in use:

- Zebra Radient eV-CL SB has only 1 acquisition path (M_DEV0).

  Zebra Radient eV-CL DB has 2 acquisition paths (M_DEV0 and M_DEV1).

  Zebra Radient eV-CL QB has 4 acquisition paths (M_DEV0 to M_DEV3).

  With both Zebra Radient eV-CL DB and QB, each acquisition path supports a video source in the Camera Link Base configuration.

- Zebra Radient eV-CL SF has only 1 acquisition path (M_DEV0).

  Zebra Radient eV-CL DF has 2 acquisition paths (M_DEV0 to M_DEV1).

  With both Zebra Radient eV-CL SF and DF, each acquisition path supports a video source in the Camera Link Medium, Full, or 80-bit configuration.

Minimum latency and grabbing all frames

When grabbing with either MdigGrab() or MdigProcess(), Zebra Radient eV-CL is optimized to have the smallest latency possible between the last pixel sent from the camera to the last pixel written into the buffer. This allows the buffer to be available for processing with a minimum amount of delay.

By default, Zebra Radient eV-CL grabs into buffers in Host memory. On-board memory is used to protect against PCIe latency. To perform real-time grabs, your rate of acquisition (grabbing bandwidth) must be lower than the PCIe maximum transfer rate (PCIe bandwidth). If the acquisition rate is higher than the transfer rate across the PCIe bus, frames will be skipped rather than allowing a grab over-run to occur. To determine the image transfer speed from Zebra Radient eV-CL to the Host, use the Zebra Radient eV-CL Bench utility.

Detecting missed frames

To be certain that no frames were missed when MdigProcess() was last used, use MdigInquire() with M_PROCESS_FRAME_MISSED. This value reads the number of frames that were transmitted by the camera when no buffer was available to receive the frame; for example, if a grab is triggered and the associated hooked function is slow, then a frame can be missed.

Efficiently copying an on-board buffer to a destination buffer in Host memory

When copying an on-board buffer into a destination buffer in Host memory, the copy operation is performed by your Zebra Radient eV-CL using the board's DMA write engine, rather than the Host. Your Zebra Radient eV-CL will perform the operation faster than the Host. To ensure that Zebra Radient eV-CL performs the operation, use the following:

- To copy an on-board buffer into a destination buffer in Host memory, use MbufCopy().
- To resize an on-board buffer while copying it to a destination buffer in Host memory, use MimResize() with a factor of 1/n, where n is a value between 1 to 16, and with an interpolation mode of M_NEAREST_NEIGHBOR. Alternatively, you can use MbufTransfer() to perform the same operation.
- To flip an on-board buffer while copying it to a destination buffer in Host memory, use MimFlip().

> Note that for your Zebra Radient eV-CL to perform these operations, the destination buffer must be allocated in non-paged memory (that is, using MbufAlloc...() with + M_IMAGE + M_NON_PAGED).

Efficiently converting buffer formats while copying

When copying an on-board buffer into a destination buffer in Host memory, the color space converter and image formatter can automatically convert the bit-depth and color format of the source buffer to the bit-depth and color format of the destination buffer.

For on-board conversion of buffers to take place, the following on-board buffer and destination buffer depth and color formats are supported.

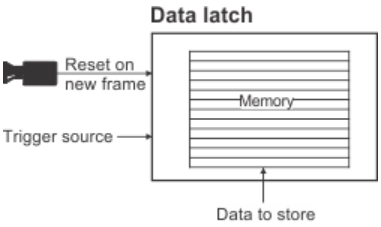
	Destination buffer depth and color format
On-board buffer depth and color	

format (specified in the DCF)	8-bit monochrome	16-bit monochrome	M_PACKED + M_BGR24	M_PACKED + M_BGR32	M_YUV16_YUYV / M_YUV16	M_PLANAR + M_RGB24	M_PLANAR + M_RGB48
8-bit monochrome	Yes	No	Yes	Yes	Yes	Yes	No
16-bit monochrome	Yes	Yes	Yes	Yes	Yes	Yes	Yes
M_PACKED + M_BGR24	Yes	No	Yes	Yes	Yes	Yes	Yes
M_RGB48 + M_PACKED	Yes	Yes	No	No	No	Yes	Yes

For information about YUV or RGB/BGR buffers, refer to Chapter 23: Data buffers. For the calculations used to convert to YUV, refer to the Zebra Radient eV-CL Hardware and Installation manual.

Using Zebra Radient eV-CL data latches

All version of Zebra Radient eV-CL has data latches to store signal state information, the quadrature decoder's counter value, or a timestamp at a specific point during a grab. Data latches are reset at every end of grabbed frame event. Data latches do not suffer the delays typical to hooking a hook-handler function to these specific points during the grab and inquiring and storing related information using software.



> Data latches are only available when using an Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0 or M_DEV1. Zebra Radient eV has 16 data latches for use with a digitizer allocated with M_DEV0 and 16 data latches for use with a digitizer allocated with M_DEV1.

You can specify the data latch trigger source and the data to record upon the trigger using MdigControl() with the M_DATA_LATCH... control types.

To retrieve data from a data latch, use MdigGetHookInfo() with M_DATA_LATCH....

Steps to configure and retrieve data from a data latch

The following steps provide a basic methodology for configuring and retrieving data from a data latch:

1. Specify what triggers storing the specified information to the specified data latch, using MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn. The trigger can be a specific auxiliary signal, a quadrature decoder's counter value, a timer active event, or one of a limited number of grab events (such as a start of frame event).

2. If necessary, specify the trigger signal transition upon which to store the information, using M_DATA_LATCH_TRIGGER_ACTIVATION + M_LATCHn. Note that this is only useful when M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn is set to M_AUX_IOn or M_TIMER_ACTIVE.

3. Specify the type of information to store, using M_DATA_LATCH_TYPE + M_LATCHn. Note that a data latch can store only 1 type of information, and each type of information can only be associated with 1 data latch, except for timestamp, which can be associated with up to 4 data latches. Aurora Imaging Library returns an error when this limitation is not respected.

4. To latch data before the start of the first grabbed frame, set M_DATA_LATCH_MODE + M_LATCHn to M_PREFETCH. Otherwise, if the data latch is triggered before the start of the first grabbed frame, no information is stored. To latch data only once the grab has started (that is, as of the start of the first grabbed frame), set M_DATA_LATCH_MODE to M_DEFAULT; this is the default value.

5. Enable the data latch using M_DATA_LATCH_STATE set to M_ENABLE.

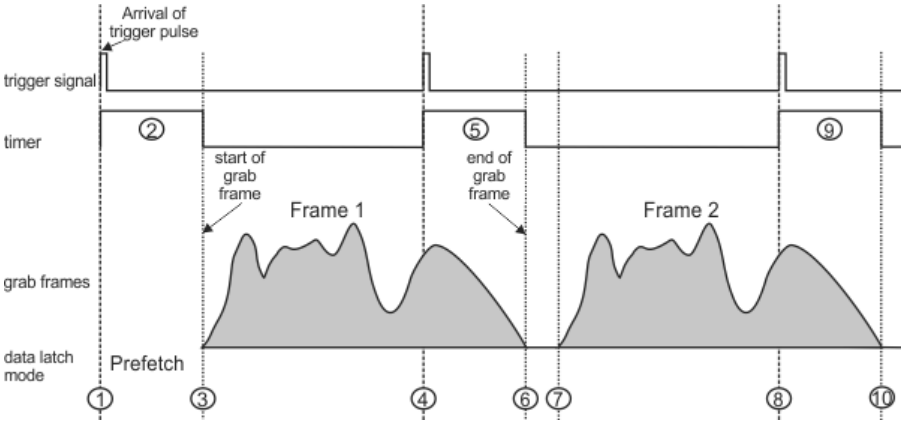
6. If necessary, repeat steps 1 to 5 for each data latch used.

7. Perform the grab. You can use MdigProcess() to grab an image; in the hook-handler function, retrieve and make use of the information stored in the data latch, using MdigGetHookInfo() with M_DATA_LATCH_VALUE. Alternatively, hook a hook-handler function to the end of grab frame event (M_GRAB_FRAME_END) using MdigHookFunction(), and call MdigGrab() to grab an image; then, to retrieve and make use of the information stored in the data latch, call MdigGetHookInfo() with M_DATA_LATCH_VALUE.

8. When done, disable the data latches using MdigControl() with M_DATA_LATCH_STATE + M_LATCHn set to M_DISABLE.

Triggering the data latch

Data latches can store data at several different points during a grab. Below are a few events that can trigger a data latch.

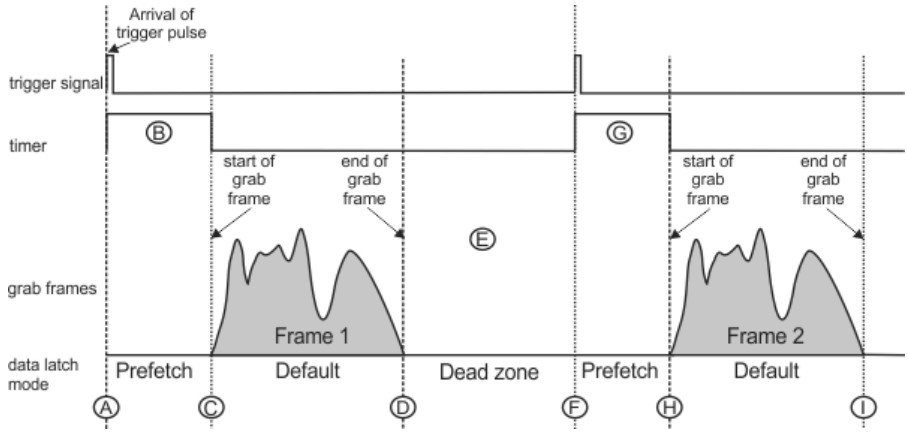


Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
1, 4, and 8	M_AUX_IOn	M_EDGE_RISING
2, 5, and 9	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
3 and 7	M_GRAB_FRAME_START	
6 and 10	M_GRAB_FRAME_END	

Events that occur before the start of the first grabbed frame of queued grabs or a grab sequence (such as, any auxiliary I/O change event, rotary decoder position change, or timer active event) can only trigger a data latch if the data latch has its mode set to prefetch (using MdigControl() with M_DATA_LATCH_MODE set to M_PREFETCH).

In the above image, if the data latch mode is set to prefetch (using M_DATA_LATCH_MODE set to M_PREFETCH), reading the data latch at the end of frame 1 could return the information latched at captions 1 through 6, depending on which event triggers the data latch. However, if the data latch mode is set to default (M_DATA_LATCH_MODE to M_DEFAULT), reading the data latch at the end of frame 1 could only return captions 3 through 6. In both cases, reading the data latch at the end of frame 2 could return captions 7 though 10. Captions 5 and 6 are returned at the end of frame 1, even though they relate to frame 2 because the data of all enabled data latches is returned at the end of every grabbed frame end event (that is, at the end of each grabbed frame).

If calls to MdigGrab() occur while no other grab is currently being performed and M_DATA_LATCH_MODE is set to M_PREFETCH, events that occur from the moment the grab command is issued until the start of the grabbed frame can also trigger a data latch and be retrieved at the end of the grabbed frame, for each MdigGrab() call.



Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
A and F	M_AUX_IOn	M_EDGE_RISING
B and G	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
C and H	M_GRAB_FRAME_START	
D and I	M_GRAB_FRAME_END	
E	No data can be latched during this period	

In the above image, if the data latch mode is set to M_PREFETCH, reading the data latch at the end of frame 1 could return the information latched at captions A through D, depending on which event triggers the data latch. In this example, no data latch can return information relating to events during caption E. Frame 2 could return the information latched at captions F through I. Note that, when using MdigProcess() or when queuing grabs using MdigGrab() in asynchronous mode, there is no dead zone.

Data latch limitations

To set the type of data that a data latch should store, use M_DATA_LATCH_TYPE. In most cases, you can set up only one data latch to record a specific type of data (for example, only one data latch can store the status of all I/O signals). The exceptions to this are data latches that store timestamps; you can set up four data latches to store timestamps. Aurora Imaging Library returns an error when this limitation is not respected.

Retrieving data latch information

Aurora Imaging Library allows each data latch to store multiple instances of the same type of data. You can retrieve a specific instance using the combination value M_VALUE_INDEX(). For example, when latching the timestamp at the end of each grabbed line of a frame with 1024 lines, to retrieve the 1024th timestamp, use MdigGetHookInfo() with M_DATA_LATCH_VALUE + M_LATCHn + M_VALUE_INDEX(1023), where *n* is the number associated with the data

latch.

To retrieve all the stored information associated with a specific data latch, use M_DATA_LATCH_VALUE_ALL + M_LATCHn. Using our previous example, this would return 1024 timestamps.

To inquire how many pieces of information are stored inside a data latch, use M_DATA_LATCH_VALUE_COUNT.

To convert a timestamp from clock ticks to seconds, use the following equation: *Timestamp* * (*TimestampFrequencyInHz*). To inquire the clock frequency, use MdigInquire() with M_DATA_LATCH_CLOCK_FREQUENCY.

Data latch example

For an example of how to use data latches, refer to the *DataLatch.cpp* example.

Zebra Radient eV-CL utilities and tools

There are two Zebra Radient eV-CL external applications (utilities) available (Usage Meter and Radient eV-CL Bench) and a series of tools integrated in the Aurora Imaging Configurator utility (such as, the PCIe Information tool and the System Monitor tool).

To access these utilities and tools, launch the Aurora Imaging Configurator utility. Then, expand the **Boards** item in the tree structure, click on the **Radient eV-series** subitem in the tree structure, and then select your board (**eV-CL**). The utilities are located in the **Launch external applications** and **Zebra Radient eV System Monitor** pages, respectively. There is a version of each of these utilities and tools for each board-type, unless otherwise specified in the text below.

Usage Meter utility

The Usage Meter utility calculates how much of the grab section and the transfer section of each DMA engine is being used. This rate of utilization is expressed as a percentage. This utility also shows the usage of on-board memory.

Zebra Radient eV-CL Bench utility

The Zebra Radient eV-CL Bench utility calculates the real-time transfer speed of the PCIe slot (in Mbytes/sec) from:

- Zebra Radient eV-CL to Host.
- Host to Zebra Radient eV-CL.

PCIe Information tool

The PCIe Information tool returns information about the board's PCIe connector.

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the System Monitor tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- Current number of PCIe lanes.
- Current speed of the PCIe lanes.
- Maximum number of PCIe lanes.
- Maximum speed of the PCIe lanes.

System Monitor tool

The System Monitor tool displays board properties of the specified digitizer (M_DEVn).

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the PCIe Information tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- The grab FPGA's current temperature, shown on a sliding scale from 0 to 85 degrees Celsius.
- The speed of the fan on the board, in RPM.

- Whether a power source is detected on the internal auxiliary 12 V power connection, if available on the Zebra Radient eV-CL board.

If your Zebra Radient eV-CL is operating in the normal range of temperatures, a check mark is presented at the bottom of the System Monitor tool. If your Zebra Radient eV-CL is operating above or below the normal range of temperatures, an X is presented.

Zebra Radient eV-CL Deserializer setup tool

The Zebra Radient eV-CL Deserializer setup tool finds the initial values needed for the deserializer alignment process when grabbing data. The values are determined for each Camera Link connection that is in-use. The resulting values are saved in the DCF, and help compensate for signal degradation. Note that we recommend using this tool only when using cables longer than 7 m and when acquired images are corrupted. For more information on using the Zebra Radient eV-CL Deserializer setup tool, refer to the Zebra Radient eV-CL installation and hardware reference manual.

Zebra Radient eV-CL SB/DB/QB connectors and signal names

This section serves as a reference to match Zebra Radient eV-CL SB, eV-CL DB, and eV-CL QB's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary/camera control signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

Zebra Radient eV-CL family members have a different number of acquisition paths and auxiliary/camera control signals. For each Zebra Radient eV-CL family member, only the auxiliary/camera control signals associated with the following digitizer device numbers are supported:

Zebra Radient eV type	Digitizer device #
Zebra Radient eV-CL SB	M_DEV0.
Zebra Radient eV-CL DB	M_DEV0, M_DEV1.
Zebra Radient eV CL-QB	M_DEV0, M_DEV1, M_DEV2, M_DEV3.

The following table lists the connectors of the auxiliary/control signals that can be used for each digitizer device number:

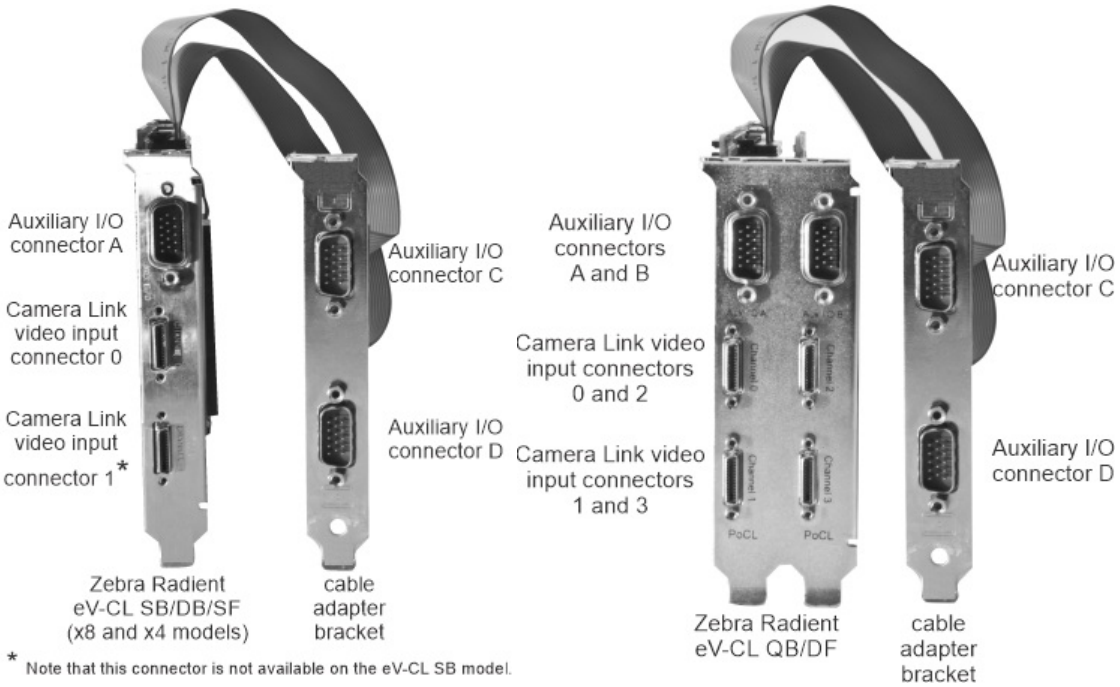
Digitizer device #	Zebra Radient eV-CL SB	Zebra Radient eV-CL DB	Zebra Radient eV-CL QB
M_DEV0	Auxiliary I/O connector A and C	Auxiliary I/O connector A and some on C	Auxiliary I/O connector A and some on C
M_DEV1		Auxiliary I/O connector C and some on A	Auxiliary I/O connector C and some on A
M_DEV2			Auxiliary I/O connector B and some on D
M_DEV3			Auxiliary I/O connector D and some on B

Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Radient eV-CL DB and eV-CL QB boards and cable adapter brackets provide several interface connectors. On the bracket of Zebra Radient eV-CL DB, there are two Camera Link video input connectors and an auxiliary I/O connector. On the double bracket of Zebra Radient eV-CL QB, there are two pairs of Camera Link video input connectors and 2 auxiliary I/O connectors. On the cable adapter bracket, there are two external auxiliary I/O connectors.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Camera Link video input connectors	SDR (0,1,2 and 3)		The Camera Link video input connectors are 26-pin high-density female mini Camera Link connectors. They are used to receive video input, timing, synchronization signals and transmit/receive communication signals between the video source and the frame grabber. There are two Camera Link video input connectors (SDR 0 and 1) on Zebra Radient eV-CL DB. There are four Camera Link video input connectors (SDR 0, 1, 2 and 3) on Zebra Radient eV-CL QB.

External auxiliary I/O connector	HD-15 [1] (A, B, C and D)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connectors A and (if present) B are located on Zebra Radient eV-CL, and external auxiliary I/O connectors C and D are located on the cable adapter bracket.

[1] Previously referred to as DBHD-15, but more accurately known as DE-15.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEV0	Indicates the following.
I/O #: M_AUX_IO0 Shared with: M_DEV1 [2]	Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 15+, 9-
Direction	Input
Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 0 or 2 on acq path 1; 2 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN8
I/O #: M_AUX_IO1 Shared with: M_DEV1 [2]	Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 12+, 11-
Direction	Input
Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN9

I/O #: M_AUX_IO2 Shared with: M_DEV1 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 0 for timer output.
	Pin information	Connector: HD-15 (A) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_6
I/O #: M_AUX_IO3 Shared with: M_DEV1 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 1 for timer output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 3 on acq path 0; 3 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4 Shared with: M_DEV1 [2]		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for quadrature input bit 0.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 or 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5 Shared with: M_DEV1 [2]		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for timer-clock input or quadrature input bit 1.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11

I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN0
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN1
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 0, which supports: user input/output, trigger input.
	Pin information	Connector: HD-15 (A) Pin: 1
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	TTL_AUX_IO_4
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 0, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (A) Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_5
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input, user input, or quadrature input bit 0.

		Pin information	Connector: HD-15 (A) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO10; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
		Hardware manual signal name	LVDS_AUX_IN2
	I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 0, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
		Pin information	Connector: HD-15 (A) Pin: 6+, 8-
		Direction	Input
		Trigger information	M_AUX_IO11; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
		Hardware manual signal name	LVDS_AUX_IN3
	I/O #: M_AUX_IO12		LVDS auxiliary signal (output) for acquisition path 0, which supports: timer output or user output.
		Pin information	Connector: HD-15 (A) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV0;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
		Hardware manual signal name	LVDS_AUX_OUT7
	I/O #: M_CC_IO1		Camera control output 1 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
		Pin information	Connector: HDR/SDR (0) Pin: 5+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
		Hardware manual signal name	CC1 (0)

I/O #: M_CC_IO2		Camera control output 2 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC2 (0)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC3 (0)

I/O #: M_CC_IO4		Camera control output 4 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC4 (0)
Digitizer device #: M_DEV1 [2]		Indicates the following.
I/O #: M_AUX_IO0 Shared with: M_DEV0		Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 0 or 2 on acq path 1; 2 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN8
I/O #: M_AUX_IO1 Shared with: M_DEV0		Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN9
I/O #: M_AUX_IO2 Shared with: M_DEV0		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 0 for timer output.
	Pin information	Connector: HD-15 (A) Pin: 3
	Direction	Input/Output

	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_6
I/O #: M_AUX_IO3 Shared with: M_DEV0		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 1 for timer output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 3 on acq path 0; 3 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4 Shared with: M_DEV0		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for quadrature input bit 0.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 or 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5 Shared with: M_DEV0		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for timer-clock input or quadrature input bit 1.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (C) Pin: 1
	Direction	Input/Output

		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV1;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV1; Trigger controller: 0 on acq path 1.
		Hardware manual signal name	TTL_AUX_IO_12
	I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (C) Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV1;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV1; Trigger controller: 1 on acq path 1.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV1;
		Hardware manual signal name	TTL_AUX_IO_13
	I/O #: M_AUX_IO12		LVDS auxiliary signal (output) for acquisition path 1, which supports: timer output or user output.
		Pin information	Connector: HD-15 (C) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
		Hardware manual signal name	LVDS_AUX_OUT15
	I/O #: M_CC_IO1		Camera control output 1 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
		Pin information	Connector: HDR/SDR (1) Pin: 5+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;

	Hardware manual signal name	CC1 (1)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
	Pin information	Connector: HDR/SDR (1) Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	CC2 (1)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
	Pin information	Connector: HDR/SDR (1) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	CC3 (1)

	I/O #: M_CC_IO4		Camera control output 4 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
		Pin information	Connector: HDR/SDR (1) Pin: 15+, 2-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
		Hardware manual signal name	CC4 (1)
Digitizer device #: M_DEV2 [3]			Indicates the following.
	I/O #: M_AUX_IO0 Shared with: M_DEV3 [3]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 15+, 9-
		Direction	Input
		Trigger information	M_AUX_IO0; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN24
	I/O #: M_AUX_IO1 Shared with: M_DEV3 [3]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 12+, 11-
		Direction	Input
		Trigger information	M_AUX_IO1; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN25
	I/O #: M_AUX_IO2 Shared with: M_DEV3 [3]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 2 for timer output.
		Pin information	Connector: HD-15 (B) [4] Pin: 3
		Direction	Input/Output

		User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ;
		Trigger information	M_AUX_IO2; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 2 on acq path 2; 2 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV2 [3] ;
		Hardware manual signal name	TTL_AUX_IO_22
	I/O #: M_AUX_IO3 Shared with: M_DEV3 [3]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 3 for timer output.
		Pin information	Connector: HD-15 (D) Pin: 3
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ;
		Trigger information	M_AUX_IO3; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 3 on acq path 2; 3 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	TTL_AUX_IO_30
	I/O #: M_AUX_IO4 Shared with: M_DEV3 [3]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for quadrature input bit 0.
		Pin information	Connector: HD-15 (D) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO4; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN26
	I/O #: M_AUX_IO5 Shared with: M_DEV3 [3]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for timer-clock input or quadrature input bit 1.
		Pin information	Connector: HD-15 (D) Pin: 6+, 8-
		Direction	Input
		Trigger information	M_AUX_IO5; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN27
	I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 2, which supports: user input or trigger input.
		Pin information	Connector: HD-15 (B) [4] Pin: 15+, 9-
		Direction	Input

		Trigger information	M_AUX_IO6; Digitizer device #: M_DEV2 [3] ; Trigger controller: 0 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN16
I/O #: M_AUX_IO7			Opto-isolated auxiliary signal (input) for acquisition path 2, which supports: user input or trigger input.
		Pin information	Connector: HD-15 (B) [4] Pin: 12+, 11-
		Direction	Input
		Trigger information	M_AUX_IO7; Digitizer device #: M_DEV2 [3] ; Trigger controller: 1 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN17
I/O #: M_AUX_IO8			TTL auxiliary signal (input/output) for acquisition path 2, which supports: trigger input, user input, or user output.
		Pin information	Connector: HD-15 (B) [4] Pin: 1
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV2 [3] ;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV2 [3] ; Trigger controller: 0 on acq path 2.
		Hardware manual signal name	TTL_AUX_IO_20
I/O #: M_AUX_IO9			TTL auxiliary signal (input/output) for acquisition path 2, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (B) [4] Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV2 [3] ;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV2 [3] ; Trigger controller: 1 on acq path 2.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV2 [3] ;
		Hardware manual signal name	TTL_AUX_IO_21
I/O #: M_AUX_IO10			LVDS auxiliary signal (input) for acquisition path 2, which supports: trigger input, user input, or quadrature input bit 0.
		Pin information	Connector: HD-15 (B) [4] Pin: 4+, 5-
		Direction	Input

	Trigger information	M_AUX_IO10; Digitizer device #: M_DEV2 [3] ; Trigger controller: 0 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN18
I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 2, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (B) [4] Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV2 [3] ; Trigger controller: 1 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN19
I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 2, which supports: timer output or user output.
	Pin information	Connector: HD-15 (B) [4] Pin: 13+, 14-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV2 [3] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [3] ;
	Hardware manual signal name	LVDS_AUX_OUT23
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [4] Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [3] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [3] ;
	Hardware manual signal name	CC1 (2)

I/O #: M_CC_IO2		Camera control output 2 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [4] Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [3] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [3] ;
	Hardware manual signal name	CC2 (2)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [4] Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [3] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [3] ;
	Hardware manual signal name	CC3 (2)

	I/O #: M_CC_IO4		Camera control output 4 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
		Pin information	Connector: HDR/SDR (2) [4] Pin: 15+, 2-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [3] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [3] ;
		Hardware manual signal name	CC4 (2)
Digitizer device #: M_DEV3 [3]			Indicates the following.
	I/O #: M_AUX_IO0 Shared with: M_DEV2 [3]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 15+, 9-
		Direction	Input
		Trigger information	M_AUX_IO0; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN24
	I/O #: M_AUX_IO1 Shared with: M_DEV2 [3]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 12+, 11-
		Direction	Input
		Trigger information	M_AUX_IO1; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN25
	I/O #: M_AUX_IO2 Shared with: M_DEV2 [3]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 2 for timer output.
		Pin information	Connector: HD-15 (B) [4] Pin: 3
		Direction	Input/Output

	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 2 on acq path 2; 2 on acq path 3.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV2 [3] ;
	Hardware manual signal name	TTL_AUX_IO_22
I/O #: M_AUX_IO3 Shared with: M_DEV2 [3]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 3 for timer output.
	Pin information	Connector: HD-15 (D) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 3 on acq path 2; 3 on acq path 3.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV3 [3] ;
	Hardware manual signal name	TTL_AUX_IO_30
I/O #: M_AUX_IO4 Shared with: M_DEV2 [3]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for quadrature input bit 0.
	Pin information	Connector: HD-15 (D) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN26
I/O #: M_AUX_IO5 Shared with: M_DEV2 [3]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for timer-clock input or quadrature input bit 1.
	Pin information	Connector: HD-15 (D) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV2 [3] , M_DEV3 [3] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN27
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 3, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (D) Pin: 1
	Direction	Input/Output

		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV3 [3] ;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV3 [3] ; Trigger controller: 0 on acq path 3.
		Hardware manual signal name	TTL_AUX_IO_28
	I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 3, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (D) Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV3 [3] ;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV3 [3] ; Trigger controller: 1 on acq path 3.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	TTL_AUX_IO_29
	I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 3, which supports: timer output or user output.
		Pin information	Connector: HD-15 (D) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV3 [3] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	LVDS_AUX_OUT31
	I/O #: M_CC_IO1		Camera control output 1 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [4] Pin: 5+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [3] ;

		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	CC1 (3)
I/O #: M_CC_IO2			Camera control output 2 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [4] Pin: 17+, 4-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [3] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	CC2 (3)
I/O #: M_CC_IO3			Camera control output 3 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [4] Pin: 3+, 16-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [3] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [3] ;
		Hardware manual signal name	CC3 (3)

	I/O #: M_CC_IO4	Camera control output 4 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
	Pin information	Connector: HDR/SDR (3) [4] Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [3] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [3] ;
	Hardware manual signal name	CC4 (3)

[2] Only applicable to Zebra Radient eV-CL DB and eV-CL QB.

[3] Only applicable to Zebra Radient eV-CL QB.

[4] Only Zebra Radient eV-CL QB has this connector; Zebra Radient eV-CL DB does not have this connector.

Zebra Radient eV-CL SF/DF connectors and signal names

This section serves as a reference to match Zebra Radient eV-CL SF and eV-CL DF's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary/camera control signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

Zebra Radient eV-CL family members have a different number of acquisition paths and auxiliary/camera control signals. For each Zebra Radient eV-CL family member, only the auxiliary/camera control signals associated with the following digitizer device numbers are supported:

Zebra Radient eV type	Digitizer device #
Zebra Radient eV-CL SF	M_DEV0.
Zebra Radient eV-CL DF	M_DEV0, M_DEV1.

The following table lists the connectors of the auxiliary/control signals that can be used for each digitizer device number:

Digitizer device #	Zebra Radient eV-CL SF	Zebra Radient eV-CL DF
M_DEV0	Auxiliary I/O connector A and C	Auxiliary I/O connector A and C
M_DEV1		Auxiliary I/O connector B and D

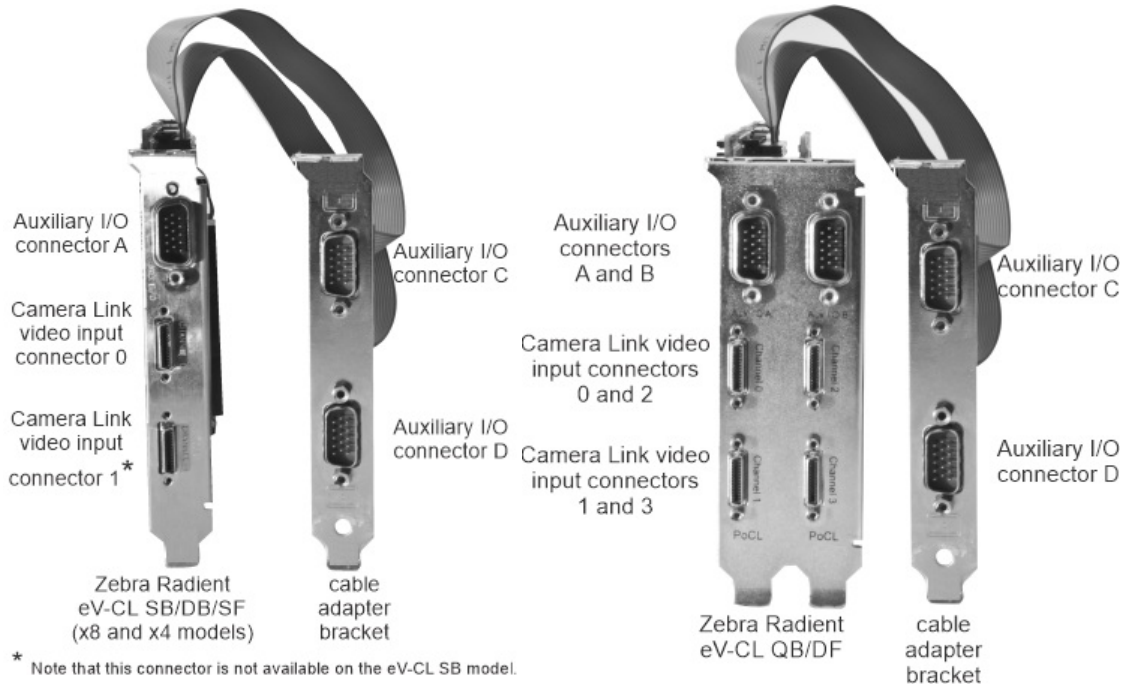
Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Radient eV-CL SF and eV-CL DF boards and cable adapter brackets provide several interface connectors. On

the bracket of Zebra Radient eVCL SF, there are two Camera Link video input connectors and an auxiliary I/O connector. On the double bracket of Zebra Radient eV-CL DF, there are two pairs of Camera Link video input connectors and two auxiliary I/O connectors. On the cable adapter bracket, there are two external auxiliary I/O connectors.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Camera Link video input connectors	SDR (0, 1, 2, and 3)		The Camera Link video input connectors are 26-pin high-density female mini Camera Link connectors. They are used to receive video input, timing, synchronization signals and transmit/receive communication signals between the video source and the frame grabber. There are two Camera Link video input connectors (SDR 0 and 1) on Zebra Radient eV-CL SF. There are four Camera Link video input connectors (SDR 0, 1, 2 and 3) on Zebra Radient eV-CL DF.
External auxiliary I/O connector	HD-15 [1] (A, B, C, and D)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connectors A and (if present) B are located on Zebra Radient eV-CL, and external auxiliary I/O connectors C and D are located on the cable adapter bracket.

[1] Previously referred to as DBHD-15, but more accurately known as DE-15.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEV0	Indicates the following.
I/O #: M_AUX_IO0	Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 15+, 9-
Direction	Input
Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN8
I/O #: M_AUX_IO1	Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 12+, 11-
Direction	Input
Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN9
I/O #: M_AUX_IO2	TTL auxiliary signal (input/output) for acquisition path 0, which supports: trigger input, user input, user output, or timer output.
Pin information	Connector: HD-15 (A) Pin: 3
Direction	Input/Output
User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0;
Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
Hardware manual signal name	TTL_AUX_IO_6

I/O #: M_AUX_IO3		TTL auxiliary signal (input/output) for acquisition path 0, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11
I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN0
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 12+, 11-
	Direction	Input

		Trigger information	M_AUX_IO7; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
		Hardware manual signal name	OPTO_AUX_IN1
	I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 0, which supports: user input/output, trigger input.
		Pin information	Connector: HD-15 (A) Pin: 1
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV0;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
		Hardware manual signal name	TTL_AUX_IO_4
	I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 0, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (A) Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV0;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV0;
		Hardware manual signal name	TTL_AUX_IO_5
	I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input, user input, or quadrature input bit 0.
		Pin information	Connector: HD-15 (A) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO10; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
		Hardware manual signal name	LVDS_AUX_IN2
	I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 0, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
		Pin information	Connector: HD-15 (A) Pin: 6+, 8-
		Direction	Input

	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN3
I/O #: M_AUX_IO12		LVDS auxiliary signal (output) for acquisition path 0, which supports: timer output or user output.
	Pin information	Connector: HD-15 (A) Pin: 13+, 14-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	LVDS_AUX_OUT7
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC1 (0)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 17+, 4-

	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC2 (0)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC3 (0)
I/O #: M_CC_IO4		Camera control output 4 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC4 (0)

Digitizer device #: M_DEV1 [2]	Indicates the following.
I/O #: M_AUX_IO0	Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
Pin information	Connector: HD-15 (D) Pin: 15+, 9-
Direction	Input
Trigger information	M_AUX_IO0; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
Hardware manual signal name	OPTO_AUX_IN24
I/O #: M_AUX_IO1	Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
Pin information	Connector: HD-15 (D) Pin: 12+, 11-
Direction	Input
Trigger information	M_AUX_IO1; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.
Hardware manual signal name	OPTO_AUX_IN25
I/O #: M_AUX_IO2	TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
Pin information	Connector: HD-15 (B) [3] Pin: 3
Direction	Input/Output
User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV1 [2] ;
Trigger information	M_AUX_IO2; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1 [2] ;
Hardware manual signal name	TTL_AUX_IO_22
I/O #: M_AUX_IO3	TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
Pin information	Connector: HD-15 (D) Pin: 3
Direction	Input/Output
User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV1 [2] ;
Trigger information	M_AUX_IO3; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.

	Hardware manual signal name	TTL_AUX_IO_30
I/O #: M_AUX_IO4		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN26
I/O #: M_AUX_IO5		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN27
I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (B) [3] Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN16
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (B) [3] Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN17
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 1

	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	TTL_AUX_IO_20
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	TTL_AUX_IO_21
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input, user input, or quadrature input bit 0.
	Pin information	Connector: HD-15 (B) [3] Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO10; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN18
I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 1, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (B) [3] Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN19
I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 1, which supports: timer output or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 13+, 14-

	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	LVDS_AUX_OUT23
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC1 (2)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC2 (2)

I/O #: M_CC_IO3		Camera control output 3 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC3 (2)
I/O #: M_CC_IO4		Camera control output 4 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC4 (2)

[2] Only applicable to Zebra Radient eV-CL DF.

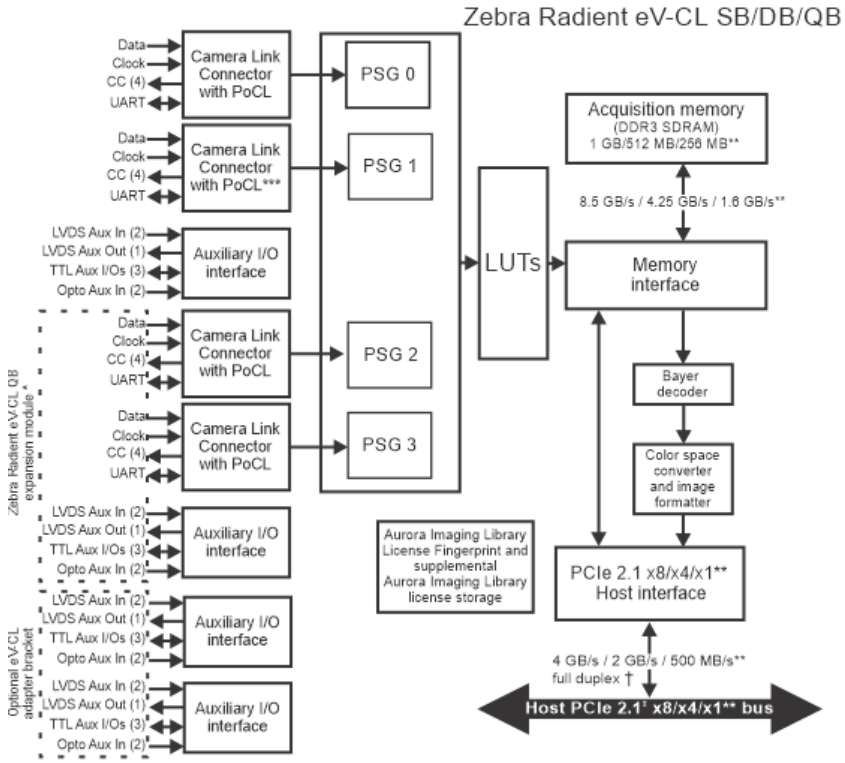
[3] Only Zebra Radient eV-CL DF has this connector; Zebra Radient eV-CL SF does not have this connector.

Zebra Radient eV-CL board flow diagrams

This section contains data flow diagrams for Zebra Radient eV-CL boards.

Zebra Radient eV-CL

The following diagram illustrates the data flow of Zebra Radient eV-CL in dual-Base or quad-Base configuration.



* Note that, without the expansion module, this is a flow diagram of Zebra Radient eV-CL SB or DB, depending on the number of camera link interfaces. With the expansion module, this is a flow diagram of Zebra Radient eV-CL QB. eV-CL SB and DB are not upgradable to eV-CL QB; the eV-CL SB and DB do not have the required connector for the eV-CL QB module.

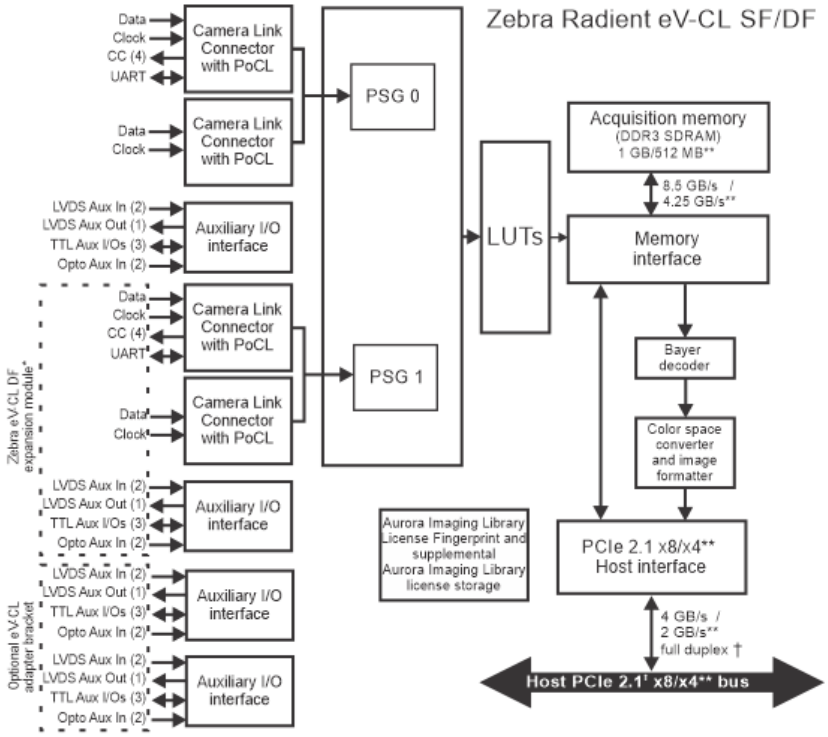
** eV-CL x8 DB and eV-CL QB models/ eV-CL x4 DB model/ eV-CL SB model.

*** Note that this connector is not available on the eV-CL SB model.

† Note that these are the theoretical maximums. The practical maximums are 3 GB/s for the eV-CL x8 DB and eV-CL QB models, 1.8 GB/s for the eV-CL x4 DB model, and 425 MB/s for the eV-CL SB model.

‡ Note that you can install Zebra Radient eV-CL in a PCIe slot 2.1 and above, but the operating speed will only run at PCIe 2.1 speeds.

The following diagram illustrates the data flow of Zebra Radient eV-CL single-Full or dual-Full configuration.



* Note that, without the expansion module, this is a flow diagram of Zebra Radient eV-CL SF. With the expansion module, this is a flow diagram of Zebra Radient eV-CL DF. eV-CL SF is not upgradable to eV-CL DF; the eV-CL SF does not have the required connector for the eV-CL DF module.

** eV-CL x8 SF and eV-CL DF models/ eV-CL x4 SF model.

† Note that these are the theoretical maximums. The practical maximums are 3 GB/s for the eV-CL x8 SF and eV-CL DF models and 1.8 GB/s for the eV-CL x4 SF model.

‡ Note that you can install Zebra Radient eV-CL in a PCIe slot 2.1 and above, but the operating speed will only run at PCIe 2.1 speeds.

Miscellaneous reference information for Zebra Radient eV-CL

This section includes additional command reference information for Zebra Radient eV-CL. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Mdig.

Mdig

This section presents information about Zebra Radient eV-CL and the digitizer module.

MdigControl/MdigInquire

The following lists additional Radient eV-CL support when using MdigControl() / MdigInquire().

- Support for **M_IO_DEBOUNCE_TIME** + **(M_AUX_IOn)**.

  Sets the amount of time that the specified auxiliary input signal is debounced. A maximum of 4 inputs can have a debounce set. You must specify a combination value to set the I/O signal to affect. Possible control/return values are:

  - **0 <= Value <= 8300000**.

    Specifies the minimum amount of time to ignore any additional signal transitions after accepting a signal transition, in nsec.

- Support for **M_SHADING_CORRECTION**.

  Sets whether to perform a gain and offset correction per pixel. When grabbing to a buffer in Host memory, your digitizer performs the shading correction operation using following equation: (Grab Image x **M_SHADING_CORRECTION_GAIN_ID**) + **M_SHADING_CORRECTION_OFFSET_ID**. Each row of gain and offset values is applied to the equivalent row of the grabbed image.

  To set the gain and provide an offset for shading correction, use **M_SHADING_CORRECTION_OFFSET_ID** and **M_SHADING_CORRECTION_GAIN_ID**, respectively. Possible control/return values are:

  - **M_ENABLE**. Enables shading correction.
  - **M_DISABLE**. Disables shading correction.

- Support for **M_SHADING_CORRECTION_GAIN_ID**.

  Sets the buffer containing the gain values that your digitizer should use when it performs a shading correction.

  To enable on-board shading correction and provide an offset, use **M_SHADING_CORRECTION** and **M_SHADING_CORRECTION_OFFSET_ID**, respectively. Possible control/return values are:

  - **M_NULL**. Specifies to not apply a gain when performing a shading correction.
  - **ShadingCorrectionGainID**. Specifies the identifier of an **M_IMAGE** buffer containing the gain values. The buffer must be allocated as a single-band unsigned 8-bit or unsigned 16-bit buffer. It is interpreted as a fixed point buffer with 2-bits for the numerical part. Use **M_SHADING_CORRECTION_GAIN_FIXED_POINT** to set the number of bits for the fractional part. It can have an X-size of up to 16384 pixels and a Y-size equal to the size of the DCF.

- Support for **M_SHADING_CORRECTION_GAIN_FIXED_POINT**.

  Sets the number of bits for the fraction part of the fixed point gain value that your digitizer should use when it performs a shading correction.

  To enable on-board shading correction and provide an offset, use **M_SHADING_CORRECTION**, **M_SHADING_CORRECTION_GAIN_ID**, and **M_SHADING_CORRECTION_OFFSET_ID**, respectively. Possible control/return values are:

  - **0<= Value <16**. Specifies the fractional part of the fixed point gain value.

- Support for **M_SHADING_CORRECTION_OFFSET_ID**.

  Sets the buffer containing the offset values that your digitizer should use when it performs a shading correction.

  To enable on-board shading correction and provide a gain, use **M_SHADING_CORRECTION** and **M_SHADING_CORRECTION_GAIN_ID**, respectively. Possible control/return values are:

  - **M_NULL**. Specifies to not apply an offset when performing a shading correction.
  - **ShadingCorrectionOffsetID**. Specifies the identifier of an **M_IMAGE** buffer containing the offset values. The buffer must be allocated as a signed 8-bit or signed 16-bit buffer.

- Support for **M_GRAB_TRIGGER_OVERLAP**.

  Sets how to deal with a new trigger that occurs while a grab is in progress. Possible control/return values are:

  - **M_DEFAULT**. Same as **M_RESET** or value specified in DCF.
  - **M_PREVIOUS_FRAME**. Specifies that a trigger received, while the grab is in progress, will be latched (stored). As soon as the current grab finishes, a new grab will be started.
  - **M_OFF**. Specifies that a new trigger is ignored.
  - **M_RESET**. Specifies that the current grab will be immediately stopped and a new grab will be started.

- Support for **M_ROTARY_ENCODER_POSITION_START_TRIGGER (+ M_ROTARY_ENCODERn)**.

  Sets the rotary decoder's counter value upon which to start generating triggers, when **M_ROTARY_ENCODER_OUTPUT_MODE** is set to **M_POSITION_TRIGGER_MULTIPLE**. If it is not set to this value, then **M_ROTARY_ENCODER_POSITION_START_TRIGGER** is ignored. Possible control/return values are:

  - **0 < Value <= 0xFFFFFFFF**. Specifies the value of the counter upon which to start generating triggers.

- Support for **M_ROTARY_ENCODER_OUTPUT_MODE (+ M_ROTARY_ENCODERn)**.

  Sets the rotary decoder's counter value and/or the direction of movement upon which the rotary decoder should output a pulse. Possible control/return values are:

  - **M_POSITION_TRIGGER_MULTIPLE**. Specifies to start triggering only when the counter value set with **M_ROTARY_ENCODER_POSITION_START_TRIGGER** is reached and then to generate a trigger at a set interval (change) in counter value. Set the interval with **M_ROTARY_ENCODER_POSITION_TRIGGER**.

- Support for **M_ROTARY_ENCODER_POSITION_TRIGGER (+ M_ROTARY_ENCODERn)**.

  Sets the rotary decoder's counter value at which a trigger is generated. Possible control/return values are:

  - **0 < Value <= 0xFFFFFFFF**. Specifies the counter value or change in counter value at which to generate a trigger.

    If **M_ROTARY_ENCODER_OUTPUT_MODE** is set to **M_POSITION_TRIGGER_MULTIPLE**, this will specify the change in counter value at which to generate a trigger, beginning at the counter value specified by

M_ROTARY_ENCODER_POSITION_START_TRIGGER.

If **M_ROTARY_ENCODER_OUTPUT_MODE** is set to **M_POSITION_TRIGGER**, this will specify the counter value at which to generate a trigger each time this value is reached.

If **M_POSITION_TRIGGER** is set to any other control value, this value will be ignored.

- Support for **M_TIMER_TRIGGER_OVERLAP** + **M_TIMERn**.

  Sets how to deal with a new trigger that occurs while the associated timer has not yet expired (both its delay and duration). Possible control/return values are:

  - **M_DEFAULT**. Same as **M_RESET** or value specified in DCF.
  - **M_LATCH**. Specifies that a trigger received, while the associated timer has not expired, will be latched (stored). As soon as the current timer expires, a new trigger is issued.
  - **M_OFF**. Specifies that a new trigger is ignored.
  - **M_RESET**. Specifies that a new trigger automatically resets the timer (regardless of whether it is in its delay or active period) and then restarts the timer. This process will repeat for each new trigger received.

- Support for **M_TIMER_TRIGGER_RATE_DIVIDER**.

  Sets the frequency to accept trigger pulses (for example, if set to 2, the first trigger pulse is ignored and the second is accepted). Possible control/return values are:

  - **M_DEFAULT**. Specifies the default value; the default value is 1.
  - **1 <= Value <= 255**. Specifies the frequency with which to accept a trigger out of a series of trigger pulses. Note that if the value is set to 1, all trigger pulses are accepted.

- Support for **M_TIMER_DELAY2** + **M_TIMERn**.

  Sets the delay between the end of the first active portion of the timer output signal and the start of the second pulse. Possible control/return values are:

  - **M_DEFAULT**. Specifies the default value. This is the same as specified in the DFC; if no value is specified, then 0.
  - **Value > 0**. Specifies the delay, in nsec.

- Support for **M_TIMER_DURATION2** + **M_TIMERn**.

  Sets the duration for the active portion of the second pulse of the timer output signal. Possible control/return values are:

  - **M_DEFAULT**. Specifies the default value. This is the same as specified in the DFC; if no value is specified, then 0.
  - **Value > 0**. Specifies the duration of the active portion of the second pulse of the timer output signal, in nsec.

- Support for **M_GRAB_FRAME_BURST_SIZE**.

  Specifies the number of frames grabbed into the same buffer at each grab command. The size Y of the grab buffer must be equal to (height of the frame * **M_GRAB_FRAME_BURST_SIZE**). Possible control/return values are:

  - **M_DEFAULT**. Same as 1.
  - **1 <= Value <= 1023**. Sets the number of frames grabbed.

- Support for **M_GRAB_FRAME_BURST_MAX_TIME**.

  Specifies the maximum amount of time to wait for all the frames to be grabbed in the multi-frame buffer.

  The timer starts when the first frame is grabbed. The number of frames in the buffer can be inquired using MdigGetHookInfo() with **M_GRAB_FRAME_BURST_COUNT**. This is useful when the camera stops sending frames and the multi-frame buffer is only partially full. Possible control/return values are:

  - **M_DEFAULT**. Same as **M_INFINITE**.
  - **0.000008 <= Value <= 1.000000**. Specifies the maximum amount of time to wait, in seconds.
  - **M_INFINITE**. Specifies to wait indefinitely.

- Support for **M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE**.

  Specifies the signal from which a rising edge signals the end of a multi-frame sequence. This is useful to force a partially completed multi-frame buffer to complete. Possible control/return values are:

  - **M_DEFAULT**. Same as **M_AUX_IO0**.
  - **M_AUX_IOn**. Specifies to use auxiliary input signal n as the trigger source, where n is the number of the auxiliary signal. Note that the specified auxiliary signal can also be a bidirectional signal set to input (using **M_IO_MODE** set to **M_INPUT**).

- Support for **M_GRAB_FRAME_BURST_END_TRIGGER_STATE**.

  Enables the **M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE** source. Possible control/return values are:

  - **M_DEFAULT**. Same as **M_DISABLE**.
  - **M_DISABLE**. Disables the grab frame burst end trigger source.
  - **M_ENABLE**. Enables the grab frame burst end trigger source.

MdigControlFeature/MdigInquireFeature

The following lists additional Radient eV-CL support when using MdigControlFeature() / MdigInquireFeature().

- Support for **M_FEATURE_ENUM_ENTRY_DISPLAY_NAME** + n.

  Inquires the display name of the specified enumeration entry of the specified feature, where n is the index of the enumerated list. For more information, see **M_FEATURE_ENUM_ENTRY_NAME** in the command reference.

- Support for **M_STRING_ARRAY_SIZE( )**.

  Specifies that the feature value is expressed as a string of a specified size. The **M_STRING_ARRAY_SIZE( )** macro passes the size of the user-allocated buffer (first passed to the MdigInquireFeature() 's UserVarPtr parameter). See **M_STRING_ARRAY_SIZE( )** in the command reference for more information.

MdigGetHookInfo

The following lists additional Radient eV-CL support when using MdigGetHookInfo().

The following information types allows you to retrieve information about a GenICam SFNC-compliant event. The information types are only available if MdigGetHookInfo() was called from a function hooked to a GenICam event using **M_GC_EVENT** + **M_FEATURE_CHANGE**. In addition, the GenICam event must be enabled using MdigControlFeature(), and the message channel must be supported by your camera.

- Support for **M_GC_FEATURE_CHANGE_NAME**.

Retrieves the name of the GenICam feature that changed. The UserVarPtr must point to a user allocated array of type **AIL_TEXT_CHAR**.

- Support for **M_GC_FEATURE_CHANGE_NAME_SIZE**.

  Retrieves the size of the name of the GenICam feature that changed. The UserVarPtr must point to an **AIL_INT** that is the required string length +1.

The following allows you to retrieve information about grab frame burst events. Unless otherwise specified, you can retrieve these information types if you call this function from within a function hooked to any digitizer event using MdigHookFunction() or MdigProcess().

- Support for **M_GRAB_FRAME_BURST_COUNT**.

  Retrieves the number of frames grabbed in the multi-frame buffer. Possible return values are:

  - **Value > 0**. Specifies the number of frames grabbed.

- Support for **M_GRAB_FRAME_BURST_END_SOURCE**.

  Retrieves the type of event that generated the end of the frame burst. Multiple events can be set at the same time. Bitwise operators must be used to verify the presence of a specific returned value. Possible return values are:

  - **M_BURST_TRIGGER**. Specifies that trigger signal generates the end of the burst sequence. To specify the source signal, use MdigControl() with **M_GRAB_FRAME_BURST_END_TRIGGER_SOURCE**.
  - **M_BURST_MAX_TIME**. Specifies that the frame burst has taken as much time to complete as the maximum frame burst time. To specify the maximum time for a frame burst to complete, use MdigControl() with **M_GRAB_FRAME_BURST_MAX_TIME**.
  - **M_BURST_COUNT**. Specifies the number of frames that have been grabbed. To specify the number of frames in a frame burst, use MdigControl() with **M_GRAB_FRAME_BURST_SIZE**.

The following allows you to retrieve information from a data latch. These information types are only available if MdigGetHookInfo() was called from a function hooked to an end of grabbed frame event using MdigHookFunction() with **M_GRAB_FRAME_END** or from the hook-handler function (callback function) of MdigProcess(). In addition, you must have enabled the data latch to store this information, using MdigProcess() with **M_DATA_LATCH_STATE**.

Note that you can only use data latches in association with the Aurora Imaging Library digitizer allocated using MdigAlloc() with **M_DEV0**. The following values require that you pass the UserVarPtr parameter the address of an AIL_INT64.

- Support for **M_DATA_LATCH_VALUE** + **M_LATCHn** + **M_VALUE_INDEX(i)**.

  Retrieves the stored data of the specified data latch. If there are multiple instances of this data (for example, your data latch is triggered at the end of each grabbed line), use + **M_VALUE_INDEX(i)**. Possible return values are:

  - **Value**. Specifies the stored data.

- Support for **M_DATA_LATCH_VALUE_ALL**.

  Retrieves all the stored data of the specified data latch. Possible return values are:

  - **Value**. Specifies all the stored data.

- Support for **M_DATA_LATCH_VALUE_COUNT**.

  Counts the number of items stored inside a data latch. Note that a data latch is reset at every frame event. Possible return values are:

  - **Value**. Specifies the number of instances. Note that a data latch is reset at every frame event.

The following is a combination constant for **M_DATA_LATCH_VALUE**. You can add the following value to **M_DATA_LATCH_VALUE** to specify the instance of the stored information to retrieve.

- + **M_VALUE_INDEX(AIL_INT64 IndexValue)**. Specifies the instance of stored information to retrieve. **IndexValue** is used to set the instance of stored information. For example, to read the information latched when the last line finishes being grabbed for a frame with 1024 lines, the instance would be 1023.

Miscellaneous user guide information for Zebra Radient eV-CL

This section includes additional user guide information for Zebra Radient eV-CL. The information found in this section might be a reiteration of content previously documented.

Supported features and standards

The Zebra Radient eV-CL supports firmware updates without power-cycling.

The Zebra Radient eV-CL supports RGB 10/12 bits Gain and Offset. For more information, see **M_SHADING_CORRECTION**.

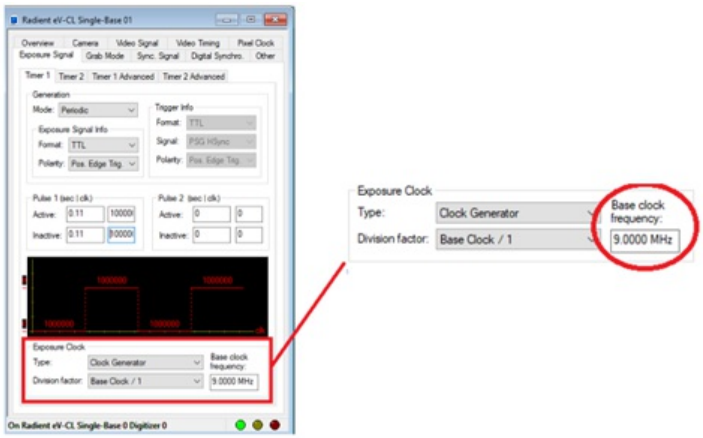
The Zebra Radient eV-CL supports the following standards:

- Camera Link version 2.1.
- GenICam GenApi version 3.4.2.
- GenICam GenCP Camera Link Module version 1.3.
- GenICam for Camera Link (CLProtocol) version 1.2. Note that this requires a third-party CLProtocol communication library (DLL) which is supplied by the camera vendor.

Particularities

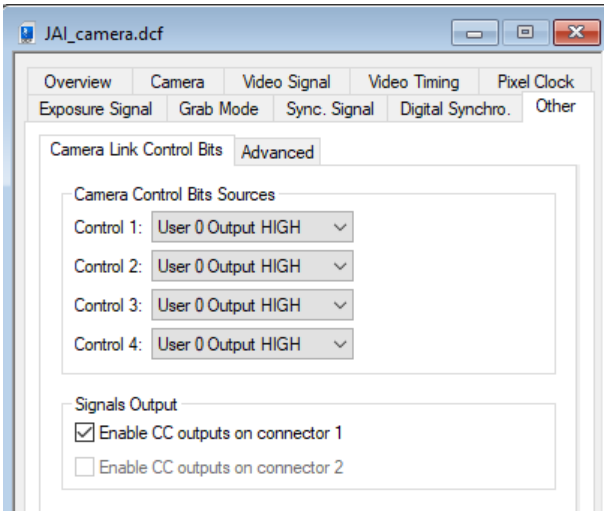
The Bayer conversion hardware supports a line width of up to 8192 bytes.

The clock generator on Zebra Radient eV-CL SB is limited to a base clock frequency range from 9 to 97 MHz. Note that using certain base clock frequencies will affect timer accuracy. For additional information, contact your Zebra technical support representative.



Data latches and shading correction are not supported on Zebra Radient eV-CL SB.

When using a JAI camera, the Camera Link Camera Control bits on the Zebra Radient eV-CL must be activated for stable communication with the camera (either with JAI or Zebra software). The bits can be activated using Aurora Imaging Intellicam in the **Other** tab of the **DCF** dialog.



When editing a DCF for Camera Link using Aurora Imaging Intellicam, Feature Browser will close and reopen. Reopening a Feature Browser window might be subject to a noticeable delay due to camera initialization. To avoid this, uncheck "Automatically compute registers" in the **Preferences Register Compute** menu tab. However, you will then need to manually compute the DCF registers using the **DCF Compute Registers** menu tab.

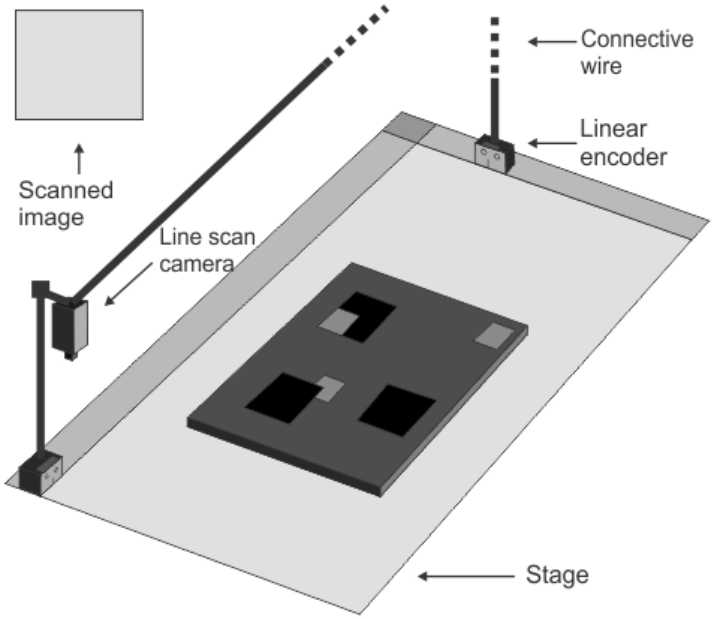
Code snippets

In the following code snippet, an encoder is used to trigger a grab when a certain position is reached. After 20 frames of 1000 lines (or 20000 lines) are grabbed, the grab moves slightly to the right and repeats the process in reverse.

```
/* The camera must be set to trigger the acquisition of a line on the rising
edge of the timer output signal. */

/* Reset the decoder counter to 0. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_POSITION, 0);

/* Program the timer to trigger on a decoder signal. */
MdigControl(AilDigitizer, M_TIMER_CLOCK_SOURCE, M_SYSCLK);
MdigControl(AilDigitizer, M_TIMER_DELAY, timerDelayInUs);
MdigControl(AilDigitizer, M_TIMER_DURATION, timerDurationInUs);
MdigControl(AilDigitizer, M_TIMER_TRIGGER_SOURCE, M_ROTARY_ENCODER);
MdigControl(AilDigitizer, M_TIMER_STATE, M_ENABLE);

/* Program the decoder to start triggering when position 1000 is reached then
continue triggering at a multiple of '1'. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_OUTPUT_MODE,
M_POSITION_TRIGGER_MULTIPLE);
MdigControl(AilDigitizer, M_ROTARY_ENCODER_POSITION_START_TRIGGER, 1000);
MdigControl(AilDigitizer, M_ROTARY_ENCODER_POSITION_TRIGGER, 1);

/* Start MdigProcess to grab a sequence (strip) of 20 frames of 1000 lines each
for a total of 20000 lines (bufferingSize == 20). */
MdigProcess(AilDigitizer, AilGrabBufferList, bufferingSize, M_SEQUENCE,
M_SYNCHRONOUS, ProcessingFunction, &UserHookData);

/* Once the end of sequence is reached, the stage will start scanning in the
other direction (the decoder counter will decrement). */
/* Program the decoder to start triggering when counter value 21000 is reached
then continue triggering at a multiple of '1'. */
```

```
MdigControl(AilDigitizer, M_ROTARY_ENCODER_POSITION_START_TRIGGER, 21000);

/*Start MdigProcess to grab a sequence of 20 frames of 1000 lines each for a
total of 20000 lines (bufferingSize == 20). */
MdigProcess(AilDigitizer, AilGrabBufferList, bufferingSize, M_SEQUENCE,
M_SYNCHRONOUS, ProcessingFunction, &UserHookData);
```

The LinearEncoderScan animation shown below illustrates this example.



In the following code snippet, the board's decoder is set to trigger a pulse only when a linear stage is moving forward. The counter decrements when the stage moves in reverse. The grab is only triggered when the counter hits a new positive value; as such, no line that was already grabbed will be grabbed again.

```
/* The camera must be set to trigger the acquisition of a line on the rising
edge of the timer output signal. */

/* Set the rotary decoder to send a trigger only when the stage is moving
forward. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_DIRECTION, M_FORWARD);

/* Specifies to set the counter to 0xFFFFFFFF upon a decrement. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_FORCE_VALUE_SOURCE,
M_STEP_BACKWARD_WHILE_POSITIVE);

/* Specifies to output a pulse upon a rotary decoder counter increment, only if
the counter value is in the range of 0x0 to 0x7FFFFFFF before the increment
occurs; when interpreting the counter value as signed, this would be the
positive counter value range. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_OUTPUT_MODE,
M_STEP_FORWARD_WHILE_POSITIVE);

/* Set the rotary decoder counter to 0. */
MdigControl(AilDigitizer, M_ROTARY_ENCODER_POSITION_TRIGGER, 0);

/* Set the signal that the timer should output at every rotary decoder trigger.
*/
MdigControl(AilDigitizer, M_TIMER_DELAY, timerDelayInUs);
```

```
MdigControl(AilDigitizer, M_TIMER_DURATION, timerDelayInUs);

/* Specifies to use the output (set with M_ROTARY_ENCODER_OUTPUT_MODE) of the
default rotary decoder as the trigger source. */
MdigControl(AilDigitizer, M_TIMER_TRIGGER_SOURCE, M_ROTARY_ENCODER);

/* The trigger rate divider can be used to reduce the trigger rate. */
MdigControl(AilDigitizer, M_TIMER_TRIGGER_RATE_DIVIDER, 1);

/* Enable the timer. */
MdigControl(AilDigitizer, M_TIMER_STATE, M_ENABLE);

/* Start the processing. The processing function is called with every frame
grabbed. */
MdigProcess(AilDigitizer, AilGrabBufferList, AilGrabBufferListSize, M_START,
M_DEFAULT, ProcessingFunction, &UserHookData);
```

Zebra Rapixo CL

This section discusses features of Aurora Imaging Library that are particular to Zebra Rapixo CL family of boards and ways that optimize the its performance. Zebra Rapixo CL family specific information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Rapixo CL overview
  - Summary of Zebra Rapixo CL and CL Pro features
  - Note on nomenclature
- Using Zebra Rapixo CL with Aurora Imaging Library
  - Performing Bayer color conversion in hardware
  - Using frame burst with a multi-frame buffer
- Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CL boards
- Minimum latency and grabbing all frames
  - Detecting missed frames when using MdigProcess
  - Quickly copying an on-board buffer to a Host destination buffer
  - Quickly converting buffer formats while copying
- Using Zebra Rapixo CL data latches
  - Steps to configure and retrieve data from a data latch
  - Triggering the data latch
  - Data latch limitations
  - Retrieving data latch information
  - Data latch example
- Zebra Rapixo Series CL tools
  - Usage Meter utility
  - Rapixo CL Bench utility
  - Board Information
  - Deserializer setup tool
  - Aurora Imaging Gecho Viewer
  - PCIe Information tool
  - System Monitor tool
- Zebra Rapixo CL DB and Pro DB/QB connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Rapixo CL SF and CL Pro SF/DF connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Rapixo CL board flow diagrams
- Miscellaneous user guide information for Zebra Rapixo CL
  - Supported standards and development kit
  - Additional features
  - Aurora Imaging Configurator
  - Particularities

Zebra Rapixo CL overview

Zebra Rapixo CL is a family of high-performance PCIe frame grabbers. There are two models:

- **Zebra Rapixo CL base model**. Zebra Rapixo CL base model supports image capture from up to 2 high-resolution, high-speed video sources that use the Camera Link (CL) communication standard. There are two versions of this model: Zebra Rapixo CL DB and Zebra Rapixo CL SF.

  Zebra Rapixo CL DB supports acquisition from two Camera Link video sources in Base configuration, and Zebra Rapixo CL SF supports acquisition from one Camera Link video source in Medium, Full, 72-bit, or 80-bit configuration.

- **Zebra Rapixo CL Pro**. Zebra Rapixo CXP Pro supports the same functionality as the base model, as well as FPGA-based processing offload capabilities (Processing FPGA). There are five versions of this model: Zebra Rapixo CL Pro SF, Zebra Rapixo CL Pro DB, Zebra Rapixo CL Pro QB, and Zebra Rapixo CL Pro DF.

  Zebra Rapixo CL Pro DB supports acquisition from two Camera Link video sources in Base configuration; Zebra Rapixo CL Pro QB supports acquisition from four of these video sources. Zebra Rapixo CL Pro SF supports acquisition from one Camera Link video source in Medium, Full, 72-bit, or 80-bit configuration; Zebra Rapixo CL Pro DF supports two of these sources.

Zebra Rapixo CL supports area-scan and line-scan monochrome and color video sources. The color video sources can be RGB video sources or video sources with a Bayer color filter. Zebra Rapixo CL can decode Bayer color-encoded images and perform color space conversions while transferring the image to the Host. Besides standard Camera Link video sources, Zebra Rapixo CL also supports additional types of video sources, including some time-multiplexed video sources.

As a Zebra Rapixo CL grabs images, it can vertically or horizontally flip them. In addition, Zebra Rapixo CL can perform color-space conversions, Bayer decoding, and image subsampling.

> You can access the latest version of the Zebra Rapixo CL and Rapixo CL Pro Installation and Hardware Reference at zebra.com/rapixo-cl-info and zebra.com/rapixo-cl-pro-info, respectively.

Summary of Zebra Rapixo CL and CL Pro features

The following table outlines the features currently available for Zebra Rapixo CL.

	Zebra Rapixo CL
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature() [1]	Yes
On-board memory	512 Mbytes DDR3 SDRAM (Zebra Rapixo CL DB) 512 Mbytes DDR3 SDRAM (Zebra Rapixo CL SF) 4 Gbytes DDR3 SDRAM (Zebra Rapixo CL Pro)

Processing FPGA	No (Zebra Rapixo CL base model)/ Yes (Zebra Rapixo CL Pro)
Digitizer LUTs	8- and 10-bit (1/digitizer) or 12-bit (shared) [2]
Asynchronous camera reset support	Yes
Acquisition section	Camera Link
Number of acquisition paths	4 (QB), 2 (DB, DF), 1 (SF)
Number of supported triggers	7/Digitizer
Number of timers	2/Digitizer
Number of UARTS	1 per CL connector
Number of auxiliary signals	32 (4 in, 1 out, and 3 in/out per I/O connector)
Number of quadrature decoders	4
Power-over-Camera Link (PoCL) support	Yes
Number of data latches	16 only available on M_DEV0 (Zebra Rapixo CL) 16/Digitizer (Zebra Rapixo CL Pro)

[1] Only if the camera supports GenICam. For more information, refer to the Using GenICam with Camera Link cameras subsection of the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

[2] When performing a color grab, one 12-bit LUT is used.

Note on nomenclature

This manual refers to all versions of the Zebra Rapixo CL family of frame grabbers as Zebra Rapixo CL and Zebra Rapixo CL **Pro** if an on-board processing FPGA is present. When necessary, this manual distinguishes between the Zebra Rapixo CL SF, Zebra Rapixo CL DB, Zebra Rapixo CL Pro SF, Zebra Rapixo CL Pro DB, Zebra Rapixo CL Pro QB, Zebra Rapixo CL Pro DF using their full names.

Using Zebra Rapixo CL with Aurora Imaging Library

To use Zebra Rapixo CL, you must allocate it using MsysAlloc() with M_SYSTEM_RAPIXOCL. This allocation opens communication with the Zebra Rapixo CL board and allows Aurora Imaging Library to use its resources. You can allocate an Aurora Imaging Library Rapixo CL system for your board in multiple processes (executables). However, different processes cannot allocate a digitizer for the same acquisition paths.

Zebra Rapixo CL features four quadrature decoders that can decode input from rotary or linear encoders with quadrature output. For more information on this feature, refer to the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. This chapter also provides information on how to configure and use the I/O signals.

Camera Link video sources, attached to Zebra Rapixo CL, can use a GenICam interface. For information regarding using Aurora Imaging Library to access the GenICam SFNC-compatible features of your camera, see the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

For information regarding the use of your Zebra Rapixo CL Pro's Processing FPGA, see Chapter 67: Using Aurora Imaging Library for FPGA processing.

Performing Bayer color conversion in hardware

When Zebra Rapixo CL grabs color images from a video source with a Bayer color filter (as specified by the DCF), it performs Bayer color conversion in hardware, as it transfers the images to the Host. If the images require white balancing, Zebra Rapixo CL can perform this automatically if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_DISABLE.

The M_BAYER... control types of MdigControl() can only be used when grabbing from a camera that has a Bayer color filter (as specified by the DCF); otherwise, an error will be generated.

Using frame burst with a multi-frame buffer

Zebra Rapixo CL supports frame burst technology. This technology allows you to grab a group of sequential frames into a multi-frame buffer with one grab command (MdigGrab(), or one grab of MdigProcess()); the defined number of frames are stored contiguously in the same buffer. The end-of-grab event only occurs once the entire group of frames has been grabbed, reducing the number of events that need to be handled. This is useful in cases where you have a high frame rate and need to ensure that no frames are missed. Note that a user-defined function hooked to the end-of-grab event (MdigProcess(), or MdigHookFunction() with M_GRAB_END) is executed only once the entire group of frames has been grabbed. For information on creating a multi-frame image buffer to store sequential frames, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers.

Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CL boards

Depending on the version of Zebra Rapixo CL, you can allocate up to 4 independent Aurora Imaging Library digitizers on the board, using MdigAlloc(). Independent digitizers have different device numbers and use different acquisition paths. The following is the number of available acquisition paths on each version of the board and the device number to use to allocate a path for the digitizer when calling MdigAlloc().

- Zebra Rapixo CL DB has two acquisition paths (M_DEV0 and M_DEV1).

  Zebra Rapixo CL Pro DB has two acquisition paths (M_DEV0 and M_DEV1).

  Zebra Rapixo CL Pro QB has four acquisition paths (M_DEV0 to M_DEV3).

  With Zebra Rapixo CL and both Zebra Rapixo CL Pro DB and QB, each acquisition path supports a video source in the Camera Link Base configuration.

- Zebra Rapixo CL SF has only one acquisition path (M_DEV0).

  Zebra Rapixo CL Pro SF has only one acquisition path (M_DEV0).

  Zebra Rapixo CL Pro DF has two acquisition paths (M_DEV0 and M_DEV1).

  With Zebra Rapixo CL SF and both Zebra Rapixo CL Pro SF and DF, each acquisition path supports a video source in any Camera Link configuration.

Minimum latency and grabbing all frames

When grabbing with either MdigGrab() or MdigProcess(), Zebra Rapixo CL is optimized to have the smallest latency possible between the last pixel sent from the camera to the last pixel written into the buffer. This allows the buffer to be available for processing with a minimum amount of delay.

On-board memory is used to protect against PCIe latency. By default, Zebra Rapixo CL grabs into off-board buffers. To perform real-time grabs, your rate of acquisition (grabbing bandwidth) must be lower than the PCIe maximum transfer rate (PCIe bandwidth). If the acquisition rate is higher than the transfer rate across the PCIe bus, frames will be skipped rather than allowing a grab over-run to occur. To determine the image transfer speed from Zebra Rapixo CL to the Host, use the Rapixo CL Bench utility.

Detecting missed frames when using MdigProcess

To be certain that no frames were missed when MdigProcess() was last used, use MdigInquire() with M_PROCESS_FRAME_MISSED. This value is based on a comparison between the frame rate (M_PROCESS_FRAME_RATE) and the frame count (M_PROCESS_FRAME_COUNT). If a grab is triggered from a temperamental trigger or if the hooked function is slow, the value returned by M_PROCESS_FRAME_MISSED will be affected. This value is most trustworthy with a camera in continuous mode. You can inquire the frame time-stamp sampled at the end of the frame using MdigGetHookInfo() with M_TIME_STAMP.

Use M_PROCESS_FRAME_COUNT to determine the total number of frames grabbed in the sequence.

Use M_PROCESS_FRAME_RATE to return an estimate of the frame rate in frames per second. Note that this value is determined based on the average of the camera's frame rate, and the total amount of time to process the user-defined hooked function(s).

Quickly copying an on-board buffer to a Host destination buffer

When copying an on-board buffer into a Host destination buffer, the copy operation should be performed by your Zebra Rapixo CL using the Zebra Rapixo CL's DMA engine, rather than the Host. Your Zebra Rapixo CL will perform the operation faster than the Host. To ensure that Zebra Rapixo CL performs the operation, use the following:

- To copy an on-board buffer into a destination buffer on the Host, use MbufCopy().
- To resize an on-board buffer while copying it to a destination buffer on the Host, use MimResize() with a factor of 1 to 16 and with an interpolation mode of M_NEAREST_NEIGHBOR. Alternatively, you can use MbufTransfer() to perform the same operation.
- To flip an on-board buffer while copying it to a destination buffer on the Host, use MimFlip().

> Note that for your Zebra Rapixo CL to perform these operations, the destination Host buffer must be allocated in non-paged memory (that is, using MbufAlloc...() with + M_IMAGE + M_NON_PAGED).

Quickly converting buffer formats while copying

When copying an on-board buffer into a destination Host buffer, the color space converter and image formatter can automatically convert the bit-depth and color format of the source buffer to the bit-depth and color format of the destination buffer.

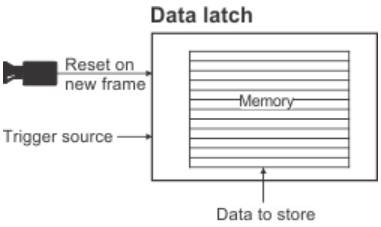
For on-board conversion of buffers to take place, the following on-board image and destination depth and color formats are supported.

On-board buffer depth and color format (specified in the DCF).	Host destination buffer depth and color format.						
	8-bit monochrome.	16-bit monochrome.	M_PACKED + M_BGR24.	M_PACKED + M_BGR32.	M_YUV16_YUYV / M_YUV16.	M_PLANAR + M_RGB24.	M_PLANAR + M_RGB48.
8-bit monochrome.	Yes.	No.	Yes.	No.	Yes.	Yes.	No.
16-bit monochrome.	Yes.	Yes.	Yes.	Yes.	Yes.	Yes.	Yes.
M_PACKED + M_BGR24.	Yes.	No.	Yes.	No.	Yes.	Yes.	No.
M_RGB48 + M_PACKED	Yes.	Yes.	Yes.	Yes.	Yes.	Yes.	Yes.

For information about YUV or RGB/BGR buffers, refer to Chapter 23: Data buffers. For the calculations used to convert to YUV, refer to the Zebra Rapixo CL Pro Hardware and Installation manual.

Using Zebra Rapixo CL data latches

Zebra Rapixo CL uses data latches to store signal state information, the rotary decoder counter value, or a timestamp at a specific point during a grab. The data latch is reset at every end of grabbed frame event. A data latch does not suffer the delays typical to hooking a hook-handler function to these specific points during the grab and inquiring and storing related information using software.



You can specify the data latch trigger source and the data to record upon the trigger using MdigControl() with the M_DATA_LATCH... control types.

To retrieve data from a data latch, use MdigGetHookInfo() with M_DATA_LATCH....

> Note that you can only use data latches in association with the Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0 or M_DEV1.

Steps to configure and retrieve data from a data latch

The following steps provide a basic methodology for configuring and retrieving data from a data latch:

1. Specify what triggers storing the specified information to the specified data latch, using MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn. The trigger can be a specific auxiliary signal, a rotary decoder's counter value, a timer active event, or one of a limited number of grab events (such as a start of frame event).

2. If necessary, specify the trigger signal transition upon which to store the information, using M_DATA_LATCH_TRIGGER_ACTIVATION + M_LATCHn. Note that this is only useful when M_DATA_LATCH_TRIGGER_SOURCE is set to M_AUX_IOn or M_TIMER_ACTIVE.

3. Specify the type of information to store, using M_DATA_LATCH_TYPE. Note that each type of information can only be associated with one data latch, except for timestamp, which can be associated with up to four data latches. Aurora Imaging Library returns an error when this limitation is not respected.

4. To latch data before the start of the first grabbed frame, set M_DATA_LATCH_MODE to M_PREFETCH. Otherwise, if the data latch is triggered before the start of the first grabbed frame, no information is stored. To latch data only once the grab has started (that is, as of the start of the first grabbed frame), set M_DATA_LATCH_MODE to M_DEFAULT; this is the default value.

5. Enable the data latch using M_DATA_LATCH_STATE set to M_ENABLE.

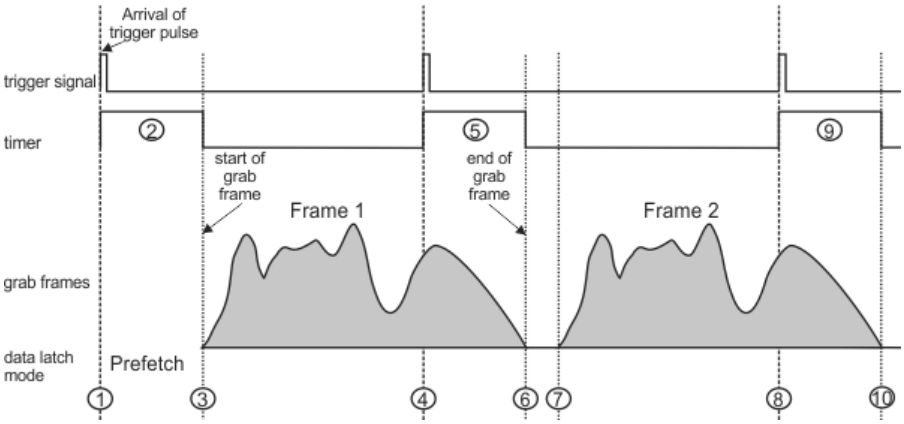
6. If necessary, repeat steps 1 to 5 for each data latch used.

7. Perform the grab. You can use MdigProcess() to grab an image; in the hook-handler function, retrieve and make use of the information stored in the data latch, using MdigGetHookInfo(). Alternatively, hook an end of grab frame event (M_GRAB_FRAME_END) to a hook-handler function using MdigHookFunction() and call MdigGrab() to grab an image; then, to retrieve and make use of the information stored in the data latch, call MdigGetHookInfo().

8. When done, disable the data latches using MdigControl() with M_DATA_LATCH_STATE set to M_DISABLE.

Triggering the data latch

Data latches can store data at several different points during a grab. Below are a few events that can trigger a data latch.

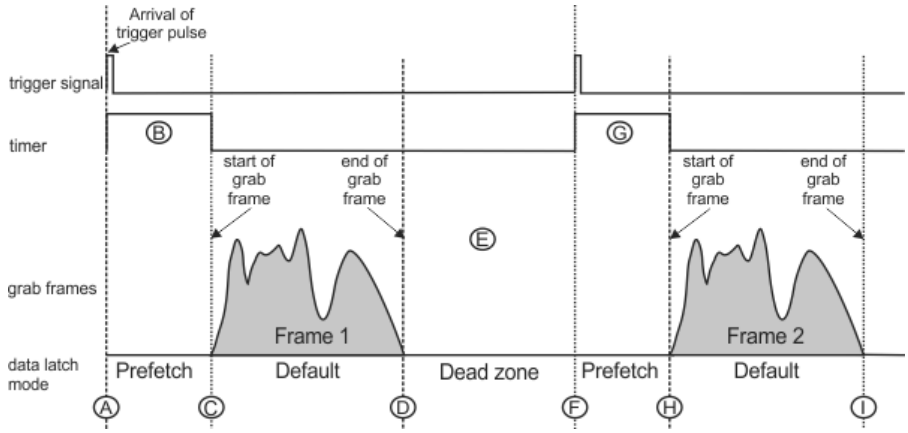


Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
1, 4, and 8	M_AUX_IOn	M_EDGE_RISING
2, 5, and 9	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
3 and 7	M_GRAB_FRAME_START	
6 and 10	M_GRAB_FRAME_END	

Events that occur before the start of the first grabbed frame of queued grabs or a grab sequence (such as, any auxiliary I/O change event, rotary decoder position change, or timer active event between when the grab command is queued until the start of the first grabbed frame) can only trigger a data latch if the data latch has its mode set to prefetch (using MdigControl() with M_DATA_LATCH_MODE set to M_PREFETCH).

In the above image, if the data latch mode is set to prefetch (using M_DATA_LATCH_MODE to M_PREFETCH), reading the data latch at the end of frame 1 could return the information latched at captions 1 through 6, depending on which event triggers the data latch. However, if the data latch mode is set to default (M_DATA_LATCH_MODE to M_DEFAULT), reading the data latch at the end of frame 1 could only return captions 3 through 6. In both cases, reading the data latch at the end of frame 2 could return captions 7 though 10. Captions 5 and 6 are returned at the end of frame 1, even though they relate to frame 2 because the data of all enabled data latches is returned at the end of every grabbed frame end event (that is, at the end of each grabbed frame).

If calls to MdigGrab() occur while no other grab is currently being performed and M_DATA_LATCH_MODE is set to M_PREFETCH, events that occur from the moment the grab command is issued until the start of the grabbed frame can also trigger a data latch and be retrieved at the end of the grabbed frame, for each MdigGrab() call.



Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
A and F	M_AUX_IOn	M_EDGE_RISING
B and G	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
C and H	M_GRAB_FRAME_START	
D and I	M_GRAB_FRAME_END	
E	No data can be latched during this period	

In the above image, if the data latch mode is set to M_PREFETCH, reading the data latch at the end of frame 1 could return the information latched at captions A through D, depending on which event triggers the data latch. In this example, no data latch can return information relating to events during caption E. Frame 2 could return the information latched at captions F through I.

> Note that, when using MdigProcess() or when queuing grabs using MdigGrab() in asynchronous mode, there is no dead zone.

Data latch limitations

To set the type of data that a data latch should store, use M_DATA_LATCH_TYPE. In most cases, you can set up only one data latch to record a specific type of data (for example, only one data latch can store the status of all I/O signals). The exceptions to this are data latches that store timestamps; you can set up four data latches to store timestamps. Aurora Imaging Library returns an error when this limitation is not respected.

Retrieving data latch information

Aurora Imaging Library allows each data latch to store multiple instances of data. For example, when latching the timestamp at the end of each grabbed line of a frame with 1024 lines, to retrieve the 1024th timestamp, use

MdigGetHookInfo() with M_DATA_LATCH_VALUE + M_LATCHn + M_VALUE_INDEX(1023), where *n* is the number associated with the data latch.

To retrieve all the stored information associated with a specific data latch, use M_DATA_LATCH_VALUE_ALL + M_LATCHn. Using our previous example, this would return 1024 timestamps.

To inquire how many pieces of information are stored inside a data latch, use M_DATA_LATCH_VALUE_COUNT.

To convert a timestamp from clock ticks to seconds, use the following equation: *Timestamp* * (*TimestampFrequencyInHz*). To inquire the clock frequency, use MdigInquire() with M_DATA_LATCH_CLOCK_FREQUENCY.

Data latch example

For an example of how to use Zebra Rapixo CL data latches, refer to the *DataLatch.cpp* example.

Zebra Rapixo Series CL tools

There are five Zebra Rapixo CL external applications (utilities) available (Usage Meter, Rapixo CL Bench, Board Information, Deserializer Setup tool, and Aurora Imaging Gecho Viewer) and a series of tools integrated in the Aurora Imaging Configurator utility (such as, the PCIe Information tool and the System Monitor tool).

To access these utilities and tools, launch the Aurora Imaging Configurator utility. Then, expand the **Boards** item in the tree structure, click on the **Rapixo series** subitem in the tree structure, and then select **CL**. The utilities are located in the **Launch External Applications** and the tools are located in the **PCIe Information** and **System Monitor** sub-entries, respectively.

Usage Meter utility

The Usage Meter utility calculates how much is being used of the grab section and the transfer section of each DMA engine. This rate of utilization is expressed as a percentage. This utility also shows the usage of on-board memory.

Rapixo CL Bench utility

The Rapixo CL Bench utility calculates the real-time transfer speed of the PCIe slot (in Mbytes/sec) from:

- Zebra Rapixo CL to Host.
- Host to Zebra Rapixo CL.

Board Information

The Board Information tool displays a feature browser window with information of each board. This utility shows information, such as:

- Hardware identification: (such as, board revision, firmware, location, and serial number).
- Hardware resources: (such as, DCF support, digitizer number, free memory, PCIe lanes, and PCIe speed).

Deserializer setup tool

The Deserializer setup tool finds the initial values needed for the deserializer alignment process when grabbing data. The values are determined for each Camera Link connection that is in-use. The resulting values are saved in the DCF, and help compensate for signal degradation.

> Note that it is recommended to use this tool only when using cables longer than 7 meters and when acquired images are corrupted. For more information on using the Deserializer setup tool, refer to the Zebra Rapixo CL installation and hardware reference manual.

Aurora Imaging Gecho Viewer

Aurora Imaging Gecho Viewer is an event logging application that gives you the ability to monitor and troubleshoot acquisition performance.

PCIe Information tool

The PCIe Information tool returns information about the board's PCIe connector.

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn).

> Note that this selector is shared with the System Monitor tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- Current number of PCIe lanes.
- Current speed of the PCIe lanes.
- Maximum number of PCIe lanes.
- Maximum speed of the PCIe lanes.

System Monitor tool

The System Monitor tool calculates the temperatures of the specified device (M_DEVn), and presents the following information inside the Aurora Imaging Configurator utility:

- An acquisition (device) selector.
- The grab FPGA current temperature, in degrees Celsius.
- The grab FPGA current temperature, as shown on a sliding scale from 0 to 85 degrees Celsius.
- The fan speed, in RPM.
- The sensor for the auxiliary power connection.
- Either a check mark (if your Zebra Rapixo CL is operating in the normal range of temperatures) or an X (if your Zebra Rapixo CL is operating above the normal range of temperatures).

Zebra Rapixo CL DB and Pro DB/QB connectors and signal names

This section serves as a reference to match Zebra Rapixo CL DB, CL Pro DB, and CL Pro QB's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary/camera control signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

Zebra Rapixo CL family members have a different number of acquisition paths and auxiliary/camera control signals. For each Zebra Rapixo CL family member, only the auxiliary/camera control signals associated with the following digitizer device numbers are supported:

Zebra Rapixo CL type	Digitizer device #
Zebra Rapixo CL DB	M_DEV0, M_DEV1.
Zebra Rapixo CL Pro DB	M_DEV0, M_DEV1.
Zebra Rapixo CL Pro QB	M_DEV0, M_DEV1, M_DEV2, M_DEV3.

The following table lists the connectors of the auxiliary/control signals that can be used for each digitizer device number:

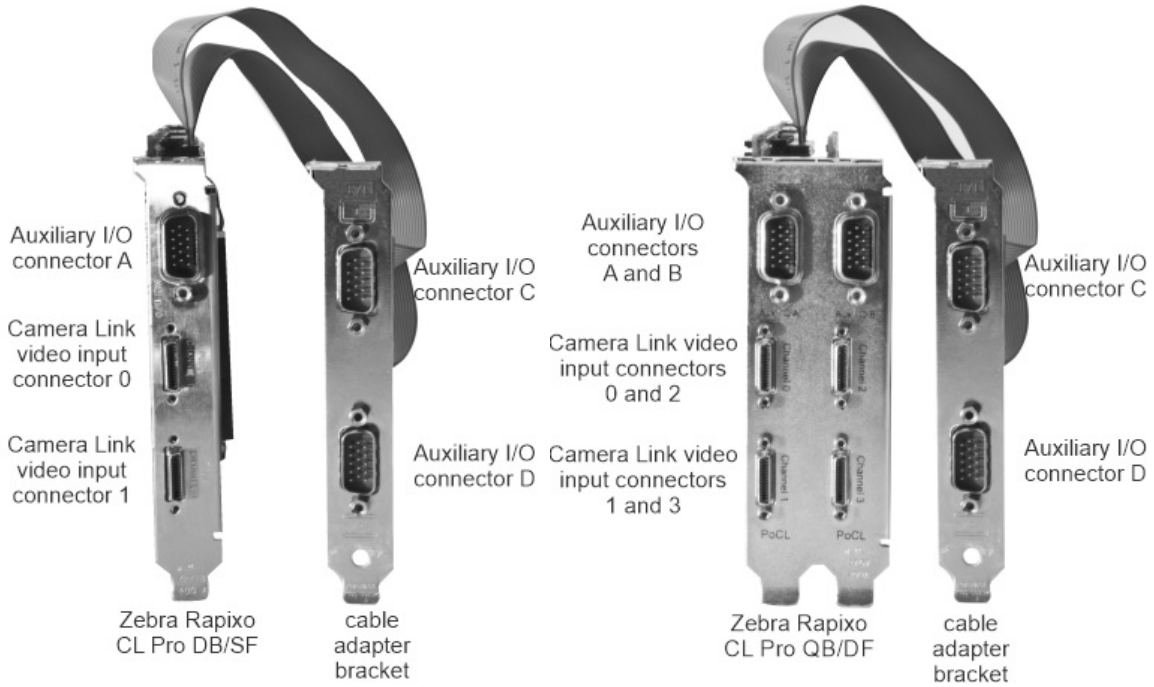
Digitizer device #	Zebra Rapixo CL DB	Zebra Rapixo CL Pro DB	Zebra Rapixo CL Pro QB
M_DEV0	Auxiliary I/O connector A and some on C	Auxiliary I/O connector A and some on C	Auxiliary I/O connector A and some on C
M_DEV1	Auxiliary I/O connector C and some on A	Auxiliary I/O connector C and some on A	Auxiliary I/O connector C and some on A
M_DEV2			Auxiliary I/O connector B and some on D
M_DEV3			Auxiliary I/O connector D and some on B

Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Rapixo CL DB, CL Pro DB, and CL Pro QB boards and cable adapter brackets provide several interface connectors. On the bracket of Zebra Rapixo CL DB and CL Pro DB, there are two Camera Link video input connectors and an auxiliary I/O connector. On the double bracket of Zebra Rapixo CL Pro QB, there are two pairs of Camera Link video input connectors and 2 auxiliary I/O connectors. On the cable adapter bracket, there are two external auxiliary I/O connectors.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Camera Link video input connectors	SDR (0,1,2 and 3)		The Camera Link video input connectors are 26-pin high-density female mini Camera Link connectors. They are used to receive video input, timing, synchronization signals and transmit/receive communication signals between the video source and the frame grabber. There are two Camera Link video input connectors (SDR 0 and 1) on Zebra Rapixo CL DB and CL Pro DB. There are four Camera Link video input connectors (SDR 0, 1, 2 and 3) on Zebra Rapixo CL Pro QB.

External auxiliary I/O connector	HD-15 [1] (A, B, C and D)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connectors A and (if present) B are located on Zebra Rapixo CL Pro, and external auxiliary I/O connectors C and D are located on the cable adapter bracket.

[1] Previously referred to as DBHD-15, but more accurately known as DE-15.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEV0	Indicates the following.
I/O #: M_AUX_IO0 Shared with: M_DEV1	Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 15+, 9-
Direction	Input
Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 0 or 2 on acq path 1; 2 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN8
I/O #: M_AUX_IO1 Shared with: M_DEV1	Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 12+, 11-
Direction	Input
Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN9

I/O #: M_AUX_IO2 Shared with: M_DEV1		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 0 for timer output.
	Pin information	Connector: HD-15 (A) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_6
I/O #: M_AUX_IO3 Shared with: M_DEV1		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 1 for timer output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 3 on acq path 0; 3 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4 Shared with: M_DEV1		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for quadrature input bit 0.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 or 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5 Shared with: M_DEV1		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for timer-clock input or quadrature input bit 1.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11

I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN0
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN1
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 0, which supports: user input/output, trigger input.
	Pin information	Connector: HD-15 (A) Pin: 1
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	TTL_AUX_IO_4
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 0, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (A) Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_5
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input, user input, or quadrature input bit 0.

		Pin information	Connector: HD-15 (A) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO10; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
		Hardware manual signal name	LVDS_AUX_IN2
	I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 0, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
		Pin information	Connector: HD-15 (A) Pin: 6+, 8-
		Direction	Input
		Trigger information	M_AUX_IO11; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
		Hardware manual signal name	LVDS_AUX_IN3
	I/O #: M_AUX_IO12		LVDS auxiliary signal (output) for acquisition path 0, which supports: timer output or user output.
		Pin information	Connector: HD-15 (A) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV0;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
		Hardware manual signal name	LVDS_AUX_OUT7
	I/O #: M_CC_IO1		Camera control output 1 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
		Pin information	Connector: HDR/SDR (0) Pin: 5+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;

	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC1 (0)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC2 (0)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC3 (0)

	I/O #: M_CC_IO4		Camera control output 4 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 or M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
		Pin information	Connector: HDR/SDR (0) Pin: 15+, 2-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
		Hardware manual signal name	CC4 (0)
Digitizer device #: M_DEV1			Indicates the following.
	I/O #: M_AUX_IO0 Shared with: M_DEV0		Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
		Pin information	Connector: HD-15 (C) Pin: 15+, 9-
		Direction	Input
		Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 0 or 2 on acq path 1; 2 on acq path 0.
		Hardware manual signal name	OPTO_AUX_IN8
	I/O #: M_AUX_IO1 Shared with: M_DEV0		Opto-isolated auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input.
		Pin information	Connector: HD-15 (C) Pin: 12+, 11-
		Direction	Input
		Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
		Hardware manual signal name	OPTO_AUX_IN9
	I/O #: M_AUX_IO2 Shared with: M_DEV0		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 0 for timer output.
		Pin information	Connector: HD-15 (A) Pin: 3
		Direction	Input/Output

	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_6
I/O #: M_AUX_IO3 Shared with: M_DEV0		TTL auxiliary signal (input/output), shared between acquisition paths 0 and 1 for trigger input, user input or user output, and dedicated to acquisition path 1 for timer output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0, M_DEV1;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 3 on acq path 0; 3 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4 Shared with: M_DEV0		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for quadrature input bit 0.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 2 on acq path 0; 2 or 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5 Shared with: M_DEV0		LVDS auxiliary signal (input), shared between acquisition paths 0 and 1 for trigger input or user input, and dedicated to acquisition path 1 for timer-clock input or quadrature input bit 1.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0, M_DEV1; Trigger controller: 1 or 3 on acq path 1; 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (C) Pin: 1
	Direction	Input/Output

		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV1;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV1; Trigger controller: 0 on acq path 1.
		Hardware manual signal name	TTL_AUX_IO_12
I/O #: M_AUX_IO9			TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (C) Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV1;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV1; Trigger controller: 1 on acq path 1.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV1;
		Hardware manual signal name	TTL_AUX_IO_13
I/O #: M_AUX_IO12			LVDS auxiliary signal (output) for acquisition path 1, which supports: timer output or user output.
		Pin information	Connector: HD-15 (C) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
		Hardware manual signal name	LVDS_AUX_OUT15
I/O #: M_CC_IO1			Camera control output 1 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
		Pin information	Connector: HDR/SDR (1) Pin: 5+, 18-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;

	Hardware manual signal name	CC1 (1)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
	Pin information	Connector: HDR/SDR (1) Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	CC2 (1)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
	Pin information	Connector: HDR/SDR (1) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
	Hardware manual signal name	CC3 (1)

	I/O #: M_CC_IO4		Camera control output 4 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV0 or M_DEV1 (OPTO_AUX_IN8). • M_AUX_IO1 on M_DEV0 and M_DEV1 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 and M_DEV1 (TTL_AUX_IO_6). • M_AUX_IO4 on M_DEV0 or M_DEV1 (LVDS_AUX_IN10). • M_AUX_IO5 on M_DEV0 and M_DEV1 (LVDS_AUX_IN11). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_12).
		Pin information	Connector: HDR/SDR (1) Pin: 15+, 2-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1;
		Hardware manual signal name	CC4 (1)
Digitizer device #: M_DEV2 [2]			Indicates the following.
	I/O #: M_AUX_IO0 Shared with: M_DEV3 [2]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 15+, 9-
		Direction	Input
		Trigger information	M_AUX_IO0; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN24
	I/O #: M_AUX_IO1 Shared with: M_DEV3 [2]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
		Pin information	Connector: HD-15 (D) Pin: 12+, 11-
		Direction	Input
		Trigger information	M_AUX_IO1; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	OPTO_AUX_IN25
	I/O #: M_AUX_IO2 Shared with: M_DEV3 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 2 for timer output.
		Pin information	Connector: HD-15 (B) [3] Pin: 3
		Direction	Input/Output

		User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ;
		Trigger information	M_AUX_IO2; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 2 on acq path 2; 2 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV2 [2] ;
		Hardware manual signal name	TTL_AUX_IO_22
	I/O #: M_AUX_IO3 Shared with: M_DEV3 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 3 for timer output.
		Pin information	Connector: HD-15 (D) Pin: 3
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ;
		Trigger information	M_AUX_IO3; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 3 on acq path 2; 3 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	TTL_AUX_IO_30
	I/O #: M_AUX_IO4 Shared with: M_DEV3 [2]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for quadrature input bit 0.
		Pin information	Connector: HD-15 (D) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO4; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN26
	I/O #: M_AUX_IO5 Shared with: M_DEV3 [2]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for timer-clock input or quadrature input bit 1.
		Pin information	Connector: HD-15 (D) Pin: 6+, 8-
		Direction	Input
		Trigger information	M_AUX_IO5; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN27
	I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 2, which supports: user input or trigger input.
		Pin information	Connector: HD-15 (B) [3] Pin: 15+, 9-
		Direction	Input

	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV2 [2] ; Trigger controller: 0 on acq path 2.
	Hardware manual signal name	OPTO_AUX_IN16
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 2, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (B) [3] Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV2 [2] ; Trigger controller: 1 on acq path 2.
	Hardware manual signal name	OPTO_AUX_IN17
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 2, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 1
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV2 [2] ;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV2 [2] ; Trigger controller: 0 on acq path 2.
	Hardware manual signal name	TTL_AUX_IO_20
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 2, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV2 [2] ;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV2 [2] ; Trigger controller: 1 on acq path 2.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	TTL_AUX_IO_21
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 2, which supports: trigger input, user input, or quadrature input bit 0.
	Pin information	Connector: HD-15 (B) [3] Pin: 4+, 5-
	Direction	Input

	Trigger information	M_AUX_IO10; Digitizer device #: M_DEV2 [2] ; Trigger controller: 0 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN18
I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 2, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (B) [3] Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV2 [2] ; Trigger controller: 1 on acq path 2.
	Hardware manual signal name	LVDS_AUX_IN19
I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 2, which supports: timer output or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 13+, 14-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV2 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	LVDS_AUX_OUT23
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	CC1 (2)

I/O #: M_CC_IO2		Camera control output 2 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	CC2 (2)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	CC3 (2)

I/O #: M_CC_IO4		Camera control output 4 for acquisition path 2, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV2 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV2 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV2 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV2 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV2 [2] ;
	Hardware manual signal name	CC4 (2)
Digitizer device #: M_DEV3 [2]		Indicates the following.
I/O #: M_AUX_IO0 Shared with: M_DEV2 [2]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO0; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
	Hardware manual signal name	OPTO_AUX_IN24
I/O #: M_AUX_IO1 Shared with: M_DEV2 [2]		Opto-isolated auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO1; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
	Hardware manual signal name	OPTO_AUX_IN25
I/O #: M_AUX_IO2 Shared with: M_DEV2 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 2 for timer output.
	Pin information	Connector: HD-15 (B) [3] Pin: 3

		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ;
		Trigger information	M_AUX_IO2; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 2 on acq path 2; 2 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV2 [2] ;
		Hardware manual signal name	TTL_AUX_IO_22
	I/O #: M_AUX_IO3 Shared with: M_DEV2 [2]		TTL auxiliary signal (input/output), shared between acquisition paths 2 and 3 for trigger input, user input or user output, and dedicated to acquisition path 3 for timer output.
		Pin information	Connector: HD-15 (D) Pin: 3
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ;
		Trigger information	M_AUX_IO3; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 3 on acq path 2; 3 on acq path 3.
		Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	TTL_AUX_IO_30
	I/O #: M_AUX_IO4 Shared with: M_DEV2 [2]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for quadrature input bit 0.
		Pin information	Connector: HD-15 (D) Pin: 4+, 5-
		Direction	Input
		Trigger information	M_AUX_IO4; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 0 or 2 on acq path 3; 2 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN26
	I/O #: M_AUX_IO5 Shared with: M_DEV2 [2]		LVDS auxiliary signal (input), shared between acquisition paths 2 and 3 for trigger input or user input, and dedicated to acquisition path 3 for timer-clock input or quadrature input bit 1.
		Pin information	Connector: HD-15 (D) Pin: 6+, 8-
		Direction	Input
		Trigger information	M_AUX_IO5; Digitizer device #: M_DEV2 [2] , M_DEV3 [2] ; Trigger controller: 1 or 3 on acq path 3; 3 on acq path 2.
		Hardware manual signal name	LVDS_AUX_IN27
	I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 3, which supports: trigger input, user input, or user output.
		Pin information	Connector: HD-15 (D) Pin: 1

		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV3 [2] ;
		Trigger information	M_AUX_IO8; Digitizer device #: M_DEV3 [2] ; Trigger controller: 0 on acq path 3.
		Hardware manual signal name	TTL_AUX_IO_28
	I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 3, which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (D) Pin: 2
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV3 [2] ;
		Trigger information	M_AUX_IO9; Digitizer device #: M_DEV3 [2] ; Trigger controller: 1 on acq path 3.
		Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	TTL_AUX_IO_29
	I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 3, which supports: timer output or user output.
		Pin information	Connector: HD-15 (D) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV3 [2] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	LVDS_AUX_OUT31
	I/O #: M_CC_IO1		Camera control output 1 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [3] Pin: 5+, 18-
		Direction	Output

		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [2] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	CC1 (3)
	I/O #: M_CC_IO2		Camera control output 2 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [3] Pin: 17+, 4-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [2] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	CC2 (3)
	I/O #: M_CC_IO3		Camera control output 3 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [3] Pin: 3+, 16-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [2] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	CC3 (3)

	I/O #: M_CC_IO4		Camera control output 4 for acquisition path 3, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO0 on M_DEV2 or M_DEV3 (OPTO_AUX_IN24). • M_AUX_IO1 on M_DEV2 and M_DEV3 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV2 and M_DEV3 (TTL_AUX_IO_22). • M_AUX_IO4 on M_DEV2 or M_DEV3 (LVDS_AUX_IN26). • M_AUX_IO5 on M_DEV2 and M_DEV3 (LVDS_AUX_IN27). • M_AUX_IO8 on M_DEV3 (TTL_AUX_IO_28).
		Pin information	Connector: HDR/SDR (3) [3] Pin: 15+, 2-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV3 [2] ;
		Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV3 [2] ;
		Hardware manual signal name	CC4 (3)

[2] Only applicable to Zebra Rapixo CL Pro.

[3] Only Zebra Rapixo CL Pro QB has this connector; Zebra Rapixo CL Pro DB does not have this connector.

Zebra Rapixo CL SF and CL Pro SF/DF connectors and signal names

This section serves as a reference to match Zebra Rapixo CL SF, Pro SF, and Pro DF's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary/camera control signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

Zebra Rapixo CL family members have a different number of acquisition paths and auxiliary/camera control signals. For each Zebra Rapixo CL family member, only the auxiliary/camera control signals associated with the following digitizer device numbers are supported:

Zebra Rapixo CL type	Digitizer device #
Zebra Rapixo CL SF	M_DEV0.
Zebra Rapixo CL Pro SF	M_DEV0.
Zebra Rapixo CL Pro DF	M_DEV0, M_DEV1.

The following table lists the connectors of the auxiliary/control signals that can be used for each digitizer device number:

Digitizer device #	Zebra Rapixo CL SF	Zebra Rapixo CL Pro SF	Zebra Rapixo CL Pro DF
M_DEV0	Auxiliary I/O connector A and C	Auxiliary I/O connector A and C	Auxiliary I/O connector A and C
M_DEV1			Auxiliary I/O connector B and D

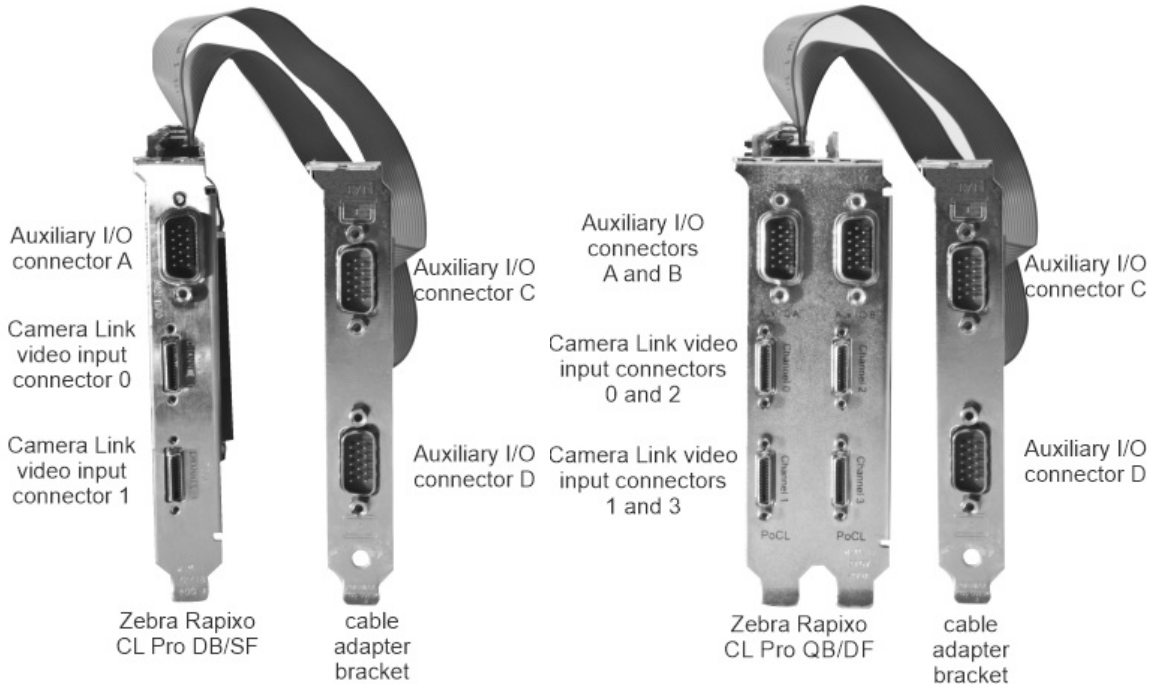
Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the

board's installation and hardware reference manual.

Board connectors

The Zebra Rapixo CL SF, Pro SF, and Pro DF boards and cable adapter brackets provide several interface connectors. On the bracket of Zebra Rapixo CL and Pro SF, there are two Camera Link video input connectors and an auxiliary I/O connector. On the double bracket of Zebra Rapixo CL Pro DF, there are two pairs of Camera Link video input connectors and two auxiliary I/O connectors. On the cable adapter bracket, there are two external auxiliary I/O connectors.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
Camera Link video input connectors	SDR (0, 1, 2, and 3)		The Camera Link video input connectors are 26-pin high-density female mini Camera Link connectors. They are used to receive video input, timing, synchronization signals and transmit/receive communication signals between the video source and the frame grabber. There are two Camera Link video input connectors (SDR 0 and 1) on Zebra Rapixo CL SF and Pro SF. There are four Camera Link video input connectors (SDR 0, 1, 2 and 3) on Zebra Rapixo CL Pro DF.
External auxiliary I/O connector	HD-15 [1] (A, B, C, and D)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connectors A and (if present) B are located on Zebra Rapixo CL Pro, and external auxiliary I/O connectors C and D are located on the cable adapter bracket.

[1] Previously referred to as DBHD-15, but more accurately known as DE-15.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEV0	Indicates the following.
I/O #: M_AUX_IO0	Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 15+, 9-
Direction	Input
Trigger information	M_AUX_IO0; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN8
I/O #: M_AUX_IO1	Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
Pin information	Connector: HD-15 (C) Pin: 12+, 11-
Direction	Input
Trigger information	M_AUX_IO1; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
Hardware manual signal name	OPTO_AUX_IN9
I/O #: M_AUX_IO2	TTL auxiliary signal (input/output) for acquisition path 0, which supports: trigger input, user input, user output, or timer output.
Pin information	Connector: HD-15 (A) Pin: 3
Direction	Input/Output
User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV0;

	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_6
I/O #: M_AUX_IO3		TTL auxiliary signal (input/output) for acquisition path 0, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (C) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
	Hardware manual signal name	TTL_AUX_IO_14
I/O #: M_AUX_IO4		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV0; Trigger controller: 2 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN10
I/O #: M_AUX_IO5		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (C) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV0; Trigger controller: 3 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN11
I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN0

I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 0, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (A) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Hardware manual signal name	OPTO_AUX_IN1
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 0, which supports: user input/output, trigger input.
	Pin information	Connector: HD-15 (A) Pin: 1
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	TTL_AUX_IO_4
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 0, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (A) Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV0;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV0;
	Hardware manual signal name	TTL_AUX_IO_5
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 0, which supports: trigger input, user input, or quadrature input bit 0.
	Pin information	Connector: HD-15 (A) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO10; Digitizer device #: M_DEV0; Trigger controller: 0 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN2

I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 0, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (A) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV0; Trigger controller: 1 on acq path 0.
	Hardware manual signal name	LVDS_AUX_IN3
I/O #: M_AUX_IO12		LVDS auxiliary signal (output) for acquisition path 0, which supports: timer output or user output.
	Pin information	Connector: HD-15 (A) Pin: 13+, 14-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	LVDS_AUX_OUT7
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC1 (0)

I/O #: M_CC_IO2		Camera control output 2 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC2 (0)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC3 (0)

I/O #: M_CC_IO4		Camera control output 4 for acquisition path 0, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV0 (OPTO_AUX_IN9). • M_AUX_IO2 on M_DEV0 (TTL_AUX_IO_6). • M_AUX_IO5 on M_DEV0 (LVDS_AUX_IN11). • M_AUX_IO7 on M_DEV0 (OPTO_AUX_IN1). • M_AUX_IO8 on M_DEV0 (TTL_AUX_IO_4). • M_AUX_IO10 on M_DEV0 (LVDS_AUX_IN2).
	Pin information	Connector: HDR/SDR (0) Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV0;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV0;
	Hardware manual signal name	CC4 (0)
Digitizer device #: M_DEV1 [2]		Indicates the following.
I/O #: M_AUX_IO0		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 15+, 9-
	Direction	Input
	Trigger information	M_AUX_IO0; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN24
I/O #: M_AUX_IO1		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO1; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN25
I/O #: M_AUX_IO2		TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 3

	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO2; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
	Timer information	Timer: M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	TTL_AUX_IO_22
I/O #: M_AUX_IO3		TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (D) Pin: 3
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO3; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.
	Hardware manual signal name	TTL_AUX_IO_30
I/O #: M_AUX_IO4		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO4; Digitizer device #: M_DEV1 [2] ; Trigger controller: 2 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN26
I/O #: M_AUX_IO5		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input or user input.
	Pin information	Connector: HD-15 (D) Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO5; Digitizer device #: M_DEV1 [2] ; Trigger controller: 3 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN27
I/O #: M_AUX_IO6		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (B) [3] Pin: 15+, 9-

	Direction	Input
	Trigger information	M_AUX_IO6; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN16
I/O #: M_AUX_IO7		Opto-isolated auxiliary signal (input) for acquisition path 1, which supports: user input or trigger input.
	Pin information	Connector: HD-15 (B) [3] Pin: 12+, 11-
	Direction	Input
	Trigger information	M_AUX_IO7; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Hardware manual signal name	OPTO_AUX_IN17
I/O #: M_AUX_IO8		TTL auxiliary signal (input/output) for acquisition path 1, which supports: trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 1
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT2; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO8; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	TTL_AUX_IO_20
I/O #: M_AUX_IO9		TTL auxiliary signal (input/output) for acquisition path 1, which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 2
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT3; Digitizer device #: M_DEV1 [2] ;
	Trigger information	M_AUX_IO9; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Timer information	Timer: M_TIMER1; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	TTL_AUX_IO_21
I/O #: M_AUX_IO10		LVDS auxiliary signal (input) for acquisition path 1, which supports: trigger input, user input, or quadrature input bit 0.

	Pin information	Connector: HD-15 (B) [3] Pin: 4+, 5-
	Direction	Input
	Trigger information	M_AUX_IO10; Digitizer device #: M_DEV1 [2] ; Trigger controller: 0 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN18
I/O #: M_AUX_IO11		LVDS auxiliary signal (input) for acquisition path 1, which supports: user input, trigger input, timer-clock input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (B) [3] Pin: 6+, 8-
	Direction	Input
	Trigger information	M_AUX_IO11; Digitizer device #: M_DEV1 [2] ; Trigger controller: 1 on acq path 1.
	Hardware manual signal name	LVDS_AUX_IN19
I/O #: M_AUX_IO13		LVDS auxiliary signal (output) for acquisition path 1, which supports: timer output or user output.
	Pin information	Connector: HD-15 (B) [3] Pin: 13+, 14-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT0; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	LVDS_AUX_OUT23
I/O #: M_CC_IO1		Camera control output 1 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 5+, 18-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;

	Hardware manual signal name	CC1 (2)
I/O #: M_CC_IO2		Camera control output 2 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 17+, 4-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC2 (2)
I/O #: M_CC_IO3		Camera control output 3 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 3+, 16-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC3 (2)

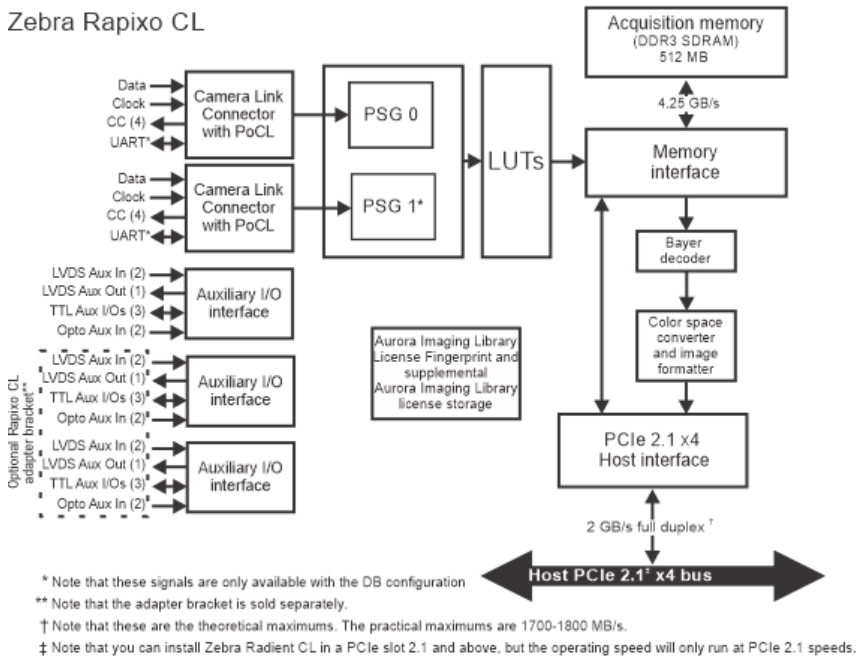
	I/O #: M_CC_IO4	Camera control output 4 for acquisition path 1, which supports: timer output, user output, VSYNC output, HSYNC output, clock output, or rerouting of specific auxiliary input signals. Only the following auxiliary input signals can be rerouted to this output signal: • M_AUX_IO1 on M_DEV1 (OPTO_AUX_IN25). • M_AUX_IO2 on M_DEV1 (TTL_AUX_IO_22). • M_AUX_IO5 on M_DEV1 (LVDS_AUX_IN27). • M_AUX_IO7 on M_DEV1 (OPTO_AUX_IN17). • M_AUX_IO8 on M_DEV1 (TTL_AUX_IO_20). • M_AUX_IO10 on M_DEV1 (LVDS_AUX_IN18).
	Pin information	Connector: HDR/SDR (2) [3] Pin: 15+, 2-
	Direction	Output
	User-bit information	User-bit #: M_USER_BIT_CC_IO0/M_USER_BIT_CC_IO1; Digitizer device #: M_DEV1 [2] ;
	Timer information	Timer: M_TIMER1/M_TIMER2; Digitizer device #: M_DEV1 [2] ;
	Hardware manual signal name	CC4 (2)

[2] Only applicable to Zebra Rapixo CL Pro DF.

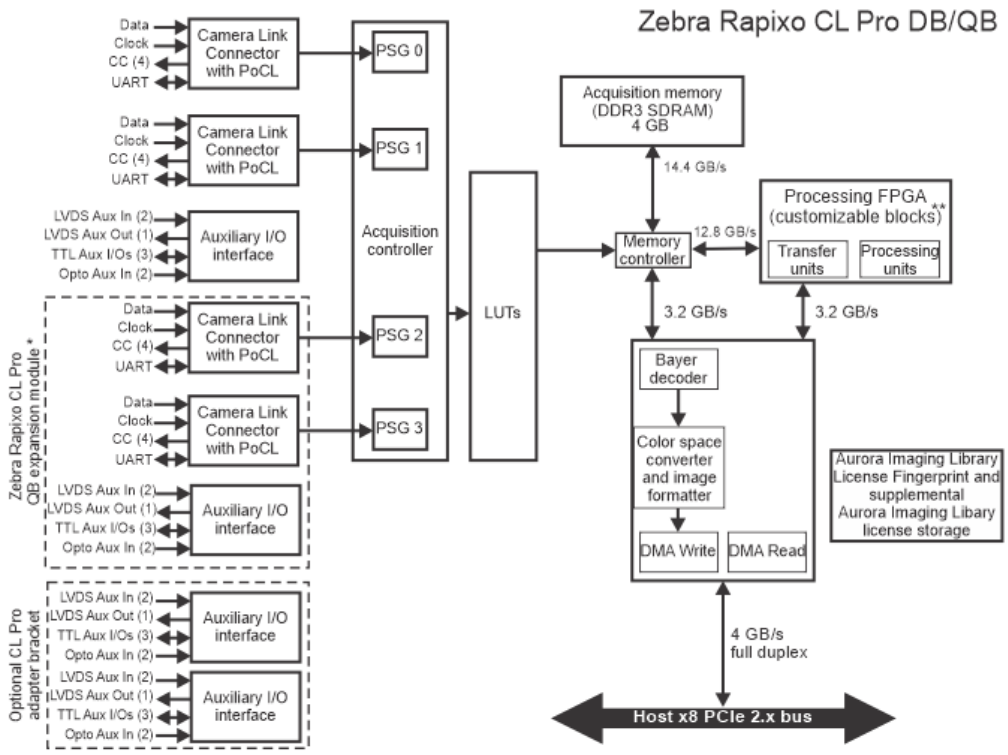
[3] Only Zebra Rapixo CL Pro DF has this connector; Zebra Rapixo CL Pro SF does not have this connector.

Zebra Rapixo CL board flow diagrams

The following diagram illustrates the data flow of Zebra Rapixo CL DB.



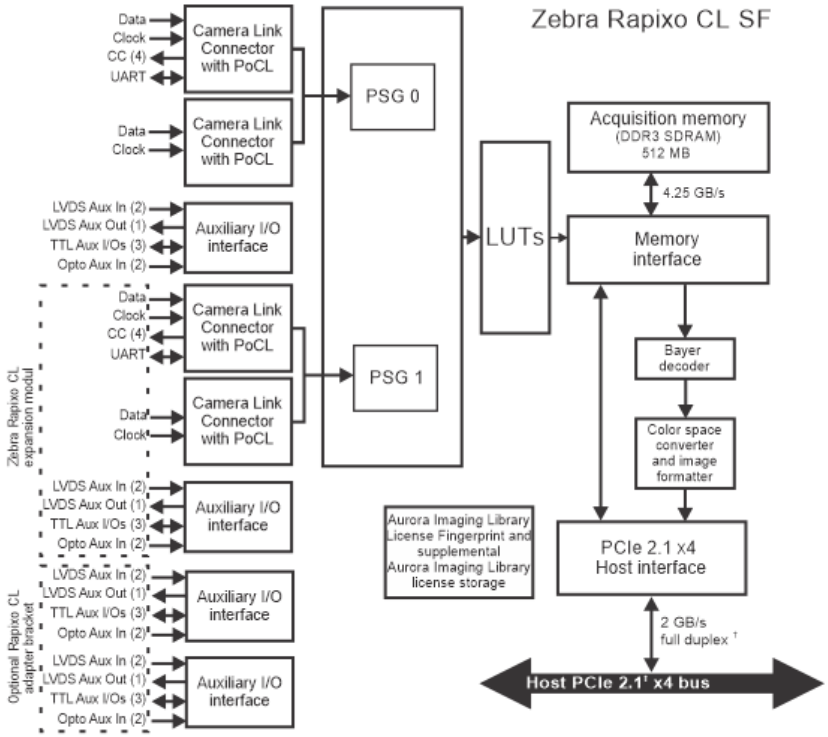
The following diagram illustrates the data flow of Zebra Rapixo CL Pro DB/QB.



* Note that, without the expansion module, this is a flow diagram of Zebra Rapixo CL Pro DB. With the expansion module, this is a flow diagram of Zebra Rapixo CL Pro QB. CL Pro DB is not upgradable to CL Pro QB; the CL Pro DB does not have the required connector for the CL Pro QB module.

**Note that this block represents only the customizable processing block of the on-board FPGA. Other functionality in this diagram is also implemented using the FPGA.

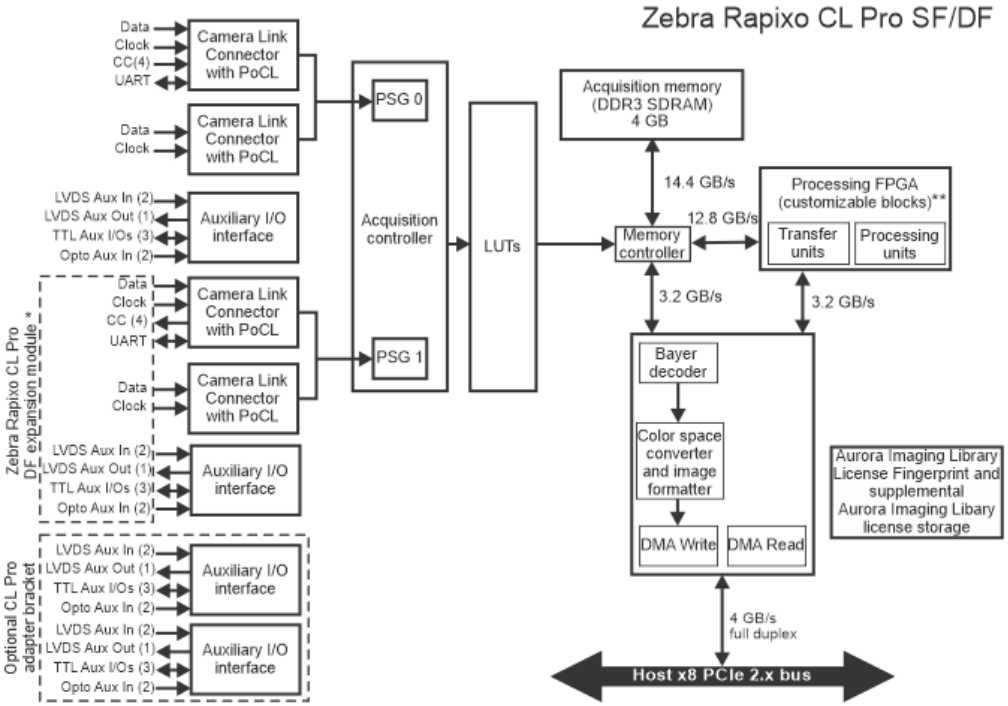
The following diagram illustrates the data flow of Zebra Rapixo CL SF.



† Note that these are the theoretical maximums. The practical maximum is 1.8 GB/s.

‡ Note that you can install Zebra Rapixo-CL in a PCIe slot 2.1 and above, but the operating speed will only run at PCIe 2.1 speeds.

The following diagram illustrates the data flow of Zebra Rapixo CL Pro SF/DF.



* Note that, without the expansion module, this is a flow diagram of Zebra Rapixo CL Pro DB. With the expansion module, this is a flow diagram of Zebra Rapixo CL Pro QB. CL Pro DB is not upgradable to CL Pro QB; the CL Pro DB does not have the required connector for the CL Pro QB module.

** Note that this block represents only the customizable processing block of the on-board FPGA. Other functionality in this diagram is also implemented using the FPGA.

Miscellaneous user guide information for Zebra Rapixo CL

This section includes additional user guide information for Zebra Rapixo CL. The information found in this section might be a reiteration of content previously documented.

Supported standards and development kit

Zebra Rapixo CL supports the following standards:

- Camera Link version 2.1.
- GenICam GenApi version 3.4.2.
- GenICam GenCP Camera Link Module version 1.3.
- GenICam for Camera Link (CLProtocol) version 1.2.

  Note that this requires a third-party CLProtocol communication library (DLL) which the camera vendor supplies.

Zebra Rapixo CL supports Zebra FPGA Development Kit (FDK) version 22.2 and later.

Additional features

The following are additional features supported for Zebra Rapixo CL.

- Firmware updates without power-cycling.
- 2 taps 10-bit RGB and 6 taps 10-bit mono.
- RGB 10/12 bits Gain and Offset. For more information, use MdigControl() with **M_SHADING_CORRECTION**.
- Data latches on all acquisition paths.
- Changing the timer duration during a grab.
- Scatter-gather DMA transfers.

Aurora Imaging Configurator

The following are two tools integrated in Aurora Imaging Configurator (*AILConfig.exe*) for Zebra Rapixo CL. To access these tools, launch the Aurora Imaging Configurator utility (*AILConfig.exe*). Expand the **Boards** item in the tree structure and then click on the **Rapixo CL** subitem from the tree structure in the presented interface.

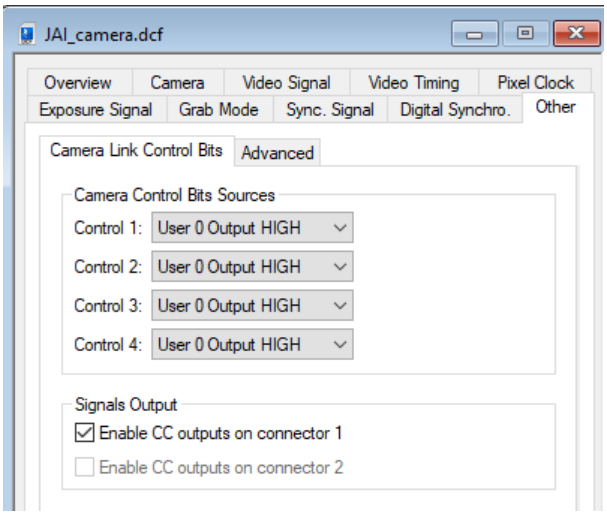
- Board Information: This tool is used to display details for each processing FPGA IP present in the current firmware.
- Deserializer Setup tool: This tool is used to calibrate the deserializer when using cables longer than 7 meters or when acquired images are corrupt.

In the same page, you can change the board type from Single-Full to Dual-Base, or vice versa.

Particularities

When using a JAI camera, the Camera Link Camera Control bits on the Zebra Rapixo CL must be activated for stable

communication with the camera (either with JAI or Zebra software). The bits can be activated using Aurora Imaging Intellicam in the **Other** tab of the **DCF** dialog.



When editing a DCF for Camera Link using Aurora Imaging Intellicam, Feature Browser will close and reopen. Reopening a Feature Browser window might be subject to a noticeable delay due to camera initialization. To avoid this, uncheck "Automatically compute registers" in the **Preferences Register Compute** menu tab. However, you will then need to manually compute the DCF registers using the **DCF Compute Registers** menu tab.

DCFs with manually edited register values for OPTO_AUX_IN0, OPTO_AUX_IN1, LVDS_AUX_IN2, LVDS_AUX_IN3, TTL_AUX_IO_4, TTL_AUX_IO_5, TTL_AUX_IO_6, OPTO_AUX_IN8, OPTO_AUX_IN9, LVDS_AUX_IN10, LVDS_AUX_IN11, TTL_AUX_IO_12, TTL_AUX_IO_13, TTL_AUX_IO_14 will need to have these values adjusted by adding 0x1000 (for example, 0x64 becomes 0x1064).

Zebra Rapixo CXP

This section discusses features of Aurora Imaging Library that are particular to Zebra Rapixo CXP family of boards and ways that optimize their performance. Zebra Rapixo CXP family specific information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Rapixo CXP overview
  - Summary of Zebra Rapixo CXP features
  - Note on nomenclature
- Using Zebra Rapixo CXP with Aurora Imaging Library
  - Grabbing and the formats to use for each camera pixel format
  - Using the CoaXPress trigger signal on Zebra Rapixo CXP
  - Performing Bayer color conversion in hardware
  - Using frame burst with a multi-frame buffer
  - Using on-board flat-field correction
  - Extracting peaks from an on-board multi-frame image buffer
  - Data forwarding with Zebra Rapixo CXP
- Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CXP boards
- Minimum latency and grabbing all frames
  - Detecting missed frames
  - Efficiently copying an on-board buffer to a destination buffer in Host memory
  - Efficiently converting buffer formats while copying
- Using Zebra Rapixo CXP data latches
  - Steps to configure and retrieve data from a data latch
  - Triggering a data latch
  - Data latch limitations
  - Retrieving data latch information
  - Data latch example
- Zebra Rapixo CXP utilities and tools
  - Usage Meter utility
  - Rapixo CXP Bench utility
  - Rapixo CXP EyeScan
  - Aurora Imaging Gecho Viewer
  - Aurora Imaging Capture Works
  - PCIe Information tool
  - System Monitor tool
- Zebra Rapixo CXP connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Rapixo CXP board flow diagrams
  - Zebra Rapixo CXP Single and Dual model
  - Zebra Rapixo CXP Quad CXP-6 and Quad CXP-12 model
  - Zebra Rapixo CXP Quad Data Forwarding model
  - Zebra Rapixo CXP Pro model
- Miscellaneous user guide information for Zebra Rapixo CXP
  - Supported standards and development kit
  - Additional features and improvements
  - Aurora Imaging Configurator
  - Aurora Imaging Capture Works
  - Aurora Imaging Gecho Viewer
  - Particularities

Zebra Rapixo CXP overview

Zebra Rapixo CXP is a family of high-performance PCIe frame grabbers. There are two models:

- **Zebra Rapixo CXP base model**. Zebra Rapixo CXP base model supports image capture from up to 4 high-resolution, high-speed video sources that use the CoaXPress (CXP) communication standard. There are five versions of the Zebra Rapixo CXP base model: Single, Dual, Quad CXP-6, Quad CXP-12, and Quad Data Forwarding.
- **Zebra Rapixo CXP Pro**. Zebra Rapixo CXP Pro supports the same functionality as the base model, as well as FPGA-based processing offload capabilities (Processing FPGA). There is only one version of this model: Pro Quad CXP-12.

Zebra Rapixo CXP supports frame and line-scan, monochrome and color video sources. The color video sources can be RGB video sources or video sources with a Bayer color filter. Zebra Rapixo CXP can decode Bayer color-encoded images and perform color space conversions while transferring the images to the Host. In addition, it can vertically and horizontally flip them and perform subsampling.

Zebra Rapixo CXP can receive images from each video source at up to the maximum CXP-6 or CXP-12 speed (depending on the model of the board and camera). Using link aggregation, the bandwidth of a link can be increased to 4 times the maximum CXP-6 and CXP-12 speeds for a single video source. Zebra Rapixo CXP provides up to 13 W of power per CoaXPress connection to any device that supports power-over-CoaXPress (PoCXP).

> You can access the latest version of the Installation and Hardware Reference at zebra.com/rapixo-cxp-info.

Summary of Zebra Rapixo CXP features

The following table outlines the features currently available for Zebra Rapixo CXP.

	Zebra Rapixo CXP
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature()	Yes
On-board memory	1 Gbytes DDR3 SDRAM (Zebra Rapixo CXP Single base model) 2 Gbytes DDR3 SDRAM (Zebra Rapixo CXP Dual base model) 4 Gbytes DDR4 SDRAM (Zebra Rapixo CXP Quad CXP-6/CXP-12 base model) 4 Gbytes DDR4 SDRAM (Zebra Rapixo CXP Quad Data Forwarding base model) 8 Gbytes DDR4 SDRAM (Zebra Rapixo CXP Pro Quad CXP-12)

Processing FPGA	No (Zebra Rapixo CXP base model)/ Yes (Zebra Rapixo CXP Pro Quad CXP-12)
Digitizer LUTs	Yes [1]
Frame burst technology	Yes
Flat-field correction	Yes [2]
Peak-extraction	Yes [3]
Acquisition section	CoaXPress
Number of acquisition paths	4
Number of auxiliary signals	32 shared (16 in, 4 out, 12 in/out)
Number of timers	4 shared
Number of quadrature decoders for input from linear or rotary encoders	4
Number of CoaXPress trigger signals (to camera only)	1/digitizer
Number of data latches	16/digitizer (allocated using M_DEV0 or M_DEV1 only)

[1] Only for 8, 10, or 12-bit data (monochrome or color). When a link is receiving color data, all bands of the data use the same specified LUT mapping. In addition, as soon as one link is receiving 12-bit data, all links (CoaXPress connectors) share the same specified LUT mapping.

[2] Only available on Zebra Rapixo CXP Pro Quad CXP-12 (with the appropriate firmware selected). Not available when using Zebra Rapixo CXP Pro Quad CXP-12 with GenTL.

[3] Only available on Zebra Rapixo CXP Pro Quad CXP-12 (with the appropriate firmware selected). Not available when using Zebra Rapixo CXP Pro Quad CXP-12 with GenTL.

Note on nomenclature

This manual refers to all versions of the board as Zebra Rapixo CXP. When necessary, this manual distinguishes between the boards using their full names (for example, Zebra Rapixo CXP Quad CXP-6, Zebra Rapixo CXP Quad CXP-

12, or Zebra Rapixo CXP Pro Quad CXP-12), or their abbreviated forms (Quad CXP-6, Quad CXP-12, and Pro Quad CXP-12).

Using Zebra Rapixo CXP with Aurora Imaging Library

To use Zebra Rapixo CXP, you must allocate it as an Aurora Imaging Library Rapixo CXP system (using MsysAlloc() with M_SYSTEM_RAPIXOCXP). This allocation opens communication with the Zebra Rapixo CXP board and allows Aurora Imaging Library to use its resources. You can allocate an Aurora Imaging Library Rapixo CXP system for your board in multiple processes (executables). However, different processes cannot allocate a digitizer for the same acquisition paths.

All Zebra Rapixo CXP boards feature 4 quadrature decoders that can decode input from linear or rotary encoders with quadrature output. For information on this feature, refer to the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. This chapter also provides information on how to configure and use the auxiliary I/O signals.

For information on how to use the Processing FPGA of your Zebra Rapixo CXP Pro, see Chapter 67: Using Aurora Imaging Library for FPGA processing.

Grabbing and the formats to use for each camera pixel format

The following table outlines which Aurora Imaging Library buffer format to use with each supported camera pixel format. Note that these are the officially supported camera pixel formats.

Camera pixel format [1]	Aurora Imaging Library buffer format (type and attribute)	Operation
Mono8, Mono10, Mono12, Mono14, Mono16, Mono32	1-band 8-bit, 1-band 16-bit, 1-band 32-bit, 3-band 8-bit [2] , 3-band 16-bit [3]	Data will automatically be bit-shifted to have most-significant bits (MSB) in destination buffer
BayerXX8, BayerXX10, BayerXX12, BayerXX14, BayerXX16 where XX = GR, GB, BG or RG	1-band 8-bit, 1-band 16-bit	Demosaicing followed by RGB to Y (bit shift for MSB)
	3-band 8-bit, 3-band 16-bit	Demosaicing (bit shift for MSB)
	3-band YUV16 + M_PACKED	Demosaicing followed by RGB to YUV
RGB8, RGBA8,	1-band 8-bit	RGB to Y
	3-band 8-bit	Direct copy
	3-band YUV16 + M_PACKED	RGB to YUV
	1-band 8-bit	RGB to Y (bit shift for MSB)

RGB10, RGBA10, RGB12, RGBA12, RGB14, RGBA14, RGB16, RGBA16	1-band 16-bit	RGB to Y
	3-band 16-bit M_PLANAR	Packed destination buffers are not supported
YUV422_8, YCbCr601_422_8, YCbCr709_422_8	1-band 8-bit	Only Y band
	3-band YUV16 + M_PACKED	Direct copy

[1] All of the values available for RGB are also available in BGR format.

[2] 3-band 8-bit buffers can be in any of the following formats: M_RGB24 + M_PLANAR, M_RGB24 + M_PACKED, M_BGR24 + M_PACKED, or M_BGR32 + M_PACKED.

[3] 3-band 16-bit buffers can be in either an M_RGB48 + M_PLANAR or M_RGB48 + M_PACKED format.

Using the CoaXPress trigger signal on Zebra Rapixo CXP

The CoaXPress trigger signal is an embedded signal transmitted along the physical transport layer connection of your camera. Typically, the CoaXPress trigger signal is reserved for trigger information and is sent with other control and data signals along the same cable. On Zebra Rapixo CXP, the CoaXPress trigger signal is only supported from the frame grabber to the camera (that is, as an output signal).

To use this signal to trigger the camera, use MdigControl() with M_IO_SOURCE + M_TL_TRIGGER set to the appropriate auxiliary input signals (for example, M_AUX_IO4).

```
/* Use CoaXPress trigger signal to trigger the camera. */
MdigControl(AilDigitizer, M_IO_SOURCE + M_TL_TRIGGER, M_AUX_IO4);
```

Note that when you set the camera to capture an image only upon a trigger, you should disable grab triggered mode on the frame grabber (M_GRAB_TRIGGER_STATE set to M_DISABLE). For more information on performing a triggered grab, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

Performing Bayer color conversion in hardware

When Zebra Rapixo CXP grabs color images from a video source with a Bayer color filter (as specified by the DCF), it performs Bayer color conversion in hardware, as it transfers the images to the Host. If the images require white balancing, Zebra Rapixo CXP can perform this automatically if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_DISABLE.

The M_BAYER... control types of MdigControl() can only be used when grabbing from a camera that has a Bayer color filter (as specified by the DCF); otherwise, an error will be generated.

Using frame burst with a multi-frame buffer

Zebra Rapixo CXP supports frame burst technology. This technology allows you to grab a group of sequential frames into a multi-frame buffer with one grab command (MdigGrab(), or one grab of MdigProcess()); the defined number of frames are stored contiguously in the same buffer. The end-of-grab event only occurs once the entire group of frames has been grabbed, reducing the number of events that need to be handled. This is useful in cases where you have a high frame rate and need to ensure that no frames are missed. Note that a user-defined function hooked to the end-of-grab event (MdigProcess(), or MdigHookFunction() with M_GRAB_END) is executed only once the entire group of frames has been grabbed. For information on creating a multi-frame image buffer to store sequential frames, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers.

Using on-board flat-field correction

Zebra Rapixo CXP Pro Quad CXP-12 supports flat-field correction on-board without Host intervention. Images taken in environments with uneven lighting can affect the quality of your processing. To correct this, an on-board flat-field correction can be applied to images so that the intensity across the image is even before processing begins. To use flat-field correction, you need to specify an on-board buffer that contains the gain values, and optionally another that contains the offset values, using MdigControl() with M_SHADING_CORRECTION_GAIN_ID and M_SHADING_CORRECTION_OFFSET_ID, respectively. The buffers should be the same size as the grabbed image, and the gain values should be in the specified fixed point format (M_SHADING_CORRECTION_GAIN_FIXED_POINT). The aggregate bandwidth to read the gain buffer and offset buffer from on-board memory cannot be bigger than the internal maximum bandwidth of the DMA port (8 Gbytes/sec). There is a line limitation of 64 K pixels per line and 1 M lines per frame. Once you have set the buffers, enable the correction using MdigControl() with M_SHADING_CORRECTION.

> Note that, flat-field correction is only available on Zebra Rapixo CXP Pro Quad CXP-12 with the appropriate firmware installed. Flat-field correction is not available when using Zebra Rapixo CXP Pro Quad CXP-12 with GenTL.

Extracting peaks from an on-board multi-frame image buffer

Zebra Rapixo CXP Pro Quad CXP-12 can perform MimLocatePeak1d() on-board without Host intervention if the specified buffer is an on-board buffer. This functionality is only available if the loaded FPGA configuration contains the Zebra PU that supports this functionality. Supporting this function on-board allows the board to perform sheet of light (laser line) extraction. The sheet of light must appear horizontally in the image. The board can extract more than one peak per lane (up to 3). In addition, the board can operate on a single frame or multi-frame on-board image buffer. For more information on how to use MimLocatePeak1d() for peak extraction from a multi-frame image buffer, see the Extraction of peaks from a multi-frame buffer subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.

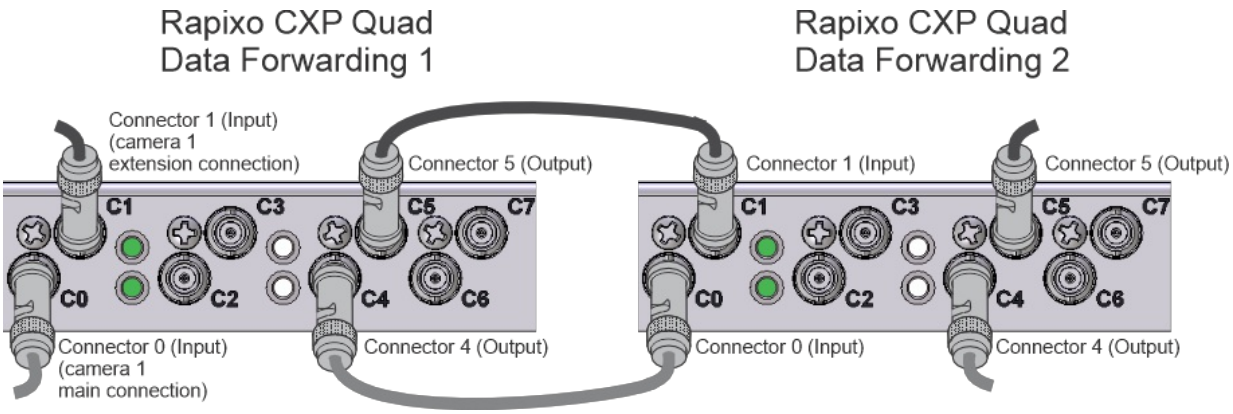
> Note that, peak-extraction is only available on Zebra Rapixo CXP Pro Quad CXP-12 with the appropriate firmware installed. Peak-extraction is not available when using Zebra Rapixo CXP Pro Quad CXP-12 with GenTL.

Data forwarding with Zebra Rapixo CXP

The Zebra Rapixo CXP Quad Data Forwarding model supports data forwarding. This allows you to distribute the image processing workload across multiple computers. This feature enables the relaying of images to another computer using up to four output connections running at up to 12.5 Gbits/sec. The number of output connections used must equal the number of input connections. Data forwarding is necessary in situations where the amount of information to process is too much for a single computer to keep up with, without dropping frames. Images can be retransmitted to

multiple computers in a daisy chain fashion by equipping each computer with a Zebra Rapixo CXP Quad Data Forwarding board; the last node in the chain does not need the Data Forwarding model. Depending on the processing that must be done, each computer in the chain can process a different portion of each incoming image, process alternate images (or nth image), or perform different independent required operations on each image.

The following diagram depicts two Data Forwarding boards in daisy chain fashion, with a main connection and an extension connection.



> Note that, you should connect the main connection to connector C0.

To use data forwarding, the Data Forwarding board in the first computer in the chain must go through the standard discovery procedure with the main connection and establish its speed. Only the first Data Forwarding board in the chain has direct communication with the camera. The subsequent Data Forwarding boards will need to have their DCFs configured with the speed, number of links, size of the image, and pixel format information. The Data Forwarding DCF Maker creates a DCF file that stores the connection details and feature settings of a CoaXpress camera connected to a Zebra Rapixo CXP. This type of DCF can only be used by additional Zebra CXP boards daisy-chained to the first Zebra Rapixo CXP DF. Once the speed is established with the camera and the DCFs are configured, start a grab on the input of the first Data Forwarding board before an output can be grabbed by the next board.

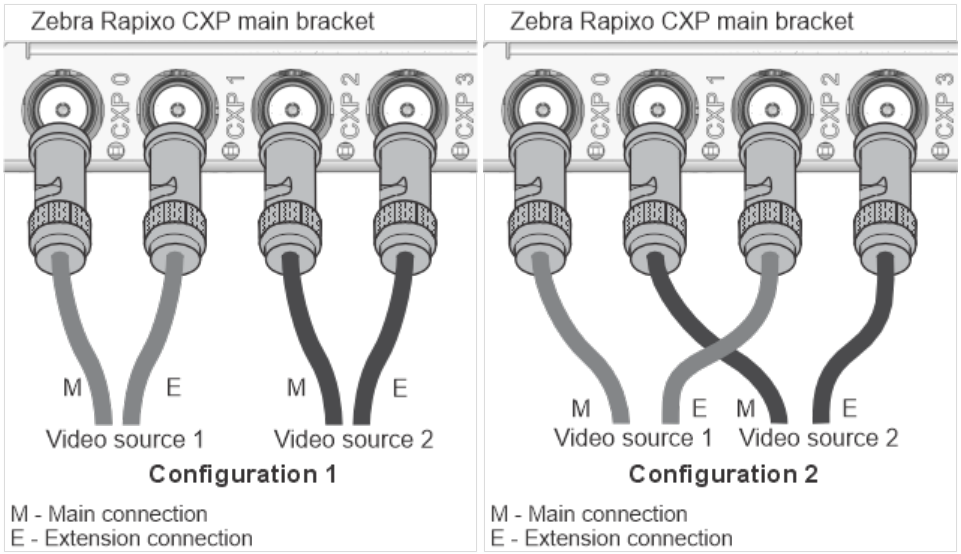
> Note that, when using PoCXP with the Data Forwarding board, only the board connected to the camera needs to have its 12 V auxiliary connector connected to a 12 V auxiliary source, since it is only this board that will provide power to the camera.

Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CXP boards

You can allocate up to four independent Aurora Imaging Library digitizers on your Zebra Rapixo CXP, using MdigAlloc(). Independent digitizers have different device numbers and use different acquisition paths. You must allocate a digitizer for each CoaXPress link. A CoaXPress link contains all the connections and components to capture from one video source.

When connecting a video source with multiple connections to Zebra Rapixo CXP, the main connection determines which device number should be used when allocating your digitizer. For example, if the main connector is plugged into CXP 0, use M_DEV0 to allocate the digitizer. If, however, the main connector is plugged into CXP 3, use M_DEV3 to allocate the digitizer. The other connectors for the same video source (called extension connectors) combine with the main connection to form a CXP link.

Both of the following configurations depict two CXP links, each with a main connection and an extension connection.



In configuration 1, you should allocate a digitizer for video source 1 using M_DEV0; whereas, you should allocate a digitizer for video source 2 using M_DEV2.

In configuration 2, you should allocate a digitizer for video source 1 using M_DEV0; whereas, you should allocate a digitizer for video source 2 using M_DEV1. Configuration 2 has a slight speed advantage to configuration 1 when the digitizer initializes, since the available devices will be scanned for initialization from 0 to 3.

Minimum latency and grabbing all frames

When grabbing with either MdigGrab() or MdigProcess(), Zebra Rapixo CXP is optimized to have the smallest latency possible between the last pixel sent from the camera to the last pixel written into the buffer. This allows the buffer to be available for processing with a minimum amount of delay.

By default, Zebra Rapixo CXP grabs into buffers in Host memory. On-board memory is used to protect against PCIe latency. To perform real-time grabs, your rate of acquisition (grabbing bandwidth) must be lower than the PCIe maximum transfer rate (PCIe bandwidth). If the acquisition rate is higher than the transfer rate across the PCIe bus, frames will be skipped rather than allowing a grab over-run to occur. To determine the image transfer speed from Zebra Rapixo CXP to the Host, use the Zebra Rapixo CXP Bench utility.

Detecting missed frames

To be certain that no frames were missed when MdigProcess() was last used, use MdigInquire() with M_PROCESS_FRAME_MISSED. This value reads the number of frames that were transmitted by the camera when no buffer was available to receive the frame; for example, if a grab is triggered and the associated hooked function is slow, then a frame can be missed.

Efficiently copying an on-board buffer to a destination buffer in Host memory

When copying an on-board buffer into a destination buffer in Host memory, the copy operation is performed by your Zebra Rapixo CXP using the board's DMA write engine, rather than the Host. Your Zebra Rapixo CXP will perform the operation faster than the Host. To ensure that Zebra Rapixo CXP performs the operation, use the following:

- To copy an on-board buffer into a destination buffer in Host memory, use MbufCopy().
- To resize an on-board buffer while copying it to a destination buffer in Host memory, use MimResize() with a factor of 1/n, where n is a value between 1 to 16, and with an interpolation mode of M_NEAREST_NEIGHBOR. Alternatively, you can use MbufTransfer() to perform the same operation.
- To flip an on-board buffer while copying it to a destination buffer in Host memory, use MimFlip().

> Note that for your Zebra Rapixo CXP to perform these operations, the destination buffer must be allocated in non-paged memory (that is, using MbufAlloc...() with + M_IMAGE + M_NON_PAGED).

Efficiently converting buffer formats while copying

When copying an on-board buffer into a destination buffer in Host memory, the color space converter and image formatter can automatically convert the bit-depth and color format of the source buffer to the bit-depth and color format of the destination buffer.

For on-board conversion of buffers to take place, the following on-board buffer and destination buffer depth and color formats are supported.

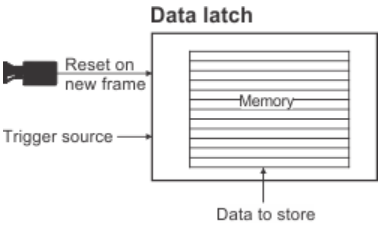
	Destination buffer depth and color format
On-board buffer depth and color	

format (specified in the DCF)	8-bit monochrome	16-bit monochrome	M_PACKED + M_BGR24	M_PACKED + M_BGR32	M_YUV16_YUYV / M_YUV16	M_PLANAR + M_RGB24	M_PLANAR + M_RGB48
8-bit monochrome	Yes	No	Yes	Yes	Yes	Yes	No
16-bit monochrome	Yes	Yes	Yes	Yes	Yes	Yes	Yes
M_PACKED + M_BGR24	Yes	No	Yes	Yes	Yes	Yes	Yes
M_RGB48 + M_PACKED	Yes	Yes	No	No	No	Yes	Yes

For information about YUV or RGB/BGR buffers, refer to Chapter 23: Data buffers. For the calculations used to convert to YUV, refer to the Zebra Rapixo CXP Hardware and Installation manual.

Using Zebra Rapixo CXP data latches

Zebra Rapixo CXP has data latches to store signal state information, the quadrature decoder's counter value, or a timestamp at a specific point during a grab. Data latches are reset at every end of grabbed frame event. Data latches do not suffer the delays typical to hooking a hook-handler function to these specific points during the grab and inquiring and storing related information using software.



> Data latches are only available when using an Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0 or M_DEV1. Zebra Rapixo has 16 data latches for use with a digitizer allocated with M_DEV0 and 16 data latches for use with a digitizer allocated with M_DEV1.

You can specify the data latch trigger source and the data to record upon the trigger using MdigControl() with the M_DATA_LATCH... control types.

To retrieve data from a data latch, use MdigGetHookInfo() with M_DATA_LATCH....

Steps to configure and retrieve data from a data latch

The following steps provide a basic methodology for configuring and retrieving data from a data latch:

1. Specify what triggers storing the specified information to the specified data latch, using MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn. The trigger can be a specific auxiliary signal, a quadrature decoder's counter value, a timer active event, or one of a limited number of grab events (such as a start of frame event or at every line of the image).

2. If necessary, specify the trigger signal transition upon which to store the information, using M_DATA_LATCH_TRIGGER_ACTIVATION + M_LATCHn. Note that this is only useful when M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn is set to M_AUX_IOn or M_TIMER_ACTIVE.

3. Specify the type of information to store, using M_DATA_LATCH_TYPE + M_LATCHn. Note that a data latch can store only one type of information, and each type of information can only be associated with one data latch, except for timestamp, which can be associated with up to four data latches. Aurora Imaging Library returns an error when this limitation is not respected.

4. To latch data before the start of the first grabbed frame, set M_DATA_LATCH_MODE + M_LATCHn to M_PREFETCH. Otherwise, if the data latch is triggered before the start of the first grabbed frame, no information is stored. To latch data only once the grab has started (that is, as of the start of the first grabbed frame), set M_DATA_LATCH_MODE to M_DEFAULT; this is the default value.

5. Enable the data latch using M_DATA_LATCH_STATE + M_LATCHn set to M_ENABLE.

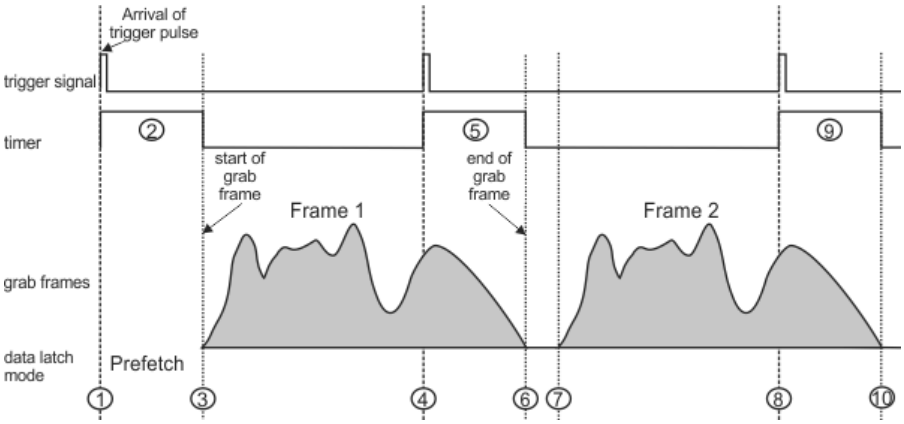
6. If necessary, repeat steps 1 to 5 for each data latch used.

7. Perform the grab. You can use MdigProcess() to grab an image; in the hook-handler function, retrieve and make use of the information stored in the data latch, using MdigGetHookInfo() with M_DATA_LATCH_VALUE. Alternatively, hook a hook-handler function to the end of grab frame event (M_GRAB_FRAME_END) using MdigHookFunction(), and call MdigGrab() to grab an image; then, to retrieve and make use of the information stored in the data latch, call MdigGetHookInfo() with M_DATA_LATCH_VALUE.

8. When done, disable the data latches using MdigControl() with M_DATA_LATCH_STATE + M_LATCHn set to M_DISABLE.

Triggering a data latch

Data latches can store data at several different points during a grab. Below are a few events that can trigger a data latch.



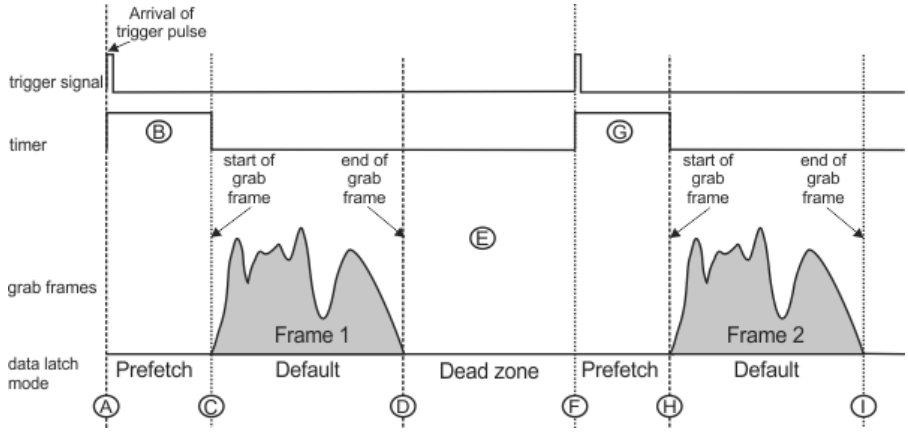
Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
1, 4, and 8	M_AUX_IOn	M_EDGE_RISING
2, 5, and 9	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
3 and 7	M_GRAB_FRAME_START	
6 and 10	M_GRAB_FRAME_END	
3 to 6 and 7 to 10	M_GRAB_LINE	

Events that occur before the start of the first grabbed frame of queued grabs or a grab sequence (such as, any auxiliary I/O change event, rotary decoder position change, or timer active event) can only trigger a data latch if the data latch has its mode set to prefetch (using MdigControl() with M_DATA_LATCH_MODE set to M_PREFETCH).

In the above image, if the data latch mode is set to prefetch (using M_DATA_LATCH_MODE set to M_PREFETCH), reading the data latch at the end of frame 1 could return the information latched at captions 1 through 6, depending on which event triggers the data latch. However, if the data latch mode is set to default (M_DATA_LATCH_MODE to M_DEFAULT), reading the data latch at the end of frame 1 could only return captions 3 through 6. In both cases, reading the data latch at the end of frame 2 could return captions 7 though 10. Captions 5 and 6 are returned at the end of frame 1, even though they relate to frame 2 because the data of all enabled data latches is returned at the end of every grabbed frame end event (that is, at the end of each grabbed frame).

If calls to MdigGrab() occur while no other grab is currently being performed and M_DATA_LATCH_MODE is set to M_PREFETCH, events that occur from the moment the grab command is issued until the start of the grabbed frame can

also trigger a data latch and be retrieved at the end of the grabbed frame, for each MdigGrab() call.



Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
A and F	M_AUX_IOn	M_EDGE_RISING
B and G	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
C and H	M_GRAB_FRAME_START	
D and I	M_GRAB_FRAME_END	
C to D and H to I	M_GRAB_LINE	
E	No data can be latched during this period	

In the above image, if the data latch mode is set to M_PREFETCH, reading the data latch at the end of frame 1 could return the information latched at captions A through D, depending on which event triggers the data latch. In this example, no data latch can return information relating to events during caption E. Frame 2 could return the information latched at captions F through I. Note that, when using MdigProcess() or when queuing grabs using MdigGrab() in asynchronous mode, there is no dead zone.

Data latch limitations

To set the type of data that a data latch should store, use M_DATA_LATCH_TYPE. In most cases, you can set up only one data latch to record a specific type of data (for example, only one data latch can store the status of all I/O signals). The exceptions to this are data latches that store timestamps; you can set up four data latches to store timestamps. Aurora Imaging Library returns an error when this limitation is not respected.

Retrieving data latch information

Aurora Imaging Library allows each data latch to store multiple instances of the same type of data. You can retrieve a

specific instance using the combination value M_VALUE_INDEX(). For example, when latching the timestamp at the end of each grabbed line of a frame with 1024 lines, to retrieve the 1024th timestamp, use MdigGetHookInfo() with M_DATA_LATCH_VALUE + M_LATCHn + M_VALUE_INDEX(1023), where *n* is the number associated with the data latch.

To retrieve all the stored information associated with a specific data latch, use M_DATA_LATCH_VALUE_ALL + M_LATCHn. Using the previous example, this would return 1024 timestamps.

To inquire how many pieces of information are stored inside a data latch, use M_DATA_LATCH_VALUE_COUNT.

To convert a timestamp from clock ticks to seconds, use the following equation: *Timestamp* * (*TimestampFrequencyInHz*). To inquire the clock frequency, use MdigInquire() with M_DATA_LATCH_CLOCK_FREQUENCY.

Data latch example

For an example of how to use the Zebra Rapixo CXP family data latches, refer to the *DataLatch.cpp* example.

Zebra Rapixo CXP utilities and tools

There are four Zebra Rapixo CXP external applications (utilities) available (Usage Meter, Rapixo CXP Bench, Rapixo CXP EyeScan, and Aurora Imaging Gecho Viewer) and a series of tools integrated in the Aurora Imaging Configurator utility (such as, the PCIe Information tool and the System Monitor tool).

To access these utilities and tools, launch the Aurora Imaging Configurator utility. Then, expand the **Boards** item in the tree structure and then click on the **Rapixo CXP** subitem from the tree structure in the presented interface. The utilities are located in the **Launch external applications** and **Zebra Rapixo CXP System Monitor** pages, respectively.

Zebra Rapixo CXP can also leverage the use of Aurora Imaging Capture Works (external application), which can be accessed using Aurora Imaging Control Center.

Usage Meter utility

The Usage Meter utility calculates how much of the grab section and the transfer section of each DMA write engine is being used. This rate of utilization is expressed as a percentage. This utility also shows the usage of on-board memory.

Rapixo CXP Bench utility

The Rapixo CXP Bench utility calculates the real-time transfer speed of the PCIe slot (in Mbytes/sec) from:

- Zebra Rapixo CXP to Host.
- Host to Zebra Rapixo CXP.

Rapixo CXP EyeScan

The Rapixo CXP EyeScan utility displays an eye diagram of the high speed incoming CoaXPress video signal as it is seen inside the FPGA's transceiver. This tool could help physical error diagnosis (for example, jitter or signal amplitude). Generally, the wider the opening of the eye in the X and Y-axis the better the quality of the signal. Be aware that differences in FPGA transceivers and external PHY might produce channel differences between signals of the same quality.

> For Zebra Rapixo Single, Dual, Quad CXP-12, and Pro Quad operating at speeds of CXP-12, CXP-6, or CXP-3, if you don't deactivate the re-timer (which smooths the data), then the signal that you see will always appear as a perfect eye. To see a non-retimed signal for these boards, call the EyeScan utility using the ByPassEq command line parameter.

Aurora Imaging Gecho Viewer

Aurora Imaging Gecho Viewer is an event logging application that gives you the ability to monitor and troubleshoot acquisition performance.

Aurora Imaging Capture Works

Aurora Imaging Capture Works is an interactive utility used to configure and test devices that make use of a GenICam-based interface standard, and supports the CoaXPress interface standard (besides other standards). Aurora Imaging Capture Works adds, in particular, support for GenICam GenDC containers and multi-part payload types. It also adds support for 3D displays. Aurora Imaging Capture Works will list all detected CoaXPress devices connected to each allocated Zebra Rapixo CXP board. It can start or stop capturing images, display acquired images, save the last grabbed

image, send a software trigger, as well as browse and control the selected device's features. You can view and change acquisition properties, and view acquisition statistics.

Aurora Imaging Capture Works key features:

- Camera-centric user interface with support for camera enumeration and presence.
- Feature browser to display and control camera, digitizer, and system settings.
- DCF creation to save camera and digitizer settings.
- Efficient display of grabbed images with many options and view modes.
- Display of line and arc profiles.
- Display of image histogram.
- Memory viewer to display pixel values.

More information is available from the Aurora Imaging Capture Works manual available online.

PCIe Information tool

The PCIe Information tool returns information about the board's PCIe connection.

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the System Monitor tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- Current number of PCIe lanes.
- Current speed of the PCIe lanes.
- Maximum number of PCIe lanes.
- Maximum speed of the PCIe lanes.

System Monitor tool

The System Monitor tool displays board properties of the specified digitizer (M_DEVn).

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the PCIe Information tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- The grab FPGA's current temperature, shown on a sliding scale from 0 to 85 degrees Celsius (for Zebra Rapixo CXP Quad CXP-6), or 0 to 100 (for all other Zebra Rapixo CXP boards), in degrees Celsius.
- The speed of the fan on the board, in RPM (Zebra Rapixo CXP Pro only).
- Whether a power source is detected on the internal auxiliary 12 V power connection.

If your Zebra Rapixo CXP is operating in the normal range of temperatures, a check mark is presented at the bottom of the System Monitor tool. If your Zebra Rapixo CXP is operating above or below the normal range of temperatures, an X is presented.

Zebra Rapixo CXP connectors and signal names

This section serves as a reference to match Zebra Rapixo CXP's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

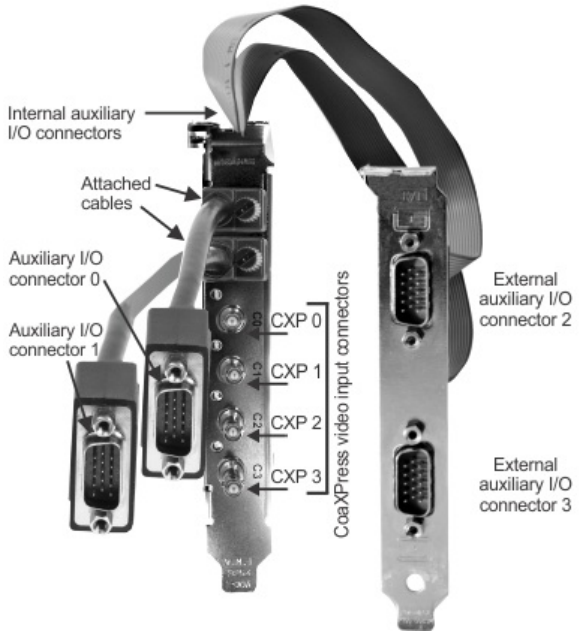
On Zebra Rapixo CXP, the auxiliary signals are acquisition path independent. Any digitizer allocated on the board (M_DEV0 through M_DEV3) can access any of the auxiliary signals.

Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below. Note that, unlike some other Zebra frame grabbers, there is no limit to the number of events that can be triggered simultaneously using the auxiliary input signals, nor is there a restriction on which auxiliary signal can be used to trigger the event.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Rapixo CXP board provides two interface options: the mDP-to-HD15 adapter cable or the HD-15 cable adapter bracket. On the mDP-to-HD15 adapter cables, which connect to the main bracket, there are two external auxiliary I/O connectors. On the cable adapter bracket, there are two external auxiliary I/O connectors.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
External auxiliary I/O connectors	HD-15 (0, 1, 2 and 3)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connector 0 and 1 are located on the mDP-to-HD15 adapter cables, and external auxiliary I/O connectors 2 and 3 are located on the cable adapter bracket.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information. The pinout for auxiliary I/O connector 0 is as follows. Auxiliary I/O connectors 1, 2, and 3 have the same pinout as auxiliary I/O connector 0, except you must add 8, 16, or 24, respectively, to the number at the end of their hardware signal name and Aurora Imaging Library constant. For example, AUX(TRIG)_TTL_IO_4 (M_AUX_IO4) on connector 0 would be AUX(TRIG)_TTL_IO_12 (M_AUX_IO12) on connector 1.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEVn [1]	Indicates the following.
I/O #: M_AUX_IO0	Opto-isolated auxiliary signal 0 (input), which supports: trigger input or user input.
Pin information	Connector: HD-15 (0) Pin: 15+, 9-
Direction	Input
Hardware manual signal name	AUX(TRIG)_OPTO_IN0
I/O #: M_AUX_IO1	Opto-isolated auxiliary signal 1 (input), which supports: trigger input or user input.
Pin information	Connector: HD-15 (0) Pin: 12+, 11-
Direction	Input

	Hardware manual signal name	AUX(TRIG)_OPTO_IN1
I/O #: M_AUX_IO2		LVDS auxiliary signal 2 (input), which supports: trigger input, user input, or quadrature input bit 0.
	Pin information	Connector: HD-15 (0) Pin: 4+, 5-
	Direction	Input
	Hardware manual signal name	AUX(TRIG)_LVDS_IN2
I/O #: M_AUX_IO3		LVDS auxiliary signal 3 (input), which supports: trigger input, user input, or quadrature input bit 1.
	Pin information	Connector: HD-15 (0) Pin: 6+, 8-
	Direction	Input
	Hardware manual signal name	AUX(TRIG)_LVDS_IN3
I/O #: M_AUX_IO4		TTL auxiliary signal 4 (input/output), which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (0) Pin: 1+
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEVn [1] ;
	Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
	Hardware manual signal name	AUX(TRIG)_TTL_IO_4
I/O #: M_AUX_IO5		TTL auxiliary signal 5 (input/output), which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (0) Pin: 2+
	Direction	Input/Output
	User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEVn [1] ;
	Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
	Hardware manual signal name	AUX(TRIG)_TTL_IO_5
I/O #: M_AUX_IO6		TTL auxiliary signal 6 (input/output), which supports: timer output, trigger input, user input, or user output.
	Pin information	Connector: HD-15 (0) Pin: 3+

		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT6; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(TRIG)_TTL_IO_6
	I/O #: M_AUX_IO7		LVDS auxiliary signal 7 (output), which supports: timer output or user output.
		Pin information	Connector: HD-15 (0) Pin: 13+, 14-
		Direction	Output
		User-bit information	User-bit #: M_USER_BIT7; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(EXP)_LVDS_OUT7

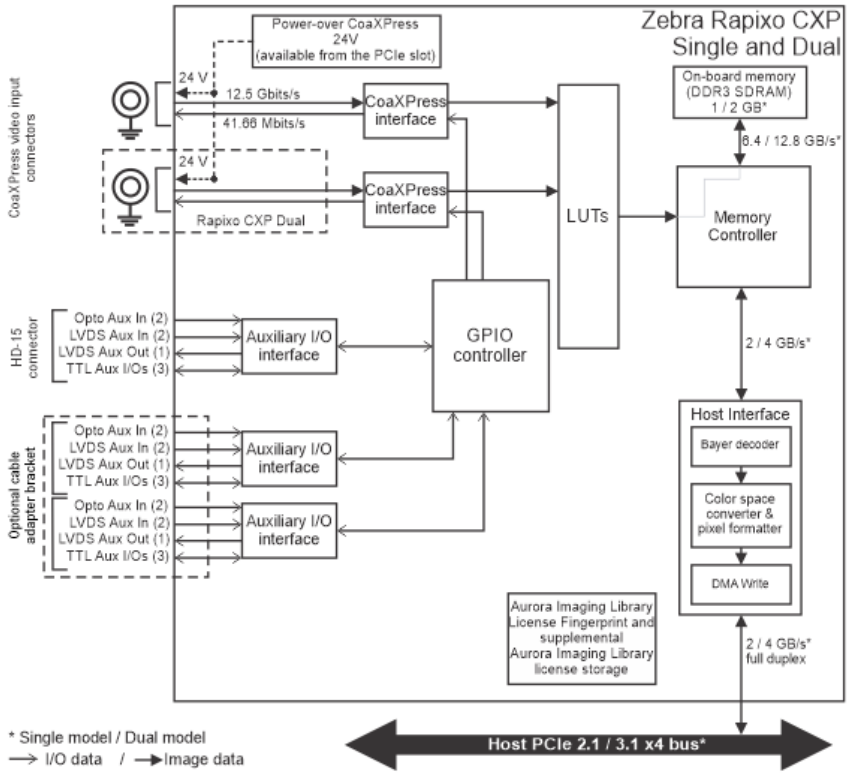
[1] Note that these are shared auxiliary signals between digitizers. If used by one digitizer, they can be simultaneously used by the other that shares this auxiliary signal. The value of *n* can be a number from 0 to 3, because those signals can be shared among the four different acquisition paths. Note that you must ensure that only one digitizer is driving an output signal.

Zebra Rapixo CXP board flow diagrams

This section contains data flow diagrams for Zebra Rapixo CXP boards.

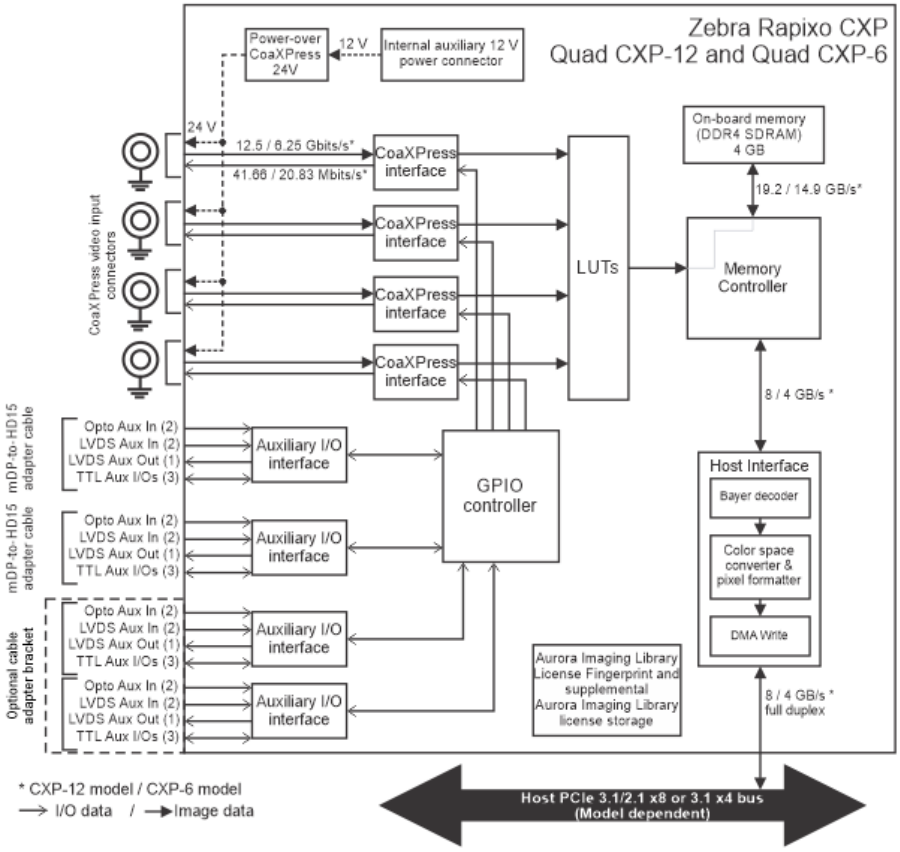
Zebra Rapixo CXP Single and Dual model

The following diagram illustrates the data flow of Zebra Rapixo base models (Single and Dual).



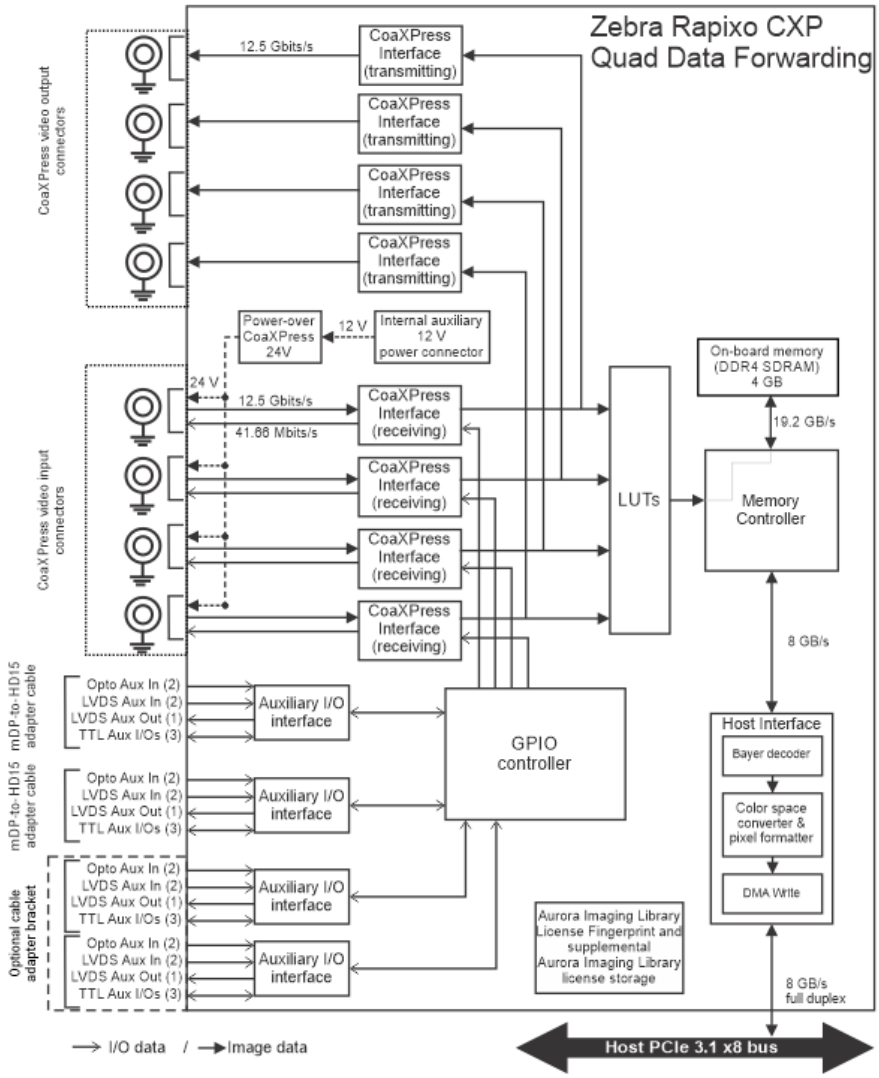
Zebra Rapixo CXP Quad CXP-6 and Quad CXP-12 model

The following diagram illustrates the data flow of Zebra Rapixo base models (Quad CXP-6 and Quad CXP-12).



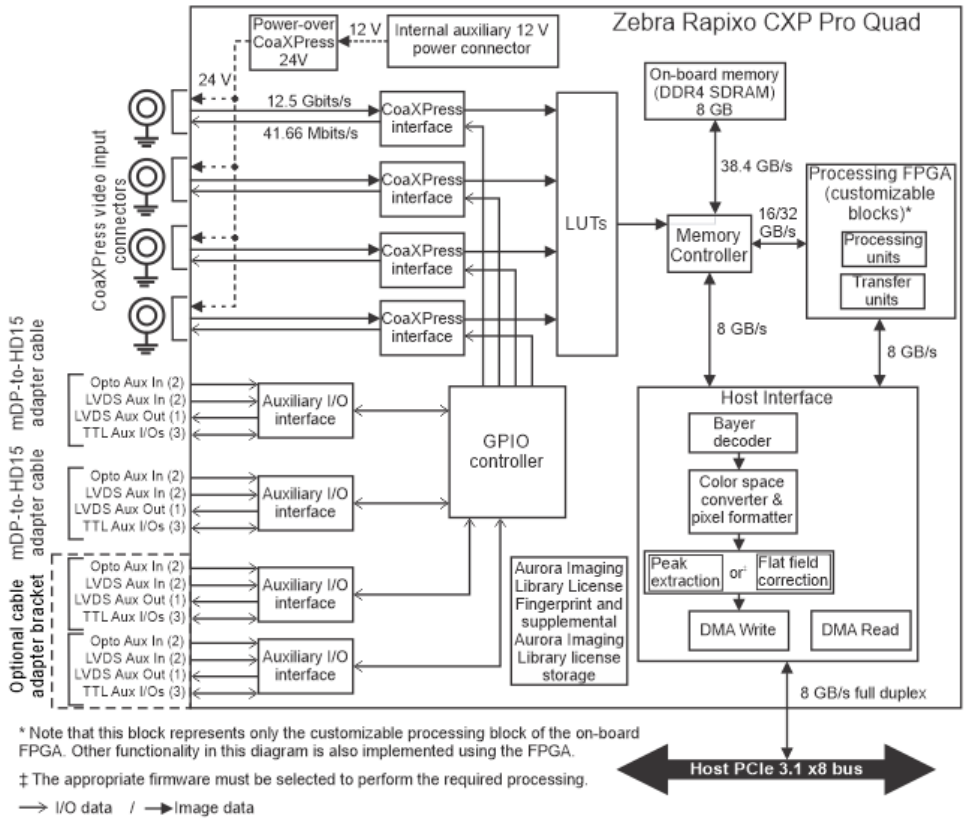
Zebra Rapixo CXP Quad Data Forwarding model

The following diagram illustrates the data flow of Zebra Rapixo base model (Quad Data Forwarding).



Zebra Rapixo CXP Pro model

The following diagram illustrates the data flow of Zebra Rapixo CXP Pro model (Pro Quad CXP-12).


Zebra Rapixo CXP Pro Quad
Power-over CoaXPress 24V
12 V
Internal auxiliary 12 V power connector
24 V
12.5 Gbits/s
41.66 Mbits/s
CoaXPress video input connectors
CoaXPress interface
CoaXPress interface
CoaXPress interface
CoaXPress interface
LUTs
On-board memory (DDR4 SDRAM) 8 GB
38.4 GB/s
Memory Controller
16/32 GB/s
Processing FPGA (customizable blocks)*
Processing units
Transfer units
8 GB/s
8 GB/s
mDP-to-HD15 adapter cable
Opto Aux In (2)
LVDS Aux In (2)
LVDS Aux Out (1)
TTL Aux I/Os (3)
Auxiliary I/O interface
GPIO controller
Host Interface
Bayer decoder
Color space converter & pixel formatter
Peak extraction
or
Flat field correction
DMA Write
DMA Read
Aurora Imaging Library License Fingerprint and supplemental Aurora Imaging Library license storage
Optional cable adapter bracket
8 GB/s full duplex
Host PCIe 3.1 x8 bus
* Note that this block represents only the customizable processing block of the on-board FPGA. Other functionality in this diagram is also implemented using the FPGA.
‡ The appropriate firmware must be selected to perform the required processing.
I/O data / Image data

Miscellaneous user guide information for Zebra Rapixo CXP

This section includes additional user guide information for Zebra Rapixo CXP. The information found in this section might be a reiteration of content previously documented.

Supported standards and development kit

Zebra Rapixo CXP supports the following standards:

- JIIA CoaXPress Standard version 2.0.
- GenICam GenApi version 3.4.2.
- GenICam Standard Feature Naming Convention (SFNC) version 2.7.
- GenICam GenTL version 1.6.

Zebra Rapixo CXP supports Zebra FPGA Development Kit (FDK) version 22.2 and later.

Additional features and improvements

The following are additional features supported for Zebra Rapixo CXP:

- Support for scatter-gather DMA transfers. Grabbing into paged buffers (**M_PAGED** attribute). By default, **M_GRAB** buffers are allocated in non-paged memory. If the non-paged memory is set to 0, then the default becomes paged memory.
- Support for acquisition from 14-bit monochrome and Bayer cameras.
- Increased Bayer demosaicing line width support to 16KB on Zebra Rapixo CXP-6 models and 32KB on Zebra Rapixo CXP-12 models.
- Support for CXP TapGeometry cameras (including 1X-2YE).
- Support for auxiliary IO debouncing.
- Support for dynamically changing the duration of a timer while it is grabbing.
- Support for GenICam GenTL Producer, specifically for the Data Stream and Buffer modules.

The following lists Zebra Rapixo CXP improvements:

- You can perform firmware updates without the need to power-cycle the computer.
- Aurora Imaging Gecho Viewer includes visualization of hardware events, such as triggers, timers, I/Os, and acquisition events.

Aurora Imaging Configurator

The Boards Information utility is integrated in Aurora Imaging Configurator for Zebra Rapixo CXP. It it located under **Boards**, then Rapixo CXP. The utility displays board information and details of the PUs (processing units) and TUs (transfer units) implemented in the firmware for Zebra Rapixo CXP Pro using the Zebra FDK.

Aurora Imaging Capture Works

The following are additional features supported for Zebra Rapixo CXP in Aurora Imaging Capture Works:

- Multi-stream cameras (up to 9 streams are available for one camera). When using more than one camera, there must be at least one stream per camera. The number of remaining streams can be distributed among rest of the cameras.
- Exchanging files with the camera.

Aurora Imaging Gecho Viewer

Aurora Imaging Gecho Viewer is an application used to control and display event logging information gathered by Aurora Imaging Gecho. To access Aurora Imaging Gecho Viewer, launch Aurora Imaging Configurator. Then, expand the **Boards** item in the tree structure, navigate to **Rapixo series**, then **CXP**. Gecho Viewer will be visible in the Launch External Applications page.

Aurora Imaging Gecho Viewer features:

- Visualize with high precision hardware events received and generated by the frame grabber, such as AuxIO changes, timers, CXP triggers, CXP protocol and connection issues, and grabbing events.
- Visualize software events, such as camera and digitizer communication, grab and hook events, issues and warnings like disconnection or missed frames.
- Example tracing (logging samples) for when the camera disconnects, frames that are missed by hardware overtriggering, and frames missed by software latency.

Particularities

The default time period from PoCXP voltage application where over-current condition is ignored is 512 ms. At the initial period following PoCXP application, the voltage might induce a punctual over-current due to the initial in-rush current (to charge the line capacitance), which might be followed by secondary over-currents caused by the activation of a camera's power supplies.

Zebra Rapixo CoF

This section discusses features of Aurora Imaging Library that are particular to Zebra Rapixo CoF family of boards and ways that optimize their performance. Zebra Rapixo CoF family specific information regarding specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Rapixo CoF overview
  - Summary of Zebra Rapixo CoF features
  - Support features and standards
  - Note on nomenclature
- Using Zebra Rapixo CoF with Aurora Imaging Library
  - Grabbing and the formats to use for each camera pixel format
  - Using the CoaXPress trigger signal on Zebra Rapixo CoF
  - Performing Bayer color conversion in hardware
  - Using frame burst with a multi-frame buffer
  - Using on-board flat-field correction
  - Extracting peaks from an on-board multi-frame image buffer
- Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CoF boards
- Minimum latency and grabbing all frames
  - Detecting missed frames
  - Efficiently copying an on-board buffer to a destination buffer in Host memory
  - Efficiently converting buffer formats while copying
- Using Zebra Rapixo CoF data latches
  - Steps to configure and retrieve data from a data latch
  - Triggering a data latch
  - Data latch limitations
  - Retrieving data latch information
  - Data latch example
- Zebra Rapixo CoF utilities and tools
  - Usage Meter utility
  - Rapixo CoF Bench utility
  - Aurora Imaging Gecho Viewer
  - Aurora Imaging Capture Works
  - PCIe Information tool
  - System Monitor tool
- Zebra Rapixo CoF connectors and signal names
  - Board connectors
  - Signal names and their matching Aurora Imaging Library constants
- Zebra Rapixo CoF board flow diagram

Zebra Rapixo CoF overview

Zebra Rapixo CoF is a family of high-performance PCIe frame grabbers. There are two models:

- **Zebra Rapixo CoF40**. Zebra Rapixo CoF40 supports image capture from high-resolution and high-speed video sources that use the CoaXPress-over-Fiber (CoF) communication standard, offering a total aggregate bandwidth of up to 40 Gbps (4 x 10 Gbits/sec) for one or multiple standard CoaXPress links. A CoaXPress link contains all the connections and components needed to capture from one video source (camera). There are two versions of the Zebra Rapixo CoF40 model: one with 4GB of on board memory and one with 8GB of on board memory.
- **Zebra Rapixo CoF100**. Zebra Rapixo CoF100 supports the same functionality as Zebra Rapixo CoF40, offering a total aggregate bandwidth of up to 100 Gbps (4 x 25 Gbits/sec) across the CoaXPress links, as well as FPGA-based processing offload capabilities (Processing FPGA). This model comes with 16GB of on-board memory.

Zebra Rapixo CoF comes with one QSFP cage to which you must connect a QSFP+ or QSFP28 transceiver. The transceiver is used to receive video streams from 10 or 25 GbE interfaces that use the CoaXPress standard. In addition, this transceiver supports transmitting trigger signals and transmitting and receiving control acknowledge messages.

Zebra Rapixo CoF supports frame and line-scan, monochrome and color video sources. The color video sources can be RGB video sources or video sources with a Bayer color filter. Zebra Rapixo CoF can decode Bayer color-encoded images and perform color space conversions while transferring the images to the Host. In addition, it can vertically and horizontally flip them and perform subsampling.

Summary of Zebra Rapixo CoF features

The following table outlines the features currently available for Zebra Rapixo CoF.

	Zebra Rapixo CoF
Can access GenICam camera features using MdigControlFeature() / MdigInquireFeature()	Yes
On-board memory	4 Gbytes DDR4 SDRAM or 8 Gbytes DDR4 SDRAM (Zebra Rapixo CoF40) 16 Gbytes DDR4 SDRAM (Zebra Rapixo CoF100)
Processing FPGA	No (Zebra Rapixo CoF40)/ Yes (Zebra Rapixo CoF100)
Digitizer LUTs	Yes [1]
Frame burst technology	Yes
Flat-field correction	Yes [2]

Peak-extraction	Yes [2]
Acquisition section	CoaXPress
Number of acquisition paths	4
Number of auxiliary signals	32 shared (16 in, 4 out, 12 in/out)
Number of timers	8 shared
Number of quadrature decoders for input from linear or rotary encoders	4
Number of CoaXPress trigger signals (to camera only)	1/digitizer
Number of data latches	16/digitizer (allocated using M_DEV0 or M_DEV1 only)

[1] Only for 8, 10, or 12-bit data (monochrome or color). When a link is receiving color data, all bands of the data use the same specified LUT mapping. In addition, as soon as one link is receiving 12-bit data, all links (CoaXPress connectors) share the same specified LUT mapping.

[2] Only available on Zebra Rapixo CoF100 (with the appropriate firmware selected). Not available when using Zebra Rapixo CoF100 with GenTL.

Support features and standards

The Zebra Rapixo CoF driver supports the following standards:

- GenICam GenTL Standard version 1.6.
- GenICam GenApi version 3.4.2.
- GenICam Standard Feature Naming Convention (SFNC) version 2.7.
- JIIA CoaXPress Standard version 2.1.
- JIIA CoaXPress over Fiber Bridge Protocol version 1.1.

Note on nomenclature

This manual refers to all versions of the board as Zebra Rapixo CoF. When necessary, this manual distinguishes between the boards using their full names (for example, Zebra Rapixo CoF40 or Zebra Rapixo CoF100), or their abbreviated forms (CoF40 and CoF100).

Using Zebra Rapixo CoF with Aurora Imaging Library

To use Zebra Rapixo CoF, you must allocate it as an Aurora Imaging Library Rapixo CoF system (using MsysAlloc() with M_SYSTEM_RAPIXOCOF). This allocation opens communication with the Zebra Rapixo CoF board and allows Aurora Imaging Library to use its resources. You can allocate an Aurora Imaging Library Rapixo CoF system for your board in multiple processes (executables). However, different processes cannot allocate a digitizer for the same acquisition paths.

All Zebra Rapixo CoF boards feature 4 quadrature decoders that can decode input from linear or rotary encoders with quadrature output. For information on this feature, refer to the Using quadrature input from a rotary encoder section of Chapter 56: I/O signals and communicating with external devices. This chapter also provides information on how to configure and use the auxiliary I/O signals.

For information on how to use the Processing FPGA of your Zebra Rapixo CoF100, see Chapter 67: Using Aurora Imaging Library for FPGA processing.

Grabbing and the formats to use for each camera pixel format

The following table outlines which Aurora Imaging Library buffer format to use with each supported camera pixel format. Note that these are the officially supported camera pixel formats.

Camera pixel format [1]	Aurora Imaging Library buffer format (type and attribute)	Operation
Mono8, Mono10, Mono12, Mono14, Mono16, Mono32	1-band 8-bit, 1-band 16-bit, 1-band 32-bit, 3-band 8-bit [2] , 3-band 16-bit [3]	Data will automatically be bit-shifted to have most-significant bits (MSB) in destination buffer
BayerXX8, BayerXX10, BayerXX12, BayerXX14, BayerXX16 where XX = GR, GB, BG or RG	1-band 8-bit, 1-band 16-bit	Demosaicing followed by RGB to Y (bit shift for MSB)
	3-band 8-bit, 3-band 16-bit	Demosaicing (bit shift for MSB)
	3-band YUV16 + M_PACKED	Demosaicing followed by RGB to YUV
RGB8, RGBA8,	1-band 8-bit	RGB to Y
	3-band 8-bit	Direct copy
	3-band YUV16 + M_PACKED	RGB to YUV
	1-band 8-bit	RGB to Y (bit shift for MSB)

RGB10, RGBA10, RGB12, RGBA12, RGB14, RGBA14, RGB16, RGBA16	1-band 16-bit	RGB to Y
	3-band 16-bit M_PLANAR	Packed destination buffers are not supported
YUV422_8, YCbCr601_422_8, YCbCr709_422_8	1-band 8-bit	Only Y band
	3-band YUV16 + M_PACKED	Direct copy

[1] All of the values available for RGB are also available in BGR format.

[2] 3-band 8-bit buffers can be in any of the following formats: M_RGB24 + M_PLANAR, M_RGB24 + M_PACKED, M_BGR24 + M_PACKED, or M_BGR32 + M_PACKED.

[3] 3-band 16-bit buffers can be in either an M_RGB48 + M_PLANAR or M_RGB48 + M_PACKED format.

Using the CoaXPress trigger signal on Zebra Rapixo CoF

The CoaXPress trigger signal is an embedded signal transmitted along the physical transport layer connection of your camera. Typically, the CoaXPress trigger signal is reserved for trigger information and is sent with other control and data signals along the same cable. On Zebra Rapixo CoF, the CoaXPress trigger signal is only supported from the frame grabber to the camera (that is, as an output signal).

To use this signal to trigger the camera, use MdigControl() with M_IO_SOURCE + M_TL_TRIGGER set to the appropriate auxiliary input signals (for example, M_AUX_IO4).

```
/* Use CoaXPress trigger signal to trigger the camera. */
MdigControl(AilDigitizer, M_IO_SOURCE + M_TL_TRIGGER, M_AUX_IO4);
```

Note that when you set the camera to capture an image only upon a trigger, you should disable grab triggered mode on the frame grabber (M_GRAB_TRIGGER_STATE set to M_DISABLE). For more information on performing a triggered grab, see the Grabbing with triggers section of Chapter 27: Grabbing with your digitizer.

Performing Bayer color conversion in hardware

When Zebra Rapixo CoF grabs color images from a video source with a Bayer color filter (as specified by the DCF), it performs Bayer color conversion in hardware, as it transfers the images to the Host. If the images require white balancing, Zebra Rapixo CoF can perform this automatically if white balancing is enabled using MdigControl() with M_WHITE_BALANCE set to M_ENABLE. If performing white balancing, you can use the default white balance coefficients, automatically have them calculated (using MdigControl() with M_WHITE_BALANCE set to M_CALCULATE), or set explicit coefficients (M_BAYER_COEFFICIENTS_ID). For information on Bayer color conversion, refer to the Using images acquired with a Bayer color filter section of Chapter 23: Data buffers.

If you don't want to perform Bayer color conversion in hardware, disable it using MdigControl() with M_BAYER_CONVERSION set to M_DISABLE.

The M_BAYER... control types of MdigControl() can only be used when grabbing from a camera that has a Bayer color filter (as specified by the DCF); otherwise, an error will be generated.

Using frame burst with a multi-frame buffer

Zebra Rapixo CoF supports frame burst technology. This technology allows you to grab a group of sequential frames into a multi-frame buffer with one grab command (MdigGrab(), or one grab of MdigProcess()); the defined number of frames are stored contiguously in the same buffer. The end-of-grab event only occurs once the entire group of frames has been grabbed, reducing the number of events that need to be handled. This is useful in cases where you have a high frame rate and need to ensure that no frames are missed. Note that a user-defined function hooked to the end-of-grab event (MdigProcess(), or MdigHookFunction() with M_GRAB_END) is executed only once the entire group of frames has been grabbed. For information on creating a multi-frame image buffer to store sequential frames, see the Specifying the dimensions of a multi-frame image buffer subsection of the Specifying the dimensions of a data buffer section of Chapter 23: Data buffers.

Using on-board flat-field correction

Zebra Rapixo CoF100 supports flat-field correction on-board without Host intervention. Images taken in environments with uneven lighting can affect the quality of your processing. To correct this, an on-board flat-field correction can be applied to images so that the intensity across the image is even before processing begins. To use flat-field correction, you need to specify an on-board buffer that contains the gain values, and optionally another that contains the offset values, using MdigControl() with M_SHADING_CORRECTION_GAIN_ID and M_SHADING_CORRECTION_OFFSET_ID, respectively. The buffers should be the same size as the grabbed image, and the gain values should be in the specified fixed point format (M_SHADING_CORRECTION_GAIN_FIXED_POINT). The aggregate bandwidth to read the gain buffer and offset buffer from on-board memory cannot be bigger than the internal maximum bandwidth of the DMA port (8 Gbytes/sec). There is a line limitation of 64 K pixels per line and 1 M lines per frame. Once you have set the buffers, enable the correction using MdigControl() with M_SHADING_CORRECTION.

Extracting peaks from an on-board multi-frame image buffer

Zebra Rapixo CoF100 can perform MimLocatePeak1d() on-board without Host intervention if the specified buffer is an on-board 8-bit buffer. Supporting this function on-board allows the board to perform sheet of light (laser line) extraction. The sheet of light must appear horizontally in the image. The board can extract more than one peak per lane (up to 3). In addition, the board can operate on a single frame or multi-frame on-board image buffer. For more information on how to use MimLocatePeak1d() for peak extraction from a multi-frame image buffer, see the Extraction of peaks from a multi-frame buffer subsection of the Peak intensity detection and depth maps section of Chapter 5: Specialized image processing.

Allocating independent Aurora Imaging Library digitizers on Zebra Rapixo CoF boards

You can allocate up to four independent Aurora Imaging Library digitizers on your Zebra Rapixo CoF, using MdigAlloc(). Independent digitizers have different device numbers and use the same acquisition path. You must allocate a digitizer for each CoaXPress link. A CoaXPress link contains all the connections and components to capture from one video source.

Minimum latency and grabbing all frames

When grabbing with either MdigGrab() or MdigProcess(), Zebra Rapixo CoF is optimized to have the smallest latency possible between the last pixel sent from the camera to the last pixel written into the buffer. This allows the buffer to be available for processing with a minimum amount of delay.

By default, Zebra Rapixo CoF grabs into buffers in Host memory. On-board memory is used to protect against PCIe latency. To perform real-time grabs, your rate of acquisition (grabbing bandwidth) must be lower than the PCIe maximum transfer rate (PCIe bandwidth). If the acquisition rate is higher than the transfer rate across the PCIe bus, frames will be skipped rather than allowing a grab over-run to occur. To determine the image transfer speed from Zebra Rapixo CoF to the Host, use the Zebra Rapixo CoF Bench utility.

Detecting missed frames

To be certain that no frames were missed when MdigProcess() was last used, use MdigInquire() with M_PROCESS_FRAME_MISSED. This value reads the number of frames that were transmitted by the camera when no buffer was available to receive the frame; for example, if a grab is triggered and the associated hooked function is slow, then a frame can be missed.

Efficiently copying an on-board buffer to a destination buffer in Host memory

When copying an on-board buffer into a destination buffer in Host memory, the copy operation is performed by your Zebra Rapixo CoF using the board's DMA write engine, rather than the Host. Your Zebra Rapixo CoF will perform the operation faster than the Host. To ensure that Zebra Rapixo CoF performs the operation, use the following:

- To copy an on-board buffer into a destination buffer in Host memory, use MbufCopy().
- To resize an on-board buffer while copying it to a destination buffer in Host memory, use MimResize() with a factor of 1/n, where n is a value between 1 to 16, and with an interpolation mode of M_NEAREST_NEIGHBOR. Alternatively, you can use MbufTransfer() to perform the same operation.
- To flip an on-board buffer while copying it to a destination buffer in Host memory, use MimFlip().

> Note that for your Zebra Rapixo CoF to perform these operations, the destination buffer must be allocated in non-paged memory (that is, using MbufAlloc...() with + M_IMAGE + M_NON_PAGED).

Efficiently converting buffer formats while copying

When copying an on-board buffer into a destination buffer in Host memory, the color space converter and image formatter can automatically convert the bit-depth and color format of the source buffer to the bit-depth and color format of the destination buffer.

For on-board conversion of buffers to take place, the following on-board buffer and destination buffer depth and color formats are supported.

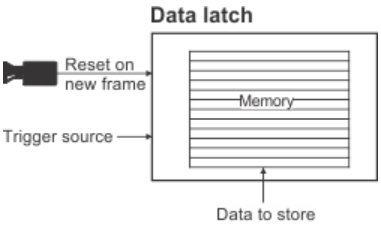
	Destination buffer depth and color format
On-board buffer depth and color	

format (specified in the DCF)	8-bit monochrome	16-bit monochrome	M_PACKED + M_BGR24	M_PACKED + M_BGR32	M_YUV16_YUYV / M_YUV16	M_PLANAR + M_RGB24	M_PLANAR + M_RGB48
8-bit monochrome	Yes	No	Yes	Yes	Yes	Yes	No
16-bit monochrome	Yes	Yes	Yes	Yes	Yes	Yes	Yes
M_PACKED + M_BGR24	Yes	No	Yes	Yes	Yes	Yes	No
M_RGB48 + M_PACKED	Yes	Yes	Yes	Yes	Yes	Yes	Yes

For information about YUV or RGB/BGR buffers, refer to Chapter 23: Data buffers. For the calculations used to convert to YUV, refer to the Zebra Rapixo CoF Hardware and Installation manual.

Using Zebra Rapixo CoF data latches

Zebra Rapixo CoF has data latches to store signal state information, the quadrature decoder's counter value, or a timestamp at a specific point during a grab. Data latches are reset at every end of grabbed frame event. Data latches do not suffer the delays typical to hooking a hook-handler function to these specific points during the grab and inquiring and storing related information using software.



> Data latches are only available when using an Aurora Imaging Library digitizer allocated using MdigAlloc() with M_DEV0. Zebra Rapixo CoF has 16 data latches for use with a digitizer allocated with M_DEV0.

You can specify the data latch trigger source and the data to record upon the trigger using MdigControl() with the M_DATA_LATCH... control types.

To retrieve data from a data latch, use MdigGetHookInfo() with M_DATA_LATCH....

Steps to configure and retrieve data from a data latch

The following steps provide a basic methodology for configuring and retrieving data from a data latch:

1. Specify what triggers storing the specified information to the specified data latch, using MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn. The trigger can be a specific auxiliary signal, a quadrature decoder's counter value, a timer active event, or one of a limited number of grab events (such as a start of frame event or at every line of the image).

2. If necessary, specify the trigger signal transition upon which to store the information, using M_DATA_LATCH_TRIGGER_ACTIVATION + M_LATCHn. Note that this is only useful when M_DATA_LATCH_TRIGGER_SOURCE + M_LATCHn is set to M_AUX_IOn or M_TIMER_ACTIVE.

3. Specify the type of information to store, using M_DATA_LATCH_TYPE + M_LATCHn. Note that a data latch can store only one type of information, and each type of information can only be associated with one data latch, except for timestamp, which can be associated with up to four data latches. Aurora Imaging Library returns an error when this limitation is not respected.

4. To latch data before the start of the first grabbed frame, set M_DATA_LATCH_MODE + M_LATCHn to M_PREFETCH. Otherwise, if the data latch is triggered before the start of the first grabbed frame, no information is stored. To latch data only once the grab has started (that is, as of the start of the first grabbed frame), set M_DATA_LATCH_MODE to M_DEFAULT; this is the default value.

5. Enable the data latch using M_DATA_LATCH_STATE + M_LATCHn set to M_ENABLE.

6. If necessary, repeat steps 1 to 5 for each data latch used.

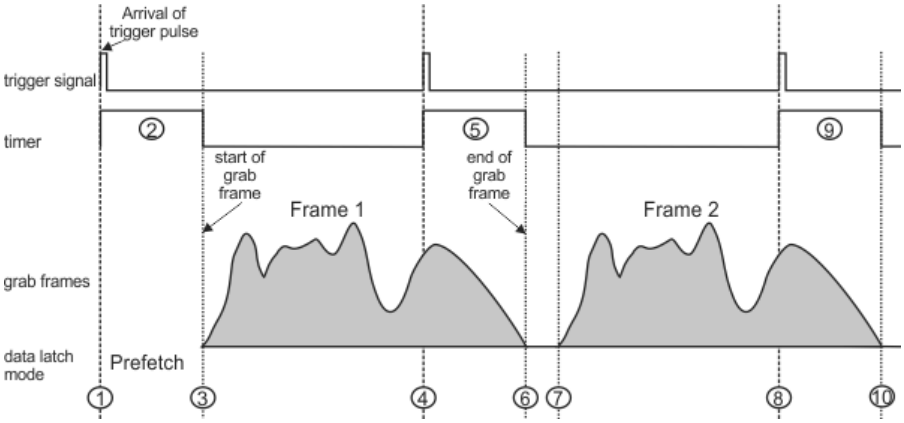
7. Perform the grab. You can use MdigProcess() to grab an image; in the hook-handler function, retrieve and make use of the information stored in the data latch, using MdigGetHookInfo() with M_DATA_LATCH_VALUE. Alternatively, hook a hook-handler function to the end of grab frame event (M_GRAB_FRAME_END) using MdigHookFunction(), and call MdigGrab() to grab an image; then, to retrieve and make use of the information stored in the data latch, call MdigGetHookInfo() with M_DATA_LATCH_VALUE.

8. When done, disable the data latches using MdigControl() with M_DATA_LATCH_STATE + M_LATCHn set to

M_DISABLE.

Triggering a data latch

Data latches can store data at several different points during a grab. Below are a few events that can trigger a data latch.

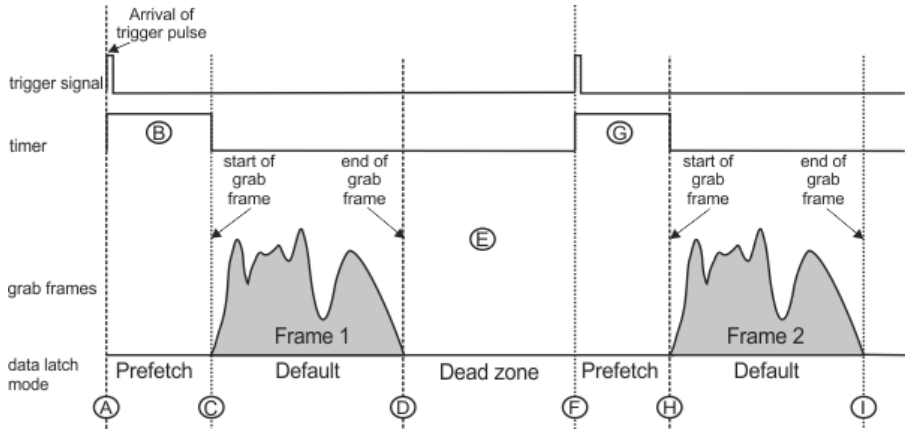


Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
1, 4, and 8	M_AUX_IOn	M_EDGE_RISING
2, 5, and 9	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
3 and 7	M_GRAB_FRAME_START	
6 and 10	M_GRAB_FRAME_END	
3 to 6 and 7 to 10	M_GRAB_LINE	

Events that occur before the start of the first grabbed frame of queued grabs or a grab sequence (such as, any auxiliary I/O change event, rotary decoder position change, or timer active event) can only trigger a data latch if the data latch has its mode set to prefetch (using MdigControl() with M_DATA_LATCH_MODE set to M_PREFETCH).

In the above image, if the data latch mode is set to prefetch (using M_DATA_LATCH_MODE set to M_PREFETCH), reading the data latch at the end of frame 1 could return the information latched at captions 1 through 6, depending on which event triggers the data latch. However, if the data latch mode is set to default (M_DATA_LATCH_MODE to M_DEFAULT), reading the data latch at the end of frame 1 could only return captions 3 through 6. In both cases, reading the data latch at the end of frame 2 could return captions 7 though 10. Captions 5 and 6 are returned at the end of frame 1, even though they relate to frame 2 because the data of all enabled data latches is returned at the end of every grabbed frame end event (that is, at the end of each grabbed frame).

If calls to MdigGrab() occur while no other grab is currently being performed and M_DATA_LATCH_MODE is set to M_PREFETCH, events that occur from the moment the grab command is issued until the start of the grabbed frame can also trigger a data latch and be retrieved at the end of the grabbed frame, for each MdigGrab() call.



Callout #	MdigControl() with M_DATA_LATCH_TRIGGER_SOURCE	MdigControl() with M_DATA_LATCH_TRIGGER_ACTIVATION
A and F	M_AUX_IOn	M_EDGE_RISING
B and G	M_TIMER_ACTIVE	M_EDGE_RISING or M_EDGE_FALLING
C and H	M_GRAB_FRAME_START	
D and I	M_GRAB_FRAME_END	
C to D and H to I	M_GRAB_LINE	
E	No data can be latched during this period	

In the above image, if the data latch mode is set to M_PREFETCH, reading the data latch at the end of frame 1 could return the information latched at captions A through D, depending on which event triggers the data latch. In this example, no data latch can return information relating to events during caption E. Frame 2 could return the information latched at captions F through I. Note that, when using MdigProcess() or when queuing grabs using MdigGrab() in asynchronous mode, there is no dead zone.

Data latch limitations

To set the type of data that a data latch should store, use M_DATA_LATCH_TYPE. In most cases, you can set up only one data latch to record a specific type of data (for example, only one data latch can store the status of all I/O signals). The exceptions to this are data latches that store timestamps; you can set up four data latches to store timestamps. Aurora Imaging Library returns an error when this limitation is not respected.

Retrieving data latch information

Aurora Imaging Library allows each data latch to store multiple instances of the same type of data. You can retrieve a specific instance using the combination value M_VALUE_INDEX(). For example, when latching the timestamp at the end

of each grabbed line of a frame with 1024 lines, to retrieve the 1024th timestamp, use MdigGetHookInfo() with M_DATA_LATCH_VALUE + M_LATCHn + M_VALUE_INDEX(1023), where *n* is the number associated with the data latch.

To retrieve all the stored information associated with a specific data latch, use M_DATA_LATCH_VALUE_ALL + M_LATCHn. Using the previous example, this would return 1024 timestamps.

To inquire how many pieces of information are stored inside a data latch, use M_DATA_LATCH_VALUE_COUNT.

To convert a timestamp from clock ticks to seconds, use the following equation: *Timestamp* * (*TimestampFrequencyInHz*). To inquire the clock frequency, use MdigInquire() with M_DATA_LATCH_CLOCK_FREQUENCY.

Data latch example

For an example of how to use the Zebra Rapixo CoF family data latches, refer to the *DataLatch.cpp* example.

Zebra Rapixo CoF utilities and tools

There are three Zebra Rapixo CoF external applications (utilities) available (Usage Meter, Rapixo CoF Bench, and Aurora Imaging Gecho Viewer) and a series of tools integrated in the Aurora Imaging Configurator utility (such as, the PCIe Information tool and the System Monitor tool).

To access these utilities and tools, launch Aurora Imaging Configurator. Then, expand the **Boards** item in the tree structure and then click on the Rapixo CoF subitem from the tree structure in the presented interface. The utilities are located in the **Launch external applications** and **System Monitor** sections, respectively.

Zebra Rapixo CoF can also leverage the use of Aurora Imaging Capture Works (external application), which can be accessed using Aurora Imaging Control Center.

Usage Meter utility

The Usage Meter utility calculates how much of the grab section and the transfer section of each DMA write engine is being used. This rate of utilization is expressed as a percentage. This utility also shows the usage of on-board memory.

Rapixo CoF Bench utility

The Rapixo CoF Bench utility calculates the real-time transfer speed of the PCIe slot (in Mbytes/sec) from:

- Zebra Rapixo CoF to Host.
- Host to Zebra Rapixo CoF.

Aurora Imaging Gecho Viewer

Aurora Imaging Gecho Viewer is an event logging application that gives you the ability to monitor and troubleshoot acquisition performance. For example, you can monitor internal and external I/O signals (such as auxiliary I/Os, timers, and CXP triggers).

Aurora Imaging Capture Works

Aurora Imaging Capture Works is an interactive utility used to configure and test devices that make use of a GenICam-based interface standard, and supports the CoaXPress-over-Fiber interface standard (besides other standards). Aurora Imaging Capture Works adds, in particular, support for GenICam GenDC containers and multi-part payload types. Aurora Imaging Capture Works will list all detected CoaXPress devices connected to each allocated Zebra Rapixo CoF board. It can start or stop capturing images, display acquired images, save the last grabbed image, send a software trigger, as well as browse and control the selected device's features. You can view and change device information (such as, the user-defined name of your device), view and change acquisition properties, and view acquisition statistics.

Aurora Imaging Capture Works key features:

- Camera-centric user interface with support for camera enumeration and presence.
- Feature browser to display and control camera, digitizer, and system settings.
- DCF creation to save camera and digitizer settings.
- Efficient display of grabbed images with many options and view modes.
- Display of line and arc profiles.

- Display of image histogram.
- Memory viewer to display pixel values.

More information is available from the Aurora Imaging Capture Works manual available online.

PCIe Information tool

The PCIe Information tool returns information about the board's PCIe connection.

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the System Monitor tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- Current number of PCIe lanes.
- Current speed of the PCIe lanes.
- Maximum number of PCIe lanes.
- Maximum speed of the PCIe lanes.

System Monitor tool

The System Monitor tool displays board properties of the specified digitizer (M_DEVn).

Use the acquisition (device) selector to set the acquisition path about which to inquire (M_DEVn). Note that this selector is shared with the PCIe Information tool.

This tool presents the following information inside the Aurora Imaging Configurator utility:

- The grab FPGA's current temperature.
- The speed of the fan on the board, in RPM.
- Whether a power source is detected on the internal auxiliary 12 V power connection.

If your Zebra Rapixo CoF is operating in the normal range of temperatures, a check mark is presented at the bottom of the System Monitor tool. If your Zebra Rapixo CoF is operating above or below the normal range of temperatures, an X is presented.

Zebra Rapixo CoF connectors and signal names

This section serves as a reference to match Zebra Rapixo CoF's connectors and auxiliary signals with Aurora Imaging Library information, such as Aurora Imaging Library auxiliary signal numbers. To set/inquire all the settings for this board's auxiliary signals (for example, signal routing and timer settings), use MdigControl() / MdigInquire(), respectively.

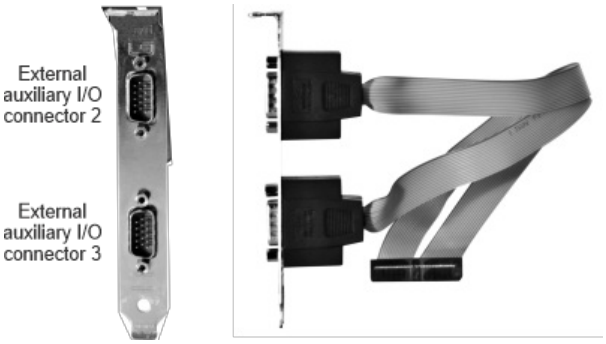
On Zebra Rapixo CoF, the auxiliary signals are acquisition path independent. Any digitizer allocated on the board (M_DEV0 through M_DEV3) can access any of the auxiliary signals.

Auxiliary I/O signals can have one or more functionalities (for example, trigger input, timer output, or user output, depending on the signal). Their possible functionalities are described in their description in the pinout table below.

Only those auxiliary signals that have matching Aurora Imaging Library information are included in this section. For information on internal connectors and a comprehensive list of all available input and output signals, refer to the board's installation and hardware reference manual.

Board connectors

The Zebra Rapixo CoF board provide several interface connectors: the QSFP transceiver cage and an HD-15 connector on the main bracket. In addition, close to the top edge of the main board, there is an internal auxiliary I/O connector that can be used with an adapter bracket to access additional I/O. On the cable adapter bracket, there are two external auxiliary I/O connectors which can connect to internal auxiliary connector 1/2 or 2/3.



Only the following connectors have auxiliary signals with matching Aurora Imaging Library information.

Connector Name	Connector Abbreviation	Image	Description
External auxiliary I/O connectors	HD-15 (0, 1, 2 and 3)		The external auxiliary I/O connectors are high-density D-subminiature 15-pin male connectors. They are used to transmit/receive auxiliary signals. External auxiliary I/O connector 0 is located on the main bracket, and external auxiliary I/O connectors 1/2 or 2/3 are located on the cable adapter bracket.

Signal names and their matching Aurora Imaging Library constants

The table below lists the auxiliary signals with their associated Aurora Imaging Library information. The pinout for auxiliary I/O connector 0 is as follows. Auxiliary I/O connectors 1, 2, and 3 have the same pinout as auxiliary I/O connector 0, except you must add 8, 16, or 24, respectively, to the number at the end of their hardware signal name and Aurora Imaging Library constant. For example, AUX(TRIG)_TTL_IO_4 (M_AUX_IO4) on connector 0 would be AUX(TRIG)_TTL_IO_12 (M_AUX_IO12) on connector 1.

Sort by: I/O information Filter by: Show all Tables: Expanded

Digitizer device #	Description
I/O #	
Pin information	
Direction	
User-bit information	
Trigger information	
Timer information	
Hardware manual signal name	
Digitizer device #: M_DEVn [1]	Indicates the following.
I/O #: M_AUX_IO0	Opto-isolated auxiliary signal 0 (input), which supports: trigger input or user input.
Pin information	Connector: HD-15 (0) Pin: 15+, 9-
Direction	Input
Hardware manual signal name	AUX(TRIG)_OPTO_IN0
I/O #: M_AUX_IO1	Opto-isolated auxiliary signal 1 (input), which supports: trigger input or user input.
Pin information	Connector: HD-15 (0) Pin: 12+, 11-
Direction	Input
Hardware manual signal name	AUX(TRIG)_OPTO_IN1
I/O #: M_AUX_IO2	LVDS auxiliary signal 2 (input), which supports: trigger input, user input, or quadrature input bit 0.
Pin information	Connector: HD-15 (0) Pin: 4+, 5-
Direction	Input
Hardware manual signal name	AUX(TRIG)_LVDS_IN2
I/O #: M_AUX_IO3	LVDS auxiliary signal 3 (input), which supports: trigger input, user input, or quadrature input bit 1.

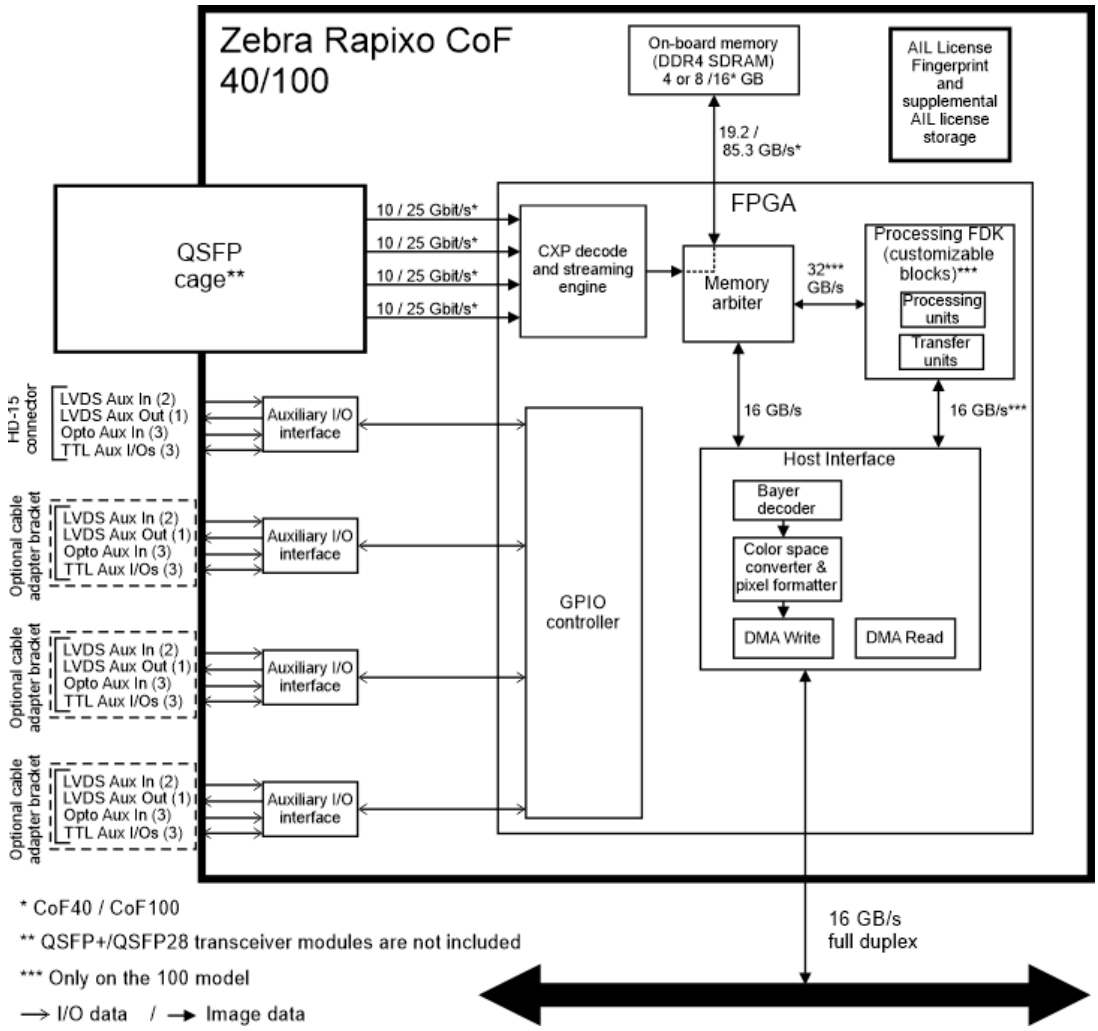
		Pin information	Connector: HD-15 (0) Pin: 6+, 8-
		Direction	Input
		Hardware manual signal name	AUX(TRIG)_LVDS_IN3
	I/O #: M_AUX_IO4		TTL auxiliary signal 4 (input/output), which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (0) Pin: 1+
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT4; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(TRIG)_TTL_IO_4
	I/O #: M_AUX_IO5		TTL auxiliary signal 5 (input/output), which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (0) Pin: 2+
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT5; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(TRIG)_TTL_IO_5
	I/O #: M_AUX_IO6		TTL auxiliary signal 6 (input/output), which supports: timer output, trigger input, user input, or user output.
		Pin information	Connector: HD-15 (0) Pin: 3+
		Direction	Input/Output
		User-bit information	User-bit #: M_USER_BIT6; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(TRIG)_TTL_IO_6
	I/O #: M_AUX_IO7		LVDS auxiliary signal 7 (output), which supports: timer output or user output.
		Pin information	Connector: HD-15 (0) Pin: 13+, 14-
		Direction	Output

		User-bit information	User-bit #: M_USER_BIT7; Digitizer device #: M_DEVn [1] ;
		Timer information	Timer: M_TIMER1/M_TIMER2/M_TIMER3/M_TIMER4; Digitizer device #: M_DEVn [1] ;
		Hardware manual signal name	AUX(EXP)_LVDS_OUT7

[1] Note that these are shared auxiliary signals between digitizers. If used by one digitizer, they can be simultaneously used by the other that shares this auxiliary signal. The value of *n* can be a number from 0 to 3, because those signals can be shared among the four different acquisition paths. Note that you must ensure that only one digitizer is driving an output signal.

Zebra Rapixo CoF board flow diagram

The following diagram illustrates the data flow of Zebra Rapixo CoF40 and CoF100.



Zebra USB3 Vision driver

This chapter discusses features of Aurora Imaging Library that are particular to the Zebra USB3 Vision driver, and ways that optimize its performance. Information regarding using this driver with specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra USB3 Vision driver overview
- Using the Zebra USB3 Vision driver with Aurora Imaging Library
  - Non-paged memory usage
- Miscellaneous reference information for Zebra USB3 Vision
  - Mbuf
    - MbufAlloc...
- Miscellaneous user guide information for Zebra USB3 Vision
  - Supported standards
  - Additional features
  - Particularities

Zebra USB3 Vision driver overview

The Zebra USB3 Vision driver allows you to use a USB 3.0 port on your computer to grab from USB3 Vision-compatible cameras with Aurora Imaging Library. You can also use the driver to set features of the camera.

Using the Zebra USB3 Vision driver with Aurora Imaging Library

To use the Zebra USB3 Vision driver with Aurora Imaging Library, you must allocate an Aurora Imaging Library USB3 Vision system (M_SYSTEM_USB3_VISION), using MsysAlloc(). This allocation opens general communication with all the USB3 Vision-compliant cameras (or devices) connected to your computer. Then, allocate a digitizer for each camera (USB3.0 camera) that you want to use to grab images and/or access directly, using MdigAlloc().

Each process (executable) can allocate only one Aurora Imaging Library USB3 Vision system; an error will occur if you try to allocate more than one. Multiple processes can allocate an Aurora Imaging Library USB3 Vision system.

To inquire the current value of the required USB3 Vision camera feature, use MdigInquireFeature(). If a USB3 Vision feature is not already set to the required value, use MdigControlFeature() to change the value of the feature. Functionality is also supported with the conventional Aurora Imaging Library functions MdigInquire() and MdigControl(). For more information on the GenICam-standard features available, refer to the GenICam standard feature naming convention (SFNC), found online at: emva.org/standards-technology/genicam/. Additional GenICam-extension features might be available; check your camera's manual and Aurora Imaging Capture Works' Feature Browser for details.

You can also use Aurora Imaging Capture Works' Feature Browser to access the USB3 Vision-compliant camera's device description file (XML), providing an interface to view and change the camera's configuration. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Zebra USB3 Vision information in the Aurora Imaging Library Reference can be found in the paragraphs and values marked as being supported by the USB Vision driver.

Non-paged memory usage

With each call to MdigAlloc(), a certain amount of memory is reserved so that Aurora Imaging Library can create internal buffers. To calculate the exact amount of memory reserved, inquire the amount of non-paged memory used before and again after you call MdigAlloc(); to perform the inquire, use MappInquire() with M_NON_PAGED_MEMORY_USED. Aurora Imaging Library uses this reserved non-paged memory when grabbing and when decoding image data received in "chunk mode".

If your application requires more non-paged memory than is currently available, increase the amount of non-paged memory accessible to Aurora Imaging Library using the Aurora Imaging Configurator utility.

Miscellaneous reference information for Zebra USB3 Vision

This section includes additional command reference information for Zebra USB3 Vision. The information found in this section might be a reiteration of content previously documented.

The following modules are mentioned in this section:

- Mbuf.

Mbuf

This section presents information about Zebra USB3 Vision and the buffer module.

MbufAlloc...

The following lists additional USB3 Vision support when using MbufAlloc...().

- Support for **M_IMAGE** + **M_GRAB** + **M_DYNAMIC** buffer attribute.

Miscellaneous user guide information for Zebra USB3 Vision

This section includes additional user guide information for Zebra USB3 Vision. The information found in this section might be a reiteration of content previously documented.

Supported standards

The Zebra USB3 Vision driver supports the following standards:

- GenICam GenApi version 3.4.2.
- GenICam Standard Feature Naming Convention (SFNC) version 2.7.
- GenCP 1.1.
- USB3 Vision 1.1.

Additional features

The following are additional features supported for Zebra USB3 Vision:

- Grab containers.
- **M_DYNAMIC** buffers.
- Device enumeration API.
- Streaming Monochrome and Bayer data of 10- and 12 bit packed pixel format.

The Zebra USB3 Vision driver allocates 10 auxiliary buffers during digitizer allocation.

Aurora Imaging Library USB3 Vision system does not require Aurora Imaging Library's non-paged memory for image acquisition.

Particularities

An error message is generated if MdigControl() is executed with **M_WHITE_BALANCE** and **M_CALCULATE** while a grab is in progress.

In the case where a synchronization lost occurs, the driver automatically attempts to resynchronize the image stream with the camera. This resets the BlockID to 0.

If two cameras are used and **M_DEV0** camera is removed from the PC, the **M_DEV1** camera will be remapped to **M_DEV0**, as if **M_DEV1** and **M_DEV0** were not allocated on their associated cameras.

BGR or RGB, packed (10 or 12 bits) pixel formats are not supported.

FLIR (Point Grey) cameras with old firmware on Windows 10 need to be unplugged before uninstalling Aurora Imaging Library. Failure to do so can cause a variety of post uninstall USB problems.

Your camera must be USB3 Vision compliant for it to work with the Zebra USB3 Vision driver. For more information, refer to automate.org/vision/vision-standards/vision-standards.

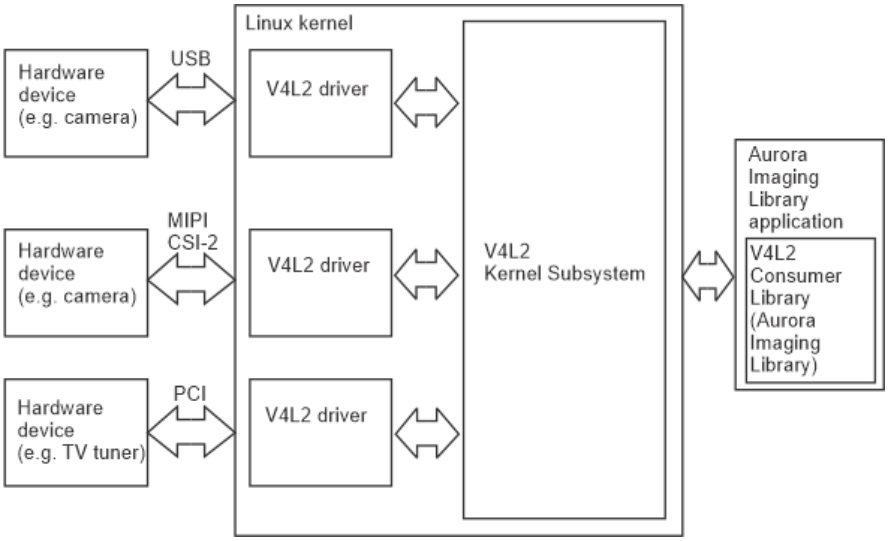
Zebra Video4Linux2 Consumer

This chapter discusses features of Aurora Imaging Library that are particular to the Zebra Video4Linux2 (V4L2) Consumer, and optimizing its performance. Information regarding using this system with specific functions is integrated into the functions' reference topics. The Aurora Imaging Library release notes, accessible from Aurora Imaging Control Center and/or the Zebra Aurora Imaging Library product support webpages (for example, zebra.com/ail-info), can provide additional documentation, such as new or modified features, systems, and products.

- Zebra Video4Linux2 Consumer (library) overview
- Using Zebra V4L2 Consumer (library) with Aurora Imaging Library

Zebra Video4Linux2 Consumer (library) overview

The Zebra Video4Linux2 Consumer (library) allows you to use a Video4Linux2 (V4L2) driver with Aurora Imaging Library. The Zebra Video4Linux2 Consumer (based on the Linux Video4Linux2 API) provides a way to communicate with any device capable of video capture (provided it has a V4L2 driver working within the running Linux kernel) and can control the capture process and move images from the driver into the user space. The Zebra Video4Linux2 Consumer allows for access and configuration of devices including video input, video output, and encoding for capture devices.



Many drivers come with the Linux kernel, but a hardware device/camera manufacturer could distribute one separately. Make sure that the required/separately distributed V4L2 driver is either installed by the software provided with the device, or by installing a compatible V4L2 driver.

Note that not all V4L2 image formats are supported by Aurora Imaging Library, and some cameras might not have any Aurora Imaging Library-supported image formats. When using cameras with auto-focus, auto-lighting, or similar features, you might need to grab a few frames before obtaining a clear picture.

Using Zebra V4L2 Consumer (library) with Aurora Imaging Library

To use the Zebra Video4Linux2 Consumer with Aurora Imaging Library, you need to allocate an Aurora Imaging Library V4L2 system (M_SYSTEM_V4L2), using MsysAlloc(). This allocation opens general communication with all devices capable of video capture found on your computer (discovers all character device special files */dev/video** , where * is a number between 0 to 63). You must then allocate a digitizer for each hardware device that you want to use to grab images and/or access directly, using MdigAlloc() with M_DEVn. For more information, refer to the Device number section of Chapter 27: Grabbing with your digitizer.

> Note that, you can only allocate one Aurora Imaging Library V4L2 system per Linux process but multiple processes can allocate a V4L2 system. Each device can only be used by one process at a time.

A conversion between Video4Linux2 and GenICam has been implemented which allows V4L2 controls to be accessible through MdigControlFeature() and MdigInquireFeature(). You can also use Aurora Imaging Capture Works' Feature Browser to access the Video4Linux2 controls, which provide an interface to view and change the camera's configuration. For more information, refer to the Using Aurora Imaging Library with GenICam section of Chapter 27: Grabbing with your digitizer.

Zebra Video4Linux2 information in the Aurora Imaging Library Reference can be found in the paragraphs and values marked as being supported by the Video4Linux2 system.